BUKU BELAJAR PEMROGRAMAN

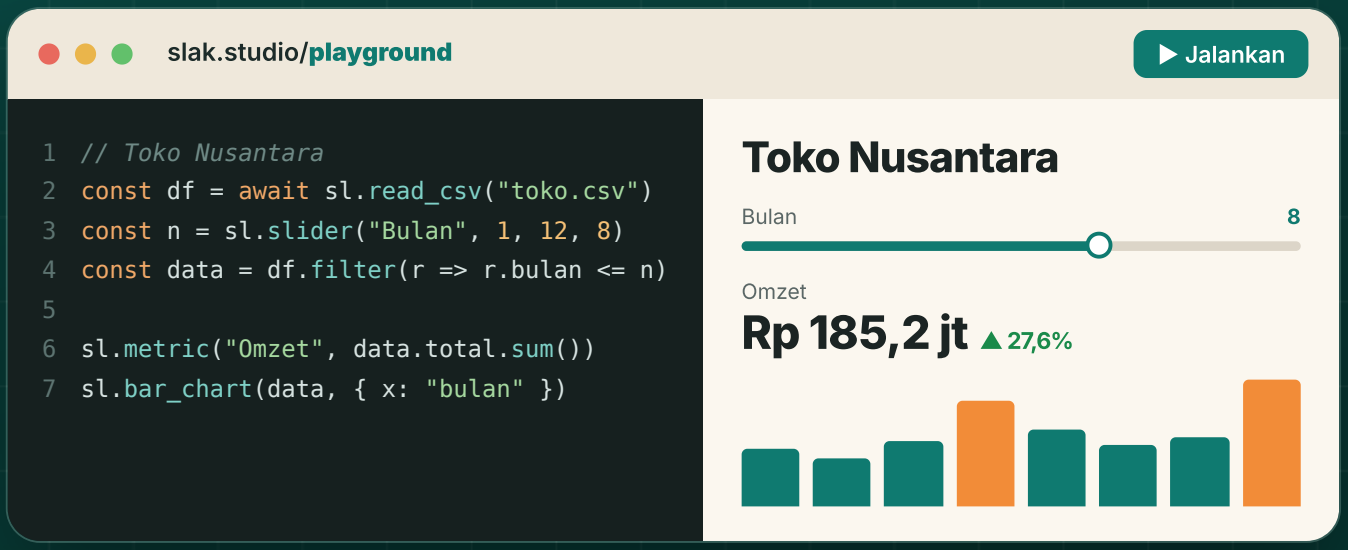
Sesuai Slak 1.9.1 · 2026

Sl.

Write and Run

Belajar JavaScript untuk Analisis Data dengan Slak.

Dari baris kode pertama hingga dasbor portofolio, langsung di browser. Tanpa instalasi, tanpa build.



JavaScript Pemula
Bab 1–10

Analis Data Pemula
Bab 11–17

Analis Data Menengah
Bab 18–26

Coba langsung
slak.studio/playground

26 bab · Latihan teruji otomatis · Data Toko Nusantara

Sebelum Mulai

Buku ini mengajarkan bahasa pemrograman JavaScript kepada orang yang belum pernah memprogram, lalu memakainya untuk menganalisis data sampai tingkat menengah. Semua contoh berjalan di browser dengan **Slak 1.9.1**, tanpa memasang server dan tanpa langkah build.

Untuk siapa buku ini

- Kamu yang belum pernah menulis program sama sekali.
- Kamu yang ingin menjadi analis data dan butuh portofolio yang bisa dibuka siapa saja lewat tautan.
- Guru dan mahasiswa yang mencari bahan belajar pemrograman berbahasa Indonesia dengan hasil yang langsung terlihat.

Kamu tidak perlu menguasai matematika lanjut. Cukup penjumlahan, persen, dan rata-rata. Konsep statistik dijelaskan saat dipakai.

Tiga tahap perjalanan

Tahap	Bab	Setelah selesai kamu bisa
1 · JavaScript Pemula	1–10	Menulis program kecil: variabel, percabangan, perulangan, fungsi, larik, objek, menangani galat, dan membuat aplikasi interaktif
2 · Analis Data Pemula	11–17	Membaca, membersihkan, meringkas, dan menggambar data, lalu menerbitkan laporan interaktif pertamamu
3 · Analis Data Menengah	18–26	Menggabungkan tabel, statistik, deret waktu, SQL, prakiraan, peta, merapikan kode, dan dasbor portofolio multi-halaman

Baca berurutan. Setiap bab memakai yang sudah dipelajari di bab sebelumnya.

Judul buku ini

Write and Run adalah filosofi resmi Slak: tulis kode, jalankan, langsung lihat hasilnya. Tidak ada terminal, tidak ada menunggu kompilasi. Begitulah kamu akan belajar di setiap halaman buku ini.

Cara membaca setiap bab

Setiap bab punya susunan yang sama:

1. **Tujuan belajar**: kemampuan yang kamu kuasai setelah bab selesai.
2. **Penjelasan konsep**: singkat, dengan contoh sehari-hari.

3. **Kode dan hasilnya**: di edisi web, setiap contoh langsung dijalankan dan hasilnya tampil tepat di bawah kodenya.
4. **Kotak "API Slak baru"**: muncul saat sebuah fungsi Slak dipakai pertama kali.
5. **Kotak "Coba di playground"**: eksperimen kecil untuk kamu ubah sendiri.
6. **Kesalahan umum**: galat yang sering muncul dan cara membacanya.
7. **Latihan**: tiga tingkat, yaitu mudah, sedang, dan tantangan.
8. **Mini-proyek**: aplikasi kecil yang menggabungkan isi bab.

> **Kotak "Fitur eksperimental"** hanya dipakai untuk fungsi yang di daftar API stabil Slak 1.9 masih berlabel eksperimental. Saat ini hanya satu: `sl.worker` (Bab 12). Cara pakainya boleh berubah sebelum Slak 2.0.

Latihan yang memeriksa dirinya sendiri

Kode awal setiap latihan sudah berisi uji otomatis dengan `sl.testing`, fitur bawaan Slak 1.9.1. Saat kamu menjalankannya, setiap uji tampil sebagai kotak hijau (lulus) atau merah (belum lulus) beserta alasannya. Tugasmu: ubah kodenya sampai semua kotak hijau.

Di edisi web, setiap contoh dan latihan punya tautan **Buka di Playground**. Tautan itu membuka kodenya di playground resmi `https://slak.studio/playground/`, siap kamu ubah. Jawaban latihan ada di Lampiran D.

Bahan buku

Folder bahan buku berisi dataset fiktif **Toko Nusantara**, jaringan toko oleh-oleh dengan sepuluh cabang dari Jayapura sampai Jakarta. Dataset ini dipakai mulai Tahap 2. Isinya dijelaskan di Lampiran E.

Berkas	Isi
`toko-nusantara.csv`	2.929 transaksi 2023–2024 yang sudah bersih
`toko-nusantara-kotor.csv`	394 transaksi yang sengaja berantakan, untuk Bab 13
`produk.csv`	20 produk beserta harga pokok dan pemasok
`cabang.csv`	10 cabang beserta kota, provinsi, dan koordinat
`target.csv`	Target omzet bulanan setiap cabang
`toko-nusantara-besar.csv`	Sekitar 250 ribu transaksi (29 MB), diunduh terpisah untuk Bab 12 dan Bab 21

Setiap berkas `.csv` punya pasangan `.csv.js` (data yang sudah dibungkus). Pasangan itu membuat contoh tetap jalan saat halaman buku dibuka dengan klik dua kali, tanpa internet.

Konvensi penulisan

- Kode ditulis tanpa titik koma di akhir baris, dengan tanda kutip ganda, sama seperti Buku Panduan Slak.
- Istilah yang dipakai: **larik** (array), **objek**, **galat** (error), **jalan ulang** (rerun), **pratinjau** (preview).
- Nama fungsi Slak selalu memakai awalan `sl.` dan sesuai daftar API stabil Slak 1.9.1.
- Fungsi yang butuh berkas di luar `slak.js` selalu disertai baris `<script>` yang perlu dimuat. Fungsi yang butuh internet diberi tanda **butuh internet**.

---

1. Mengenal JavaScript lewat Slak →

Bab 1 · Mengenal JavaScript lewat Slak

> **Tujuan belajar**
>
> - Memahami apa itu program dan apa itu JavaScript.
> - Menjalankan kode pertamamu di playground, di Slak Studio, atau di satu berkas HTML.
> - Membedakan pekerjaan JavaScript dan pekerjaan Slak.
> - Membagikan kodemu lewat tautan.

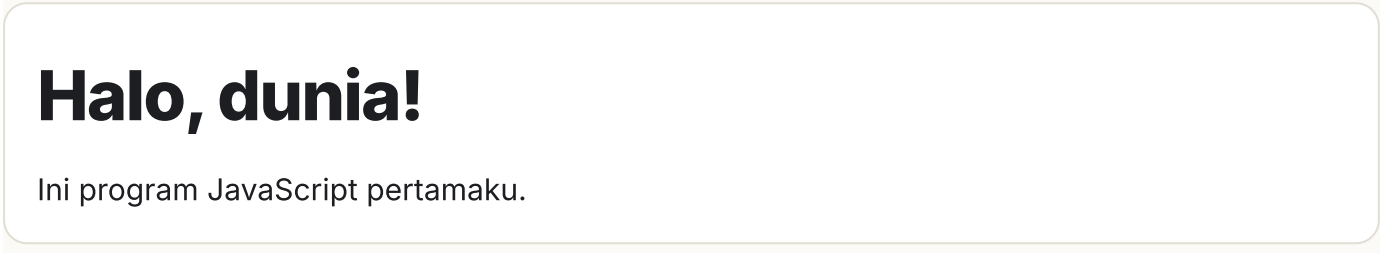
1.1 Program itu resep

Program adalah daftar perintah yang dikerjakan komputer satu per satu, dari atas ke bawah, persis seperti resep masakan. Bahasa pemrograman adalah bahasa yang dipakai untuk menulis resep itu.

JavaScript adalah bahasa pemrograman yang dipahami setiap browser: Chrome, Edge, Firefox, dan Safari. Artinya, untuk belajar JavaScript kamu tidak perlu memasang apa pun. Browser di laptop atau ponselmu sudah cukup.

Biasanya, membuat tampilan di browser dengan JavaScript cukup rumit: kamu harus mengurus HTML, CSS, dan cara tombol bereaksi saat diklik. **Slak** mengurus semua itu untukmu. Kamu cukup menulis perintah seperti "tampilkan judul" atau "buat penggeser angka", dan Slak menggambarnya.

```
sl.title("Halo, dunia!")
sl.write("Ini program JavaScript pertamaku.")
```



Buka di Playground ↗

Dua baris itu adalah program lengkap. Baris pertama menampilkan judul besar, baris kedua menampilkan kalimat biasa.

> **API Slak baru**
>
> - `sl.title(teks)` menampilkan judul besar di bagian atas.
> - `sl.write(nilai)` menampilkan apa saja: teks, angka, tabel, atau objek. Teks boleh memakai Markdown, misalnya `**tebal**`.

1.2 Write and Run

Filosofi Slak adalah **Write and Run**: tulis kode, jalankan, langsung lihat hasilnya. Tidak ada build, tidak ada `npm install`, tidak ada server. Inilah cara web dibuat pada masa awalnya, dan inilah cara kamu belajar di buku ini.

Ada tiga tempat untuk menjalankan kode Slak. Pilih yang paling mudah untukmu.

Cara 1: playground (tanpa memasang apa pun)

Buka **https://slak.studio/playground/** di browser. Layarnya terbagi dua:

- **Kiri**: editor, tempat kamu menulis kode.
- **Kanan**: pratinjau, tempat hasilnya tampil.

Saat kotak **Jalankan otomatis** dicentang, hasil diperbarui setiap kali kamu berhenti mengetik. Kamu juga bisa menekan **Ctrl+Enter** atau tombol **Jalankan**. Daftar di kiri atas berisi contoh siap pakai. Kodemu disimpan otomatis di browser, jadi tidak hilang saat halaman ditutup.

Inilah cara tercepat untuk memulai. Semua contoh Tahap 1 bisa kamu coba di sini.

Cara 2: Slak Studio (aplikasi desktop Windows)

Slak Studio adalah editor lengkap untuk Slak. Memasangnya cukup mengekstrak satu berkas zip:

1. Unduh `SlakStudio-1.9.1-Windows-x64.zip`.
2. Klik kanan berkas zip, pilih **Extract All...** (Ekstrak Semua), lalu pilih lokasi tetap, misalnya `Documents\Slak Studio`. Jangan menjalankan aplikasi dari dalam zip.
3. Buka folder hasil ekstrak, lalu klik dua kali **Slak Studio.exe**.
4. Jika muncul layar biru **Windows protected your PC**, klik **More info** (Info selengkapnya), lalu **Run anyway** (Tetap jalankan). Peringatan ini muncul karena aplikasinya belum bertanda tangan digital, dan hanya muncul pada pembukaan pertama.
5. Slak Studio terbuka dengan berkas `selamat-datang.slak` dan panel **Belajar** di kiri.

Kebutuhannya: Windows 10 atau 11 (64-bit), sekitar 370 MB ruang disk, dan 4 GB RAM atau lebih.

Cara 3: satu berkas HTML

Tanpa playground dan tanpa Slak Studio pun Slak tetap jalan. Simpan `slak.js` (dari paket Slak 1.9.1) dan berkas berikut di satu folder:

```
<!doctype html>
<html lang="id">
<head>
<meta charset="utf-8">
<title>Aplikasiku</title>
<script src="slak.js"></script>
</head>
<body>
<script type="text/slak">
sl.title("Halo, dunia!")
sl.write("Ini program JavaScript pertamaku.")
</script>
</body>
</html>
```

Beri nama `index.html`, lalu klik dua kali. Browser terbuka dan aplikasinya langsung jalan. Kode JavaScript-mu ditulis di dalam tag `<script type="text/slak">`.

1.3 Berkeliling Slak Studio

Kalau kamu memakai Slak Studio, kenali empat bagian ini dulu:

Bagian	Fungsi
Editor	Tempat menulis kode, lengkap dengan nomor baris, pewarnaan, dan saran fungsi
Pratinjau	Aplikasimu yang sedang berjalan; bisa diklik dan diisi seperti aplikasi sungguhan
Konsol	Keluaran `console.log()`, galat, dan penjelasan galat dalam bahasa Indonesia
Panel samping	Berkas, **Belajar** (49 pelajaran), **Contoh**, **Referensi** (tekan **F1**), dan Pintasan

Beberapa pintasan yang langsung berguna:

- **Ctrl+N**: berkas baru. **Ctrl+S**: simpan sebagai berkas `.slak`.
- **Ctrl+Enter**: jalankan. **Ctrl+Shift+Enter**: jalankan dari awal.
- **Ctrl+E**: ekspor menjadi satu berkas `.html` yang bisa dikirim ke siapa saja.

Slak Studio juga punya **mode sel**. Baris komentar `// %%` membagi kode menjadi beberapa sel. Saat kamu mengubah sel di bawah, sel di atasnya tidak dijalankan ulang, jadi mencoba kode sepotong demi sepotong terasa cepat. Tekan **Shift+Enter** untuk menjalankan sampai akhir sel tempat kursor berada. Di luar Slak Studio, baris `// %%` hanya dianggap komentar biasa.

```
// %% Judul
sl.title("Mode sel")

// %% Isi
sl.write("Sel ini bisa diubah tanpa menjalankan ulang sel di atasnya.")
```

Mode sel

Sel ini bisa diubah tanpa menjalankan ulang sel di atasnya.

Buka di Playground ↗

1.4 Apa yang dikerjakan JavaScript, apa yang dikerjakan Slak

Setiap baris di buku ini adalah JavaScript asli. Bedanya, perintah yang diawali `sl.` adalah fungsi Slak.

Dikerjakan JavaScript	Dikerjakan Slak
Menyimpan nilai di variabel	Menampilkan teks, tabel, dan grafik
Menghitung, membandingkan, mengulang	Membuat tombol, penggeser, kotak isian
Mengolah teks, tanggal, dan larik	Menjalankan ulang program saat pengguna berinteraksi
Memutuskan jalan mana yang diambil	Menyimpan data di browser, membaca berkas CSV

```
const harga = 25000
const jumlah = 3
const total = harga * jumlah
sl.write("Total belanja: " + total)
```

Total belanja: 75000

Buka di Playground ↗

Tiga baris pertama murni JavaScript: menyimpan dua angka, lalu mengalikannya. Baris terakhir memakai Slak untuk menampilkan hasilnya. Sepanjang buku, kamu akan belajar JavaScript lewat pola seperti ini: **hitung dengan JavaScript, tampilkan dengan Slak**.

1.5 Mengatur halaman

Judul tab browser dan ikon halaman diatur dengan `sl.set_page_config`. Panggil di baris paling atas.

```
sl.set_page_config({ page_title: "Perkenalan", page_icon: "👋" })
sl.title("Perkenalan")
```

> **API Slak baru**
>
> - `sl.set_page_config({ page_title, page_icon, layout })` mengatur judul tab, ikon, dan tata letak. `layout: "wide"` membuat konten lebih lebar.

Contoh ini ditandai *tidak dijalankan di halaman buku* karena akan mengganti judul halaman buku itu sendiri. Coba di playground.

1.6 Membagikan kodemu

Di playground, tombol 🔗 **Bagikan** menyalin tautan yang berisi kodemu, misalnya `https://slak.studio/playground/#kode=K87R…`. Kodenya dimampatkan ke bagian setelah tanda `#`, jadi tidak dikirim ke server mana pun. Siapa pun yang membuka tautan itu melihat kode dan hasilnya. Di Slak Studio, pilih menu **Berkas › Salin tautan berbagi**.

Tautan yang sama bisa dibuat dari kode dengan `Slak.share.link`:

```
const kode = 'sl.title("Halo dari temanmu")'
const tautan = await Slak.share.link(kode, "https://slak.studio/playground/")
sl.write("Tautan berbagi:")
sl.code(tautan)
```

Tautan berbagi:

```
https://slak.studio/playground/#kode=K87RK8ksyUnVUPJIzMlXSEksylQoSc1NzMstVdIEAA
```

Buka di Playground ↗

> **API Slak baru**
>
> - `Slak.share.link(kode, alamat)` membuat tautan playground yang berisi kode. Perhatikan awalannya `Slak.`, bukan `sl.`: ini fungsi milik Slak sebagai alat, bukan perintah untuk menampilkan sesuatu.

> - `sl.code(teks)` menampilkan teks sebagai kode.

Kata `await` di depan `Slak.share.link` artinya "tunggu sampai hasilnya siap". Kamu akan memahaminya di Bab 12.

> **Coba di playground** Ganti teks di dalam `sl.write` pada contoh 1.4, lalu ubah angka `harga` dan `jumlah`. Perhatikan totalnya berubah sendiri.

Kesalahan umum

Gejala	Penyebab	Perbaikan
`sl is not defined`	Kode ditulis di `<script>` biasa, bukan `<script type="text/slak">`	Pakai `type="text/slak"`
Halaman kosong	`slak.js` tidak ada di folder yang sama	Salin `slak.js` ke folder halaman
`Unexpected string`	Tanda kutip tidak berpasangan, misalnya `"Halo`	Tutup dengan `"`
Fungsi sl.Title tidak ada	Huruf besar dan kecil berbeda di JavaScript	Tulis `sl.title`, huruf kecil semua

Latihan

Latihan 1.1 (mudah). Tampilkan judul berisi namamu dan satu kalimat tentang hobimu.

```
sl.title("Ganti dengan namamu")
// Tulis satu kalimat tentang hobimu di bawah ini
```



Buka di Playground ↗

Latihan 1.2 (sedang). Sebuah toko menjual buku seharga 45000 dan pena seharga 7500. Hitung total belanja 2 buku dan 3 pena di variabel `total`, lalu tampilkan. Uji di bawah memeriksa nilainya.

```
const hargaBuku = 45000
const hargaPena = 7500
const total = 0   // ganti 0 dengan rumus yang benar
sl.write("Total: " + total)

await sl.testing.test("total belanja benar", () =>
sl.testing.expect(total).toBe(112500))
```

Total: 0

✕ **total belanja benar** — Diharapkan: sama persis dengan 112500 — nilai sebenarnya: 0

Buka di Playground ↗

Latihan 1.3 (tantangan). Buat tautan playground untuk kode `sl.title("Kuis Ibu Kota")`, simpan di variabel `tautan`, lalu tampilkan dengan `sl.code`.

```
let tautan = ""
// isi variabel tautan dengan Slak.share.link(...)

sl.code(tautan)
await sl.testing.test("tautan mengarah ke playground resmi", () =>
sl.testing.expect(tautan).toMatch("https://slak.studio/playground/#"))
```

✕ **tautan mengarah ke playground resmi** — Diharapkan: cocok dengan https://slak.studio/playground/# — nilai sebenarnya: ""

Buka di Playground ↗

Mini-proyek: halaman perkenalan diri

Buat halaman perkenalan diri, lalu bagikan lewat tombol **Bagikan** di playground.

```
sl.title("Halo, saya Rina 👋")
sl.write("Saya sedang belajar **JavaScript** untuk menjadi analis data.")
sl.write("Kota asal: Jayapura")
sl.write("Hobi: membaca, bersepeda, dan mencoba resep baru.")

const tahunLahir = 2004
const tahunIni = 2026
sl.write("Umur saya sekitar " + (tahunIni - tahunLahir) + " tahun.")
```

Halo, saya Rina 👋

Saya sedang belajar **JavaScript** untuk menjadi analis data.

Kota asal: Jayapura

Hobi: membaca, bersepeda, dan mencoba resep baru.

Umur saya sekitar 22 tahun.

Buka di Playground ↗

Ganti isinya dengan dirimu sendiri. Setelah Bab 2, kamu bisa mempercantiknya dengan warna dan emoji.

← Sebelum Mulai

2. Nilai, Variabel, dan Tipe Data →

Bab 2 · Nilai, Variabel, dan Tipe Data

> **Tujuan belajar**
>
> - Menyimpan nilai di variabel dengan `let` dan `const`.
> - Mengenal tiga tipe data dasar: angka, teks, dan boolean.
> - Memahami nilai kosong `null`, `undefined`, dan `NaN`, yang sering muncul di data nyata.
> - Memeriksa dan mengubah tipe data.
> - Mempercantik teks dengan Markdown, warna, dan emoji.

2.1 Variabel adalah kotak berlabel

Bayangkan sebuah kotak dengan label di luarnya. Kamu menaruh sesuatu di dalamnya, lalu memanggilnya lewat labelnya. Itulah **variabel**.

```
const namaToko = "Toko Nusantara"
let jumlahCabang = 10

sl.write(namaToko)
sl.write(jumlahCabang)
```

```
Toko Nusantara
10
```

Buka di Playground ↗

Ada dua cara membuat variabel:

Kata kunci	Arti	Kapan dipakai
`const`	Isinya tidak boleh diganti	Pilihan utama. Pakai ini kecuali kamu memang perlu mengganti isinya
`let`	Isinya boleh diganti	Untuk nilai yang berubah, misalnya penghitung

```
let stok = 50
stok = stok - 3      // terjual 3
stok = stok + 20     // datang barang baru
sl.write("Stok sekarang: " + stok)
```

```
Stok sekarang: 67
```

Buka di Playground ↗

Tanda `=` di JavaScript berarti **mengisi**, bukan "sama dengan". Baris `stok = stok - 3` artinya "ambil isi `stok`, kurangi 3, lalu masukkan hasilnya kembali ke `stok`".

Teks setelah `//` adalah **komentar**. Komentar diabaikan komputer dan hanya ditulis untuk manusia yang membaca kode.

Aturan memberi nama

- Boleh huruf, angka, `_`, dan `$`, tetapi tidak boleh diawali angka: `harga2` boleh, `2harga` tidak.
- Tidak boleh ada spasi. Untuk nama dari beberapa kata, tulis kata berikutnya diawali huruf besar: `hargaSatuan`, `jumlahCabang`. Gaya ini disebut *camelCase*.
- Huruf besar dan kecil dibedakan: `stok` dan `Stok` adalah dua variabel berbeda.
- Pilih nama yang menjelaskan isinya. `totalBelanja` jauh lebih jelas daripada `x`.

2.2 Tiga tipe data dasar

Setiap nilai punya **tipe**. Tipe menentukan apa yang bisa dilakukan terhadap nilai itu.

Tipe	Contoh	Keterangan
number (angka)	`25000`, `3.5`, `-7`	Desimal memakai titik, bukan koma
string (teks)	`"Jayapura"`, `'kopi'`	Diapit tanda kutip ganda atau tunggal
boolean	`true`, `false`	Hanya dua nilai: benar atau salah

```
const harga = 85000          // number
const produk = "Kopi Wamena" // string
const tersedia = true        // boolean

sl.write(typeof harga)
sl.write(typeof produk)
sl.write(typeof tersedia)
```

```
number

string

boolean
```

Buka di Playground ↗

`typeof` memberi tahu tipe sebuah nilai. Gunakan saat kamu ragu.

Angka

```
const a = 10
const b = 3
sl.write(a + b)   // tambah
sl.write(a - b)   // kurang
sl.write(a * b)   // kali
sl.write(a / b)   // bagi
sl.write(a % b)   // sisa bagi: 10 dibagi 3 sisanya 1
sl.write(a ** 2)  // pangkat
```

```
13
7
30
3,3333
1
100
```

Buka di Playground ↗

Saat menampilkan angka, `sl.write` memakai gaya Indonesia: titik untuk ribuan dan koma untuk desimal. Jadi `10 / 3` tampil `3,3333` (dibulatkan empat desimal), dan `25000` tampil `25.000`. Nilainya sendiri tidak berubah; hanya tampilannya yang diatur. Angka yang disambung ke dalam teks, misalnya `"Total: " + 25000`, tampil apa adanya.

Satu keanehan yang perlu kamu tahu sejak awal:

```
sl.write(String(0.1 + 0.2))
sl.write(0.1 + 0.2 === 0.3)
```

```
0.30000000000000004
false
```

Buka di Playground ↗

`String(...)` mengubah angka menjadi teks supaya tampil apa adanya, tanpa pembulatan tampilan. Hasilnya `0.30000000000000004`, bukan `0.3`, sehingga perbandingan di baris kedua bernilai `false`. Komputer menyimpan pecahan dalam bentuk biner, sehingga beberapa pecahan tidak bisa disimpan tepat. Untuk uang, hitung dalam rupiah bulat. Untuk menampilkan, bulatkan (Bab 8).

Teks

Teks bisa disambung dengan `+`:

```
const kota = "Jayapura"
sl.write("Cabang " + kota + " buka pukul 08.00")
```

```
Cabang Jayapura buka pukul 08.00
```

Buka di Playground ↗

Cara yang lebih rapi adalah **template literal**: teks diapit tanda backtick `` ` `` dan nilai disisipkan dengan `${...}`.

```
const kotaCabang = "Sorong"
const jam = 8
sl.write(`Cabang ${kotaCabang} buka pukul ${jam}.00`)
```

```
Cabang Sorong buka pukul 8.00
```

Buka di Playground ↗

Di dalam `${...}` boleh ada perhitungan, misalnya `` `Total: ${3 * 25000}` ``.

2.3 Nilai kosong: null, undefined, dan NaN

Data nyata hampir selalu punya lubang: sel yang lupa diisi, harga yang ditulis sebagai teks, kolom yang hilang. JavaScript punya tiga cara menyatakan "tidak ada":

Nilai	Arti	Contoh muncul
`undefined`	Belum pernah diisi	Variabel dibuat tanpa nilai: `let x`
`null`	Sengaja dikosongkan	Sel kosong di CSV sering dibaca menjadi `null`
`NaN`	*Not a Number*, hasil hitungan yang gagal	`Number("Rp 5.000")`

```
let belumDiisi
const sengajaKosong = null
const gagal = Number("Rp 5.000")

sl.write(belumDiisi)
sl.write(sengajaKosong)
sl.write(gagal)
sl.write(gagal + 1000)   // NaN menular ke hitungan berikutnya
```

```
undefined

null

NaN

NaN
```

Buka di Playground ↗

Perhatikan baris terakhir: sekali sebuah nilai menjadi `NaN`, semua hitungan yang memakainya ikut menjadi `NaN`. Inilah sebabnya analis data selalu memeriksa nilai kosong lebih dulu. Kamu akan membersihkannya di Bab 13.

Untuk memeriksa `NaN`, gunakan `Number.isNaN(x)`. Membandingkan `x === NaN` selalu menghasilkan `false`, bahkan bila `x` memang `NaN`.

2.4 Mengubah tipe

```
const dariTeks = Number("1250")
const keTeks = String(1250)
const desimal = parseFloat("3.75 kg")
const bulat = parseInt("12 buah")

sl.write(dariTeks + 50)    // 1300: angka ditambah angka
sl.write(keTeks + 50)      // "125050": teks disambung teks
sl.write(desimal)
sl.write(bulat)
```

```
1.300

125050

3,75
```

```
12
```

Buka di Playground ↗

`Number()` menolak teks yang tidak murni angka dan memberi `NaN`. `parseFloat()` dan `parseInt()` membaca angka di awal teks lalu berhenti di huruf pertama.

Baris kedua menunjukkan jebakan yang sering terjadi: `+` antara teks dan angka **menyambung**, bukan menjumlah. Bila hasil penjumlahanmu tampak aneh, periksa dulu apakah salah satu nilainya teks.

2.5 sl.write atau console.log?

	`sl.write(x)`	`console.log(x)`
Tampil di	Halaman aplikasi	Konsol (Slak Studio) atau DevTools browser (F12)
Untuk	Pengguna aplikasimu	Kamu sendiri, saat memeriksa isi variabel

Gunakan `sl.write` untuk hasil yang ingin dilihat pengguna, dan `console.log` untuk mengintip isi variabel saat mencari kesalahan (Bab 9).

2.6 Mempercantik teks

```
sl.markdown("Teks **tebal**, *miring*, dan `kode`.")
sl.markdown("Omzet naik :green[12%], biaya naik :red[3%].")
sl.markdown("Laporan bulanan :bar_chart: dari cabang Jayapura :jempol:")
sl.text("sl.text menampilkan teks apa adanya, **tanpa** Markdown.")
```

Teks **tebal**, *miring*, dan `kode`.

Omzet naik 12%, biaya naik 3%.

Laporan bulanan 📊 dari cabang Jayapura 👍

```
sl.text menampilkan teks apa adanya, **tanpa** Markdown.
```

Buka di Playground ↗

> **API Slak baru**
>
> - `sl.markdown(teks)` menampilkan teks berformat Markdown: tebal, miring, daftar, tabel, tautan, dan warna `:green[...]`, `:red[...]`, `:blue[...]`, `:orange[...]`, `:violet[...]`, `:gray[...]`.
> - `sl.text(teks)` menampilkan teks persis seperti ditulis.

> - Nama emoji ditulis di antara titik dua. Slak mengenal semua nama emoji Streamlit, misalnya `:bar_chart:` dan `:rocket:`, ditambah nama berbahasa Indonesia seperti `:jempol:`, `:senyum:`, dan `:kantung_uang:`.

Mau mencari nama emoji? `sl.emoji.search("uang")` mengembalikan daftar nama yang cocok.

```
sl.write(sl.emoji.search("uang").slice(0, 5))
```

	emoji	nama	kode
0	💰	uang	:uang:
1	💸	uang_bersayap	:uang_bersayap:
2	💴	uang_kertas_yen	:uang_kertas_yen:
3	💶	uang_kertas_euro	:uang_kertas_euro:
4	💵	uang_kertas_dolar	:uang_kertas_dolar:

5 baris × 3 kolom

Buka di Playground ↗

> **Coba di playground** Ubah `:green[12%]` menjadi `:orange[12%]`, lalu coba emoji `:rocket:` dan `:senyum:`.

Kesalahan umum

Gejala	Penyebab	Perbaikan
`Assignment to constant variable`	Mengganti isi variabel `const`	Pakai `let` bila isinya memang berubah
`stok is not defined`	Salah ketik nama, atau beda huruf besar-kecil	Samakan persis dengan nama saat dibuat
Hasil `"10050"` padahal ingin `150`	Salah satu nilai berupa teks	Ubah dengan `Number(...)`
Hasil `NaN`	Teks seperti `"Rp 5.000"` diubah ke angka	Buang dulu `Rp` dan titiknya (Bab 8)

Latihan

Latihan 2.1 (mudah). Buat variabel `const` bernama `produk` berisi `"Sagu Lempeng 1kg"` dan variabel `harga` berisi `30000`. Tampilkan kalimat `Sagu Lempeng 1kg harganya 30000` dengan template literal.

```
// buat variabel produk dan harga di sini

const kalimat = ""   // ganti dengan template literal
sl.write(kalimat)

await sl.testing.test("kalimat benar", () => sl.testing.expect(kalimat).toBe("Sagu Lempeng 1kg harganya 30000"))
```

✕ **kalimat benar** — Diharapkan: sama persis dengan "Sagu Lempeng 1kg harganya 30000" — nilai sebenarnya: ""

Buka di Playground ↗

Latihan 2.2 (sedang). Data dari formulir selalu berupa teks. Ubah `jumlahTeks` dan `hargaTeks` menjadi angka, lalu hitung `total`.

```
const jumlahTeks = "4"
const hargaTeks = "22000"
const total = jumlahTeks * hargaTeks + ""   // perbaiki baris ini

sl.write(total)
await sl.testing.test("total berupa angka", () => sl.testing.expect(typeof total).toBe("number"))
await sl.testing.test("total = 88000", () => sl.testing.expect(total).toBe(88000))
```

88000

✕ **total berupa angka** — Diharapkan: sama persis dengan "number" — nilai sebenarnya: "string"

✕ **total = 88000** — Diharapkan: sama persis dengan 88000 — nilai sebenarnya: "88000"

Buka di Playground ↗

Latihan 2.3 (tantangan). Variabel `isian` berisi teks dari pengguna. Buat variabel `sah` yang bernilai `true` bila `isian` bisa diubah menjadi angka, dan `false` bila tidak. Petunjuk: `Number.isNaN`.

```
const isian = "12a"
const sah = true   // ganti dengan pemeriksaan yang benar

sl.write(sah)
await sl.testing.test('"12a" bukan angka', () =>
sl.testing.expect(sah).toBe(false))
```

true

✕ **"12a" bukan angka** — Diharapkan: sama persis dengan false — nilai sebenarnya: true

Buka di Playground ↗

Mini-proyek: kartu data diri

Kumpulkan data dirimu di variabel, lalu tampilkan sebagai kartu yang rapi.

```
const nama = "Rina Wenda"
const kotaAsal = "Jayapura"
const tahunLahir = 2004
const sedangBelajar = true
const tahunSekarang = 2026

sl.title(`:wave: ${nama}`)
sl.markdown(`**Kota asal:** ${kotaAsal}`)
sl.markdown(`**Umur:** sekitar ${tahunSekarang - tahunLahir} tahun`)
sl.markdown(`**Sedang belajar JavaScript:** ${sedangBelajar ? ":green[ya]" :
":red[belum]"}`)
sl.markdown("Cita-cita: menjadi analis data :bar_chart:")
```

👋 Rina Wenda

Kota asal: Jayapura

Umur: sekitar 22 tahun

Sedang belajar JavaScript: ya

Cita-cita: menjadi analis data 📊

Buka di Playground ↗

Tanda `? :` pada baris keempat memilih salah satu dari dua nilai. Kamu akan mempelajarinya di bab berikutnya.

← 1. Mengenal JavaScript lewat Slak

3. Operator dan Percabangan →

Bab 3 · Operator dan Percabangan

> **Tujuan belajar**
>
> - Membandingkan nilai dengan `===`, `!==`, `<`, `>`, `<=`, `>=`.
> - Menggabungkan syarat dengan `&&`, `||`, dan `!`.
> - Membuat program memilih jalan dengan `if`, `else if`, `else`, `switch`, dan operator ternary.
> - Menerima masukan pengguna lewat widget Slak.

3.1 Perbandingan menghasilkan boolean

Setiap perbandingan menghasilkan `true` atau `false`.

```
const nilai = 78
sl.write(nilai > 75)     // lebih besar
sl.write(nilai <= 60)    // lebih kecil atau sama dengan
sl.write(nilai === 78)   // sama persis
sl.write(nilai !== 100)  // tidak sama
```

```
true
false
true
true
```

Buka di Playground ↗

Mengapa tiga tanda sama dengan?

JavaScript punya dua cara membandingkan:

```
sl.write(5 == "5")    // true: tipenya diam-diam diubah dulu
sl.write(5 === "5")   // false: angka dan teks tidak sama
sl.write(0 == "")     // true, mengejutkan
sl.write(0 === "")    // false
```

```
true
false
```

true

false

Buka di Playground ↗

`==` mengubah tipe data sebelum membandingkan, sehingga hasilnya sering mengejutkan. `===` baru bernilai `true` bila nilai **dan** tipenya sama persis. **Selalu pakai `===` dan `!==`.**

Ingat juga: `=` (satu) berarti *mengisi*, sedangkan `===` (tiga) berarti *membandingkan*.

3.2 Menggabungkan syarat

Operator	Arti	Contoh	Hasil
`&&`	dan: keduanya harus benar	`nilai >= 75 && hadir >= 80`	`true` bila kedua syarat terpenuhi
`|\`	|	`	atau: salah satu cukup
`!`	bukan	`!lunas`	kebalikan dari `lunas`

```
const nilaiUjian = 82
const kehadiran = 76
sl.write(nilaiUjian >= 75 && kehadiran >= 80)   // false: kehadiran kurang
sl.write(nilaiUjian >= 75 || kehadiran >= 80)   // true: nilainya cukup
```

false

true

Buka di Playground ↗

3.3 if, else if, else

`if` menjalankan sebuah blok kode hanya bila syaratnya benar.

```
const saldo = 120000
const harga = 150000

if (saldo >= harga) {
  sl.success("Pembayaran berhasil")
} else {
  sl.error(`Saldo kurang Rp ${harga - saldo}`)
}
```

Saldo kurang Rp 30000

Buka di Playground ↗

Untuk lebih dari dua kemungkinan, sambung dengan `else if`. Syarat diperiksa dari atas ke bawah, dan yang pertama benar yang dijalankan.

```
const suhu = 31
if (suhu >= 35) {
  sl.error("Sangat panas")
} else if (suhu >= 30) {
  sl.warning("Panas")
} else if (suhu >= 20) {
  sl.info("Sejuk")
} else {
  sl.info("Dingin")
}
```

Panas

Buka di Playground ↗

> **API Slak baru**
>
> - `sl.success(teks)`, `sl.info(teks)`, `sl.warning(teks)`, `sl.error(teks)` menampilkan kotak pesan hijau, biru, kuning, dan merah.

3.4 Menerima masukan pengguna

Sejauh ini nilai ditulis langsung di kode. Aplikasi sungguhan meminta nilai dari pengguna. Di Slak, **widget mengembalikan nilainya** seperti fungsi biasa:

```
const namaPembeli = sl.text_input("Nama pembeli")
const jumlahBeli = sl.number_input("Jumlah barang", { min: 0, value: 1, step: 1 })

if (namaPembeli === "") {
  sl.info("Isi nama pembeli dulu.")
} else {
  sl.write(`${namaPembeli} membeli ${jumlahBeli} barang.`)
}
```

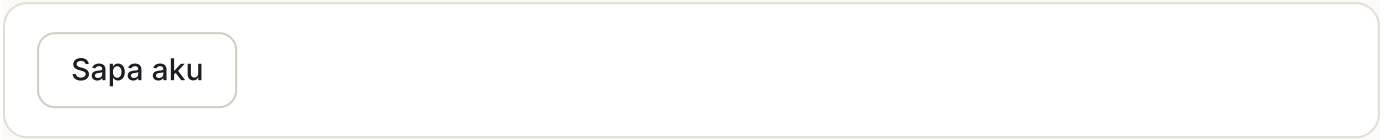


Buka di Playground ↗

Ketik nama, tekan Enter, lalu ubah jumlahnya. Setiap kali kamu mengubah widget, **seluruh program dijalankan ulang dari atas** dengan nilai baru. Itulah cara Slak bekerja, dan kamu akan memahaminya lebih dalam di Bab 10.

Tombol bekerja dengan cara yang sama: `sl.button` bernilai `true` hanya sekali, tepat setelah diklik.

```
if (sl.button("Sapa aku")) {
  sl.success("Halo! Semangat belajar :senyum:")
}
```



Buka di Playground ↗

> **API Slak baru**
>
> - `sl.text_input(label, nilaiAwal)` mengembalikan teks. Nilainya dipakai setelah kamu menekan Enter atau pindah ke kotak lain.
> - `sl.number_input(label, { min, max, value, step })` mengembalikan angka.
> - `sl.button(label)` mengembalikan `true` pada satu jalan ulang setelah diklik, lalu `false` lagi.

3.5 switch

Bila kamu membandingkan satu nilai dengan banyak pilihan pasti, `switch` lebih rapi daripada deretan `else if`.

```
const kodeCabang = "SOQ"
let namaKota
switch (kodeCabang) {
  case "JYP": namaKota = "Jayapura"; break
  case "SOQ": namaKota = "Sorong"; break
  case "MKW": namaKota = "Manokwari"; break
  default: namaKota = "Tidak dikenal"
}
sl.write(namaKota)
```

Sorong

Buka di Playground ↗

Jangan lupa `break` di setiap `case`. Tanpa `break`, program terus menjalankan `case` berikutnya. Di sini titik koma dipakai karena dua perintah ditulis di satu baris.

3.6 Operator ternary

Untuk memilih satu dari dua nilai, ada bentuk singkat `syarat ? nilaiBilaBenar : nilaiBilaSalah`.

```
const totalBelanja = 260000
const ongkir = totalBelanja >= 250000 ? 0 : 15000
sl.write(`Ongkos kirim: Rp ${ongkir}`)
```

Ongkos kirim: Rp 0

Buka di Playground ↗

Pakai ternary hanya untuk pilihan sederhana. Bila logikanya panjang, `if` lebih mudah dibaca.

Kesalahan umum

Gejala	Penyebab	Perbaikan
Syarat selalu benar	Menulis `if (x = 5)`	Pakai `===` untuk membandingkan
`Assignment to constant variable` di dalam `if`	`=` yang seharusnya `===` pada variabel `const`	Ganti dengan `===`
Semua cabang `switch` ikut jalan	Lupa `break`	Tambahkan `break` di setiap `case`

Gejala	Penyebab	Perbaikan
`else if` yang benar tidak pernah jalan	Syarat yang lebih luas ditulis lebih dulu	Urutkan dari syarat paling sempit atau paling tinggi

Latihan

Latihan 3.1 (mudah). Lengkapi fungsi `lulus(nilai)` agar mengembalikan `"Lulus"` bila nilai 75 ke atas dan `"Belum lulus"` bila kurang dari 75. Fungsi dibahas di Bab 5; untuk sekarang cukup isi bagian `if`.

```
function lulus(nilai) {
  // tulis if di sini
  return ""
}

await sl.testing.test("80 lulus", () => sl.testing.expect(lulus(80)).toBe("Lulus"))
await sl.testing.test("75 lulus", () => sl.testing.expect(lulus(75)).toBe("Lulus"))
await sl.testing.test("74 belum lulus", () =>
sl.testing.expect(lulus(74)).toBe("Belum lulus"))
```

- ✕ **80 lulus** — Diharapkan: sama persis dengan "Lulus" — nilai sebenarnya: ""
- ✕ **75 lulus** — Diharapkan: sama persis dengan "Lulus" — nilai sebenarnya: ""
- ✕ **74 belum lulus** — Diharapkan: sama persis dengan "Belum lulus" — nilai sebenarnya: ""

Buka di Playground ↗

Latihan 3.2 (sedang). Toko Nusantara memberi diskon 10% untuk belanja minimal Rp 500.000 **atau** bila pembeli adalah anggota. Lengkapi `diskon(total, anggota)` agar mengembalikan besar diskon dalam rupiah.

```
function diskon(total, anggota) {
  return 0
}

await sl.testing.test("600 ribu, bukan anggota", () =>
sl.testing.expect(diskon(600000, false)).toBe(60000))
await sl.testing.test("200 ribu, anggota", () => sl.testing.expect(diskon(200000,
true)).toBe(20000))
await sl.testing.test("200 ribu, bukan anggota", () =>
sl.testing.expect(diskon(200000, false)).toBe(0))
```

- ✕ **600 ribu, bukan anggota** — Diharapkan: sama persis dengan 60000 — nilai sebenarnya: 0
- ✕ **200 ribu, anggota** — Diharapkan: sama persis dengan 20000 — nilai sebenarnya: 0
- ✓ 200 ribu, bukan anggota (0 md)

Buka di Playground ↗

Latihan 3.3 (tantangan). Tahun kabisat habis dibagi 4, kecuali tahun yang habis dibagi 100; tahun yang habis dibagi 400 tetap kabisat. Lengkapi `kabisat(tahun)`. Petunjuk: operator sisa bagi `%`.

```
function kabisat(tahun) {
  return false
}

await sl.testing.test("2024 kabisat", () =>
sl.testing.expect(kabisat(2024)).toBe(true))
await sl.testing.test("2023 bukan", () =>
sl.testing.expect(kabisat(2023)).toBe(false))
await sl.testing.test("1900 bukan", () =>
sl.testing.expect(kabisat(1900)).toBe(false))
await sl.testing.test("2000 kabisat", () =>
sl.testing.expect(kabisat(2000)).toBe(true))
```

✗ **2024 kabisat** — Diharapkan: sama persis dengan true — nilai sebenarnya: false

✓ 2023 bukan (0 md)

✓ 1900 bukan (0 md)

✗ **2000 kabisat** — Diharapkan: sama persis dengan true — nilai sebenarnya: false

Buka di Playground ↗

Mini-proyek: kalkulator nilai rapor

Pengguna mengisi tiga nilai. Program menghitung rata-rata, lalu menentukan predikat A sampai E.

```
sl.title("Kalkulator Nilai Rapor :memo:")

const tugas = sl.number_input("Nilai tugas", { min: 0, max: 100, value: 80 })
const uts = sl.number_input("Nilai UTS", { min: 0, max: 100, value: 75 })
const uas = sl.number_input("Nilai UAS", { min: 0, max: 100, value: 70 })

const rata = (tugas + uts + uas) / 3

let predikat
if (rata >= 85) predikat = "A"
else if (rata >= 75) predikat = "B"
else if (rata >= 65) predikat = "C"
else if (rata >= 50) predikat = "D"
else predikat = "E"

sl.write(`Rata-rata: **${rata.toFixed(1)}**, predikat **${predikat}**`)

if (predikat === "A" || predikat === "B") {
  sl.success("Hebat, pertahankan!")
} else if (predikat === "C") {
  sl.info("Sudah cukup, masih bisa ditingkatkan.")
} else {
  sl.warning("Ayo belajar lebih giat lagi.")
}
```

Kalkulator Nilai Rapor 📝

Nilai tugas

80 − +

Nilai UTS

75 − +

Nilai UAS

70 − +

Rata-rata: **75.0**, predikat **B**

✅ Hebat, pertahankan!

Buka di Playground ↗

Bila isi `if` hanya satu baris, kurung kurawal boleh ditiadakan seperti pada bagian predikat. `rata.toFixed(1)` membulatkan angka menjadi satu desimal untuk ditampilkan.

Bab 4 · Perulangan

> **Tujuan belajar**
>
> - Mengulang perintah dengan `for`, `while`, dan `for...of`.
> - Menghentikan atau melewati putaran dengan `break` dan `continue`.
> - Menghitung total, rata-rata, nilai terbesar, dan nilai terkecil secara manual.
> - Menghindari perulangan tanpa akhir.

4.1 Mengapa perlu perulangan

Bayangkan kamu harus menjumlahkan omzet 365 hari. Menulis 365 baris penjumlahan jelas tidak masuk akal. Perulangan membuat komputer mengerjakan hal yang sama berulang kali untukmu. Inilah inti analisis data: mengerjakan satu langkah yang sama pada ribuan baris.

4.2 for

```
for (let i = 1; i <= 5; i++) {
  sl.write(`Putaran ke-${i}`)
}
```

Putaran ke-1

Putaran ke-2

Putaran ke-3

Putaran ke-4

Putaran ke-5

Buka di Playground ↗

Bagian di dalam kurung `for` terdiri dari tiga potong yang dipisah titik koma:

Bagian	Contoh	Arti
Awal	`let i = 1`	Buat penghitung `i`, mulai dari 1
Syarat	`i <= 5`	Ulangi selama syarat ini benar
Langkah	`i++`	Setelah setiap putaran, tambah `i` dengan 1

`i++` adalah cara singkat menulis `i = i + 1`. Ada juga `i--` (kurangi 1) dan `i += 5` (tambah 5).

```
// hitung mundur dari 10, lompat 2
for (let i = 10; i > 0; i -= 2) {
  sl.write(i)
}
```

```
10
8
6
4
2
```

Buka di Playground ↗

4.3 for...of: mengunjungi setiap isi daftar

Daftar nilai disebut **larik** (array), ditulis di antara kurung siku. Bab 6 membahasnya lengkap; untuk sekarang cukup tahu cara mengunjungi setiap isinya.

```
const kota = ["Jayapura", "Sorong", "Manokwari"]
for (const k of kota) {
  sl.write(`Cabang ${k}`)
}
```

```
Cabang Jayapura
Cabang Sorong
Cabang Manokwari
```

Buka di Playground ↗

`for...of` membaca "untuk setiap `k` di dalam `kota`". Kamu tidak perlu mengurus penghitung sama sekali.

4.4 Pola akumulator: total, rata-rata, terbesar, terkecil

Hampir semua ringkasan data dibuat dengan pola yang sama: siapkan sebuah variabel penampung, lalu perbarui isinya di setiap putaran.

```
const omzetHarian = [1250000, 980000, 1430000, 1100000, 1675000, 2010000, 1890000]

let total = 0
let terbesar = omzetHarian[0]
let terkecil = omzetHarian[0]

for (const x of omzetHarian) {
  total += x
  if (x > terbesar) terbesar = x
  if (x < terkecil) terkecil = x
}

const rataRata = total / omzetHarian.length

sl.write(`Total seminggu: ${total}`)
sl.write(`Rata-rata per hari: ${rataRata}`)
sl.write(`Tertinggi: ${terbesar}, terendah: ${terkecil}`)
```

Total seminggu: 10335000

Rata-rata per hari: 1476428.5714285714

Tertinggi: 2010000, terendah: 980000

Buka di Playground ↗

- `omzetHarian[0]` adalah isi pertama larik. Penghitungan posisi di JavaScript dimulai dari 0.
- `omzetHarian.length` adalah banyaknya isi larik.
- `total += x` sama dengan `total = total + x`.

Pola ini akan kamu temui terus. Di Bab 6 kamu belajar cara yang lebih singkat, dan di Bab 14 Slak Data mengerjakannya dalam satu perintah. Tetapi di baliknya, semuanya bekerja seperti kode di atas.

4.5 while

`while` mengulang selama syaratnya benar. Pakai bila kamu tidak tahu sejak awal berapa kali harus mengulang.

```
// Berapa bulan sampai tabungan mencapai 5 juta bila setiap bulan menabung 400 ribu
// dan saldo bertambah 0,5% per bulan?
let saldo = 0
let bulan = 0
while (saldo < 5000000) {
  saldo = saldo * 1.005 + 400000
  bulan++
}
sl.write(`Butuh ${bulan} bulan, saldo akhir ${Math.round(saldo)}`)
```

```
Butuh 13 bulan, saldo akhir 5358896
```

Buka di Playground ↗

`Math.round` membulatkan angka ke bilangan bulat terdekat.

4.6 break dan continue

- `break` menghentikan perulangan seketika.
- `continue` melewati sisa putaran ini dan lanjut ke putaran berikutnya.

```
const stokGudang = [12, 0, 7, 0, 3, 25]
for (let i = 0; i < stokGudang.length; i++) {
  if (stokGudang[i] === 0) continue          // lewati rak kosong
  if (stokGudang[i] > 20) {
    sl.write(`Rak ${i} penuh, berhenti memeriksa.`)
    break
  }
  sl.write(`Rak ${i}: ${stokGudang[i]} barang`)
}
```

```
Rak 0: 12 barang
Rak 2: 7 barang
Rak 4: 3 barang
Rak 5 penuh, berhenti memeriksa.
```

Buka di Playground ↗

4.7 Perulangan tanpa akhir

Bila syarat perulangan tidak pernah menjadi salah, program berputar selamanya:

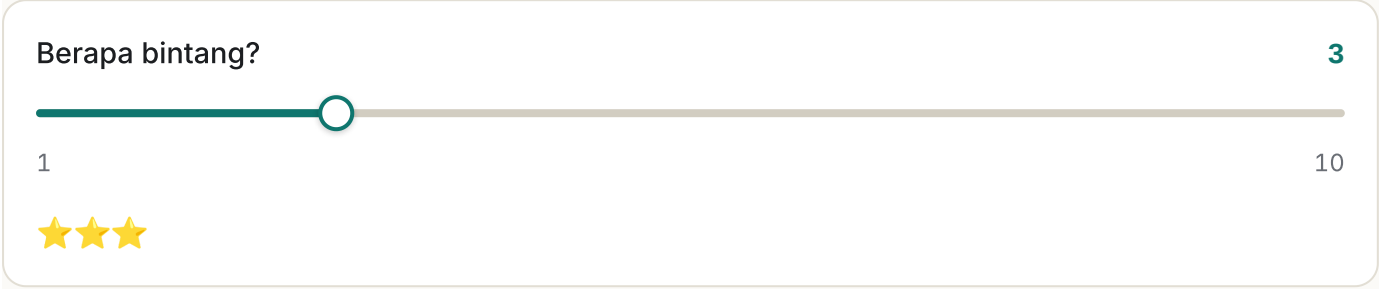
```
let n = 0
while (n < 10) {
  sl.write(n)
  // lupa n++, jadi n selalu 0
}
```

Slak Studio menyisipkan pengaman ke setiap perulangan. Bila satu perulangan berjalan lebih dari 2,5 detik tanpa jeda, perulangan itu dihentikan dengan pesan yang menunjuk barisnya. Di browser biasa tidak ada pengaman seperti ini, jadi selalu periksa: **apakah ada baris yang membuat syaratnya suatu saat salah?**

4.8 Penggeser angka

Perulangan menjadi lebih menarik bila banyaknya putaran dipilih pengguna.

```
const n = sl.slider("Berapa bintang?", 1, 10, 3)
let bintang = ""
for (let i = 0; i < n; i++) bintang += "⭐"
sl.write(bintang)
```



Buka di Playground ↗

> **API Slak baru**
>
> - `sl.slider(label, min, max, nilaiAwal, langkah)` mengembalikan angka yang dipilih. Urutan argumennya: label, nilai terkecil, nilai terbesar, nilai awal, lalu langkah (boleh dilewati).

> **Coba di playground** Ganti `"⭐"` dengan `":rocket: "`, lalu ubah batas slider menjadi 1 sampai 20.

Kesalahan umum

Gejala	Penyebab	Perbaikan
Aplikasi membeku atau Studio menghentikan perulangan	Syarat tidak pernah salah	Pastikan penghitung berubah setiap putaran
Satu isi terakhir tidak ikut dihitung	Syarat `i < panjang - 1`	Pakai `i < larik.length`
`undefined` di akhir	Syarat `i <= larik.length`	Posisi terakhir adalah `length - 1`, jadi pakai `<`
Total selalu 0	Variabel `total` dibuat di dalam perulangan	Buat penampung di luar perulangan

Latihan

Latihan 4.1 (mudah). Hitung jumlah semua bilangan dari 1 sampai 100 dengan `for`, simpan di `jumlah`.

```
let jumlah = 0
// tulis perulangan di sini

sl.write(jumlah)
await sl.testing.test("1 + 2 + ... + 100 = 5050", () =>
sl.testing.expect(jumlah).toBe(5050))
```

0

✕ **1 + 2 + ... + 100 = 5050** — Diharapkan: sama persis dengan 5050 — nilai sebenarnya: 0

Buka di Playground ↗

Latihan 4.2 (sedang). Hitung berapa hari omzet di atas 1,5 juta, simpan di `hariRamai`.

```
const omzet = [1250000, 980000, 1430000, 1100000, 1675000, 2010000, 1890000]
let hariRamai = 0
// tulis perulangan di sini

sl.write(hariRamai)
await sl.testing.test("ada 3 hari ramai", () =>
sl.testing.expect(hariRamai).toBe(3))
```

0

✕ **ada 3 hari ramai** — Diharapkan: sama persis dengan 3 — nilai sebenarnya: 0

Buka di Playground ↗

Latihan 4.3 (tantangan). Cari posisi (indeks) hari dengan omzet terendah, simpan di `posisiTerendah`.

```
const omzet2 = [1250000, 980000, 1430000, 1100000, 1675000, 2010000, 1890000]
let posisiTerendah = -1
// tulis perulangan di sini

sl.write(posisiTerendah)
await sl.testing.test("hari terendah ada di indeks 1", () =>
sl.testing.expect(posisiTerendah).toBe(1))
```

-1

✕ **hari terendah ada di indeks 1** — Diharapkan: sama persis dengan 1 — nilai sebenarnya: -1

Buka di Playground ↗

Mini-proyek: tabel perkalian

Ukuran tabel dipilih dengan slider. Tabelnya disusun sebagai teks Markdown dengan perulangan bersarang: satu perulangan untuk baris, satu lagi untuk kolom.

```
sl.title("Tabel Perkalian :abacus:")
const ukuran = sl.slider("Ukuran tabel", 2, 12, 5)

let kepala = "| × |"
let garis = "|---|"
for (let k = 1; k <= ukuran; k++) {
  kepala += ` ${k} |`
  garis += "---|"
}

let isi = ""
for (let b = 1; b <= ukuran; b++) {
```

```
  let baris = `| **${b}** |`
  for (let k = 1; k <= ukuran; k++) {
    baris += ` ${b * k} |`
  }
  isi += baris + "\n"
}

sl.markdown(kepala + "\n" + garis + "\n" + isi)
```

Tabel Perkalian 🧮

Ukuran tabel 5

2 12

×	1	2	3	4	5
1	1	2	3	4	5
2	2	4	6	8	10
3	3	6	9	12	15
4	4	8	12	16	20
5	5	10	15	20	25

Buka di Playground ↗

`"\n"` adalah tanda ganti baris di dalam teks. Markdown membutuhkan setiap baris tabel di baris tersendiri.

Bab 5 · Fungsi

> **Tujuan belajar**
>
> - Membuat dan memanggil fungsi, serta mengembalikan nilai dengan `return`.
> - Menulis fungsi panah (arrow function) dan parameter bawaan.
> - Memahami scope: variabel di dalam dan di luar fungsi.
> - Membuat fungsi statistik sendiri: `jumlah`, `rataRata`, `median`, dan `modus`.

5.1 Fungsi adalah resep yang diberi nama

Kalau sebuah hitungan dipakai berkali-kali, bungkus menjadi **fungsi**. Kamu menulisnya sekali, lalu memanggilnya kapan saja.

```
function hargaSetelahDiskon(harga, persen) {
  const potongan = harga * persen / 100
  return harga - potongan
}

sl.write(hargaSetelahDiskon(85000, 10))
sl.write(hargaSetelahDiskon(120000, 25))
```

76.500

90.000

Buka di Playground ↗

Bagian	Contoh	Arti
Nama	`hargaSetelahDiskon`	Dipakai saat memanggil
Parameter	`harga, persen`	Nilai yang diterima fungsi, seperti bahan resep
Isi	di antara `{ }`	Langkah-langkahnya
`return`	`return harga - potongan`	Hasil yang dikirim kembali ke pemanggil

Saat dipanggil dengan `hargaSetelahDiskon(85000, 10)`, nilai `85000` masuk ke `harga` dan `10` masuk ke `persen`. Nilai yang dikirim saat memanggil disebut **argumen**.

`return` juga menghentikan fungsi. Baris setelah `return` tidak dijalankan. Fungsi tanpa `return` mengembalikan `undefined`.

5.2 Parameter bawaan

Parameter boleh diberi nilai bawaan, yang dipakai bila argumennya tidak dikirim.

```
function hargaJual(hargaPokok, margin = 30) {
  return Math.round(hargaPokok * (1 + margin / 100))
}
sl.write(hargaJual(50000))       // memakai margin 30
sl.write(hargaJual(50000, 45))   // memakai margin 45
```

```
65.000
72.500
```

Buka di Playground ↗

5.3 Fungsi panah

JavaScript modern punya cara menulis fungsi yang lebih ringkas, yaitu **fungsi panah** (arrow function):

```
const pajak = (harga) => harga * 0.11
const keRibuan = (x) => Math.round(x / 1000) + " rb"

sl.write(pajak(200000))
sl.write(keRibuan(1250000))
```

```
22.000
1250 rb
```

Buka di Playground ↗

Bila isinya hanya satu ekspresi, tidak perlu `{ }` dan `return`. Bila isinya beberapa baris, tulis lengkap:

```
const ongkir = (beratKg, kota) => {
  const tarif = kota === "Jayapura" ? 8000 : 15000
  return Math.ceil(beratKg) * tarif
}
sl.write(ongkir(2.3, "Sorong"))
```

```
45.000
```

Buka di Playground ↗

`Math.ceil` membulatkan ke atas: 2,3 kg dihitung 3 kg. Pasangannya `Math.floor` membulatkan ke bawah.

Kedua bentuk fungsi sama-sama benar. Buku ini memakai `function` untuk fungsi yang panjang dan penting, dan fungsi panah untuk fungsi pendek, terutama di Bab 6 saat fungsi dikirim ke fungsi lain.

5.4 Scope: variabel punya wilayah

Variabel yang dibuat di dalam fungsi hanya hidup di dalam fungsi itu.

```
const kotaUtama = "Jayapura"   // di luar: bisa dibaca di mana saja

function sapa(nama) {
  const salam = "Selamat datang"   // di dalam: hanya ada di sini
  return `${salam}, ${nama}, di cabang ${kotaUtama}`
}

sl.write(sapa("Yohanes"))
sl.write(typeof salam)   // "undefined": salam tidak dikenal di luar fungsi
```

```
Selamat datang, Yohanes, di cabang Jayapura

undefined
```

Buka di Playground ↗

Aturan ini melindungi kodemu. Dua fungsi boleh sama-sama punya variabel bernama `total` tanpa saling mengganggu.

5.5 Fungsi statistik buatan sendiri

Analis data bekerja dengan ringkasan: jumlah, rata-rata, nilai tengah, nilai yang paling sering muncul. Mari menulisnya sendiri dengan JavaScript murni, supaya nanti kamu paham apa yang dikerjakan Slak Data di balik layar.

```
function jumlah(data) {
  let total = 0
  for (const x of data) total += x
  return total
}

function rataRata(data) {
  if (data.length === 0) return null
  return jumlah(data) / data.length
}

function median(data) {
  if (data.length === 0) return null
  const urut = [...data].sort((a, b) => a - b)
  const tengah = Math.floor(urut.length / 2)
  if (urut.length % 2 === 1) return urut[tengah]
  return (urut[tengah - 1] + urut[tengah]) / 2
}

function modus(data) {
  let terbaik = null
  let frekTerbaik = 0
  for (const x of data) {
    let frek = 0
    for (const y of data) if (y === x) frek++
    if (frek > frekTerbaik) {
      frekTerbaik = frek
      terbaik = x
    }
  }
  return terbaik
}

const jumlahBeli = [1, 2, 1, 3, 1, 5, 2, 1, 4, 2]
sl.write(`Jumlah: ${jumlah(jumlahBeli)}`)
sl.write(`Rata-rata: ${rataRata(jumlahBeli)}`)
sl.write(`Median: ${median(jumlahBeli)}`)
sl.write(`Modus: ${modus(jumlahBeli)}`)
```

Jumlah: 22

Rata-rata: 2.2

Median: 2

Modus: 1

Buka di Playground ↗

Beberapa hal baru di `median`:

- `[...data]` membuat salinan larik, supaya larik aslinya tidak ikut berubah urutannya.
- `.sort((a, b) => a - b)` mengurutkan angka dari kecil ke besar. Fungsi panah di dalamnya memberi tahu cara membandingkan dua angka. Bab 6 menjelaskannya.

Perhatikan juga bahwa `rataRata` memanggil `jumlah`. Fungsi kecil yang benar bisa dirangkai menjadi fungsi yang lebih besar.

Mengapa median penting? Rata-rata mudah tertarik oleh nilai ekstrem. Bila sembilan pelanggan belanja Rp 50.000 dan satu pelanggan belanja Rp 5.000.000, rata-ratanya Rp 545.000, padahal hampir semua pelanggan belanja jauh lebih sedikit. Mediannya tetap Rp 50.000. Untuk data uang, analis hampir selalu melihat keduanya.

5.6 Fungsi dan widget

```
const satuan = sl.selectbox("Ubah ke", ["gram", "ons", "kilogram"])
const kg = sl.number_input("Berat (kg)", { min: 0, value: 1.5, step: 0.5 })

function dariKg(nilai, ke) {
  if (ke === "gram") return nilai * 1000
  if (ke === "ons") return nilai * 10
  return nilai
}

const [kiri, kanan] = sl.columns(2)
kiri.metric("Kilogram", kg)
kanan.metric(satuan, dariKg(kg, satuan))
```

Ubah ke

gram

Berat (kg)

1.5 − +

Kilogram
1,5

gram
1.500

Buka di Playground ↗

> **API Slak baru**
>
> - `sl.selectbox(label, pilihan)` menampilkan daftar pilihan dan mengembalikan pilihan yang dipilih.
> - `sl.columns(n)` membagi baris menjadi `n` kolom dan mengembalikan larik kolom. Setiap kolom punya semua fungsi tampilan, misalnya `kiri.write(...)`.
> - `sl.metric(label, nilai)` menampilkan angka besar dengan label. Kamu akan sering memakainya untuk dasbor.

Baris `const [kiri, kanan] = sl.columns(2)` mengambil dua isi larik sekaligus ke dua variabel. Cara ini disebut *destructuring* dan dibahas di Bab 7.

Kesalahan umum

Gejala	Penyebab	Perbaikan
Hasil fungsi `undefined`	Lupa `return`	Tambahkan `return hasil`
`hitung is not a function`	Memanggil sesuatu yang bukan fungsi, atau salah nama	Periksa nama fungsi dan tanda kurungnya
Fungsi panah multi-baris memberi `undefined`	Memakai `{ }` tetapi lupa `return`	Tambahkan `return`, atau buang `{ }` untuk satu ekspresi
Larik asli ikut berubah urutan	`data.sort(...)` mengurutkan di tempat	Salin dulu: `[...data].sort(...)`

Latihan

Latihan 5.1 (mudah). Lengkapi fungsi `luasPersegiPanjang(p, l)`.

```
function luasPersegiPanjang(p, l) {

}

await sl.testing.test("4 × 5 = 20", () => sl.testing.expect(luasPersegiPanjang(4,
5)).toBe(20))
await sl.testing.test("2,5 × 2 = 5", () =>
sl.testing.expect(luasPersegiPanjang(2.5, 2)).toBe(5))
```

✗ **4 × 5 = 20** — Diharapkan: sama persis dengan 20 — nilai sebenarnya: undefined

✗ **2,5 × 2 = 5** — Diharapkan: sama persis dengan 5 — nilai sebenarnya: undefined

Buka di Playground ↗

Latihan 5.2 (sedang). Tulis fungsi panah `rentang` yang mengembalikan selisih nilai terbesar dan terkecil sebuah larik. Petunjuk: `Math.max(...data)` dan `Math.min(...data)`.

```
const rentang = (data) => 0

await sl.testing.test("rentang [3, 9, 4] = 6", () => sl.testing.expect(rentang([3,
9, 4])).toBe(6))
await sl.testing.test("rentang [5] = 0", () =>
sl.testing.expect(rentang([5])).toBe(0))
```

✗ **rentang [3, 9, 4] = 6** — Diharapkan: sama persis dengan 6 — nilai sebenarnya: 0

✓ rentang [5] = 0 (0 md)

Buka di Playground ↗

Latihan 5.3 (tantangan). Tulis fungsi `rataRataTerpangkas(data)` yang membuang satu nilai terbesar dan satu nilai terkecil, lalu menghitung rata-rata sisanya. Cara ini dipakai juri lomba agar satu nilai ekstrem tidak menentukan hasil.

```
function rataRataTerpangkas(data) {
  return 0
}

await sl.testing.test("[7, 8, 9, 10, 2] → 8", () =>
sl.testing.expect(rataRataTerpangkas([7, 8, 9, 10, 2])).toBe(8))
await sl.testing.test("data asli tidak berubah", () => {
  const d = [5, 1, 9]
  rataRataTerpangkas(d)
  sl.testing.expect(d).toEqual([5, 1, 9])
})
```

[7, 8, 9, 10, 2] → 8 — Diharapkan: sama persis dengan 8 — nilai sebenarnya: 0

data asli tidak berubah (0 md)

Buka di Playground ↗

Mini-proyek: konverter satuan

```
sl.title("Konverter Satuan :straight_ruler:")

const jenis = sl.selectbox("Jenis", ["Suhu", "Panjang", "Mata uang"])

function celsiusKe(c, ke) {
  if (ke === "Fahrenheit") return c * 9 / 5 + 32
  if (ke === "Kelvin") return c + 273.15
  return c
}

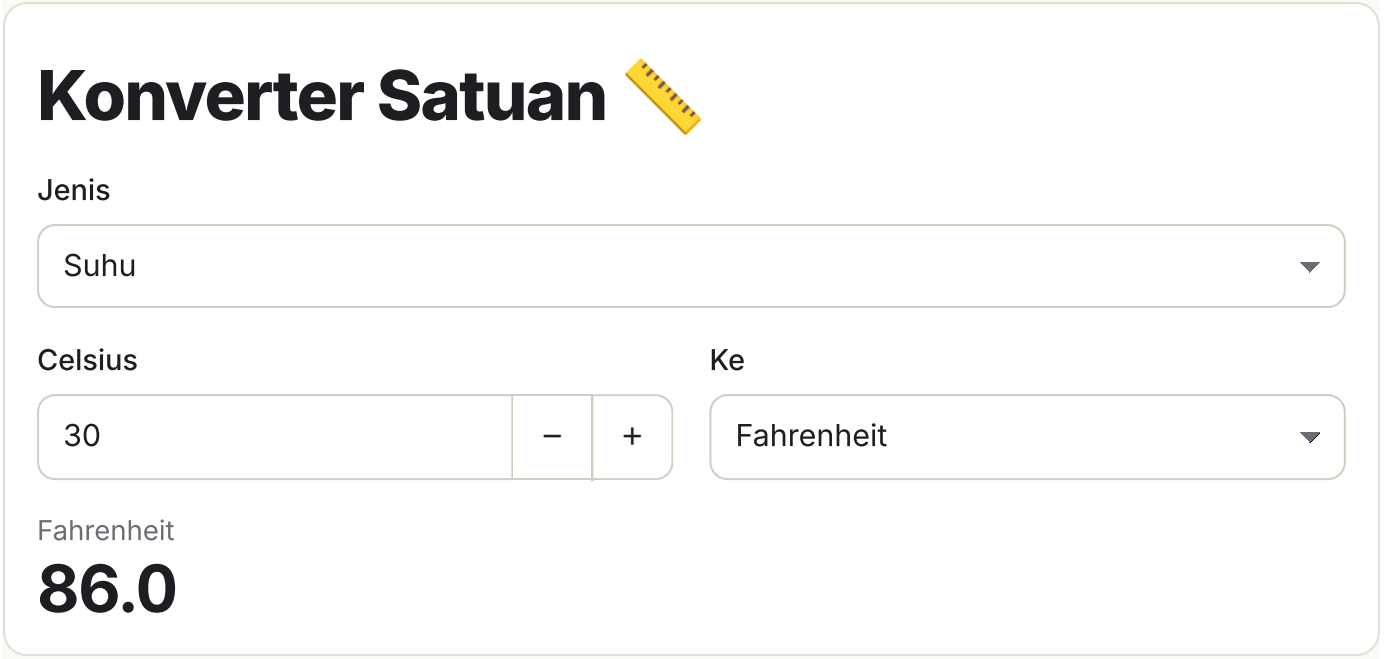
function meterKe(m, ke) {
  const faktor = { Sentimeter: 100, Kilometer: 0.001, Kaki: 3.28084 }
  return m * faktor[ke]
}

function rupiahKe(rp, ke, kurs) {
  return rp / kurs[ke]
}

const [kiri, kanan] = sl.columns(2)

if (jenis === "Suhu") {
  const c = kiri.number_input("Celsius", { value: 30 })
  const ke = kanan.selectbox("Ke", ["Fahrenheit", "Kelvin"], { key: "ke_suhu" })
  sl.metric(ke, celsiusKe(c, ke).toFixed(1))
} else if (jenis === "Panjang") {
  const m = kiri.number_input("Meter", { value: 1500 })
  const ke = kanan.selectbox("Ke", ["Kilometer", "Sentimeter", "Kaki"], { key:
"ke_panjang" })
  sl.metric(ke, meterKe(m, ke).toFixed(2))
} else {
  const kurs = { USD: 16200, SGD: 12100, AUD: 10500 }   // contoh kurs, bukan kurs
hari ini
  const rp = kiri.number_input("Rupiah", { value: 1000000, step: 50000 })
```

```
  const ke = kanan.selectbox("Ke", Object.keys(kurs), { key: "ke_uang" })
  sl.metric(ke, rupiahKe(rp, ke, kurs).toFixed(2))
  sl.caption("Kurs contoh, bukan kurs hari ini.")
}
```



Buka di Playground ↗

Tiga kotak pilihan "Ke" punya isi berbeda, jadi masing-masing diberi `key` sendiri. Slak mengenali widget dari jenis dan labelnya; `key` memberi identitas tetap sehingga pilihan di satu jenis tidak terbawa ke jenis lain. Bab 10 membahas `key` lebih jauh.

`faktor[ke]` dan `kurs[ke]` mengambil nilai dari objek berdasarkan nama. Objek dibahas di Bab 7.

← 4. Perulangan

6. Larik (Array) →

Bab 6 · Larik (Array)

> **Tujuan belajar**
>
> - Membuat, membaca, dan mengubah larik.
> - Memakai `push`, `pop`, `slice`, `includes`, dan `indexOf`.
> - Menguasai lima metode terpenting analis data: `map`, `filter`, `reduce`, `sort`, dan `find`.
> - Merangkai beberapa metode dalam satu baris.

Bab ini adalah bab terpenting di Tahap 1. Hampir semua pekerjaan analis data adalah mengubah, menyaring, dan meringkas daftar nilai. Bacalah pelan-pelan dan coba setiap contohnya.

6.1 Membuat dan membaca larik

Larik (array) adalah daftar nilai berurutan, ditulis di antara kurung siku dan dipisah koma.

```
const kota = ["Jayapura", "Sorong", "Manokwari", "Makassar"]

sl.write(kota[0])             // isi pertama
sl.write(kota[2])             // isi ketiga
sl.write(kota.length)         // banyaknya isi
sl.write(kota[kota.length - 1])  // isi terakhir
sl.write(kota[10])            // tidak ada: undefined
```

Jayapura

Manokwari

4

Makassar

`undefined`

Buka di Playground ↗

Posisi isi disebut **indeks** dan dimulai dari **0**. Jadi larik berisi 4 kota punya indeks 0, 1, 2, dan 3.

6.2 Mengubah larik

```
const keranjang = ["kopi", "sagu"]
keranjang.push("abon")          // tambah di akhir
sl.write(keranjang)
const keluar = keranjang.pop()  // ambil dan buang yang terakhir
sl.write(`Dikeluarkan: ${keluar}`)
keranjang[0] = "kopi robusta"   // ganti isi indeks 0
sl.write(keranjang)
```

```
[
  "kopi",
  "sagu",
  "abon"
]
```

Dikeluarkan: abon

```
[
  "kopi robusta",
  "sagu"
]
```

Buka di Playground ↗

Larik yang dibuat dengan `const` tetap boleh diubah isinya. Yang dilarang `const` hanyalah mengganti larik itu dengan larik lain.

6.3 Mencari di dalam larik

```
const produk = ["Kopi Wamena", "Sagu Lempeng", "Abon Cakalang", "Sambal Roa"]
sl.write(produk.includes("Sambal Roa"))   // ada atau tidak
sl.write(produk.indexOf("Abon Cakalang")) // di indeks berapa
sl.write(produk.indexOf("Teh"))           // -1 berarti tidak ada
sl.write(produk.slice(1, 3))              // potong dari indeks 1 sampai sebelum 3
```

true

2

-1

```
[
  "Sagu Lempeng",
  "Abon Cakalang"
]
```

Buka di Playground ↗

`slice` tidak mengubah larik asli; ia membuat larik baru.

6.4 map: ubah setiap isi

`map` menjalankan sebuah fungsi untuk setiap isi larik dan mengumpulkan hasilnya menjadi larik baru yang sama panjang.

```
const harga = [85000, 72000, 35000, 48000]
const hargaBaru = harga.map(h => h * 1.05)        // naik 5%
const dalamRibu = harga.map(h => `${h / 1000} rb`)
sl.write(hargaBaru)
sl.write(dalamRibu)
```

```
[
  89250,
  75600,
  36750,
  50400
]
```

```
[
  "85 rb",
  "72 rb",
  "35 rb",
  "48 rb"
]
```

Buka di Playground ↗

Fungsi panah `h => h * 1.05` dipanggil empat kali, sekali untuk setiap harga. `h` berisi harga yang sedang diproses.

Bila kamu juga butuh indeksnya, terima parameter kedua: `harga.map((h, i) => ...)`.

6.5 filter: ambil yang memenuhi syarat

`filter` menyimpan isi yang membuat fungsinya bernilai `true`, dan membuang sisanya.

```
const omzet = [1250000, 980000, 1430000, 1100000, 1675000, 2010000, 1890000]
const hariRamai = omzet.filter(x => x > 1500000)
sl.write(hariRamai)
sl.write(`${hariRamai.length} dari ${omzet.length} hari ramai`)
```

```
[
  1675000,
  2010000,
  1890000
]
```

3 dari 7 hari ramai

Buka di Playground ↗

Bandingkan dengan Latihan 4.2: pekerjaan yang dulu butuh lima baris perulangan kini cukup satu baris.

6.6 reduce: ringkas menjadi satu nilai

`reduce` adalah pola akumulator dari Bab 4 dalam bentuk fungsi.

```
const omzetMinggu = [1250000, 980000, 1430000, 1100000, 1675000, 2010000, 1890000]
const total = omzetMinggu.reduce((acc, x) => acc + x, 0)
const tertinggi = omzetMinggu.reduce((acc, x) => x > acc ? x : acc, omzetMinggu[0])
sl.write(`Total: ${total}`)
sl.write(`Tertinggi: ${tertinggi}`)
```

Total: 10335000

Tertinggi: 2010000

Buka di Playground ↗

Cara membacanya:

- `acc` (akumulator) adalah penampung. Isi awalnya argumen kedua `reduce`, yaitu `0`.
- Untuk setiap `x`, fungsi mengembalikan nilai `acc` yang baru.

- Setelah semua isi diproses, `reduce` mengembalikan `acc` terakhir.

Putaran	`acc` sebelum	`x`	`acc` sesudah
1	0	1.250.000	1.250.000
2	1.250.000	980.000	2.230.000
3	2.230.000	1.430.000	3.660.000
…	…	…	…

6.7 sort: mengurutkan

```
const angka = [10, 9, 100, 25]
sl.write([...angka].sort())                  // salah untuk angka!
sl.write([...angka].sort((a, b) => a - b))   // kecil ke besar
sl.write([...angka].sort((a, b) => b - a))   // besar ke kecil
```

```
[
  10,
  100,
  25,
  9
]
```

```
[
  9,
  10,
  25,
  100
]
```

```
[
  100,
  25,
  10,
  9
]
```

Buka di Playground ↗

Tanpa fungsi pembanding, `sort` mengurutkan sebagai **teks**, sehingga `100` dianggap lebih kecil dari `25` (karena `"1"` lebih awal daripada `"2"`). Untuk angka, selalu beri pembanding:

- `(a, b) => a - b` mengurutkan naik.
- `(a, b) => b - a` mengurutkan turun.

`sort` mengubah larik aslinya. Karena itu contoh di atas menyalin larik dulu dengan `[...angka]`.

Untuk teks berbahasa Indonesia, pakai `localeCompare`:

```
const nama = ["yosef", "Agus", "ani", "Benny"]
sl.write([...nama].sort((a, b) => a.localeCompare(b, "id")))
```

```
[
  "Agus",
  "ani",
  "Benny",
  "yosef"
]
```

Buka di Playground ↗

6.8 find dan some

```
const stok = [12, 0, 7, 0, 3, 25]
sl.write(stok.find(x => x > 10))        // isi pertama yang cocok
sl.write(stok.findIndex(x => x === 0))  // indeks pertama yang cocok
sl.write(stok.some(x => x === 0))       // adakah yang kosong?
sl.write(stok.every(x => x >= 0))       // semuanya tidak negatif?
```

```
12

1

true

true
```

Buka di Playground ↗

6.9 Merangkai metode

Karena `map` dan `filter` mengembalikan larik baru, metode bisa dirangkai. Bacalah dari kiri ke kanan seperti kalimat.

```
const transaksi = [120000, 45000, 310000, 89000, 560000, 22000, 410000]

const totalBesarSetelahPajak = transaksi
  .filter(t => t >= 100000)     // ambil transaksi besar
  .map(t => t * 1.11)           // tambahkan PPN 11%
  .reduce((a, t) => a + t, 0)   // jumlahkan

sl.write(Math.round(totalBesarSetelahPajak))
```

1.554.000

Buka di Playground ↗

"Ambil transaksi minimal 100 ribu, tambahkan pajak, lalu jumlahkan." Pola **saring → ubah → ringkas** ini adalah inti analisis data. Di Tahap 2, Slak Data memberi nama lain untuk langkah yang sama: `filter`, `assign`, dan `agg`.

6.10 Memilih banyak sekaligus

```
const semuaKota = ["Jayapura", "Sorong", "Manokwari", "Makassar", "Ambon"]
const dipilih = sl.multiselect("Pilih cabang", semuaKota, { default: ["Jayapura", "Sorong"] })
sl.write(`Kamu memilih ${dipilih.length} cabang: ${dipilih.join(", ")}`)
```

Pilih cabang

Jayapura × Sorong ×

Pilih opsi...

Kamu memilih 2 cabang: Jayapura, Sorong

Buka di Playground ↗

> **API Slak baru**
>
> - `sl.multiselect(label, pilihan, { default })` mengembalikan **larik** berisi pilihan yang dipilih.

`join(", ")` menyambung isi larik menjadi satu teks dengan pemisah koma.

> **Coba di playground** Pada contoh 6.9, ganti batas 100000 menjadi 300000, lalu ganti `.reduce(...)` dengan `.length` untuk menghitung banyaknya transaksi besar.

Kesalahan umum

Gejala	Penyebab	Perbaikan
Urutan angka aneh, misalnya 10, 100, 25, 9	`sort()` tanpa pembanding	`sort((a, b) => a - b)`
`map` menghasilkan larik berisi `undefined`	Fungsi panah memakai `{ }` tanpa `return`	Hapus `{ }` atau tambahkan `return`
`reduce of empty array with no initial value`	Lupa nilai awal pada larik kosong	Selalu beri argumen kedua, misalnya `0`
`Cannot read properties of undefined`	Indeks di luar batas	Ingat indeks terakhir adalah `length - 1`

Latihan

Latihan 6.1 (mudah). Dari daftar harga, buat larik `hargaPromo` berisi harga yang dipotong 20%.

```
const harga = [50000, 80000, 125000]
const hargaPromo = harga   // ganti dengan map

sl.write(hargaPromo)
await sl.testing.test("dipotong 20%", () =>
sl.testing.expect(hargaPromo).toEqual([40000, 64000, 100000]))
```

```
[
  50000,
  80000,
  125000
]
```

✕ **dipotong 20%** — Diharapkan: setara dengan [40000,64000,100000] — nilai sebenarnya: [50000,80000,125000]

Buka di Playground ↗

Latihan 6.2 (sedang). Hitung rata-rata nilai siswa yang **lulus** (nilai minimal 75) dengan `filter` dan `reduce`.

```
const nilai = [80, 62, 91, 75, 58, 88]
const rataLulus = 0   // ganti

sl.write(rataLulus)
await sl.testing.test("rata-rata yang lulus = 83.5", () =>
sl.testing.expect(rataLulus).toBe(83.5))
```

0

✕ **rata-rata yang lulus = 83.5** — Diharapkan: sama persis dengan 83.5 — nilai sebenarnya: 0

Buka di Playground ↗

Latihan 6.3 (tantangan). Buat larik `tigaTerbesar` berisi tiga omzet tertinggi, urut dari yang terbesar, tanpa mengubah larik `omzet`.

```
const omzet = [1250000, 980000, 1430000, 1100000, 1675000, 2010000, 1890000]
const tigaTerbesar = []   // ganti

sl.write(tigaTerbesar)
await sl.testing.test("tiga terbesar", () =>
sl.testing.expect(tigaTerbesar).toEqual([2010000, 1890000, 1675000]))
await sl.testing.test("omzet asli tidak berubah", () =>
sl.testing.expect(omzet[0]).toBe(1250000))
```

```
[]
```

✕ **tiga terbesar** — Diharapkan: setara dengan [2010000,1890000,1675000] — nilai sebenarnya: []

✓ omzet asli tidak berubah (0 md)

Buka di Playground ↗

Mini-proyek: daftar nilai siswa

```
sl.title("Daftar Nilai Siswa :books:")

const nama = ["Agus", "Benny", "Citra", "Dewi", "Elsa", "Frans", "Gita", "Hana"]
const nilaiSiswa = [78, 65, 92, 88, 71, 55, 83, 95]

const batas = sl.slider("Nilai minimum", 0, 100, 70)
const urutan = sl.selectbox("Urutkan", ["Nilai tertinggi", "Nilai terendah", "Nama
A–Z"])

// gabungkan nama dan nilai menjadi pasangan [nama, nilai]
let daftar = nama.map((n, i) => [n, nilaiSiswa[i]])

daftar = daftar.filter(p => p[1] >= batas)

if (urutan === "Nilai tertinggi") daftar.sort((a, b) => b[1] - a[1])
else if (urutan === "Nilai terendah") daftar.sort((a, b) => a[1] - b[1])
else daftar.sort((a, b) => a[0].localeCompare(b[0], "id"))

const rata = daftar.length ? daftar.reduce((s, p) => s + p[1], 0) / daftar.length :
0
sl.write(`**${daftar.length} siswa**, rata-rata ${rata.toFixed(1)}`)

for (const [n, v] of daftar) {
  sl.markdown(`- ${n}: ${v >= 75 ? `:green[${v}]` : `:orange[${v}]`}`)
}
```

Daftar Nilai Siswa 📚

Nilai minimum **70**

0 100

Urutkan

Nilai tertinggi

6 siswa, rata-rata 84.5

- Hana: 95
- Citra: 92

- Dewi: 88
- Gita: 83
- Agus: 78
- Elsa: 71

Buka di Playground ↗

Menyimpan nama dan nilai dalam dua larik terpisah lalu memasangkannya terasa repot. Di Bab 7 kamu belajar cara yang jauh lebih rapi: satu larik berisi objek.

← 5. Fungsi

7. Objek dan Larik Objek →

Bab 7 · Objek dan Larik Objek

> **Tujuan belajar**
>
> - Membuat objek dan membaca propertinya dengan titik dan kurung siku.
> - Memakai destructuring dan spread.
> - Memahami bahwa satu baris data adalah satu objek, dan satu tabel adalah larik objek.
> - Menampilkan larik objek sebagai tabel dengan Slak.

7.1 Objek: satu benda, banyak keterangan

Di Bab 6, nama dan nilai siswa disimpan di dua larik terpisah. Repot dan mudah tertukar. **Objek** menyimpan beberapa keterangan tentang satu benda di satu tempat, masing-masing diberi nama.

```
const produk = {
  nama: "Kopi Arabika Wamena 250g",
  kategori: "Minuman",
  harga: 85000,
  stok: 24,
  tersedia: true
}

sl.write(produk.nama)
sl.write(produk["harga"])
sl.write(produk)
```

Kopi Arabika Wamena 250g

85.000


```
{
  "nama": "Kopi Arabika Wamena 250g",
  "kategori": "Minuman",
  "harga": 85000,
  "stok": 24,
  "tersedia": true
}
```


Buka di Playground ↗

- Isi objek ditulis sebagai pasangan `nama: nilai`, dipisah koma, di antara kurung kurawal.
- Setiap pasangan disebut **properti**.
- Baca properti dengan titik (`produk.nama`) atau kurung siku (`produk["harga"]`).

Kurung siku wajib dipakai bila nama propertinya disimpan di variabel:

```
const barang = { nama: "Sagu Lempeng", harga: 30000, stok: 40 }
const kolom = sl.selectbox("Tampilkan kolom", ["nama", "harga", "stok"])
sl.write(barang[kolom])
```

Tampilkan kolom

nama

Sagu Lempeng

Buka di Playground ↗

7.2 Mengubah dan menambah properti

```
const item = { nama: "Sambal Roa 200g", harga: 45000 }
item.harga = 47000          // ubah
item.stok = 30              // tambah properti baru
delete item.stok            // hapus
sl.write(item)
sl.write("stok" in item)    // apakah properti ada?
```

```
{
  "nama": "Sambal Roa 200g",
  "harga": 47000
}
```

false

Buka di Playground ↗

`Object.keys`, `Object.values`, dan `Object.entries` mengubah objek menjadi larik, sehingga semua metode Bab 6 bisa dipakai:

```
const omzetKota = { Jayapura: 4200000, Sorong: 3100000, Manokwari: 1800000 }
sl.write(Object.keys(omzetKota))
sl.write(Object.values(omzetKota).reduce((a, b) => a + b, 0))
for (const [kota, omzet] of Object.entries(omzetKota)) {
  sl.write(`${kota}: ${omzet}`)
}
```

```
[
  "Jayapura",
  "Sorong",
  "Manokwari"
]
```

9.100.000

Jayapura: 4200000

Sorong: 3100000

Manokwari: 1800000

Buka di Playground ↗

7.3 Destructuring dan spread

Destructuring mengambil beberapa properti sekaligus ke variabel:

```
const trx = { id: "TN000123", kota: "Ambon", total: 170000, kanal: "Online" }
const { kota, total } = trx
sl.write(`${kota}: ${total}`)

const [pertama, kedua] = ["Jayapura", "Sorong", "Manokwari"]
sl.write(pertama + " dan " + kedua)
```

Ambon: 170000

Jayapura dan Sorong

Buka di Playground ↗

Spread (`...`) menyalin isi objek atau larik ke tempat baru:

```
const dasar = { kategori: "Minuman", pemasok: "Koperasi Baliem" }
const kopi = { ...dasar, nama: "Kopi Wamena", harga: 85000 }
const kopiPromo = { ...kopi, harga: 76500 }   // salinan dengan harga berbeda
sl.write(kopi)
sl.write(kopiPromo)
```


```
{
  "kategori": "Minuman",
  "pemasok": "Koperasi Baliem",
  "nama": "Kopi Wamena",
  "harga": 85000
}
```

```
{
  "kategori": "Minuman",
  "pemasok": "Koperasi Baliem",
  "nama": "Kopi Wamena",
  "harga": 76500
}
```


Buka di Playground ↗

Salinan dengan spread tidak mengubah objek asli. Kebiasaan ini membuat kode analisis aman: data mentah tetap utuh, hasil olahan disimpan di variabel baru.

7.4 Larik objek adalah tabel

Inilah gagasan terpenting bab ini: **satu baris data = satu objek, satu tabel = larik objek.**

```
const penjualan = [
  { tanggal: "2024-03-01", kota: "Jayapura", produk: "Kopi Wamena", jumlah: 2, harga: 85000 },
  { tanggal: "2024-03-01", kota: "Sorong", produk: "Sagu Lempeng", jumlah: 5, harga: 30000 },
  { tanggal: "2024-03-02", kota: "Jayapura", produk: "Sambal Roa", jumlah: 3, harga: 45000 },
  { tanggal: "2024-03-02", kota: "Manokwari", produk: "Kopi Wamena", jumlah: 1, harga: 85000 }
]

sl.table(penjualan)
```

	tanggal	kota	produk	jumlah	harga
0	1 Mar 2024	Jayapura	Kopi Wamena	2	85.000
1	1 Mar 2024	Sorong	Sagu Lempeng	5	30.000
2	2 Mar 2024	Jayapura	Sambal Roa	3	45.000
3	2 Mar 2024	Manokwari	Kopi Wamena	1	85.000

Buka di Playground ↗

Setiap objek menjadi satu baris, dan setiap nama properti menjadi judul kolom. Semua yang kamu pelajari di Bab 6 langsung berlaku:

```
const jual = [
  { kota: "Jayapura", produk: "Kopi Wamena", jumlah: 2, harga: 85000 },
  { kota: "Sorong", produk: "Sagu Lempeng", jumlah: 5, harga: 30000 },
  { kota: "Jayapura", produk: "Sambal Roa", jumlah: 3, harga: 45000 },
  { kota: "Manokwari", produk: "Kopi Wamena", jumlah: 1, harga: 85000 }
]

// tambah kolom total
```

```
const denganTotal = jual.map(r => ({ ...r, total: r.jumlah * r.harga }))

// saring Jayapura
const jayapura = denganTotal.filter(r => r.kota === "Jayapura")

// ringkas
const omzetJayapura = jayapura.reduce((a, r) => a + r.total, 0)

sl.dataframe(denganTotal)
sl.metric("Omzet Jayapura", omzetJayapura)
```

	kota	produk	jumlah	harga	total
0	Jayapura	Kopi Wamena	2	85.000	170.000
1	Sorong	Sagu Lempeng	5	30.000	150.000
2	Jayapura	Sambal Roa	3	45.000	135.000
3	Manokwari	Kopi Wamena	1	85.000	85.000

4 baris × 5 kolom

Omzet Jayapura

305.000

Buka di Playground ↗

Perhatikan `r => ({ ...r, total: ... })`. Objek yang dikembalikan fungsi panah harus diapit kurung biasa, supaya JavaScript tidak mengira `{` sebagai awal blok kode.

> **API Slak baru**
>
> - `sl.table(data)` menampilkan tabel statis.
> - `sl.dataframe(data)` menampilkan tabel interaktif: kolomnya bisa diurutkan dengan mengeklik judul, dan ada tombol unduh CSV.
> - `sl.metric(label, nilai, delta)` menampilkan angka besar. Argumen ketiga (opsional) adalah perubahan, misalnya `"+12%"`, yang tampil hijau atau merah.

7.5 Dataset bawaan Slak

Slak Data menyediakan beberapa dataset contoh yang selalu tersedia, tanpa berkas apa pun. Salah satunya `penjualan`: 3.278 transaksi toko di Papua tahun 2023–2024.

```
<script src="slak.js"></script>
<script src="slak-data.js"></script>
```

Dataset bawaan butuh `slak-data.js`. Playground, Slak Studio, dan halaman buku ini sudah memuatnya.

```
const df = sl.dataset("penjualan")
const baris = df.to_records()     // ubah menjadi larik objek biasa

sl.write(`Banyak baris: ${baris.length}`)
sl.write(baris[0])
sl.dataframe(baris.slice(0, 5))
```

Banyak baris: 3278

```
{
  "id_transaksi": "T00166",
  "tanggal": "2022-12-31T15:00:00.000Z",
  "pelanggan_id": "P043",
  "kota": "Sorong",
  "kanal": "Online",
  "kategori": "Minuman",
  "produk": "Kopi Papua 250g",
  "jumlah": 1,
  "harga": 65000,
  "total": 65000
}
```


	id_transaksi	tanggal	pelanggan_id	kota	kanal	kategori	produk	jumlah	harga	total
0	T00166	1 Jan 2023	P043	Sorong	Online	Minuman	Kopi Papua 250g	1	65.000	65.000
1	T00166	1 Jan 2023	P043	Sorong	Toko	Minuman	Kopi Papua 250g	2	65.000	130.000
2	T00166	1 Jan 2023	P043	Sorong	Toko	Makanan	Keripik Keladi	2	18.000	36.000
3	T00931	3 Jan 2023	P191	Jayapura	Toko	Snack	Cokelat Batang	2	22.000	44.000
4	T00931	3 Jan 2023	P191	Jayapura	Toko	Snack	Cokelat Batang	1	22.000	22.000

5 baris × 10 kolom

Buka di Playground ↗

> **API Slak baru**
>
> - `sl.dataset(nama)` mengembalikan dataset contoh sebagai DataFrame, yaitu tabel milik Slak Data (Bab 14).
> - `df.to_records()` mengubah DataFrame menjadi larik objek biasa, supaya bisa kamu olah dengan JavaScript murni.

Sampai Bab 13, kamu mengolah data dengan JavaScript murni: `filter`, `map`, `reduce`. Itu disengaja. Setelah paham caranya, Slak Data akan terasa wajar, bukan sihir.

```
const rows = sl.dataset("penjualan").to_records()
const online = rows.filter(r => r.kanal === "Online")
const omzetOnline = online.reduce((a, r) => a + r.total, 0)
const omzetSemua = rows.reduce((a, r) => a + r.total, 0)

const [k1, k2] = sl.columns(2)
k1.metric("Omzet online", omzetOnline)
k2.metric("Porsi online", (omzetOnline / omzetSemua * 100).toFixed(1) + "%")
```

Omzet online
160.074.000

Porsi online
44.2%

Buka di Playground ↗

> **Coba di playground** Ganti `"Online"` dengan `"Toko"`, lalu coba saring berdasarkan kota, misalnya `r.kota === "Sorong"`.

Kesalahan umum

Gejala	Penyebab	Perbaikan
`Cannot read properties of undefined (reading 'harga')`	Objeknya tidak ada, misalnya `find` tidak menemukan apa-apa	Periksa dulu: `if (p) { ... }` atau pakai `p?.harga`
Properti bernilai `undefined`	Salah ketik nama properti, atau beda huruf besar-kecil	Lihat isi objek dengan `sl.write(obj)`
`map` mengembalikan `undefined` saat membuat objek	Menulis `r => { ...r }`	Apit dengan kurung: `r => ({ ...r })`

Gejala	Penyebab	Perbaikan
Data asli ikut berubah	Mengubah `r.total = ...` di dalam `map`	Buat salinan: `({ ...r, total: ... })`

Latihan

Latihan 7.1 (mudah). Buat objek `cabang` dengan properti `kota` bernilai `"Ambon"` dan `buka` bernilai `2021`.

```
const cabang = {}

sl.write(cabang)
await sl.testing.test("kota Ambon", () => sl.testing.expect(cabang.kota).toBe("Ambon"))
await sl.testing.test("buka 2021", () => sl.testing.expect(cabang.buka).toBe(2021))
```

```
{}
```

✕ **kota Ambon** — Diharapkan: sama persis dengan "Ambon" — nilai sebenarnya: undefined

✕ **buka 2021** — Diharapkan: sama persis dengan 2021 — nilai sebenarnya: undefined

Buka di Playground ↗

Latihan 7.2 (sedang). Dari dataset `penjualan`, hitung omzet (jumlah kolom `total`) untuk kategori `"Pakaian"`.

```
const rows = sl.dataset("penjualan").to_records()
const omzetPakaian = 0   // ganti

sl.metric("Omzet pakaian", omzetPakaian)
await sl.testing.test("omzet pakaian benar", () => sl.testing.expect(omzetPakaian).toBe(
  rows.filter(r => r.kategori === "Pakaian").reduce((a, r) => a + r.total, 0)))
await sl.testing.test("tidak nol", () => sl.testing.expect(omzetPakaian).toBeGreaterThan(0))
```

Omzet pakaian

0

✕ **omzet pakaian benar** — Diharapkan: sama persis dengan 57535000 — nilai sebenarnya: 0

✕ **tidak nol** — Diharapkan: lebih besar dari 0 — nilai sebenarnya: 0

Buka di Playground ↗

Latihan 7.3 (tantangan). Buat objek `omzetPerKota` yang berisi omzet setiap kota, misalnya `{ Jayapura: ..., Sorong: ... }`. Petunjuk: mulai dari objek kosong, lalu tambahkan dengan `obj[kota] = (obj[kota] || 0) + r.total`.

```
const data = sl.dataset("penjualan").to_records()
const omzetPerKota = {}
// isi dengan perulangan

sl.write(omzetPerKota)
await sl.testing.test("semua kota ada", () => sl.testing.expect(Object.keys(omzetPerKota).length).toBe(new
Set(data.map(r => r.kota)).size))
await sl.testing.test("jumlahnya sama dengan total omzet", () =>
  sl.testing.expect(Object.values(omzetPerKota).reduce((a, b) => a + b, 0)).toBe(data.reduce((a, r) => a + r.total,
0)))
```

```
{}
```

semua kota ada — Diharapkan: sama persis dengan 6 — nilai sebenarnya: 0

jumlahnya sama dengan total omzet — Diharapkan: sama persis dengan 362287000 — nilai sebenarnya: 0

Buka di Playground ↗

Pola di latihan 7.3 disebut **mengelompokkan** (group by). Di Bab 15 Slak Data melakukannya dengan satu perintah `groupby`.

Mini-proyek: katalog produk dengan pencarian

```
sl.title("Katalog Toko Nusantara :shopping:")

const katalog = [
  { produk: "Kopi Arabika Wamena 250g", kategori: "Minuman", harga: 85000, stok: 24 },
  { produk: "Teh Kayu Manis 100g", kategori: "Minuman", harga: 35000, stok: 0 },
  { produk: "Keripik Keladi 200g", kategori: "Makanan", harga: 22000, stok: 80 },
  { produk: "Abon Ikan Cakalang 250g", kategori: "Makanan", harga: 55000, stok: 15 },
  { produk: "Noken Anyaman Kecil", kategori: "Kerajinan", harga: 150000, stok: 6 },
  { produk: "Gelang Manik Papua", kategori: "Kerajinan", harga: 40000, stok: 32 },
  { produk: "Minyak Kayu Putih 60ml", kategori: "Kebutuhan Rumah", harga: 38000, stok: 50 }
]

const cari = sl.text_input("Cari produk", "")
const kategoriDipilih = sl.multiselect("Kategori", ["Minuman", "Makanan", "Kerajinan", "Kebutuhan Rumah"])
const hanyaAda = sl.checkbox("Hanya yang tersedia")

const hasil = katalog
  .filter(p => p.produk.toLowerCase().includes(cari.toLowerCase()))
  .filter(p => kategoriDipilih.length === 0 || kategoriDipilih.includes(p.kategori))
  .filter(p => !hanyaAda || p.stok > 0)

sl.caption(`${hasil.length} dari ${katalog.length} produk`)
sl.dataframe(hasil)

if (hasil.length > 0) {
  const termurah = hasil.reduce((a, p) => p.harga < a.harga ? p : a)
  sl.info(`Termurah: ${termurah.produk} (${termurah.harga})`)
}
```

Katalog Toko Nusantara 🛍

Cari produk

Kategori

Pilih opsi...

Hanya yang tersedia

7 dari 7 produk

	produk	kategori	harga	stok
0	Kopi Arabika Wamena 250g	Minuman	85.000	24
1	Teh Kayu Manis 100g	Minuman	35.000	0
2	Keripik Keladi 200g	Makanan	22.000	80
3	Abon Ikan Cakalang 250g	Makanan	55.000	15
4	Noken Anyaman Kecil	Kerajinan	150.000	6
5	Gelang Manik Papua	Kerajinan	40.000	32
6	Minyak Kayu Putih 60ml	Kebutuhan Rumah	38.000	50

7 baris × 4 kolom

ⓘ Termurah: Keripik Keladi 200g (22000)

Buka di Playground ↗

> **API Slak baru**
>
> - `sl.checkbox(label)` mengembalikan `true` atau `false`.

`toLowerCase()` mengubah teks menjadi huruf kecil, sehingga pencarian "kopi" tetap menemukan "Kopi". Teks dibahas di bab berikutnya.

Bab 8 · Teks, Angka, dan Tanggal

> **Tujuan belajar**
>
> - Mengolah teks: `trim`, `toLowerCase`, `split`, `replace`, `includes`, `padStart`.
> - Membulatkan angka dan menampilkan Rupiah dengan format Indonesia.
> - Membuat, membaca, dan menghitung selisih tanggal dengan `Date`.
> - Menampilkan tanggal berbahasa Indonesia.

8.1 Metode teks yang paling sering dipakai

Teks (string) punya banyak metode bawaan. Semuanya **mengembalikan teks baru**; teks aslinya tidak berubah.

```
const mentah = "  Kopi Arabika WAMENA  "
sl.write(`[${mentah.trim()}]`)          // buang spasi di awal dan akhir
sl.write(mentah.trim().toLowerCase())    // huruf kecil semua
sl.write(mentah.trim().toUpperCase())    // huruf besar semua
sl.write(mentah.includes("Arabika"))     // memuat kata?
sl.write(mentah.trim().length)           // panjang teks
```

```
[Kopi Arabika WAMENA]
kopi arabika wamena
KOPI ARABIKA WAMENA
true
19
```

Buka di Playground ↗

```
const baris = "TN000123;2024-03-01;Jayapura;85000"
const bagian = baris.split(";")          // pecah menjadi larik
sl.write(bagian)
sl.write(bagian[2])

const kode = "7"
sl.write(kode.padStart(4, "0"))          // "0007"
sl.write("Kopi Wamena".replace("Wamena", "Toraja"))
sl.write("a-b-c".replaceAll("-", "/"))
```

```
[
  "TN000123",
  "2024-03-01",
  "Jayapura",
  "85000"
]
```

Jayapura

0007

Kopi Toraja

a/b/c

Buka di Playground ↗

`split` sangat penting untuk analis data. Berkas CSV pada dasarnya adalah teks yang dipecah per baris, lalu per koma. Di Bab 11 kamu akan membaca CSV dengan `split` sebelum memakai pembaca CSV milik Slak.

Membersihkan teks dengan pola

Kadang yang ingin dibuang bukan satu kata, tetapi "semua yang bukan angka". Untuk itu ada **ekspresi reguler** (regex), pola yang ditulis di antara garis miring. Kamu cukup mengenal satu pola ini:

```
const hargaTeks = "Rp 1.250.000"
const hanyaAngka = hargaTeks.replace(/[^0-9]/g, "")
sl.write(hanyaAngka)
sl.write(Number(hanyaAngka) + 50000)
```

1250000

1.300.000

Buka di Playground ↗

`/[^0-9]/g` artinya "setiap karakter yang bukan 0 sampai 9, di seluruh teks". Menggantinya dengan `""` menyisakan angkanya saja.

8.2 Membulatkan angka

```
const x = 1234.5678
sl.write(Math.round(x))        // 1235
sl.write(Math.floor(x))        // 1234
sl.write(Math.ceil(x))         // 1235
sl.write(x.toFixed(2))         // "1234.57" (teks!)
sl.write(Math.round(x * 100) / 100)  // 1234.57 (angka)
```

1.235

1.234

1.235

1234.57

1.234,57

Buka di Playground ↗

Perhatikan hasilnya di layar: `sl.write` menampilkan **angka** dengan gaya Indonesia (`1.235` dan `1.234,57`), sedangkan **teks** hasil `toFixed` tampil apa adanya (`1234.57`).

`toFixed` mengembalikan **teks**. Pakai untuk menampilkan, bukan untuk dihitung lagi.

8.3 Format Rupiah dan angka Indonesia

Orang Indonesia menulis ribuan dengan titik dan desimal dengan koma: `1.250.000,50`. JavaScript menyediakan `Intl.NumberFormat` untuk menulis angka sesuai kebiasaan negara mana pun.

```
const rupiah = new Intl.NumberFormat("id-ID", { style: "currency", currency: "IDR",
maximumFractionDigits: 0 })
const angkaID = new Intl.NumberFormat("id-ID")
const persen = new Intl.NumberFormat("id-ID", { style: "percent",
maximumFractionDigits: 1 })

sl.write(rupiah.format(1250000))
sl.write(angkaID.format(3278.5))
sl.write(persen.format(0.1234))
sl.write((1250000).toLocaleString("id-ID"))   // cara singkat
```

Rp 1.250.000

3.278,5

12,3%

1.250.000

Buka di Playground ↗

Buat formatnya sekali di variabel, lalu pakai berulang. Membuatnya ulang di setiap baris memperlambat program saat datanya ribuan.

`sl.metric` juga bisa memformat sendiri. Opsi `format: "rp"` menampilkan Rupiah:

```
sl.metric("Omzet Maret", 48250000, "+8%", { format: "rp" })
```

Omzet Maret

Rp 48.250.000

▲ +8%

Buka di Playground ↗

8.4 Tanggal dengan Date

```
const hariIni = new Date()
const tanggalTetap = new Date("2024-04-10")      // dari teks ISO: tahun-bulan-hari
const dariAngka = new Date(2024, 3, 10)          // bulan dihitung dari 0: 3 =
April!

sl.write(`Tahun ini: ${hariIni.getFullYear()}`)
sl.write(`Bulan: ${tanggalTetap.getMonth() + 1}`)   // tambah 1 agar menjadi 1–12
sl.write(`Tanggal: ${dariAngka.getDate()}`)
sl.write(`Hari ke: ${dariAngka.getDay()}`)          // 0 = Minggu, 1 = Senin, ...
```

Tahun ini: 2026

Bulan: 4

Tanggal: 10

Hari ke: 3

Buka di Playground ↗

Dua jebakan yang paling sering:

1. **Bulan dihitung dari 0.** Januari adalah 0, Desember adalah 11. Selalu tambah 1 saat menampilkan.
2. **Teks tanggal sebaiknya berformat ISO** `"2024-04-10"`. Format lain seperti `"10/04/2024"` dibaca berbeda-beda oleh tiap browser. Slak Data membaca format Indonesia dengan benar (Bab 11), tetapi `new Date` biasa tidak.

Selisih hari

Di balik layar, tanggal disimpan sebagai jumlah milidetik sejak 1 Januari 1970. Selisih dua tanggal adalah selisih milidetiknya.

```
const MS_SEHARI = 24 * 60 * 60 * 1000
const awal = new Date("2024-01-01")
const lebaran = new Date("2024-04-10")
const selisih = Math.round((lebaran - awal) / MS_SEHARI)
sl.write(`${selisih} hari dari awal tahun sampai Lebaran 2024`)
```

100 hari dari awal tahun sampai Lebaran 2024

Buka di Playground ↗

Tanggal berbahasa Indonesia

```
const t = new Date("2024-04-10")
const panjang = new Intl.DateTimeFormat("id-ID", { weekday: "long", day: "numeric",
month: "long", year: "numeric" })
const pendek = new Intl.DateTimeFormat("id-ID", { day: "2-digit", month: "short",
year: "numeric" })
sl.write(panjang.format(t))
sl.write(pendek.format(t))
sl.write(t.toISOString().slice(0, 10))   // kembali ke format ISO
```

Rabu, 10 April 2024

10 Apr 2024

2024-04-10

Buka di Playground ↗

Slak juga punya `sl.strftime` bila kamu terbiasa dengan format gaya Python:

```
sl.write(sl.strftime(new Date("2024-04-10"), "%A, %d %B %Y"))
```

Rabu, 10 April 2024

Buka di Playground ↗

8.5 Memilih tanggal

```
const tgl = sl.date_input("Tanggal transaksi", "2024-04-10")
sl.write(typeof tgl)
sl.write(tgl)
```

Tanggal transaksi

04/10/2024

string

2024-04-10

Buka di Playground ↗

> **API Slak baru**
>
> - `sl.date_input(label, nilaiAwal)` mengembalikan tanggal sebagai **teks** `"YYYY-MM-DD"`. Ubah dengan `new Date(tgl)` bila ingin menghitung.
> - `sl.strftime(tanggal, pola)` memformat tanggal dengan pola seperti `%d`, `%B`, `%Y`, `%A`.

> **Coba di playground** Ganti `weekday: "long"` menjadi `weekday: "short"`, dan `month: "long"` menjadi `month: "2-digit"`. Lihat bedanya.

Kesalahan umum

Gejala	Penyebab	Perbaikan
Tanggal mundur satu bulan	Lupa bulan dihitung dari 0	`getMonth() + 1`, atau `new Date(2024, 3, 10)` untuk April
`Invalid Date`	Format teks tidak dikenali, misalnya `"10-04-2024"`	Pakai ISO `"2024-04-10"`
`toFixed is not a function`	Memanggilnya pada teks, bukan angka	Ubah dulu dengan `Number(...)`
Rupiah tampil `1250000` tanpa titik	Angka ditampilkan apa adanya	Pakai `Intl.NumberFormat("id-ID")`

Latihan

Latihan 8.1 (mudah). Ubah teks `" makassar "` menjadi `"Makassar"`: tanpa spasi berlebih, huruf pertama besar.

```
const mentah = "  makassar "
const rapi = mentah   // ganti

sl.write(`[${rapi}]`)
await sl.testing.test("menjadi Makassar", () =>
sl.testing.expect(rapi).toBe("Makassar"))
```

[makassar]

✕ **menjadi Makassar** — Diharapkan: sama persis dengan "Makassar" — nilai sebenarnya: " makassar "

Buka di Playground ↗

Latihan 8.2 (sedang). Lengkapi `keAngka(teks)` yang mengubah teks harga seperti `"Rp 42.000"` atau `"42000"` menjadi angka `42000`.

```
function keAngka(teks) {
  return teks
}

await sl.testing.test('"Rp 42.000" → 42000', () => sl.testing.expect(keAngka("Rp
42.000")).toBe(42000))
await sl.testing.test('"42000" → 42000', () =>
sl.testing.expect(keAngka("42000")).toBe(42000))
await sl.testing.test('"Rp 1.250.000" → 1250000', () =>
sl.testing.expect(keAngka("Rp 1.250.000")).toBe(1250000))
```

- ✕ **"Rp 42.000" → 42000** — Diharapkan: sama persis dengan 42000 — nilai sebenarnya: "Rp 42.000"
- ✕ **"42000" → 42000** — Diharapkan: sama persis dengan 42000 — nilai sebenarnya: "42000"
- ✕ **"Rp 1.250.000" → 1250000** — Diharapkan: sama persis dengan 1250000 — nilai sebenarnya: "Rp 1.250.000"

Buka di Playground ↗

Latihan 8.3 (tantangan). Lengkapi `selisihHari(a, b)` untuk dua teks tanggal ISO, lalu `namaHari(teks)` yang mengembalikan nama hari berbahasa Indonesia.

```
function selisihHari(a, b) {
  return 0
}
function namaHari(teks) {
  return ""
}

await sl.testing.test("1 Jan → 10 Apr 2024 = 100 hari", () =>
sl.testing.expect(selisihHari("2024-01-01", "2024-04-10")).toBe(100))
await sl.testing.test("17 Agustus 1945 hari Jumat", () =>
sl.testing.expect(namaHari("1945-08-17")).toBe("Jumat"))
```

✕ **1 Jan → 10 Apr 2024 = 100 hari** — Diharapkan: sama persis dengan 100 — nilai sebenarnya: 0

✕ **17 Agustus 1945 hari Jumat** — Diharapkan: sama persis dengan "Jumat" — nilai sebenarnya: ""

Buka di Playground ↗

Mini-proyek: kalkulator umur dan hitung mundur

```
sl.title("Kalkulator Umur dan Hitung Mundur :calendar:")

const MS_HARI = 24 * 60 * 60 * 1000
const fmt = new Intl.DateTimeFormat("id-ID", { weekday: "long", day: "numeric",
month: "long", year: "numeric" })

const lahir = new Date(sl.date_input("Tanggal lahir", "2004-05-21"))
const acara = new Date(sl.date_input("Tanggal acara", "2026-12-25"))
const hari = new Date(sl.date_input("Anggap hari ini", "2026-11-09"))

let umur = hari.getFullYear() - lahir.getFullYear()
const sudahUlangTahun =
  hari.getMonth() > lahir.getMonth() ||
  (hari.getMonth() === lahir.getMonth() && hari.getDate() >= lahir.getDate())
if (!sudahUlangTahun) umur--

const sisa = Math.ceil((acara - hari) / MS_HARI)

const [a, b] = sl.columns(2)
a.metric("Umur", `${umur} tahun`)
b.metric("Menuju acara", sisa >= 0 ? `${sisa} hari` : "sudah lewat")

sl.write(`Kamu lahir pada hari **${fmt.format(lahir).split(",")[0]}**.`)
sl.write(`Acara jatuh pada ${fmt.format(acara)}.`)
```

Kalkulator Umur dan Hitung Mundur 📆

Tanggal lahir

05/21/2004

Tanggal acara

12/25/2026

Anggap hari ini

11/09/2026

Umur

22 tahun

Menuju acara

46 hari

Kamu lahir pada hari **Jumat**.

Acara jatuh pada Jumat, 25 Desember 2026.

Buka di Playground ↗

"Anggap hari ini" sengaja dibuat bisa diubah, supaya kamu bisa menguji perhitungannya. Dalam aplikasi sungguhan, ganti dengan `new Date()`.

← 7. Objek dan Larik Objek

9. Galat dan Debugging →

Bab 9 · Galat dan Debugging

> **Tujuan belajar**
>
> - Mengenali tiga jenis galat: sintaks, referensi, dan tipe.
> - Membaca pesan galat di playground, Slak Studio, dan DevTools browser.
> - Menangani galat dengan `try`, `catch`, `finally`, dan `throw`.
> - Mencari kesalahan dengan teknik yang teratur.
> - Menguji fungsi secara otomatis dengan `sl.testing`.

9.1 Galat itu teman

Setiap pemrogram, dari pemula sampai yang berpengalaman puluhan tahun, membuat kesalahan setiap hari. Bedanya hanya satu: yang berpengalaman tahu cara membaca pesan galat. Pesan galat bukan hukuman. Ia petunjuk yang menunjuk ke baris yang perlu kamu perbaiki.

9.2 Tiga jenis galat

Jenis	Kapan terjadi	Contoh pesan	Penyebab umum
SyntaxError	Sebelum program jalan	`Unexpected token`, `missing ) after argument list`	Kurung, kutip, atau kurawal tidak berpasangan
ReferenceError	Saat program jalan	`total is not defined`	Salah ketik nama variabel, atau variabel dipakai di luar wilayahnya
TypeError	Saat program jalan	`Cannot read properties of undefined`, `x.map is not a function`	Memakai nilai dengan cara yang tidak cocok dengan tipenya

Galat sintaks menghentikan program sebelum satu baris pun dijalankan. Dua jenis lainnya menghentikan program di baris tempat galat terjadi; baris di atasnya tetap sudah jalan.

9.3 Membaca galat di Slak

Saat kode salah, Slak menampilkan kotak galat merah di pratinjau, lengkap dengan tombol **Detail** untuk melihat urutan pemanggilannya (*stack trace*).

Slak Studio menunjukkan masalah yang sama di empat tempat sekaligus:

1. Baris yang bermasalah diberi latar merah muda di editor.
2. Pratinjau menampilkan kotak galat.

3. Konsol mencatat galat dalam bahasa aslinya, lalu menambahkan baris 💡 berisi penjelasan dalam bahasa Indonesia.
4. Bilah status menampilkan nomor baris dan penjelasannya. Klik untuk melompat ke baris itu.

Untuk salah ketik nama fungsi, Slak Studio bahkan menebak maksudmu, misalnya "Fungsi sl.metrik tidak ada. Maksudmu sl.metric?". Saat kamu mengetik `sl.`, editor menampilkan saran fungsi lengkap dengan urutan argumennya, dan panel **Referensi** (tekan **F1**) berisi semua fungsi Slak beserta contohnya.

Di playground atau browser biasa, buka **DevTools** dengan **F12**, lalu pilih tab **Console**. Pesan dari `console.log` dan galat muncul di sana.

9.4 Dua kesalahan klasik pemula

Satu `=` yang seharusnya tiga.

```
const status = "lunas"
if (status = "belum") {          // salah: mengisi, bukan membandingkan
  sl.warning("Tagih pembayaran")
}
```

Karena `status` dibuat dengan `const`, JavaScript menolak mengisinya dan muncul galat **Assignment to constant variable**. Pesan itu adalah petunjuk: kamu menulis `=` padahal maksudnya `===`. Bila variabelnya dibuat dengan `let`, tidak ada galat sama sekali. Kondisinya diam-diam selalu benar, dan kesalahan seperti ini jauh lebih sulit ditemukan. Satu alasan lagi untuk memilih `const`.

Membaca properti dari sesuatu yang tidak ada.

```
const produk = [{ nama: "Kopi Wamena", harga: 85000 }]
const dicari = produk.find(p => p.nama === "Kopi Toraja")   // tidak ketemu:
undefined

try {
  sl.write(dicari.harga)
} catch (e) {
  sl.error(e.message)
}

sl.write(dicari?.harga)          // aman: undefined, tanpa galat
sl.write(dicari?.harga ?? 0)     // aman, dengan nilai cadangan 0
```

Cannot read properties of undefined (reading 'harga')

```
undefined
```

0

Buka di Playground ↗

- `?.` (*optional chaining*) berhenti dengan tenang bila nilai di kirinya `undefined` atau `null`.
- `??` memberi nilai cadangan bila nilai di kirinya `undefined` atau `null`.

Kedua operator ini sangat berguna untuk data nyata yang penuh lubang.

9.5 try, catch, finally

Bungkus kode yang mungkin gagal dengan `try`. Bila terjadi galat, program tidak berhenti, melainkan lompat ke `catch`.

```
const teksJson = '{"kota": "Ambon", "omzet": 1800000'   // kurung tutup hilang

try {
  const data = JSON.parse(teksJson)
  sl.write(data)
} catch (e) {
  sl.warning("Data tidak bisa dibaca: " + e.message)
} finally {
  sl.caption("Pemeriksaan selesai.")   // selalu dijalankan, ada galat atau tidak
}
```

Data tidak bisa dibaca: Expected ',' or '}' after property value in JSON at position 34 (line 1 column 35)

Pemeriksaan selesai.

Buka di Playground ↗

`JSON.parse` mengubah teks JSON menjadi objek. Bab 11 membahasnya.

Kamu juga bisa **melempar** galat sendiri dengan `throw`, misalnya saat masukan tidak masuk akal:

```
function hargaSatuan(total, jumlah) {
  if (jumlah <= 0) throw new Error("Jumlah harus lebih dari 0")
  return total / jumlah
}

try {
  sl.write(hargaSatuan(170000, 2))
  sl.write(hargaSatuan(170000, 0))
} catch (e) {
  sl.exception(e)
}
```

85.000

Error: Jumlah harus lebih dari 0

▶ Detail

Buka di Playground ↗

> **API Slak baru**
>
> - `sl.warning(teks)` menampilkan kotak peringatan kuning.
> - `sl.exception(galat)` menampilkan galat lengkap dengan tombol **Detail** berisi stack trace, tanpa menghentikan aplikasi.

9.6 Teknik mencari kesalahan

Saat hasil program salah tetapi tidak ada pesan galat, gunakan langkah-langkah ini:

1. **Baca pesannya sampai habis.** Nomor baris dan nama variabel biasanya sudah menjawab.
2. **Intip isi variabel.** Tulis `console.log(x)` atau `sl.write(x)` sebelum baris yang dicurigai. Apakah isinya seperti dugaanmu? Apakah tipenya benar?
3. **Perkecil masalahnya.** Jadikan baris yang dicurigai komentar (**Ctrl+/** di Slak Studio) satu per satu, sampai galatnya hilang.
4. **Coba dengan data kecil.** Lebih mudah memeriksa 3 baris daripada 3.000 baris.
5. **Jelaskan kodenya keras-keras**, baris demi baris, seolah kepada teman. Sering kali kesalahannya terlihat saat dijelaskan.

Di DevTools browser, kamu juga bisa memasang **breakpoint**: klik nomor baris di tab **Sources**, dan program berhenti di sana supaya isi setiap variabel bisa diperiksa. Cara yang lebih cepat: tulis `debugger` di baris mana pun, dan program berhenti di situ selama DevTools terbuka.

9.7 Menguji kode secara otomatis

Memeriksa hasil dengan mata melelahkan dan mudah terlewat. Slak punya `sl.testing`, alat uji yang sama yang dipakai latihan di buku ini.

```
function diskonAnggota(total) {
  return total >= 500000 ? total * 0.1 : 0
}

const { test, expect } = sl.testing

await test("600 ribu dapat diskon 60 ribu", () =>
expect(diskonAnggota(600000)).toBe(60000))
await test("tepat 500 ribu dapat diskon", () =>
expect(diskonAnggota(500000)).toBe(50000))
await test("200 ribu tidak dapat diskon", () =>
expect(diskonAnggota(200000)).toBe(0))
```

✓ 600 ribu dapat diskon 60 ribu (0 md)

✓ tepat 500 ribu dapat diskon (0 md)

✓ 200 ribu tidak dapat diskon (0 md)

Buka di Playground ↗

Setiap `test` menampilkan kotak hijau bila lulus, atau merah beserta nilai yang diharapkan dan nilai sebenarnya bila gagal.

Pernyataan	Lulus bila
`expect(x).toBe(y)`	`x` sama persis dengan `y` (angka, teks, boolean)
`expect(x).toEqual(y)`	Isi larik atau objek `x` sama dengan `y`
`expect(x).toBeCloseTo(y, 2)`	`x` mendekati `y` sampai 2 desimal (untuk pecahan)
`expect(x).toBeGreaterThan(y)`	`x > y`

Pernyataan	Lulus bila
`expect(x).toContain(y)`	Larik atau teks `x` memuat `y`
`expect(x).toHaveLength(n)`	Panjang `x` adalah `n`
`expect(fn).toThrow()`	Memanggil `fn` melempar galat
`expect(x).not.toBe(y)`	Kebalikan dari pernyataan sesudahnya

> **API Slak baru**
>
> - `await sl.testing.test(judul, fungsi)` menjalankan satu uji dan menampilkan hasilnya.
> - `sl.testing.expect(nilai)` membuat pernyataan yang diperiksa, misalnya `.toBe(...)`.

Mengapa `toBeCloseTo` perlu? Ingat Bab 2: `0.1 + 0.2` tidak tepat sama dengan `0.3`.

```
await sl.testing.test("0,1 + 0,2 mendekati 0,3", () => sl.testing.expect(0.1 + 0.2).toBeCloseTo(0.3, 10))
```

✓ 0,1 + 0,2 mendekati 0,3 (0 md)

Buka di Playground ↗

Kebiasaan menulis uji untuk setiap fungsi penting akan sangat berguna di Bab 24, saat kodemu tumbuh besar.

> **Coba di playground** Ubah fungsi `diskonAnggota` menjadi `total > 500000` (tanpa `=`). Uji mana yang menjadi merah?

Kesalahan umum

Pesan	Arti	Perbaikan
`x is not defined`	Variabel `x` belum dibuat atau salah ketik	Periksa ejaan dan huruf besar-kecil
`Cannot read properties of undefined (reading 'y')`	Kamu menulis `sesuatu.y`, padahal `sesuatu` kosong	Periksa asal `sesuatu`; pakai `?.` bila memang bisa kosong
`x.map is not a function`	`x` bukan larik	Lihat isi `x` dengan `sl.write(x)`
`Unexpected end of input`	Kurung kurawal atau kurung biasa belum ditutup	Hitung pasangan `{ }` dan `( )`

Pesan	Arti	Perbaikan
`Assignment to constant variable`	Mengisi ulang variabel `const`, sering karena `=` yang seharusnya `===`	Ganti `===`, atau pakai `let` bila memang perlu diganti

Latihan

Latihan 9.1 (mudah). Fungsi ini seharusnya mengembalikan nama hari, tetapi ada satu kesalahan. Temukan dan perbaiki.

```
function namaHari(nomor) {
  const hari = ["Minggu", "Senin", "Selasa", "Rabu", "Kamis", "Jumat", "Sabtu"]
  return hari[nomor + 1]
}

await sl.testing.test("0 = Minggu", () =>
sl.testing.expect(namaHari(0)).toBe("Minggu"))
await sl.testing.test("5 = Jumat", () =>
sl.testing.expect(namaHari(5)).toBe("Jumat"))
```

✕ **0 = Minggu** — Diharapkan: sama persis dengan "Minggu" — nilai sebenarnya: "Senin"

✕ **5 = Jumat** — Diharapkan: sama persis dengan "Jumat" — nilai sebenarnya: "Sabtu"

Buka di Playground ↗

Latihan 9.2 (sedang). Lengkapi `bacaHarga(teks)` agar mengembalikan objek hasil `JSON.parse` bila teksnya JSON yang sah, dan `null` bila tidak, tanpa menghentikan program.

```
function bacaHarga(teks) {
  return JSON.parse(teks)
}

await sl.testing.test("JSON sah terbaca", () =>
sl.testing.expect(bacaHarga('{"harga": 5000}')).toEqual({ harga: 5000 }))
await sl.testing.test("JSON rusak menjadi null", () =>
sl.testing.expect(bacaHarga('{"harga": ')).toBeNull())
```

JSON sah terbaca (0 md)

JSON rusak menjadi null — Unexpected end of JSON input

Buka di Playground ↗

Latihan 9.3 (tantangan). Tulis fungsi `persen(bagian, keseluruhan)` yang mengembalikan persentase dibulatkan satu desimal, dan **melempar galat** bila `keseluruhan` bernilai 0.

```
function persen(bagian, keseluruhan) {
  return 0
}

await sl.testing.test("25 dari 200 = 12.5", () => sl.testing.expect(persen(25, 200)).toBe(12.5))
await sl.testing.test("1 dari 3 = 33.3", () => sl.testing.expect(persen(1, 3)).toBe(33.3))
await sl.testing.test("pembagi 0 melempar galat", () => sl.testing.expect(() => persen(5, 0)).toThrow())
```

25 dari 200 = 12.5 — Diharapkan: sama persis dengan 12.5 — nilai sebenarnya: 0

1 dari 3 = 33.3 — Diharapkan: sama persis dengan 33.3 — nilai sebenarnya: 0

pembagi 0 melempar galat — Diharapkan: melempar galat — nilai sebenarnya: () ⇒ persen(5, 0)

Buka di Playground ↗

Mini-proyek: perbaiki aplikasi kasir

Aplikasi kasir kecil ini berisi **lima bug**. Programnya tetap jalan, tetapi hasilnya salah. Lima uji di bagian bawah menunjukkan apa yang seharusnya terjadi. Temukan dan perbaiki semua bug sampai kelima uji lulus. Jawabannya ada di Lampiran D.

```
sl.title("Kasir Toko Nusantara :convenience_store:")

const keranjang = [
  { produk: "Kopi Arabika Wamena 250g", harga: 85000, jumlah: 2 },
  { produk: "Keripik Keladi 200g", harga: 22000, jumlah: 3 },
  { produk: "Gelang Manik Papua", harga: "40000", jumlah: 1 }
]

// Bug ada di fungsi-fungsi berikut
function subtotal(item) {
  return item.harga + item.jumlah
}

function totalBelanja(daftar) {
  let total = 0
  for (let i = 1; i < daftar.length; i++) total += subtotal(daftar[i])
  return total
}

function diskon(total) {
  if (total > 250000) return total * 0.05
  return 0
}

function ppn(setelahDiskon) {
  return setelahDiskon * 11
}

const total = totalBelanja(keranjang)
const pot = diskon(total)
const pajak = ppn(total - pot)

sl.dataframe(keranjang)
sl.metric("Total", total, null, { format: "rp" })
sl.metric("Bayar", total - pot + pajak, null, { format: "rp" })

const { test, expect } = sl.testing
await test("subtotal kopi = 170000", () =>
expect(subtotal(keranjang[0])).toBe(170000))
await test("subtotal gelang = 40000", () =>
expect(subtotal(keranjang[2])).toBe(40000))
await test("total belanja = 276000", () =>
```

```
expect(totalBelanja(keranjang)).toBe(276000))
await test("diskon 5% mulai tepat 250000", () =>
expect(diskon(250000)).toBe(12500))
await test("PPN 11% dari 100000 = 11000", () => expect(ppn(100000)).toBe(11000))
```

Kasir Toko Nusantara 🏪

	produk	harga	jumlah
0	Kopi Arabika Wamena 250g	85.000	2
1	Keripik Keladi 200g	22.000	3
2	Gelang Manik Papua	40000	1

3 baris × 3 kolom

Total

22003400001

Bayar

Rp 250.838.760.011

✕ **subtotal kopi = 170000** — Diharapkan: sama persis dengan 170000 — nilai sebenarnya: 85002

✕ **subtotal gelang = 40000** — Diharapkan: sama persis dengan 40000 — nilai sebenarnya: "400001"

✕ **total belanja = 276000** — Diharapkan: sama persis dengan 276000 — nilai sebenarnya: "22003400001"

✕ **diskon 5% mulai tepat 250000** — Diharapkan: sama persis dengan 12500 — nilai sebenarnya: 0

✕ **PPN 11% dari 100000 = 11000** — Diharapkan: sama persis dengan 11000 — nilai sebenarnya: 1100000

Buka di Playground ↗

← 8. Teks, Angka, dan Tanggal

10. Cara Slak Menjalankan Skrip →

Bab 10 · Cara Slak Menjalankan Skrip

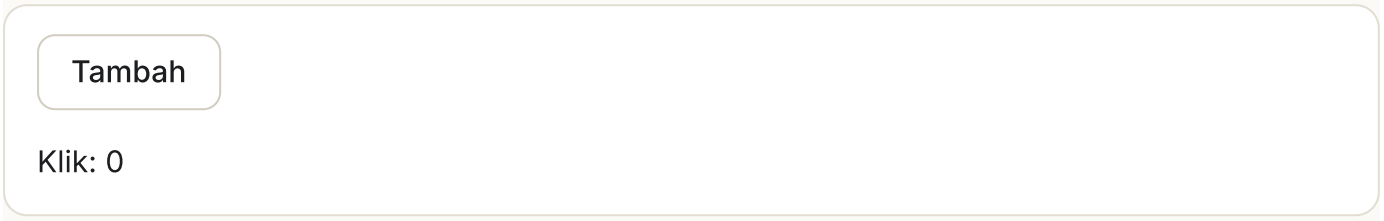
> **Tujuan belajar**
>
> - Memahami jalan ulang: setiap interaksi menjalankan ulang seluruh skrip dari atas.
> - Menyimpan nilai antar-interaksi dengan `sl.session_state` dan `key`.
> - Menyimpan data permanen di browser dengan `sl.local_state`.
> - Mengumpulkan beberapa isian sekaligus dengan `sl.form`.
> - Membandingkan cara Slak dengan event listener JavaScript biasa.
> - Menyembunyikan kunci jawaban kuis dengan `sl.crypto`.

10.1 Jalan ulang

Sejak Bab 3 kamu sudah merasakannya: setiap kali widget diubah, hasil di layar ikut berubah. Di balik itu, Slak melakukan satu hal sederhana: **menjalankan ulang seluruh skrip dari baris pertama**, kali ini dengan nilai widget yang baru. Lalu Slak membandingkan hasil baru dengan tampilan lama dan hanya mengubah bagian yang berbeda, sehingga posisi gulir dan kursor tetap terjaga.

Akibatnya, kode Slak bisa dibaca seperti resep dari atas ke bawah. Tetapi ada satu akibat yang sering mengejutkan pemula:

```
let klik = 0
if (sl.button("Tambah")) klik++
sl.write(`Klik: ${klik}`)
```



Buka di Playground ↗

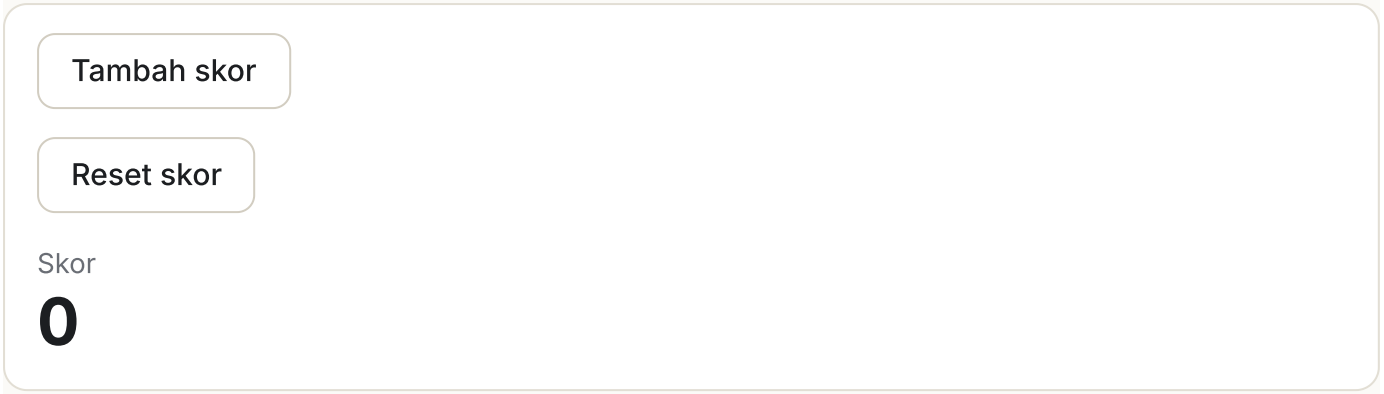
Klik tombolnya beberapa kali. Angkanya tidak pernah lebih dari 1. Mengapa? Setiap kali tombol diklik, skrip dijalankan ulang dari atas, dan baris pertama mengembalikan `klik` ke 0. Variabel biasa **lupa** setiap jalan ulang.

10.2 session_state: ingatan selama halaman terbuka

`sl.session_state` adalah objek yang isinya **bertahan antar-jalan ulang** selama halaman terbuka.

```
const ss = sl.session_state
if (!("skor" in ss)) ss.skor = 0     // isi nilai awal hanya bila belum ada

if (sl.button("Tambah skor")) ss.skor++
if (sl.button("Reset skor")) ss.skor = 0
sl.metric("Skor", ss.skor)
```



Buka di Playground ↗

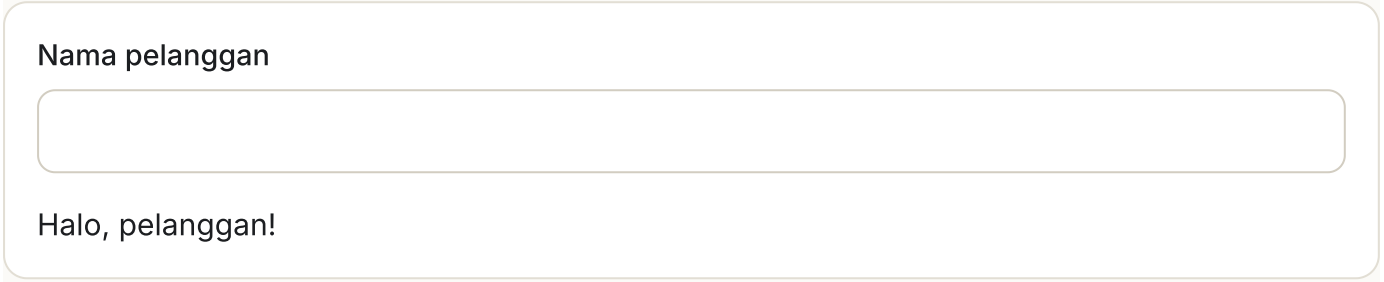
Pola `if (!("skor" in ss)) ss.skor = 0` sangat penting. Tanpa `if`, nilai awal akan menimpa skor di setiap jalan ulang, dan kamu kembali ke masalah 10.1.

> **API Slak baru**
>
> - `sl.session_state` menyimpan nilai selama halaman terbuka. Isinya hilang bila halaman dimuat ulang.

key: widget yang ikut tersimpan

Widget yang diberi `key` otomatis menyimpan nilainya di `session_state` dengan nama itu:

```
sl.text_input("Nama pelanggan", { key: "pelanggan" })
sl.write(`Halo, ${sl.session_state.pelanggan || "pelanggan"}!`)
```



Buka di Playground ↗

`key` juga memberi widget identitas tetap. Tanpa `key`, Slak mengenali widget dari jenis dan labelnya, jadi mengganti label membuat widget dianggap baru dan nilainya kembali ke awal. Setiap `key` harus unik; memakai `key` yang sama dua kali menghasilkan galat "Key widget

dipakai lebih dari sekali".

Callback: on_click

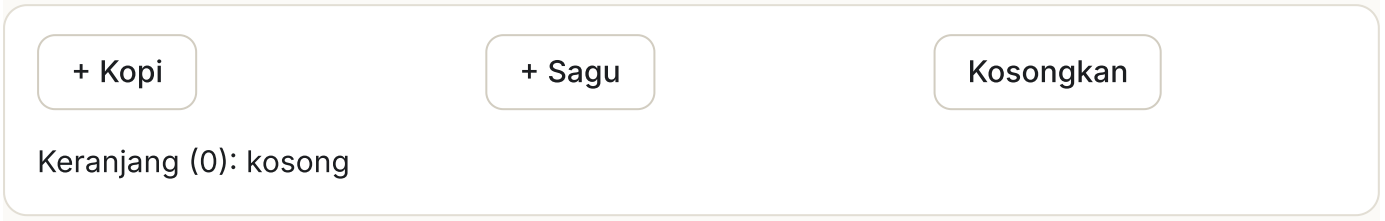
Tombol bisa diberi fungsi yang dipanggil **sebelum** skrip dijalankan ulang. Cara ini membuat perubahan langsung terlihat di jalan ulang yang sama.

```
const s = sl.session_state
if (!s.keranjang) s.keranjang = []

const tambah = (nama) => { s.keranjang.push(nama) }
const kosongkan = () => { s.keranjang = [] }

const [k1, k2, k3] = sl.columns(3)
k1.button("+ Kopi", { on_click: tambah, args: ["Kopi"] })
k2.button("+ Sagu", { on_click: tambah, args: ["Sagu"] })
k3.button("Kosongkan", { on_click: kosongkan })

sl.write(`Keranjang (${s.keranjang.length}): ${s.keranjang.join(", ") ||
"kosong"}`)
```



Buka di Playground ↗

`args` adalah larik argumen yang dikirim ke fungsi callback.

10.3 local_state: ingatan permanen di browser

`session_state` hilang saat halaman dimuat ulang. Untuk data yang harus bertahan, misalnya catatan atau pengaturan, pakai `sl.local_state`. Cara pakainya sama, tetapi isinya disimpan di browser dan tetap ada besok.

```
const ls = sl.local_state
if (!("kunjungan" in ls)) ls.kunjungan = 0
if (sl.button("Catat kunjungan")) ls.kunjungan++
sl.metric("Kunjungan tercatat", ls.kunjungan)
sl.caption("Muat ulang halaman: angkanya tetap.")
```

Catat kunjungan

Kunjungan tercatat

0

Muat ulang halaman: angkanya tetap.

Buka di Playground ↗

> **API Slak baru**
>
> - `sl.local_state` seperti `session_state`, tetapi permanen. Data disimpan per browser dan hanya cocok untuk data JSON: angka, teks, boolean, larik, dan objek biasa.
> - `sl.clear_local_state()` mengosongkannya.

Dua hal yang perlu kamu tahu:

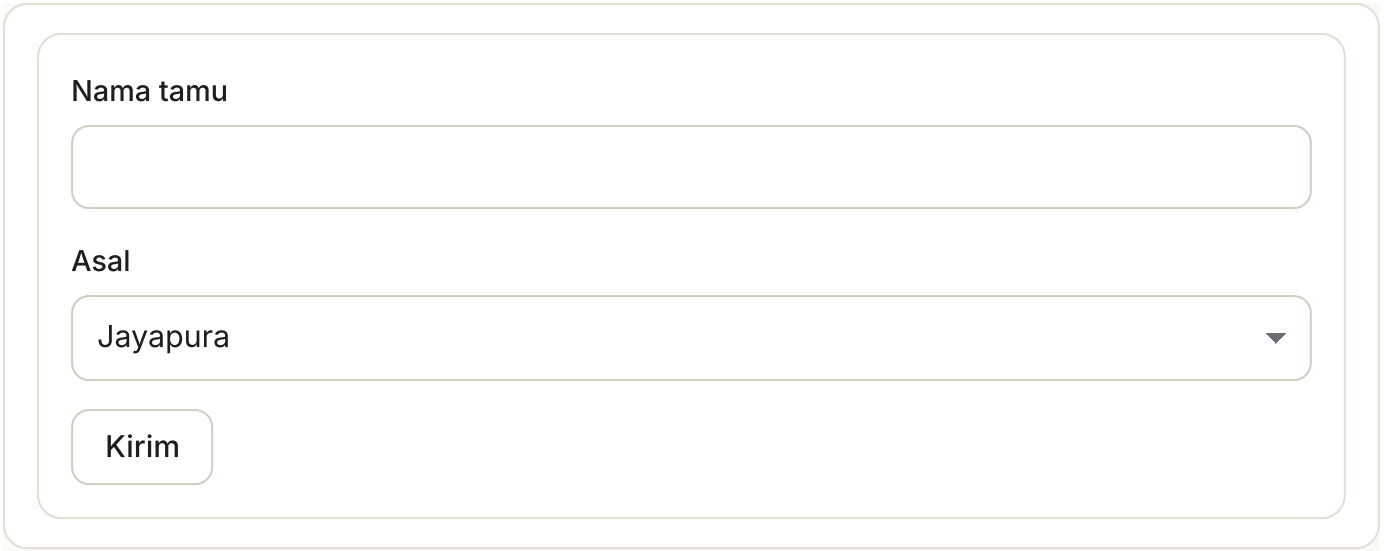
- Data tersimpan **di browser pengguna**, bukan di server. Pengguna lain tidak melihatnya, dan data hilang bila pengguna menghapus data situs.
- Jangan menyimpan password atau data rahasia di sana. Siapa pun yang memakai komputer itu bisa membacanya.

10.4 Form: kirim sekaligus

Setiap perubahan widget memicu jalan ulang. Untuk formulir dengan banyak isian, itu tidak perlu: kita ingin semua isian diproses sekali saat tombol **Kirim** ditekan. Gunakan `sl.form`.

```
const f = sl.form("tamu", { clear_on_submit: true })
const namaTamu = f.text_input("Nama tamu")
const asal = f.selectbox("Asal", ["Jayapura", "Sorong", "Manokwari", "Lainnya"])
const kirim = f.form_submit_button("Kirim")

if (kirim) {
  if (!namaTamu) sl.error("Nama wajib diisi.")
  else sl.success(`Terima kasih, ${namaTamu} dari ${asal}!`)
}
```



Buka di Playground ↗

> **API Slak baru**
>
> - `sl.form(key, { clear_on_submit })` membuat formulir. Widget di dalamnya ditulis dengan awalan formulir, misalnya `f.text_input(...)`, dan tidak memicu jalan ulang sampai tombol kirim ditekan.
> - `f.form_submit_button(label)` bernilai `true` sekali setelah ditekan. `clear_on_submit: true` mengosongkan isian setelah dikirim.

10.5 Bandingkan dengan JavaScript biasa

Bagaimana cara membuat penghitung klik tanpa Slak? Dengan JavaScript biasa di browser, kamu bekerja dengan **DOM** (struktur halaman) dan **event listener** (fungsi yang menunggu kejadian):

```
<button id="tombol">Tambah</button>
<p id="hasil">Klik: 0</p>
<script>
  let klik = 0
  const tombol = document.getElementById("tombol")
  const hasil = document.getElementById("hasil")
  tombol.addEventListener("click", () => {
    klik++
    hasil.textContent = "Klik: " + klik
  })
</script>
```

	JavaScript biasa	Slak
Alur	Pasang fungsi yang menunggu kejadian	Skrip dijalankan ulang dari atas
Variabel bertahan?	Ya, skrip hanya jalan sekali	Tidak; pakai `session_state`
Mengubah tampilan	Kamu sendiri yang mengubah DOM	Slak menggambar ulang yang berubah
Cocok untuk	Situs dan aplikasi web umum	Aplikasi data, dasbor, alat kecil

Keduanya JavaScript yang sama. Slak hanya memilih gaya yang lebih mudah untuk aplikasi data: kamu memikirkan **apa yang ditampilkan**, bukan **kapan harus mengubah tampilan**. Di Bab 26 kamu akan melihat seberapa banyak yang dikerjakan Slak untukmu.

10.6 Menyembunyikan kunci jawaban kuis

Kode aplikasi Slak bisa dibaca siapa saja yang membuka halamannya. Bila kunci jawaban kuis ditulis apa adanya, peserta bisa mengintipnya. `sl.crypto.quiz_hash` mengubah jawaban menjadi kode yang tidak bisa dibaca, dan `sl.crypto.quiz_check` memeriksa jawaban peserta terhadap kode itu.

Langkah pertama, buat kodenya sekali (di playground atau Slak Studio):

```
sl.code(sl.crypto.quiz_hash("Jayapura"))
```

```
sq1$966f57a9$7L0feSUQLImxHqA4foG045FG
```

Buka di Playground ↗

Setiap kali dijalankan, kodenya berbeda karena ada bumbu acak, tetapi semuanya sama-sama cocok dengan "Jayapura". Salin hasilnya ke aplikasi kuis:

```
const KUNCI = "sq1$1dfd355a$dEQmiN6xy_-AgzCbOolSOwc5"   // jawaban asli tidak
tertulis di kode
const jawab = sl.text_input("Ibu kota Provinsi Papua?")
if (jawab) {
  if (sl.crypto.quiz_check(jawab, KUNCI)) sl.success("Benar!")
  else sl.error("Belum tepat, coba lagi.")
}
```

Ibu kota Provinsi Papua?

Buka di Playground ↗

`quiz_check` mengabaikan huruf besar-kecil, spasi berlebih, dan titik di akhir, jadi "jayapura" dan "Jayapura." juga benar.

> **API Slak baru**
>
> - `sl.crypto.quiz_hash(jawaban)` membuat kode pelindung. Bisa juga larik beberapa jawaban yang sama-sama benar.
> - `sl.crypto.quiz_check(jawaban, kode)` mengembalikan `true` bila jawaban cocok.

Hash menyembunyikan jawaban dari mata, bukan dari tebakan. Untuk pilihan ganda dengan empat opsi, peserta tetap bisa mencoba semuanya. Pakai untuk isian singkat.

> **Coba di playground** Buat kode untuk dua jawaban yang sama-sama benar: `sl.crypto.quiz_hash(["Jakarta", "DKI Jakarta"])`. Lalu periksa keduanya dengan `quiz_check`.

Kesalahan umum

Gejala	Penyebab	Perbaikan
Penghitung tidak pernah lebih dari 1	Memakai variabel biasa	Simpan di `sl.session_state`
Nilai kembali ke awal setiap klik	Menulis `ss.skor = 0` tanpa `if`	`if (!("skor" in ss)) ss.skor = 0`
Isian widget hilang setelah label diubah	Widget dianggap baru	Beri `key` yang tetap
`Key widget dipakai lebih dari sekali`	Dua widget memakai `key` sama	Buat setiap `key` unik
Data `local_state` berupa tanggal berubah menjadi teks	`local_state` hanya menyimpan JSON	Simpan tanggal sebagai teks ISO

Latihan

Latihan 10.1 (mudah). Lengkapi agar `ss.hitung` bertambah 1 setiap tombol diklik dan tidak kembali ke 0. Uji di bawah menjalankan aplikasimu di balik layar, mengeklik tombolnya tiga kali, lalu memeriksa angkanya.

```
const kode = `
const ss = sl.session_state
ss.hitung = 0
if (sl.button("Tambah")) ss.hitung++
sl.metric("Hitungan", ss.hitung)
`
sl.code(kode)

await sl.testing.test("tiga klik menjadi 3", async () => {
  const at = await sl.testing.run(kode)
  for (let i = 0; i < 3; i++) await at.button("Tambah").click().run()
  sl.testing.expect(at.metric("Hitungan").value).toBe(3)
})
```

```
const ss = sl.session_state
ss.hitung = 0
if (sl.button("Tambah")) ss.hitung++
sl.metric("Hitungan", ss.hitung)
```

✕ **tiga klik menjadi 3** — Diharapkan: sama persis dengan 3 — nilai sebenarnya: 1

Buka di Playground ↗

Perbaiki teks di dalam `kode`. Contoh ini sekaligus memperkenalkan `sl.testing.run(kode)`: menjalankan aplikasi tersembunyi, lalu mengeklik tombol dari kode uji.

Latihan 10.2 (sedang). Lengkapi fungsi callback `hapusTerakhir` agar membuang isi terakhir `daftar`.

```
const kode2 = `
const s = sl.session_state
if (!s.daftar) s.daftar = ["kopi", "sagu", "abon"]
const hapusTerakhir = () => { }
sl.button("Hapus terakhir", { on_click: hapusTerakhir })
sl.write(s.daftar.length + " barang")
`
sl.code(kode2)

await sl.testing.test("satu kali hapus menyisakan 2 barang", async () => {
  const at = await sl.testing.run(kode2)
  await at.button("Hapus terakhir").click().run()
  sl.testing.expect(at.session_state.daftar).toEqual(["kopi", "sagu"])
})
```

```
const s = sl.session_state
if (!s.daftar) s.daftar = ["kopi", "sagu", "abon"]
const hapusTerakhir = () => { }
sl.button("Hapus terakhir", { on_click: hapusTerakhir })
sl.write(s.daftar.length + " barang")
```

✕ **satu kali hapus menyisakan 2 barang** — Diharapkan: setara dengan ["kopi","sagu"] — nilai sebenarnya: ["kopi","sagu","abon"]

Buka di Playground ↗

Latihan 10.3 (tantangan). Ganti `KUNCI` agar kuis menerima "Manokwari" sebagai jawaban benar, tanpa menulis kata "Manokwari" di dalam `KUNCI`.

```
const KUNCI = ""   // isi dengan hasil sl.crypto.quiz_hash("Manokwari")

await sl.testing.test("Manokwari benar", () =>
sl.testing.expect(sl.crypto.quiz_check("Manokwari", KUNCI)).toBe(true))
await sl.testing.test("Sorong salah", () =>
sl.testing.expect(sl.crypto.quiz_check("Sorong", KUNCI)).toBe(false))
await sl.testing.test("kunci tidak memuat jawabannya", () =>
sl.testing.expect(KUNCI.toLowerCase().includes("manokwari")).toBe(false))
```

Manokwari benar — Diharapkan: sama persis dengan true — nilai sebenarnya: false

Sorong salah (0 md)

kunci tidak memuat jawabannya (0 md)

Buka di Playground ↗

Mini-proyek 1: daftar tugas yang tidak hilang

```
sl.title("Daftar Tugas :white_check_mark:")

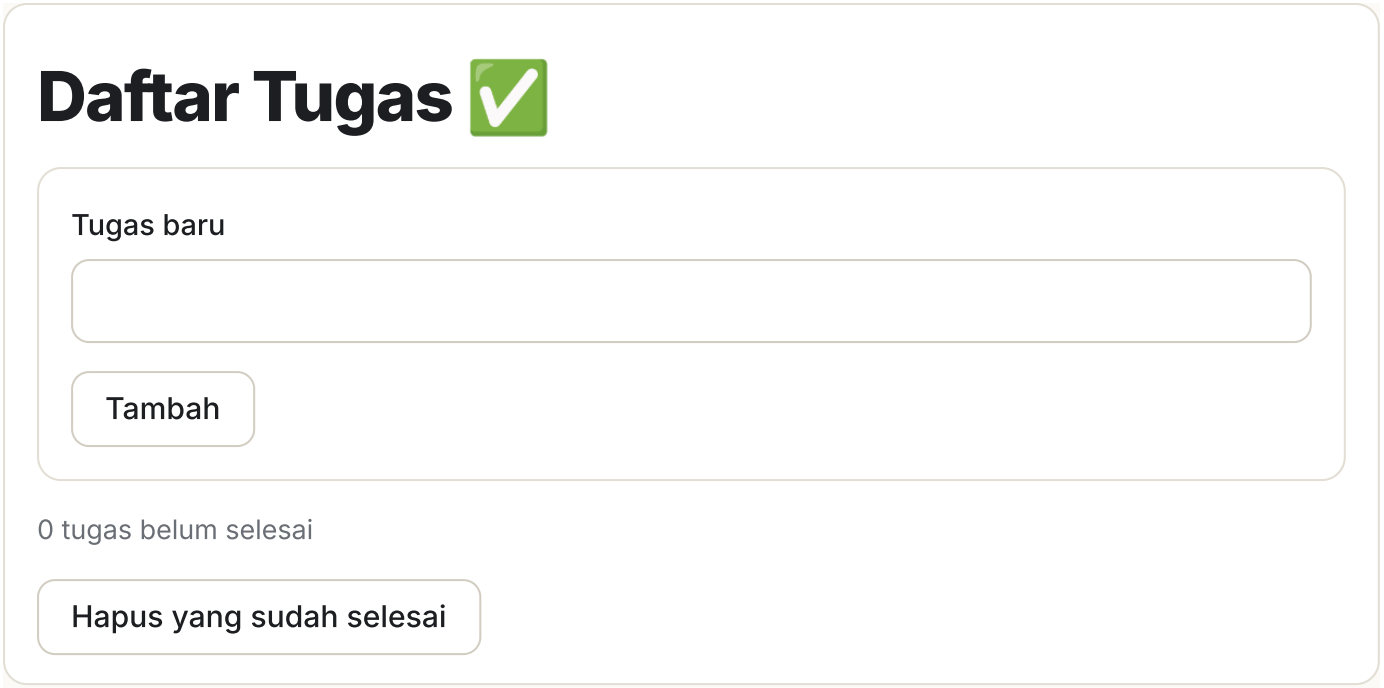
const ls = sl.local_state
if (!ls.tugas) ls.tugas = []

const f = sl.form("tugas-baru", { clear_on_submit: true })
const judul = f.text_input("Tugas baru")
if (f.form_submit_button("Tambah") && judul.trim()) {
  ls.tugas = [...ls.tugas, { judul: judul.trim(), selesai: false }]
}

const ubah = (i) => {
  ls.tugas = ls.tugas.map((t, j) => j === i ? { ...t, selesai: !t.selesai } : t)
}
const hapusSelesai = () => { ls.tugas = ls.tugas.filter(t => !t.selesai) }

ls.tugas.forEach((t, i) => {
  sl.checkbox(t.judul, { value: t.selesai, key: "tugas-" + i, on_change: ubah,
args: [i] })
})

const sisa = ls.tugas.filter(t => !t.selesai).length
sl.caption(`${sisa} tugas belum selesai`)
sl.button("Hapus yang sudah selesai", { on_click: hapusSelesai })
```



Buka di Playground ↗

Tutup halamannya, buka lagi besok: tugasmu masih ada.

Perhatikan bahwa `ls.tugas` selalu diganti dengan larik baru (`[...ls.tugas, ...]`, `map`, `filter`), bukan diubah di tempat. Mengganti nilainya memastikan Slak menyimpan perubahan ke browser.

Mini-proyek 2: kuis ibu kota

```
sl.title("Kuis Ibu Kota Provinsi :round_pushpin:")

const SOAL = [
  { tanya: "Papua", kunci: "sq1$1dfd355a$dEQmiN6xy_-AgzCbOolSOwc5" },
  { tanya: "Papua Barat", kunci: "sq1$d3b9602d$w8Z83V5uRJtt1rylukrGlCYb" },
  { tanya: "Maluku", kunci: "sq1$7545a5b7$1yKypXkI2LzXuP6012Wgl2KS" }
]

const f = sl.form("kuis")
const jawaban = SOAL.map((s, i) => f.text_input(`${i + 1}. Ibu kota ${s.tanya}?`, {
key: "jawab-" + i }))
const periksa = f.form_submit_button("Periksa jawaban")

if (periksa) {
  let skor = 0
  SOAL.forEach((s, i) => {
    if (sl.crypto.quiz_check(jawaban[i], s.kunci)) skor++
```

```
  })
  sl.metric("Skor", `${skor} / ${SOAL.length}`)
  if (skor === SOAL.length) sl.balloons()
}
```

Kuis Ibu Kota Provinsi 📍

1. Ibu kota Papua?

2. Ibu kota Papua Barat?

3. Ibu kota Maluku?

Periksa jawaban

Buka di Playground ↗

`sl.balloons()` menerbangkan balon di layar. Pantas untuk jawaban sempurna.

← 9. Galat dan Debugging

11. Dari Data Mentah ke Struktur →

Bab 11 · Dari Data Mentah ke Struktur

> **Tujuan belajar**
>
> - Mengenal format data yang paling sering ditemui analis: CSV, JSON, dan spreadsheet.
> - Membaca CSV secara manual dengan `split`, lalu mengubah teks menjadi angka.
> - Membaca dan menulis JSON dengan `JSON.parse` dan `JSON.stringify`.
> - Membaca berkas dengan `sl.read_csv` dan berkas unggahan pengguna.
> - Memahami cara Slak membaca berkas saat halaman dibuka dengan klik dua kali, dan batas ukuran data di browser.

Selamat datang di Tahap 2. Mulai bab ini, kamu bekerja dengan data Toko Nusantara: 2.929 transaksi dari sepuluh cabang selama 2023–2024.

11.1 Tiga format data

Format	Bentuk	Contoh pemakaian
CSV	Teks; setiap baris satu rekaman, kolom dipisah koma atau titik koma	Ekspor dari Excel, sistem kasir, portal data pemerintah
JSON	Teks dengan bentuk persis seperti objek dan larik JavaScript	Data dari API di internet, berkas pengaturan
Excel (.xlsx)	Berkas biner berisi beberapa lembar	Laporan kantor sehari-hari

CSV adalah format yang paling sering dipakai analis karena sederhana dan bisa dibuka program apa pun. Inilah lima baris pertama `cabang.csv` dari bahan buku:

```
kode,cabang,kota,provinsi,lat,lon,tanggal_buka
JYP,Nusantara Jayapura,Jayapura,Papua,-2.5337,140.7181,2019-03-01
SOQ,Nusantara Sorong,Sorong,Papua Barat Daya,-0.8762,131.2558,2020-07-15
MKW,Nusantara Manokwari,Manokwari,Papua Barat,-0.8615,134.062,2021-02-01
MKS,Nusantara Makassar,Makassar,Sulawesi Selatan,-5.1477,119.4327,2019-09-10
```

Baris pertama adalah **kepala** (nama kolom). Baris berikutnya adalah data.

11.2 Membaca CSV dengan tangan

Sebelum memakai alat jadi, mari membaca CSV dengan JavaScript murni. Semua yang dibutuhkan sudah kamu pelajari: `split`, `map`, dan objek.

```
const teksCsv = `kode,kota,omzet,buka
JYP,Jayapura,4200000,2019-03-01
SOQ,Sorong,3100000,2020-07-15
MKW,Manokwari,1800000,2021-02-01`

const baris = teksCsv.trim().split("\n")       // pecah per baris
const kepala = baris[0].split(",")             // nama kolom
const data = baris.slice(1).map(b => {
  const sel = b.split(",")
  const obj = {}
  kepala.forEach((k, i) => { obj[k] = sel[i] })
  return obj
})

sl.write(data)
sl.write(typeof data[0].omzet)
```

	kode	kota	omzet	buka
0	JYP	Jayapura	4200000	1 Mar 2019
1	SOQ	Sorong	3100000	15 Jul 2020
2	MKW	Manokwari	1800000	1 Feb 2021

3 baris × 4 kolom

string

Buka di Playground ↗

Hasilnya larik objek, persis seperti di Bab 7. Tetapi perhatikan baris terakhir: `omzet` masih **teks**. CSV tidak menyimpan tipe data; semua isinya teks. Tugas pertama analis adalah mengubah setiap kolom ke tipe yang benar.

```
const mentah = [
  { kode: "JYP", kota: "Jayapura", omzet: "4200000", buka: "2019-03-01" },
  { kode: "SOQ", kota: "Sorong", omzet: "3100000", buka: "2020-07-15" }
]

const rapi = mentah.map(r => ({
  ...r,
  omzet: Number(r.omzet),
  buka: new Date(r.buka)
}))

sl.write(rapi[0].omzet + rapi[1].omzet)
sl.write(rapi[0].buka.getFullYear() < rapi[1].buka.getFullYear())
```

7.300.000

true

Buka di Playground ↗

Cara manual ini cukup untuk CSV sederhana. Sayangnya CSV dunia nyata jarang sederhana:

- Teks berisi koma diapit tanda kutip: `"Kopi, Teh, dan Gula"`. `split(",")` akan memotongnya di tempat yang salah.
- Berkas dari Excel Indonesia memakai titik koma dan desimal koma: `1.250.000,50`.
- Sel kosong, tanda `-`, atau tulisan `NA` berarti "tidak ada data".

Karena itu, untuk berkas sungguhan kita memakai pembaca CSV milik Slak.

11.3 sl.read_csv

```
const df = await sl.read_csv("toko-nusantara.csv")
sl.write(df.shape)            // [banyak baris, banyak kolom]
sl.dataframe(df.head(5))
```

```
[
  2929,
  13
]
```

	id_transaksi	tanggal	pelanggan_id	cabang	kota	provinsi	kanal	kategori	produk
0	TN000001	1 Jan 2023	P0622	Nusantara Ambon	Ambon	Maluku	Toko	Pakaian	Topi Rotan
1	TN000002	1 Jan 2023	P0039	Nusantara Surabaya	Surabaya	Jawa Timur	Online	Minuman	Kopi Arabika Wamena 2
2	TN000003	1 Jan 2023	P0399	Nusantara Surabaya	Surabaya	Jawa Timur	Online	Kerajinan	Tenun Ikat Sumba Seler
3	TN000004	1 Jan 2023	P0608	Nusantara Jayapura	Jayapura	Papua	Toko	Kerajinan	Tenun Ikat Sumba Seler
4	TN000005	2 Jan 2023	P0187	Nusantara Ambon	Ambon	Maluku	Online	Makanan	Sambal Roa 200g

5 baris × 13 kolom

Buka di Playground ↗

> **API Slak baru**
>
> - `await sl.read_csv(nama)` membaca berkas CSV atau TSV dan mengembalikan **DataFrame**, tabel milik Slak Data. Butuh `slak-data.js`.
> - `df.head(n)` mengambil `n` baris pertama. `df.shape` berisi `[banyak baris, banyak kolom]`.

Kata `await` wajib di depannya karena membaca berkas butuh waktu. Bab 12 menjelaskannya.

`read_csv` mengerjakan semua yang tadi kita lakukan dengan tangan, dan lebih banyak lagi:

- Mengenali pemisah koma, titik koma, atau tab.
- Mengubah kolom angka menjadi angka dan kolom tanggal ISO menjadi tanggal.

- Mengenali format Indonesia `1.250.000,50` dan tanggal `31/12/2024` atau `5 Januari 2024`.
- Menganggap sel kosong, `NA`, `-`, dan `null` sebagai nilai kosong.

```
const toko = await sl.read_csv("toko-nusantara.csv")
sl.write(toko.dtypes)    // tipe setiap kolom
```

```
{
  "id_transaksi": "string",
  "tanggal": "date",
  "pelanggan_id": "string",
  "cabang": "string",
  "kota": "string",
  "provinsi": "string",
  "kanal": "string",
  "kategori": "string",
  "produk": "string",
  "jumlah": "int",
  "harga": "int",
  "diskon_persen": "int",
  "total": "int"
}
```

Buka di Playground ↗

Kamu belum perlu memahami DataFrame. Sampai Bab 13 kita mengubahnya menjadi larik objek biasa dengan `to_records()`, lalu mengolahnya dengan JavaScript murni:

```
const rows = (await sl.read_csv("toko-nusantara.csv")).to_records()
const total = rows.reduce((a, r) => a + r.total, 0)
sl.metric("Omzet 2023–2024", total, null, { format: "rp" })
sl.write(rows[0].tanggal instanceof Date)
```

Omzet 2023–2024

Rp 330.315.950

true

Buka di Playground ↗

Baris terakhir menunjukkan bahwa kolom `tanggal` sudah menjadi objek `Date`, siap dihitung.

Opsi yang sering dipakai

```
const sebagian = await sl.read_csv("toko-nusantara.csv", { usecols: ["tanggal", "kota", "total"], nrows: 100 })
sl.write(sebagian.shape)
```

```
[
  100,
  3
]
```

Buka di Playground ↗

Opsi	Fungsi
`sep`	Pemisah kolom, misalnya `";"`
`decimal`	Tanda desimal, misalnya `","`
`usecols`	Hanya membaca kolom tertentu
`nrows`	Hanya membaca sejumlah baris pertama
`dtype`	Memaksa tipe kolom, misalnya `{ kode: "str" }`
`parse_dates`	Kolom yang harus dibaca sebagai tanggal

11.4 Di mana Slak mencari berkas

Browser melarang halaman yang dibuka dengan klik dua kali membaca berkas lain di komputer. Slak mengakalinya tanpa kamu mengubah kode. Saat kamu menulis `sl.read_csv("toko-nusantara.csv")`, Slak mencari berurutan:

1. **Berkas yang dibungkus**: `toko-nusantara.csv.js` di folder yang sama. Buat lewat Slak Studio (**Alat › Bungkus data**) atau `bungkus.html` dari paket Slak.
2. **Data di dalam halaman**, misalnya `<script type="text/csv" id="toko-nusantara.csv">`. Ekspor HTML Slak Studio memakai cara ini.
3. **Folder proyek**, saat kodemu dijalankan di Slak Studio.
4. **Hosting** atau server lokal, dengan cara biasa.
5. **Tombol pilih berkas.** Bila semua gagal, Slak meminta pengguna memilih berkasnya sekali dan mengingatnya di browser itu.

Folder bahan buku sudah berisi berkas `.csv.js` yang dibungkus, jadi semua contoh buku tetap jalan saat halaman buku dibuka dengan klik dua kali.

> **Catatan untuk pengguna playground** Playground tidak punya folder proyek. Saat contoh yang membaca `toko-nusantara.csv` dibuka di playground, Slak akan memintamu memilih berkasnya (bila playground dibuka dari komputer), atau kamu bisa mengunggahnya dengan `sl.file_uploader` seperti di 11.6. Cara paling nyaman untuk Tahap 2 dan 3 adalah halaman buku ini sendiri atau Slak Studio dengan folder bahan buku.

11.5 JSON

JSON (*JavaScript Object Notation*) adalah teks yang bentuknya persis seperti objek dan larik JavaScript. Karena itu JSON adalah format bawaan internet: hampir semua layanan data mengirim JSON.

```
const teks = '{"kota": "Ambon", "omzet": 1800000, "produk": ["Kopi", "Pala"]}'
const obj = JSON.parse(teks)          // teks → objek
sl.write(obj.produk[1])

const kembali = JSON.stringify({ kota: "Kupang", omzet: 950000 })   // objek → teks
sl.write(kembali)
sl.json(obj)
```

Pala

{"kota":"Kupang","omzet":950000}

```
{
  "kota": "Ambon",
  "omzet": 1800000,
  "produk": [
    "Kopi",
    "Pala"
  ]
}
```

Buka di Playground ↗

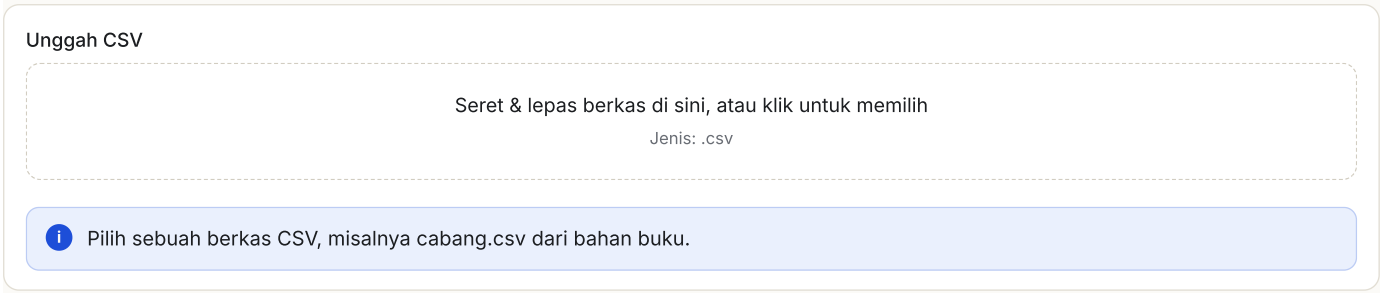
> **API Slak baru**
>
> - `sl.json(objek)` menampilkan objek atau larik dengan rapi dan bisa dilipat.
> - `await sl.read_json(nama)` membaca berkas JSON menjadi DataFrame.

Perbedaan JSON dengan objek JavaScript: nama properti **wajib** diapit tanda kutip ganda, dan tidak boleh ada fungsi, `undefined`, atau koma di akhir.

11.6 Berkas dari pengguna

Aplikasi analisis yang baik membiarkan pengguna memakai datanya sendiri.

```
const berkas = sl.file_uploader("Unggah CSV", { type: ["csv"] })
if (berkas) {
  const data = await sl.read_csv(berkas)
  sl.success(`${berkas.name}: ${data.shape[0]} baris, ${data.shape[1]} kolom`)
  sl.dataframe(data.head(10))
} else {
  sl.info("Pilih sebuah berkas CSV, misalnya cabang.csv dari bahan buku.")
}
```



Buka di Playground ↗

> **API Slak baru**
>
> - `sl.file_uploader(label, { type })` mengembalikan objek berkas (`name`, `size`, `text`, `bytes`) atau `null` bila belum ada berkas. Berkasnya tidak pernah dikirim ke server mana pun; semuanya diproses di browser.

11.7 Seberapa besar data yang sanggup diolah browser?

Slak berjalan di browser, jadi memori yang tersedia adalah memori komputer atau ponsel pengunjung. Slak Data menetapkan batas resmi:

Banyak baris	Rasanya
Sampai 100.000 baris (sekitar 5 MB CSV)	Nyaman, juga di laptop murah dan ponsel
100.000 sampai 1.000.000 baris	Berat; Slak menampilkan peringatan dan menyarankan DuckDB atau SQL
Di atas 1.000.000 baris	Ringkas dulu dengan DuckDB (Bab 21), atau di luar Slak

Data Toko Nusantara yang kita pakai jauh di bawah batas nyaman. Versi besarnya (250 ribu baris) dipakai di Bab 12 dan Bab 21 untuk merasakan bedanya.

Kesalahan umum

Gejala	Penyebab	Perbaikan
`await is only valid in async functions`	`await` dipakai di dalam fungsi biasa	Tambah `async` di depan fungsinya (Bab 12)
Muncul permintaan memilih berkas	Halaman dibuka dengan klik dua kali dan tidak ada `.csv.js`	Bungkus datanya, atau buka lewat Slak Studio atau hosting
Semua angka terbaca sebagai teks di cara manual	CSV tidak menyimpan tipe	Ubah dengan `Number(...)`, atau pakai `sl.read_csv`
`Unexpected token` di `JSON.parse`	Teks bukan JSON yang sah, misalnya kutip tunggal	Periksa kutip ganda dan koma

Latihan

Latihan 11.1 (mudah). Ubah teks CSV berikut menjadi larik objek `produk`, dengan kolom `harga` berupa angka.

```
const csv = `nama,harga
Kopi Wamena,85000
Sagu Lempeng,30000
Sambal Roa,45000`

const produk = []   // ganti

sl.write(produk)
await sl.testing.test("ada 3 produk", () => sl.testing.expect(produk).toHaveLength(3))
await sl.testing.test("harga berupa angka", () => sl.testing.expect(produk[1]).toEqual({ nama: "Sagu Lempeng", harga: 30000 }))
```

```
[]
```

✗ **ada 3 produk** — Diharapkan: panjangnya 3 — nilai sebenarnya: []

✗ **harga berupa angka** — Diharapkan: setara dengan {"nama":"Sagu Lempeng","harga":30000} — nilai sebenarnya: undefined

Buka di Playground ↗

Latihan 11.2 (sedang). Baca `cabang.csv`, lalu buat larik `kotaPapua` berisi nama kota cabang yang provinsinya diawali kata "Papua".

```
const cabang = (await sl.read_csv("cabang.csv")).to_records()
const kotaPapua = []   // ganti

sl.write(kotaPapua)
await sl.testing.test("tiga kota di tanah Papua", () => sl.testing.expect(kotaPapua).toEqual(["Jayapura", "Sorong",
"Manokwari"]))
```

```
[]
```

✗ **tiga kota di tanah Papua** — Diharapkan: setara dengan ["Jayapura","Sorong","Manokwari"] — nilai sebenarnya: []

Buka di Playground ↗

Latihan 11.3 (tantangan). Simpan objek `laporan` sebagai teks JSON di `teks`, lalu baca kembali ke `salinan`. Pastikan `salinan` setara dengan aslinya tetapi bukan objek yang sama.

```
const laporan = { bulan: "2024-04", omzet: 48250000, cabang: ["Jayapura", "Sorong"] }
const teks = ""
const salinan = null

await sl.testing.test("teks adalah JSON", () => sl.testing.expect(teks.startsWith("{")).toBe(true))
await sl.testing.test("isi salinan sama", () => sl.testing.expect(salinan).toEqual(laporan))
await sl.testing.test("bukan objek yang sama", () => sl.testing.expect(salinan === laporan).toBe(false))
```

✗ **teks adalah JSON** — Diharapkan: sama persis dengan true — nilai sebenarnya: false

✗ **isi salinan sama** — Diharapkan: setara dengan {"bulan":"2024-04","omzet":48250000,"cabang":["Jayapura","Sorong"]} — nilai sebenarnya: null

✓ bukan objek yang sama (0 md)

Buka di Playground ↗

`JSON.parse(JSON.stringify(x))` adalah cara singkat menyalin objek sampai ke isi terdalamnya.

Mini-proyek: penampil CSV apa pun

```
sl.title("Penampil CSV :page_facing_up:")

const unggahan = sl.file_uploader("Pilih berkas CSV", { type: ["csv", "tsv", "txt"] })
const contoh = sl.checkbox("Pakai contoh: toko-nusantara.csv", { value: !unggahan })

let df = null
if (unggahan) df = await sl.read_csv(unggahan)
else if (contoh) df = await sl.read_csv("toko-nusantara.csv")

if (!df) {
  sl.info("Unggah berkas, atau centang contoh di atas.")
} else {
  const rows = df.to_records()
  const kolom = df.columns

  const [a, b, c] = sl.columns(3)
  a.metric("Baris", rows.length)
  b.metric("Kolom", kolom.length)
  c.metric("Sel kosong", rows.reduce((n, r) => n + kolom.filter(k => r[k] == null).length, 0))

  const dipilih = sl.multiselect("Kolom yang ditampilkan", kolom, { default: kolom.slice(0, 6) })
  const banyak = sl.slider("Banyak baris", 5, 100, 20, 5)
  sl.dataframe(rows.slice(0, banyak).map(r => Object.fromEntries(dipilih.map(k => [k, r[k]]))))
}
```

Penampil CSV 📄

Pilih berkas CSV

Seret & lepas berkas di sini, atau klik untuk memilih

Jenis: .csv, .tsv, .txt

☑ Pakai contoh: toko-nusantara.csv

Baris	Kolom	Sel kosong
2.929	13	0

Kolom yang ditampilkan

id_transaksi × tanggal × pelanggan_id × cabang × kota × provinsi ×

Pilih opsi...

Banyak baris 20

5 100

	id_transaksi	tanggal	pelanggan_id	cabang	kota	provinsi
0	TN000001	1 Jan 2023	P0622	Nusantara Ambon	Ambon	Maluku
1	TN000002	1 Jan 2023	P0039	Nusantara Surabaya	Surabaya	Jawa Timur
2	TN000003	1 Jan 2023	P0399	Nusantara Surabaya	Surabaya	Jawa Timur
3	TN000004	1 Jan 2023	P0608	Nusantara Jayapura	Jayapura	Papua
4	TN000005	2 Jan 2023	P0187	Nusantara Ambon	Ambon	Maluku
5	TN000006	2 Jan 2023	P0540	Nusantara Surabaya	Surabaya	Jawa Timur
6	TN000007	2 Jan 2023	P0623	Nusantara Manado	Manado	Sulawesi Utara
7	TN000008	3 Jan 2023	P0204	Nusantara Makassar	Makassar	Sulawesi Selatan
8	TN000009	3 Jan 2023	P0527	Nusantara Manokwari	Manokwari	Papua Barat
9	TN000010	3 Jan 2023	P0011	Nusantara Surabaya	Surabaya	Jawa Timur
10	TN000011	4 Jan 2023	P0409	Nusantara Ambon	Ambon	Maluku
11	TN000012	4 Jan 2023	P0588	Nusantara Kupang	Kupang	Nusa Tenggara Timur
12	TN000013	4 Jan 2023	P0150	Nusantara Surabaya	Surabaya	Jawa Timur
13	TN000014	5 Jan 2023	P0590	Nusantara Makassar	Makassar	Sulawesi Selatan
14	TN000015	5 Jan 2023	P0588	Nusantara Kupang	Kupang	Nusa Tenggara Timur
15	TN000016	5 Jan 2023	P0266	Nusantara Makassar	Makassar	Sulawesi Selatan
16	TN000017	5 Jan 2023	P0389	Nusantara Makassar	Makassar	Sulawesi Selatan
17	TN000018	6 Jan 2023	P0010	Nusantara Ambon	Ambon	Maluku
18	TN000019	6 Jan 2023	P0163	Nusantara Makassar	Makassar	Sulawesi Selatan
19	TN000020	6 Jan 2023	P0323	Nusantara Jayapura	Jayapura	Papua

20 baris × 6 kolom

Buka di Playground ↗

`df.columns` berisi larik nama kolom. `Object.fromEntries` adalah kebalikan dari `Object.entries`: larik pasangan `[nama, nilai]` menjadi objek.

← 10. Cara Slak Menjalankan Skrip

12. Asinkron dan Mengambil Data →

Bab 12 · Asinkron dan Mengambil Data

> **Tujuan belajar**
>
> - Memahami mengapa sebagian pekerjaan tidak selesai seketika.
> - Memakai Promise, `async`, dan `await`.
> - Mengambil data dari internet dengan `fetch` dan `sl.read_url`.
> - Memakai dataset bawaan Slak dan dataset terbuka: GitHub, Kaggle, Hugging Face, Bank Dunia, dan katalog data terbuka.
> - Membaca Google Sheets dan berkas Excel.
> - Menampilkan proses yang lama dengan `sl.spinner` dan `sl.progress`, serta mengenal `sl.worker`.

12.1 Memesan makanan

Bayangkan kamu memesan kopi di kedai. Kasir tidak menyuruhmu berdiri diam di depan mesin sampai kopinya jadi. Kamu diberi **nomor antrean**, lalu bebas duduk. Saat kopinya siap, nomormu dipanggil.

Mengambil data dari internet atau membaca berkas besar bekerja dengan cara yang sama. Pekerjaannya butuh waktu, dan JavaScript tidak mau seluruh halaman membeku selama menunggu. Maka fungsi seperti `fetch` tidak langsung mengembalikan data. Ia mengembalikan **Promise**, semacam nomor antrean: janji bahwa hasilnya akan datang nanti.

```
const janji = sl.sleep(500)   // menunggu setengah detik, mengembalikan Promise
sl.write(janji instanceof Promise)
```

```
true
```

Buka di Playground ↗

12.2 await: tunggu sampai siap

`await` berarti "tunggu janji ini terpenuhi, lalu berikan hasilnya".

```
sl.write("Memesan kopi…")
await sl.sleep(1000)
sl.write("Kopi siap :coffee:")
```

```
Memesan kopi...
Kopi siap ☕
```

Buka di Playground ↗

Tanpa `await`, baris ketiga langsung jalan tanpa menunggu.

Di skrip Slak, `await` boleh langsung dipakai di tingkat atas. Di dalam fungsi, fungsinya harus ditandai `async`:

```
async function hargaKopi() {
  await sl.sleep(300)          // pura-pura bertanya ke server
  return 85000
}

const harga = await hargaKopi()
sl.write(`Harga kopi: ${harga}`)
```

```
Harga kopi: 85000
```

Buka di Playground ↗

Fungsi `async` **selalu** mengembalikan Promise, jadi pemanggilnya juga perlu `await`.

Menunggu beberapa hal sekaligus

Bila tiga pesanan tidak saling bergantung, jangan tunggu satu per satu. `Promise.all` menunggu semuanya bersamaan:

```
const t0 = performance.now()
const [a, b, c] = await Promise.all([sl.sleep(400), sl.sleep(400), sl.sleep(400)])
sl.write(`Selesai dalam ${Math.round(performance.now() - t0)} md, bukan 1200 md`)
```

Selesai dalam 400 md, bukan 1200 md

Buka di Playground ↗

12.3 fetch: mengambil dari internet

`fetch` adalah fungsi bawaan browser untuk mengambil apa pun dari internet. Contoh berikut **butuh internet**: ia mengambil jumlah penduduk Indonesia dari API Bank Dunia.

```
try {
  const url = "https://api.worldbank.org/v2/country/IDN/indicator/SP.POP.TOTL?format=json&date=2019:2023"
  const respons = await fetch(url)
  const json = await respons.json()      // dua kali menunggu: header, lalu isi
  const baris = json[1].map(r => ({ tahun: r.date, penduduk: r.value }))
  sl.table(baris)
} catch (e) {
  sl.warning("Tidak bisa mengambil data: " + e.message)
}
```

	tahun	penduduk
0	2023	281.190.067
1	2022	278.830.529
2	2021	276.758.053
3	2020	274.814.866
4	2019	272.489.381

Buka di Playground ↗

Perhatikan dua `await`: pertama menunggu jawaban server datang, kedua menunggu isinya selesai dibaca. Bungkus selalu dengan `try...catch`, karena internet bisa putus kapan saja.

Untuk CSV, JSON, Excel, atau zip di alamat mana pun, `sl.read_url` lebih praktis karena langsung menghasilkan DataFrame:

```
try {
  const df = await sl.read_url("https://raw.githubusercontent.com/plotly/datasets/master/2014_usa_states.csv")
  sl.write(`${df.shape[0]} baris`)
  sl.dataframe(df.head(5))
} catch (e) {
  sl.warning(e.message)
}
```

52 baris

	Rank	State	Postal	Population
0	1	Alabama	AL	4.849.377
1	2	Alaska	AK	736.732
2	3	Arizona	AZ	6.731.484
3	4	Arkansas	AR	2.966.369
4	5	California	CA	38.802.500

5 baris × 4 kolom

Buka di Playground ↗

> **API Slak baru**
>
> - `await sl.read_url(alamat)` membaca CSV, TSV, JSON, Excel, atau zip dari alamat web dan mengembalikan DataFrame.

Beberapa server menolak dibaca dari halaman web lain (aturan keamanan browser bernama CORS). Bila itu terjadi, Slak menampilkan pesan yang menjelaskannya.

12.4 Dataset bawaan dan dataset terbuka

Slak menyediakan dua jenis sumber data siap pakai.

Dataset bawaan ada di dalam `slak-data.js`, selalu tersedia tanpa internet:

```
sl.dataframe(sl.datasets())
```

	nama	keterangan	baris
0	penjualan	Transaksi toko di Papua 2023–2024: tanggal, pelanggan, kota, kanal, kategori, ...	3.278
1	penjualan_bulanan	Omzet bulanan 2019–2024 dengan tren naik dan pola musiman (puncak Dese...	72
2	kunjungan_harian	Kunjungan harian situs web selama dua tahun dengan pola mingguan. Cocok un...	730
3	pelanggan	Data 1.000 pelanggan layanan internet dan apakah mereka berhenti (churn). Co...	1.000
4	ab_test	Eksperimen A/B halaman checkout dengan 4.000 pengunjung: grup, perangkat,...	4.000
5	karyawan	Data 300 karyawan: pengalaman, pendidikan, departemen, jam pelatihan, nilai k...	300
6	nilai_ujian	Nilai 90 siswa sebelum dan sesudah program belajar, dari tiga metode berbeda....	90
7	data_kotor	Data pesanan yang sengaja berantakan: spasi berlebih, huruf besar-kecil camp...	62
8	iris	Data bunga iris klasik (Fisher, 1936): 150 bunga, 4 ukuran, 3 spesies. Data asli, ...	150

9 baris × 3 kolom

Buka di Playground ↗

Dataset terbuka diunduh dari sumbernya saat dipakai, lalu disimpan di browser selama 7 hari sehingga tetap jalan saat internet putus:

Fungsi	Sumber	Contoh
`sl.open_data`	Katalog 15 dataset pilihan beserta lisensinya	`await sl.open_data("penguins")`
`sl.github`	Berkas di GitHub	`await sl.github("plotly/datasets/2014_usa_states.csv")`
`sl.worldbank`	Indikator Bank Dunia	`await sl.worldbank("SP.POP.TOTL", { negara: ["IDN"] })`
`sl.huggingface`	Dataset Hugging Face	`await sl.huggingface("scikit-learn/iris", { file: "Iris.csv" })`
`sl.kaggle`	Dataset Kaggle	`await sl.kaggle("uciml/iris", { file: "Iris.csv" })`

```
try {
  const katalog = sl.open_data.list()
  sl.dataframe(katalog.select(["id", "judul", "baris", "lisensi"]))
} catch (e) {
  sl.warning(e.message)
}
```

	id	judul	baris	lisensi
0	penguins	Penguin Palmer	344	CC0 (domain publik)
1	co2	Emisi CO_2 dunia (Our World in Data)	50.000	CC BY 4.0 (sebutkan Our World in Data)
2	gapminder	Gapminder	1.704	CC BY 4.0 (sebutkan Gapminder)
3	iris	Iris	150	CC BY 4.0 (UCI Machine Learning Repository)
4	tips	Tip restoran	244	Dataset klasik pendidikan; lisensi tidak dinyatakan d…
5	titanic	Penumpang Titanic	891	Dataset klasik pendidikan; lisensi tidak dinyatakan d…
6	flights	Penumpang pesawat bulanan	144	Dataset klasik pendidikan; lisensi tidak dinyatakan d…
7	mpg	Efisiensi bahan bakar mobil	398	CC BY 4.0 (UCI Machine Learning Repository)
8	diamonds	Harga berlian	53.940	Dataset contoh ggplot2; lisensi tidak dinyatakan di …
9	taxis	Taksi New York	6.433	Data terbuka Kota New York (NYC TLC)
10	saham_apple	Saham Apple 2014	240	Data contoh Plotly
11	negara_bagian_as	Penduduk negara bagian AS 2014	50	Data contoh Plotly (sumber: Sensus AS)
12	supermarket	Penjualan supermarket	1.000	Data contoh Plotly
13	gempa	Gempa bumi dunia	23.412	Data contoh Plotly (sumber: USGS, domain publik)
14	minuman_iowa	Penjualan minuman Iowa 2021	50.000	Data terbuka Negara Bagian Iowa

15 baris × 4 kolom

Buka di Playground ↗

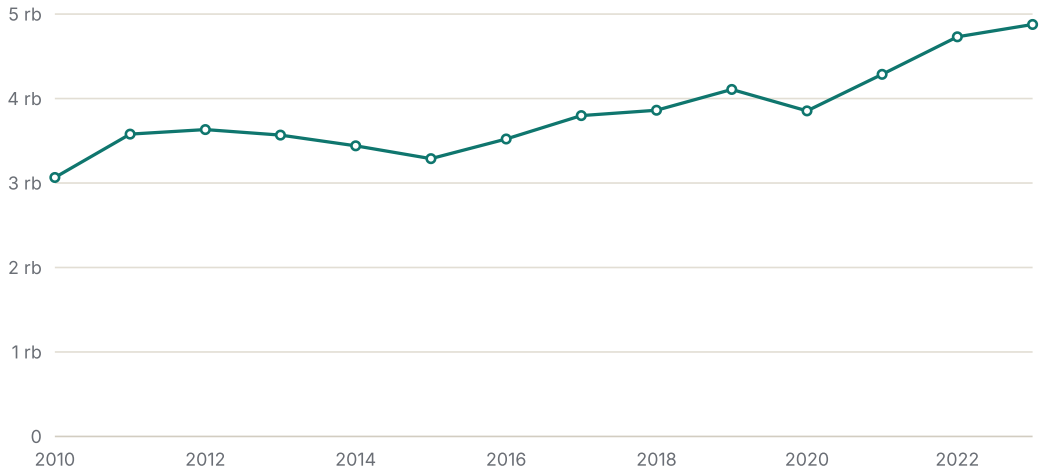
Selalu periksa kolom **lisensi** sebelum memakai data untuk tujuan komersial atau portofolio. Beberapa dataset klasik tidak menyatakan lisensi di sumbernya.

Kaggle tidak mengizinkan browser mengunduh langsung. Di Slak Studio, unduhan berjalan biasa. Di browser biasa, Slak memintamu memilih berkas zip yang kamu unduh sendiri dari situs Kaggle, lalu mengingatnya.

Bank Dunia untuk Indonesia

```
try {
  const wb = await sl.worldbank("NY.GDP.PCAP.CD", { negara: ["IDN"], tahun: [2010, 2023] })
  const rows = wb.to_records().filter(r => r.nilai != null)
  sl.line_chart(rows, { x: "tahun", y: "nilai", title: "PDB per kapita Indonesia (US$)" })
} catch (e) {
  sl.warning("Butuh internet: " + e.message)
}
```

PDB per kapita Indonesia (US$)



Buka di Playground ↗

> **API Slak baru**
>
> - `sl.open_data(id)`, `sl.open_data.list()`, `sl.github(jalur)`, `sl.worldbank(indikator, { negara, tahun })`, `sl.huggingface(dataset, { file })`, `sl.kaggle(dataset, { file })` mengambil dataset terbuka sebagai DataFrame. Semuanya ada di `slak-data.js`.
> - `sl.datasets()` menampilkan daftar dataset bawaan.
> - `sl.line_chart(data, { x, y, title })` menggambar grafik garis. Grafik dibahas di Bab 16.

Supaya aplikasi tetap jalan di komputer mana pun tanpa internet, tambahkan `sl.offline_button()`. Tombol itu mengunduh data sebagai berkas `.csv.js`; taruh di folder halaman, dan data dibaca dari sana.

12.5 Google Sheets dan Excel

Banyak data kantor tinggal di Google Sheets. Bagikan lembarnya sebagai **Siapa saja yang memiliki link dapat melihat**, lalu:

```
const df = await sl.read_sheets("https://docs.google.com/spreadsheets/d/ID-LEMBAR-KAMU/edit")
sl.dataframe(df)
```

`sl.read_sheets` tetap jalan saat halaman dibuka dengan klik dua kali, asalkan ada internet.

Berkas Excel dibaca tanpa pustaka tambahan:

```
const laporan = await sl.read_excel("laporan.xlsx", { sheet: "Penjualan" })
const lembar = await sl.excel_sheets("laporan.xlsx")   // daftar nama lembar
```

12.6 Memberi tahu pengguna bahwa kamu sedang bekerja

Pengguna yang melihat layar diam akan mengira aplikasimu rusak. Tunjukkan bahwa sesuatu sedang dikerjakan.

```
const angka = await sl.spinner("Menghitung omzet…", async () => {
  await sl.sleep(800)
  return 48250000
})
sl.metric("Omzet", angka, null, { format: "rp" })
```

Omzet

Rp 48.250.000

Buka di Playground ↗

```
if (sl.button("Proses data")) {
  const bar = sl.progress(0, "Mulai…")
  const langkah = ["Membaca", "Membersihkan", "Meringkas", "Menggambar"]
  for (let i = 0; i < langkah.length; i++) {
    await sl.sleep(400)
    bar.progress((i + 1) * 25, langkah[i])
  }
  sl.success("Selesai")
}
```

Proses data

Buka di Playground ↗

> **API Slak baru**
>
> - `await sl.spinner(teks, fungsiAsync)` menampilkan putaran selama fungsi bekerja, lalu mengembalikan hasilnya.
> - `sl.progress(nilai, teks)` menampilkan bilah kemajuan berskala 0–100. Perbarui dengan `.progress(nilai, teks)`.

12.7 Pekerjaan berat di latar belakang

JavaScript di halaman bekerja di satu jalur. Saat ada hitungan berat, misalnya memproses ratusan ribu baris, halaman ikut membeku: tombol tidak bisa diklik, penggeser tidak bisa digeser.

Kabar baiknya, `sl.read_csv` sudah menangani ini sendiri. Untuk berkas di atas 5 MB, pekerjaan mengurai teks, angka, dan tanggal otomatis dipindah ke **Web Worker**, jalur kedua di belakang layar. Hasilnya sama persis, dan halaman hanya membeku sesaat.

```
if (sl.button("Muat 250 ribu transaksi")) {
  const t0 = performance.now()
  const besar = await sl.spinner("Membaca toko-nusantara-besar.csv…", () => sl.read_csv("toko-nusantara-besar.csv"))
  sl.success(`${besar.shape[0].toLocaleString("id-ID")} baris dibaca dalam ${((performance.now() - t0) /
1000).toFixed(1)} detik`)
}
```

Muat 250 ribu transaksi

Buka di Playground ↗

Contoh ini butuh `toko-nusantara-besar.csv` (29 MB) dari bahan buku di folder yang sama. Berkas itu tidak dibungkus menjadi `.csv.js` karena terlalu besar; buka halaman buku lewat Slak Studio atau hosting, atau pilih berkasnya saat diminta.

> **Fitur eksperimental:** `sl.worker` Untuk hitunganmu sendiri, Slak 1.9 menyediakan `sl.worker(fungsi, ...data)`. Di daftar API stabil, fungsi ini berlabel **eksperimental**: cara pakainya boleh berubah sebelum Slak 2.0. Pakai bila memang butuh, dan periksa catatan rilis saat memperbarui Slak.
>
> Aturannya: fungsi dikirim ke worker sebagai teks, jadi ia **tidak boleh memakai variabel dari luar fungsi**, termasuk `sl`. Semua yang dibutuhkan dikirim sebagai argumen. DataFrame dikirim sebagai larik objek. Bila browser menolak worker, fungsinya dijalankan biasa.

```
const df = sl.dataset("penjualan")
const besarSekali = await sl.worker((rows, batas) => rows.filter(r => r.total > batas), df, 500000)
sl.write(`${besarSekali.length} transaksi di atas 500 ribu`)
sl.dataframe(sl.DataFrame(besarSekali).head(5))
```

24 transaksi di atas 500 ribu

	id_transaksi	tanggal	pelanggan_id	kota	kanal	kategori	produk	jumlah	harga	total
0	T00434	25 Jan 2023	P097	Jayapura	Online	Pakaian	Topi Rimba	7	85.000	595.000
1	T01229	29 Apr 2023	P256	Jayapura	Online	Minuman	Kopi Papua 250g	8	65.000	520.000
2	T00253	8 Mei 2023	P063	Jayapura	Online	Minuman	Kopi Papua 250g	11	65.000	715.000
3	T00818	2 Jun 2023	P164	Jayapura	Online	Pakaian	Topi Rimba	6	85.000	510.000
4	T01378	21 Agu 2023	P285	Biak	Online	Minuman	Kopi Papua 250g	9	65.000	585.000

5 baris × 10 kolom

Buka di Playground ↗

> **Coba di playground** Ganti `penguins` di contoh katalog dengan `id` lain dari daftar `sl.open_data.list()`, lalu lihat kolom-kolomnya.

Kesalahan umum

Gejala	Penyebab	Perbaikan
Muncul `[object Promise]`	Lupa `await`	Tulis `await` di depan pemanggilan
`await is only valid in async functions`	`await` di dalam fungsi biasa	Tandai fungsinya `async`
`Failed to fetch`	Tidak ada internet, alamat salah, atau server menolak (CORS)	Periksa alamat; bungkus dengan `try...catch`
`sl.worker : x is not defined`	Fungsi worker memakai variabel dari luar	Kirim semua yang dibutuhkan sebagai argumen

Latihan

Latihan 12.1 (mudah). Fungsi `ambilStok` mengembalikan Promise. Ambil hasilnya ke `stok`.

```
async function ambilStok() {
  await sl.sleep(100)
  return 24
}
const stok = ambilStok()   // perbaiki

sl.write(stok)
await sl.testing.test("stok = 24", () => sl.testing.expect(stok).toBe(24))
```

```
{}
```

✕ **stok = 24** — Diharapkan: sama persis dengan 24 — nilai sebenarnya: {}

Buka di Playground ↗

Latihan 12.2 (sedang). Jalankan ketiga fungsi bersamaan dengan `Promise.all`, lalu jumlahkan hasilnya ke `total`. Uji kedua memeriksa bahwa waktunya kurang dari 500 md.

```
const omzetCabang = async (n) => { await sl.sleep(300); return n }
const t0 = performance.now()

const total = 0   // ganti: tunggu omzetCabang(100), omzetCabang(200), omzetCabang(300) bersamaan

const lama = performance.now() - t0
await sl.testing.test("total = 600", () => sl.testing.expect(total).toBe(600))
await sl.testing.test("berjalan bersamaan", () => sl.testing.expect(lama).toBeLessThan(500))
```

✗ **total = 600** — Diharapkan: sama persis dengan 600 — nilai sebenarnya: 0

✓ berjalan bersamaan (0 md)

Buka di Playground ↗

Latihan 12.3 (tantangan). Lengkapi `ambilAman(fn)` yang menjalankan fungsi async `fn` dan mengembalikan hasilnya, atau `null` bila `fn` melempar galat.

```
async function ambilAman(fn) {
  return await fn()
}

const berhasil = async () => "data"
const gagal = async () => { throw new Error("internet putus") }

await sl.testing.test("berhasil → data", async () => sl.testing.expect(await ambilAman(berhasil)).toBe("data"))
await sl.testing.test("gagal → null", async () => sl.testing.expect(await ambilAman(gagal)).toBeNull())
```

✓ berhasil → data (0 md)

✗ **gagal → null** — internet putus

Buka di Playground ↗

Mini-proyek: Indonesia dalam angka

Aplikasi ini **butuh internet**. Ia mengambil tiga indikator Bank Dunia sekaligus, lalu menampilkan perkembangan Indonesia dibanding negara tetangga.

```
sl.title("Indonesia dalam Angka :earth_asia:")

const INDIKATOR = {
  "PDB per kapita (US$)": "NY.GDP.PCAP.CD",
  "Penduduk": "SP.POP.TOTL",
  "Pengguna internet (% penduduk)": "IT.NET.USER.ZS"
}

const pilih = sl.selectbox("Indikator", Object.keys(INDIKATOR))
const negara = sl.multiselect("Bandingkan dengan", ["MYS", "PHL", "THA", "VNM"], { default: ["MYS", "VNM"] })

try {
  const df = await sl.spinner("Mengambil data Bank Dunia…", () =>
    sl.worldbank(INDIKATOR[pilih], { negara: ["IDN", ...negara], tahun: [2005, 2023] }))
  const rows = df.to_records().filter(r => r.nilai != null)

  const idn = rows.filter(r => r.kode === "IDN").sort((a, b) => a.tahun - b.tahun)
  const awal = idn[0], akhir = idn[idn.length - 1]
  sl.metric(`${pilih}, ${akhir.tahun}`, Math.round(akhir.nilai * 10) / 10,
    `${((akhir.nilai / awal.nilai - 1) * 100).toFixed(0)}% sejak ${awal.tahun}`)

  sl.line_chart(rows, { x: "tahun", y: "nilai", color: "negara" })
  sl.caption("Sumber: Bank Dunia, World Development Indicators.")
} catch (e) {
  sl.warning("Data tidak bisa diambil. Periksa koneksi internet. " + e.message)
}
```

Indonesia dalam Angka 🌏

Indikator

PDB per kapita (US$)

Bandingkan dengan

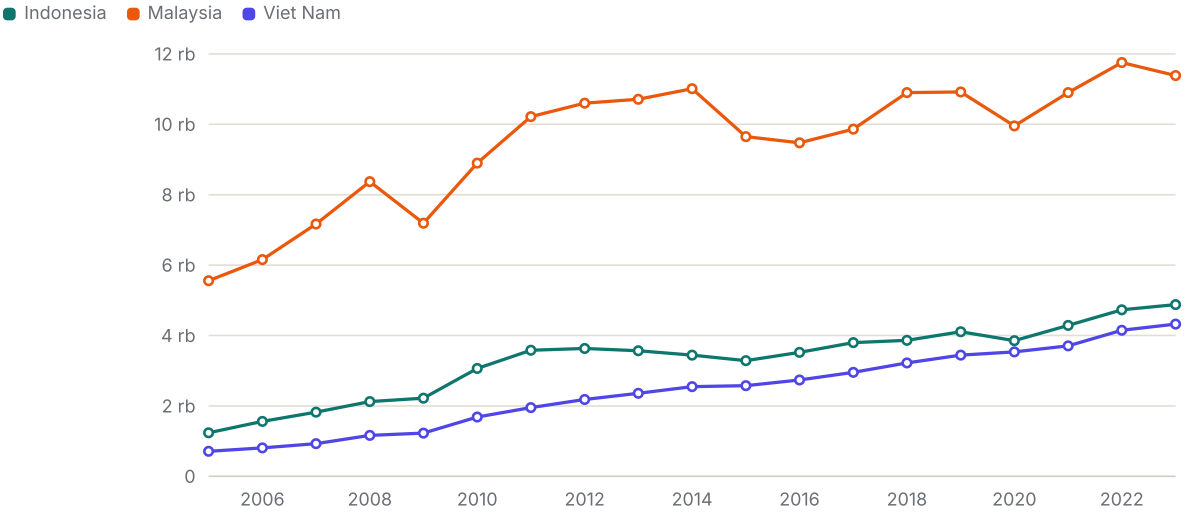
MYS × VNM ×

Pilih opsi...

PDB per kapita (US$), 2023

4.876,3

▲ 294% sejak 2005



Sumber: Bank Dunia, World Development Indicators.

Buka di Playground ↗

Kode negara yang dipakai Bank Dunia: IDN Indonesia, MYS Malaysia, PHL Filipina, THA Thailand, VNM Vietnam.

Bab 13 · Membersihkan Data dengan JavaScript Murni

> **Tujuan belajar**
>
> - Memeriksa kualitas data: nilai kosong, duplikat, format campur, salah ketik, dan pencilan.
> - Menulis fungsi pembersih yang bisa dipakai ulang dengan JavaScript murni.
> - Menyeragamkan teks, angka, dan tanggal.
> - Memperbaiki sel secara manual dengan tabel yang bisa diedit.

Kata orang, 80% pekerjaan analis data adalah membersihkan data. Angka itu tidak berlebihan. Data dari kasir, formulir, atau gabungan beberapa cabang hampir selalu berantakan, dan analisis dari data kotor menghasilkan kesimpulan yang salah.

Di bab ini kamu membersihkan `toko-nusantara-kotor.csv` dengan JavaScript murni: `map`, `filter`, objek, dan teks. Di bab berikutnya kamu mengulang semua langkah ini dengan Slak Data, dalam jauh lebih sedikit baris.

13.1 Kenali dulu kotorannya

```
const mentah = (await sl.read_csv("toko-nusantara-kotor.csv")).to_records()
sl.write(`${mentah.length} baris`)
sl.code(JSON.stringify(mentah.slice(0, 6), null, 1))
```

394 baris


```
[
 {
  "id_transaksi": "TN001510",
  "tanggal": "2024-02-14",
  "kota": "Makassar",
  "kanal": "Online",
  "kategori": "Kerajinan",
  "produk": "gelang manik papua ",
  "jumlah": 3,
  "harga": "Rp 42.000"
 },
 {
  "id_transaksi": "TN001785",
  "tanggal": "2024-04-13",
  "kota": "Jayapura",
  "kanal": "Toko",
  "kategori": "Kebutuhan Rumah",
  "produk": "Minyak Kayu Putih 60ml",
  "jumlah": 1,
  "harga": "Rp 40.000"
 },
 {
  "id_transaksi": "TN002217",
  "tanggal": "2024-07-29",
  "kota": "Jayapura",
  "kanal": "Toko",
  "kategori": "Makanan",
  "produk": "Sagu Lempeng 1kg",
  "jumlah": 1,
  "harga": "31500"
 },
 {
  "id_transaksi": "TN002245",
  "tanggal": "2024-08-04",
  "kota": "Surabaya",
  "kanal": "Toko",
  "kategori": "Minuman",
  "produk": "Kopi Arabika Wamena 250g",
  "jumlah": 1,
  "harga": "89000"
```

```
 },
 {
  "id_transaksi": "TN001425",
  "tanggal": "2024-01-21",
  "kota": "Makassar",
  "kanal": "Toko",
  "kategori": "Makanan",
  "produk": "Sambal Roa 200g",
  "jumlah": 5,
  "harga": "47000"
 },
 {
  "id_transaksi": "TN002331",
  "tanggal": "2024-08-26",
  "kota": "Makassar",
  "kanal": "Toko",
  "kategori": "Kerajinan",
  "produk": "Gelang Manik Papua",
  "jumlah": 1,
  "harga": "42000"
 }
]
```


Buka di Playground ↗

`JSON.stringify(x, null, 1)` menampilkan isi data **apa adanya**, termasuk spasi dan tanda kutip, sehingga kotorannya terlihat jelas. Tabel `sl.dataframe` sengaja merapikan tampilan, jadi untuk memeriksa data mentah, lihat teksnya.

Mari menghitung masalahnya satu per satu:

```
const data = (await sl.read_csv("toko-nusantara-kotor.csv")).to_records()
const kolom = Object.keys(data[0])

const kosong = {}
for (const k of kolom) kosong[k] = data.filter(r => r[k] == null || r[k] === "").length
sl.write("Nilai kosong per kolom:")
sl.json(kosong)

const id = data.map(r => r.id_transaksi)
sl.write(`ID ganda: ${id.length - new Set(id).size}`)

const ejaanKota = [...new Set(data.map(r => r.kota))]
sl.write(`Ejaan kota yang berbeda: ${ejaanKota.length}`)
sl.code(JSON.stringify(ejaanKota))

const contohHarga = [...new Set(data.map(r => typeof r.harga))]
sl.write(`Tipe kolom harga: ${contohHarga.join(", ")}`)
```

Nilai kosong per kolom:


```
{
  "id_transaksi": 0,
  "tanggal": 0,
  "kota": 0,
  "kanal": 19,
  "kategori": 0,
  "produk": 0,
  "jumlah": 16,
  "harga": 0
}
```


ID ganda: 14

Ejaan kota yang berbeda: 13

```
["Makassar","Jayapura","Surabaya","jayapura"," Jayapura","surabaya","Makasar","Jayapra","Surabya","makassar ","
SURABAYA ","MAKASSAR","JAYAPURA"]
```

Tipe kolom harga: string

Buka di Playground ↗

- `new Set(larik)` membuat himpunan: nilai yang sama hanya disimpan sekali. `set.size` adalah banyaknya nilai unik.
- `[...new Set(larik)]` mengubahnya kembali menjadi larik berisi nilai-nilai unik.

Hanya tiga kota, tetapi ejaannya lebih dari sepuluh. Ada transaksi ganda, nilai kosong, dan harga yang sebagian berupa teks `"Rp 42.000"`. Daftar pekerjaan kita:

1. Rapikan teks: spasi berlebih dan huruf besar-kecil.
2. Seragamkan nama kota, termasuk salah ketik.
3. Ubah harga menjadi angka.
4. Ubah tanggal menjadi `Date`, dari tiga format berbeda.
5. Tangani nilai kosong.
6. Buang transaksi ganda.
7. Tangani pencilan.

13.2 Fungsi pembersih teks

```
function rapikan(teks) {
  if (teks == null) return null
  return String(teks).trim().replace(/\s+/g, " ")
}

function hurufJudul(teks) {
  if (teks == null) return null
  return rapikan(teks).toLowerCase().split(" ")
    .map(kata => kata.charAt(0).toUpperCase() + kata.slice(1))
    .join(" ")
}

sl.write(`[${rapikan("  Kopi   Wamena ")}]`)
sl.write(hurufJudul("  GELANG manik papua "))
```

[Kopi Wamena]

Gelang Manik Papua

Buka di Playground ↗

`/\s+/g` adalah pola untuk "satu atau lebih spasi". Menggantinya dengan satu spasi merapikan spasi ganda di tengah teks.

13.3 Menyeragamkan nama kota

Setelah huruf judul, sisa masalahnya adalah salah ketik. Untuk itu, buat **kamus perbaikan**: objek yang memetakan ejaan salah ke ejaan benar.

```
const PERBAIKAN_KOTA = { Jayapra: "Jayapura", Makasar: "Makassar", Surabya: "Surabaya" }

function normalKota(teks) {
  if (teks == null) return null
  const t = teks.trim().toLowerCase().replace(/^\w/, c => c.toUpperCase())
  return PERBAIKAN_KOTA[t] || t
}

for (const k of ["jayapura", " Jayapura", "JAYAPURA", "Jayapra", "makassar ", "Makasar", " SURABAYA "]) {
  sl.write(`"${k}" → ${normalKota(k)}`)
}
```

"jayapura" → Jayapura

" Jayapura" → Jayapura

"JAYAPURA" → Jayapura

"Jayapra" → Jayapura

"makassar " → Makassar

"Makasar" → Makassar

" SURABAYA " → Surabaya

Buka di Playground ↗

`PERBAIKAN_KOTA[t] || t` berarti "ambil perbaikannya bila ada, bila tidak pakai `t` itu sendiri". Kamus seperti ini lebih mudah dirawat daripada deretan `if`: menambah perbaikan baru cukup menambah satu pasangan.

13.4 Angka dari teks

Kamu sudah menulis fungsi ini di Latihan 8.2. Kini tambahkan penanganan nilai kosong:

```
function keAngka(x) {
  if (x == null || x === "") return null
  if (typeof x === "number") return x
  const bersih = String(x).replace(/[^0-9,-]/g, "").replace(",", ".")
  const n = Number(bersih)
  return Number.isNaN(n) ? null : n
}

sl.write(keAngka("Rp 42.000"))
sl.write(keAngka(31500))
sl.write(keAngka("Rp 1.250.000,50"))
sl.write(keAngka(""))
```

42.000

31.500

1.250.000,5

`null`

Buka di Playground ↗

Titik ribuan dibuang, dan koma desimal diganti titik supaya dipahami JavaScript.

13.5 Tanggal dari tiga format

Kolom `tanggal` berisi tiga gaya: `2024-02-14`, `14/02/2024`, dan `14 Februari 2024`. Fungsi berikut mengenali ketiganya dan selalu mengembalikan `Date`.

```
const BULAN = ["januari", "februari", "maret", "april", "mei", "juni", "juli",
               "agustus", "september", "oktober", "november", "desember"]

function keTanggal(teks) {
  if (teks == null || teks === "") return null
  const t = String(teks).trim()
  let m
  if ((m = t.match(/^(\d{4})-(\d{2})-(\d{2})$/))) return new Date(+m[1], +m[2] - 1, +m[3])
  if ((m = t.match(/^(\d{1,2})\/(\d{1,2})\/(\d{4})$/))) return new Date(+m[3], +m[2] - 1, +m[1])
  if ((m = t.match(/^(\d{1,2}) (\w+) (\d{4})$/))) {
    const b = BULAN.indexOf(m[2].toLowerCase())
    if (b >= 0) return new Date(+m[3], b, +m[1])
  }
```

```
  return null
}

const fmt = new Intl.DateTimeFormat("id-ID", { dateStyle: "medium" })
for (const x of ["2024-02-14", "14/02/2024", "14 Februari 2024", "kemarin"]) {
  const d = keTanggal(x)
  sl.write(`${x} → ${d ? fmt.format(d) : "tidak dikenali"}`)
}
```

```
2024-02-14 → 14 Feb 2024
14/02/2024 → 14 Feb 2024
14 Februari 2024 → 14 Feb 2024
kemarin → tidak dikenali
```

Buka di Playground ↗

Hal baru di sini:

- `t.match(pola)` mencocokkan teks dengan pola dan mengembalikan bagian-bagiannya. `\d{4}` berarti empat angka, dan bagian dalam kurung biasa ditangkap ke `m[1]`, `m[2]`, dan seterusnya.
- `+m[1]` adalah cara singkat mengubah teks menjadi angka, sama dengan `Number(m[1])`.
- `new Date(tahun, bulan, tanggal)` membuat tanggal di zona waktu setempat. Ingat, bulannya dihitung dari 0.

Fungsi yang tidak mengenali format mengembalikan `null`, bukan menebak. Lebih baik jujur "tidak tahu" daripada diam-diam salah.

13.6 Nilai kosong, duplikat, dan pencilan

Untuk nilai kosong, tidak ada satu jawaban yang selalu benar. Pilih sesuai arti kolomnya:

Kolom	Keputusan	Alasan
`kanal` kosong	Isi `"Tidak diketahui"`	Transaksinya tetap sah; kanalnya saja yang tidak tercatat
`jumlah` kosong	Buang barisnya	Tanpa jumlah, omzetnya tidak bisa dihitung
`harga` tidak terbaca	Buang barisnya	Sama seperti di atas

Transaksi ganda dikenali dari `id_transaksi`. Simpan yang pertama, buang sisanya:

```
function buangGanda(data, kunci) {
  const sudah = new Set()
  return data.filter(r => {
    if (sudah.has(r[kunci])) return false
    sudah.add(r[kunci])
    return true
  })
}

const contoh = [{ id: "A", n: 1 }, { id: "B", n: 2 }, { id: "A", n: 1 }]
sl.write(buangGanda(contoh, "id"))
```

	id	n
0	A	1
1	B	2

2 baris × 2 kolom

Buka di Playground ↗

Pencilan (outlier) adalah nilai yang jauh berbeda dari kebanyakan data. Di data kita ada beberapa transaksi dengan `jumlah` 250, padahal pembeli biasanya membeli 1 sampai 5 barang. Kemungkinan besar salah ketik. Untuk sekarang kita pakai aturan sederhana: jumlah di atas 50 dianggap salah dan dibuang. Bab 19 mengajarkan cara menentukan batasnya secara statistik dengan metode IQR.

Selalu **catat** keputusan pembersihanmu. Orang yang membaca laporanmu berhak tahu berapa baris yang dibuang dan mengapa.

13.7 Menyatukan semuanya

```
const BLN = ["januari", "februari", "maret", "april", "mei", "juni", "juli", "agustus", "september", "oktober", "november",
"desember"]
const KOTA = { Jayapra: "Jayapura", Makasar: "Makassar", Surabya: "Surabaya" }

const rapi = t => t == null ? null : String(t).trim().replace(/\s+/g, " ")
const judul = t => t == null ? null : rapi(t).toLowerCase().replace(/(^|\s)\S/g, c => c.toUpperCase())
const kota = t => { const j = judul(t); return KOTA[j] || j }
const angka = x => {
  if (x == null || x === "") return null
  if (typeof x === "number") return x
  const n = Number(String(x).replace(/[^0-9,-]/g, "").replace(",", "."))
  return Number.isNaN(n) ? null : n
}
const tanggal = t => {
  if (t == null) return null
  const s = String(t).trim(); let m
  if ((m = s.match(/^(\d{4})-(\d{2})-(\d{2})$/))) return new Date(+m[1], +m[2] - 1, +m[3])
  if ((m = s.match(/^(\d{1,2})\/(\d{1,2})\/(\d{4})$/))) return new Date(+m[3], +m[2] - 1, +m[1])
  if ((m = s.match(/^(\d{1,2}) (\w+) (\d{4})$/)) && BLN.includes(m[2].toLowerCase())) return new Date(+m[3],
BLN.indexOf(m[2].toLowerCase()), +m[1])
  return null
}

function bersihkan(data) {
  const catatan = []
  let hasil = data.map(r => ({
    id_transaksi: rapi(r.id_transaksi),
    tanggal: tanggal(r.tanggal),
    kota: kota(r.kota),
    kanal: rapi(r.kanal) || "Tidak diketahui",
    kategori: rapi(r.kategori),
    produk: judul(r.produk),
    jumlah: angka(r.jumlah),
    harga: angka(r.harga)
  }))

  const n0 = hasil.length
  hasil = hasil.filter(r => r.jumlah != null && r.harga != null && r.tanggal != null)
  catatan.push(`${n0 - hasil.length} baris dibuang karena jumlah, harga, atau tanggal kosong`)

  const n1 = hasil.length
  const sudah = new Set()
  hasil = hasil.filter(r => !sudah.has(r.id_transaksi) && sudah.add(r.id_transaksi))
  catatan.push(`${n1 - hasil.length} transaksi ganda dibuang`)

  const n2 = hasil.length
  hasil = hasil.filter(r => r.jumlah <= 50)
  catatan.push(`${n2 - hasil.length} pencilan jumlah (> 50) dibuang`)

  hasil = hasil.map(r => ({ ...r, total: r.jumlah * r.harga }))
  return { hasil, catatan }
}

const mentah = (await sl.read_csv("toko-nusantara-kotor.csv")).to_records()
const { hasil, catatan } = bersihkan(mentah)

sl.write(`Dari ${mentah.length} baris menjadi ${hasil.length} baris bersih.`)
for (const c of catatan) sl.markdown(`- ${c}`)
sl.write(`Kota: ${[...new Set(hasil.map(r => r.kota))].join(", ")}`)
sl.dataframe(hasil.slice(0, 8))
```

Dari 394 baris menjadi 362 baris bersih.

- 16 baris dibuang karena jumlah, harga, atau tanggal kosong
- 13 transaksi ganda dibuang
- 3 pencilan jumlah (> 50) dibuang

Kota: Makassar, Jayapura, Surabaya

	id_transaksi	tanggal	kota	kanal	kategori	produk	jumlah	harga	total
0	TN001510	14 Feb 2024	Makassar	Online	Kerajinan	Gelang Manik Papua	3	42.000	126.000
1	TN001785	13 Apr 2024	Jayapura	Toko	Kebutuhan Rumah	Minyak Kayu Putih 60ml	1	40.000	40.000
2	TN002217	29 Jul 2024	Jayapura	Toko	Makanan	Sagu Lempeng 1kg	1	31.500	31.500
3	TN002245	4 Agu 2024	Surabaya	Toko	Minuman	Kopi Arabika Wamena 250g	1	89.000	89.000
4	TN001425	21 Jan 2024	Makassar	Toko	Makanan	Sambal Roa 200g	5	47.000	235.000
5	TN002331	26 Agu 2024	Makassar	Toko	Kerajinan	Gelang Manik Papua	1	42.000	42.000
6	TN002173	17 Jul 2024	Surabaya	Toko	Pakaian	Topi Rotan	2	89.000	178.000
7	TN001909	14 Mei 2024	Surabaya	Online	Pakaian	Kain Pantai Bali	1	100.000	100.000

8 baris × 9 kolom

Buka di Playground ↗

Dua trik di dalam `bersihkan`:

- `.replace(/(^|\s)\S/g, c => c.toUpperCase())` membesarkan huruf pertama setiap kata dalam satu langkah.
- `!sudah.has(id) && sudah.add(id)` memeriksa sekaligus mencatat. `add` mengembalikan himpunannya sendiri, yang selalu dianggap benar.

Fungsi `bersihkan` bisa dipakai ulang untuk data bulan depan dengan format yang sama. Itulah gunanya menulis pembersihan sebagai fungsi, bukan langkah sekali jalan.

13.8 Memperbaiki dengan tangan

Kadang perbaikan paling cepat adalah mengedit beberapa sel langsung, seperti di Excel. `sl.data_editor` menampilkan tabel yang bisa diedit dan mengembalikan data hasil editan.

```
const stokAwal = [
  { produk: "Kopi Arabika Wamena 250g", stok: 24, rak: "A1" },
  { produk: "Keripik keladi", stok: null, rak: "B3" },
  { produk: "Sabun Sereh Wangi", stok: 50, rak: "" }
]

const diedit = sl.data_editor(stokAwal, { num_rows: "dynamic", key: "edit-stok" })
const belumLengkap = diedit.filter(r => r.stok == null || !r.rak).length
sl.caption(belumLengkap ? `${belumLengkap} baris belum lengkap` : "Semua baris lengkap")
```

		produk	stok	rak
×	0	Kopi Arabika Wamena 250g	24	A1
×	1	Keripik keladi	—	B3
×	2	Sabun Sereh Wangi	50	

3 baris + Tambah baris

2 baris belum lengkap

Buka di Playground ↗

> **API Slak baru**

> - `sl.data_editor(data, { num_rows })` menampilkan tabel yang bisa diedit, lalu mengembalikan larik hasil editan. `num_rows: "dynamic"` mengizinkan pengguna menambah dan menghapus baris.

Klik dua kali sel kosong, isi nilainya, lalu tekan Enter. Keterangan di bawahnya langsung berubah.

> **Coba di playground** Tambahkan `"Surabya": "Surabaya"` yang sudah ada di kamus dengan ejaan salah baru, misalnya `"Jaya Pura": "Jayapura"`. Ingat bahwa kuncinya harus ditulis dalam bentuk setelah `hurufJudul`.

Kesalahan umum

Gejala	Penyebab	Perbaikan
Kota yang sama terhitung dua kali	Beda spasi atau huruf besar-kecil	Rapikan dan seragamkan sebelum mengelompokkan
Hasil penjumlahan menjadi `NaN`	Ada satu harga yang tidak terbaca	Ubah dengan fungsi yang mengembalikan `null`, lalu saring
Tanggal 03/04 terbaca 4 Maret	Format bulan-hari ala Amerika	Tentukan sendiri urutannya, seperti di `keTanggal`
Baris hilang terlalu banyak	Aturan pembersihan terlalu ketat	Hitung dan catat yang dibuang di setiap langkah

Latihan

Latihan 13.1 (mudah). Lengkapi `bersihkanKanal(k)`: rapikan spasi dan huruf, dan isi `"Tidak diketahui"` bila kosong.

```
function bersihkanKanal(k) {
  return k
}

const { test, expect } = sl.testing
await test('" online " → Online', () => expect(bersihkanKanal(" online ")).toBe("Online"))
await test('"TOKO" → Toko', () => expect(bersihkanKanal("TOKO")).toBe("Toko"))
await test("null → Tidak diketahui", () => expect(bersihkanKanal(null)).toBe("Tidak diketahui"))
await test('"" → Tidak diketahui', () => expect(bersihkanKanal("")).toBe("Tidak diketahui"))
```

- ✕ **" online " → Online** — Diharapkan: sama persis dengan "Online" — nilai sebenarnya: " online "
- ✕ **"TOKO" → Toko** — Diharapkan: sama persis dengan "Toko" — nilai sebenarnya: "TOKO"
- ✕ **null → Tidak diketahui** — Diharapkan: sama persis dengan "Tidak diketahui" — nilai sebenarnya: null
- ✕ **"" → Tidak diketahui** — Diharapkan: sama persis dengan "Tidak diketahui" — nilai sebenarnya: ""

Buka di Playground ↗

Latihan 13.2 (sedang). Lengkapi `buangGanda(data)` yang membuang baris dengan `id` ganda, **menyimpan yang terakhir**, bukan yang pertama. Urutan hasil boleh mengikuti kemunculan terakhir.

```
function buangGanda(data) {
  return data
}

const d = [{ id: 1, v: "lama" }, { id: 2, v: "x" }, { id: 1, v: "baru" }]
await sl.testing.test("tersisa 2 baris", () => sl.testing.expect(buangGanda(d)).toHaveLength(2))
await sl.testing.test("id 1 berisi yang baru", () => sl.testing.expect(buangGanda(d).find(r => r.id === 1).v).toBe("baru"))
```

- ✕ **tersisa 2 baris** — Diharapkan: panjangnya 2 — nilai sebenarnya: [{"id":1,"v":"lama"},{"id":2,"v":"x"},{"id":1,"v":"baru"}]

✗ **id 1 berisi yang baru** — Diharapkan: sama persis dengan "baru" — nilai sebenarnya: "lama"

Buka di Playground ↗

Latihan 13.3 (tantangan). Tambahkan format keempat ke `keTanggal`: `"2024.02.14"` (titik). Fungsi harus tetap mengenali format ISO.

```
function keTanggal(t) {
  let m
  if ((m = t.match(/^(\d{4})-(\d{2})-(\d{2})$/))) return new Date(+m[1], +m[2] - 1, +m[3])
  return null
}

const iso = d => d ? `${d.getFullYear()}-${d.getMonth() + 1}-${d.getDate()}` : null
await sl.testing.test("ISO tetap jalan", () => sl.testing.expect(iso(keTanggal("2024-02-14"))).toBe("2024-2-14"))
await sl.testing.test("format titik dikenali", () => sl.testing.expect(iso(keTanggal("2024.02.14"))).toBe("2024-2-14"))
```

✓ ISO tetap jalan (0 md)

✗ **format titik dikenali** — Diharapkan: sama persis dengan "2024-2-14" — nilai sebenarnya: null

Buka di Playground ↗

Mini-proyek: laporan sebelum dan sesudah

Buat laporan yang menunjukkan kualitas data sebelum dan sesudah dibersihkan, lengkap dengan dampaknya pada angka omzet. Laporan seperti ini membuat atasan atau klien percaya pada analisismu.

```
sl.title("Laporan Kebersihan Data :broom:")

const KAMUS = { Jayapra: "Jayapura", Makasar: "Makassar", Surabya: "Surabaya" }
const judul = t => t == null ? null : String(t).trim().replace(/\s+/g, " ").toLowerCase().replace(/(^|\s)\S/g, c =>
c.toUpperCase())
const angka = x => x == null || x === "" ? null : typeof x === "number" ? x : Number(String(x).replace(/[^0-9]/g, ""))

const mentah = (await sl.read_csv("toko-nusantara-kotor.csv")).to_records()

const sudah = new Set()
const bersih = mentah
  .map(r => ({ id: r.id_transaksi, kota: KAMUS[judul(r.kota)] || judul(r.kota), jumlah: angka(r.jumlah), harga:
angka(r.harga) }))
  .filter(r => r.jumlah != null && r.harga != null)
  .filter(r => !sudah.has(r.id) && sudah.add(r.id))
  .filter(r => r.jumlah <= 50)
  .map(r => ({ ...r, total: r.jumlah * r.harga }))

const omzetMentah = mentah.reduce((a, r) => a + (Number(r.jumlah) || 0) * (angka(r.harga) || 0), 0)
const omzetBersih = bersih.reduce((a, r) => a + r.total, 0)

const [a, b, c] = sl.columns(3)
a.metric("Baris", bersih.length, `${bersih.length - mentah.length} dari mentah`)
b.metric("Ejaan kota", new Set(bersih.map(r => r.kota)).size, `${new Set(bersih.map(r => r.kota)).size - new
Set(mentah.map(r => r.kota)).size}`)
c.metric("Omzet", omzetBersih, null, { format: "rp" })

sl.warning(`Bila data tidak dibersihkan, omzet terhitung ${Math.round((omzetMentah / omzetBersih - 1) * 100)}% lebih tinggi
dari sebenarnya, karena transaksi ganda dan salah ketik jumlah.`)

const perKota = {}
for (const r of bersih) perKota[r.kota] = (perKota[r.kota] || 0) + r.total
sl.table(Object.entries(perKota).map(([kota, omzet]) => ({ kota, omzet })))
```

Laporan Kebersihan Data 🧹

Baris
362
▼ -32 dari mentah

Ejaan kota
3
▼ -10

Omzet
Rp 40.399.000

! Bila data tidak dibersihkan, omzet terhitung 106% lebih tinggi dari sebenarnya, karena transaksi ganda dan salah ketik jumlah.

	kota	omzet
0	Makassar	9.684.500
1	Jayapura	15.984.000
2	Surabaya	14.730.500

Buka di Playground ↗

Kalimat peringatan di laporan ini adalah temuan yang sesungguhnya. Data kotor tidak hanya jelek dilihat; ia membuat angka keputusan menjadi salah.

← 12. Asinkron dan Mengambil Data

14. Dari JavaScript Murni ke Slak Data →

Bab 14 · Dari JavaScript Murni ke Slak Data

> **Tujuan belajar**
>
> - Mengenal `sl.DataFrame` dan `sl.Series`, tabel dan kolom gaya pandas.
> - Mengulang semua langkah Bab 13 dengan Slak Data.
> - Memilih kolom, menyaring, mengurutkan, dan menambah kolom.
> - Melihat isi dan ringkasan data dengan `head`, `info`, `describe`, dan `shape`.
> - Menimbang kapan memakai JavaScript murni dan kapan memakai Slak Data.

14.1 DataFrame dan Series

Sejak Bab 11, `sl.read_csv` mengembalikan **DataFrame** dan kita langsung mengubahnya menjadi larik objek dengan `to_records()`. Kini saatnya mengenal DataFrame itu sendiri.

- **DataFrame** adalah tabel: setiap kolom punya nama dan tipe.
- **Series** adalah satu kolom: deretan nilai bertipe sama.

Namanya sengaja dibuat sama dengan **pandas**, pustaka analisis data paling terkenal di Python. Bila suatu hari kamu belajar Python, sebagian besar yang kamu pelajari di sini langsung berlaku.

```
const df = sl.DataFrame([
  { kota: "Jayapura", omzet: 4200000, cabang: 1 },
  { kota: "Sorong", omzet: 3100000, cabang: 1 },
  { kota: "Manokwari", omzet: 1800000, cabang: 1 }
])

sl.write(df.shape)            // [baris, kolom]
sl.write(df.columns)          // nama kolom
sl.write(df.omzet.sum())      // satu kolom adalah Series
sl.write(df.omzet.mean())
sl.dataframe(df)
```

```
[
  3,
  3
]
```

```
[
  "kota",
  "omzet",
  "cabang"
]
```

9.100.000

3.033.333,3333

	kota	omzet	cabang
0	Jayapura	4.200.000	1
1	Sorong	3.100.000	1
2	Manokwari	1.800.000	1

3 baris × 3 kolom

Buka di Playground ↗

> **API Slak baru**
>
> - `sl.DataFrame(larikObjek)` membuat DataFrame dari larik objek. Kebalikannya: `df.to_records()`.
> - `df.namaKolom` atau `df["nama kolom"]` mengambil satu kolom sebagai Series.
> - Series punya ringkasan siap pakai: `sum()`, `mean()`, `median()`, `min()`, `max()`, `std()`, `nunique()`, `quantile(0.9)`.

sl. atau nama tabel?

Satu aturan yang perlu kamu ingat sepanjang Tahap 2 dan 3:

- Awalan `sl.` dipakai untuk menyuruh Slak **menampilkan** sesuatu, **membuat widget**, atau **memuat data**: `sl.dataframe(df)`, `sl.slider(...)`, `sl.read_csv(...)`.
- **Nama tabel** dipakai untuk **mengolah** tabel itu: `df.groupby(...)`, `df.query(...)`.

Nama `df` hanyalah kebiasaan; tabelmu boleh bernama `penjualan`. Bila awalannya tertukar, misalnya `sl.groupby(...)`, Slak memberi pesan yang menunjuk tulisan yang benar.

14.2 Mengenal data dalam empat perintah

```
const toko = await sl.read_csv("toko-nusantara.csv")
sl.dataframe(toko.head())     // 5 baris pertama
sl.dataframe(toko.info())     // tipe, terisi, kosong, unik per kolom
sl.dataframe(toko.describe()) // statistik ringkas kolom angka
sl.dataframe(toko.value_counts("kota"))
```

	id_transaksi	tanggal	pelanggan_id	cabang	kota	provinsi	kanal	kategori	produk
0	TN000001	1 Jan 2023	P0622	Nusantara Ambon	Ambon	Maluku	Toko	Pakaian	Topi Rotan
1	TN000002	1 Jan 2023	P0039	Nusantara Surabaya	Surabaya	Jawa Timur	Online	Minuman	Kopi Arabika Wamena 2
2	TN000003	1 Jan 2023	P0399	Nusantara Surabaya	Surabaya	Jawa Timur	Online	Kerajinan	Tenun Ikat Sumba Seler
3	TN000004	1 Jan 2023	P0608	Nusantara Jayapura	Jayapura	Papua	Toko	Kerajinan	Tenun Ikat Sumba Seler
4	TN000005	2 Jan 2023	P0187	Nusantara Ambon	Ambon	Maluku	Online	Makanan	Sambal Roa 200g

5 baris × 13 kolom

	kolom	tipe	terisi	kosong	unik	contoh
0	id_transaksi	string	2.929	0	2.929	TN000001
1	tanggal	date	2.929	0	731	1/1/2023
2	pelanggan_id	string	2.929	0	631	P0622
3	cabang	string	2.929	0	10	Nusantara Ambon
4	kota	string	2.929	0	10	Ambon
5	provinsi	string	2.929	0	10	Maluku
6	kanal	string	2.929	0	2	Toko
7	kategori	string	2.929	0	5	Pakaian
8	produk	string	2.929	0	20	Topi Rotan
9	jumlah	int	2.929	0	5	2
10	harga	int	2.929	0	37	85000
11	diskon_persen	int	2.929	0	2	0
12	total	int	2.929	0	190	170000

13 baris × 6 kolom

	statistik	jumlah	harga	diskon_persen	total
0	count	2.929	2.929	2.929	2.929
1	mean	1,6709	65.797,2004	0,2731	112.774,3086
2	std	0,9977	57.366,8988	1,6302	149.395,2652
3	min	1	18.000	0	16.200
4	25%	1	35.000	0	40.000
5	50%	1	48.000	0	72.000
6	75%	2	85.000	0	126.000
7	max	5	367.500	10	1.837.500

8 baris × 5 kolom

	kota	jumlah
0	Surabaya	387
1	Jakarta	384
2	Makassar	350
3	Jayapura	340
4	Ambon	301
5	Denpasar	273
6	Sorong	246
7	Manado	245
8	Kupang	202
9	Manokwari	201

10 baris × 2 kolom

Buka di Playground ↗

Empat baris ini adalah langkah pertama setiap analisis. Dalam semenit kamu tahu banyaknya baris, tipe setiap kolom, ada tidaknya nilai kosong, dan sebaran angkanya. Bandingkan dengan Bab 13.1, yang butuh beberapa perulangan untuk sebagian kecil informasi ini.

14.3 Bab 13 dalam beberapa baris

Inilah pembersihan Bab 13, langkah demi langkah, berdampingan dengan versi JavaScript murninya:

Langkah	JavaScript murni (Bab 13)	Slak Data
Rapikan spasi	`map` + `trim` + regex	`df.strip_strings()`
Huruf judul	`split` + `map` + `join`	`df.kota.str.title()`
Salah ketik	kamus + `KAMUS[t] |	|
Harga dari teks	fungsi `keAngka` dengan regex	`df.to_numeric("harga")`
Tanggal tiga format	fungsi `keTanggal` dengan `match`	`df.to_datetime("tanggal")`
Isi nilai kosong	`r.kanal |	|
Buang baris kosong	`filter(r => r.jumlah != null)`	`df.dropna({ subset: ["jumlah"] })`
Buang ganda	`Set` + `filter`	`df.drop_duplicates({ subset: ["id_transaksi"] })`
Tambah kolom	`map(r => ({ ...r, total }))`	`df.assign({ total: r => r.jumlah * r.harga })`

```
const SALAH_KETIK = { Jayapra: "Jayapura", Makasar: "Makassar", Surabya: "Surabaya" }

let df = await sl.read_csv("toko-nusantara-kotor.csv")
df = df.strip_strings()
df = df.assign({
  kota: () => df.kota.str.title().replace(SALAH_KETIK),
  produk: () => df.produk.str.title()
```

```
})
df = df.to_numeric("harga").to_datetime("tanggal")
df = df.fillna({ kanal: "Tidak diketahui" }).dropna({ subset: ["jumlah", "harga", "tanggal"] })
df = df.drop_duplicates({ subset: ["id_transaksi"] })
df = df.filter(df.jumlah.le(50))
df = df.assign({ total: r => r.jumlah * r.harga })

sl.write(`${df.shape[0]} baris bersih, omzet ${df.total.sum().toLocaleString("id-ID")}`)
sl.write(df.kota.unique())
sl.dataframe(df.head(8))
```

362 baris bersih, omzet 40.399.000

```
[
  "Makassar",
  "Jayapura",
  "Surabaya"
]
```

	id_transaksi	tanggal	kota	kanal	kategori	produk	jumlah	harga	total
0	TN001510	14 Feb 2024	Makassar	Online	Kerajinan	Gelang Manik Papua	3	42.000	126.000
1	TN001785	13 Apr 2024	Jayapura	Toko	Kebutuhan Rumah	Minyak Kayu Putih 60ml	1	40.000	40.000
2	TN002217	29 Jul 2024	Jayapura	Toko	Makanan	Sagu Lempeng 1kg	1	31.500	31.500
3	TN002245	4 Agu 2024	Surabaya	Toko	Minuman	Kopi Arabika Wamena 250g	1	89.000	89.000
4	TN001425	21 Jan 2024	Makassar	Toko	Makanan	Sambal Roa 200g	5	47.000	235.000
5	TN002331	26 Agu 2024	Makassar	Toko	Kerajinan	Gelang Manik Papua	1	42.000	42.000
6	TN002173	17 Jul 2024	Surabaya	Toko	Pakaian	Topi Rotan	2	89.000	178.000
7	TN001909	14 Mei 2024	Surabaya	Online	Pakaian	Kain Pantai Bali	1	100.000	100.000

8 baris × 9 kolom

Buka di Playground ↗

Hasilnya sama persis dengan Bab 13: 362 baris dengan omzet yang sama. Bedanya, kode ini bisa dibaca seperti daftar langkah, dan setiap langkahnya sudah teruji.

Hal-hal yang perlu diperhatikan:

- **Setiap perintah menghasilkan DataFrame baru.** Data asli tidak berubah. Karena itu hasilnya selalu disimpan lagi: `df = df.strip_strings()`.
- Di `assign`, fungsi tanpa parameter `() => df.kota...` mengembalikan **Series** untuk seluruh kolom, sedangkan fungsi dengan parameter `r => r.jumlah * r.harga` dipanggil **per baris**, seperti `map`.
- `df.jumlah.le(50)` menghasilkan Series benar/salah: *le* artinya *less or equal*, kurang dari atau sama dengan. Ada juga `lt`, `gt`, `ge`, `eq`, dan `ne`.

> **API Slak baru**
>
> - Membersihkan: `strip_strings()`, `clean_names()` (merapikan nama kolom), `to_numeric(kolom)`, `to_datetime(kolom)`, `fillna({ kolom: nilai })`, `dropna({ subset })`, `drop_duplicates({ subset })`, `missing()` (nilai kosong per kolom).
> - Teks per kolom lewat `.str`: `.str.title()`, `.str.lower()`, `.str.upper()`, `.str.contains()`, `.str.replace()`.
> - `series.replace({ lama: baru })` mengganti nilai lewat kamus.

14.4 Memilih, menyaring, mengurutkan

```
const t = await sl.read_csv("toko-nusantara.csv")

const kecil = t.select(["tanggal", "kota", "produk", "total"])   // pilih kolom
const papua = kecil.filter(r => ["Jayapura", "Sorong", "Manokwari"].includes(r.kota))
const besar = papua.query("total >= 300000")
const urut = besar.sort_values("total", { ascending: false })

sl.write(`${urut.shape[0]} transaksi besar di tanah Papua`)
sl.dataframe(urut.head(10))
```

50 transaksi besar di tanah Papua

	tanggal	kota	produk	total
0	12 Mei 2024	Manokwari	Tenun Ikat Sumba Selendang	1.837.500
1	19 Jan 2023	Manokwari	Tenun Ikat Sumba Selendang	1.750.000
2	19 Mei 2024	Jayapura	Tenun Ikat Sumba Selendang	1.470.000
3	5 Jan 2024	Manokwari	Ukiran Asmat Mini	1.445.000
4	19 Okt 2024	Sorong	Ukiran Asmat Mini	1.445.000
5	1 Jan 2023	Jayapura	Tenun Ikat Sumba Selendang	1.050.000
6	27 Nov 2023	Sorong	Tenun Ikat Sumba Selendang	1.050.000
7	9 Apr 2024	Jayapura	Ukiran Asmat Mini	867.000
8	7 Jan 2024	Sorong	Tenun Ikat Sumba Selendang	735.000
9	26 Feb 2024	Jayapura	Tenun Ikat Sumba Selendang	735.000

10 baris × 4 kolom

Buka di Playground ↗

Ada tiga cara menyaring, pilih yang paling mudah dibaca untukmu:

Cara	Contoh	Cocok untuk
Fungsi per baris	`df.filter(r => r.kanal === "Online")`	Logika apa pun, gaya JavaScript
Series benar/salah	`df.filter(df.total.gt(100000))`	Syarat sederhana pada satu kolom
Ekspresi teks	`df.query("total > 100000 and kota in ['Ambon', 'Kupang']")`	Syarat panjang, gaya pandas

`query` bisa memakai nilai dari variabel atau widget dengan awalan `@`:

```
const t2 = await sl.read_csv("toko-nusantara.csv")
const minimal = sl.number_input("Total minimal", { value: 500000, step: 50000 })
const hasil = t2.query("total >= @minimal", { minimal })
sl.write(`${hasil.shape[0]} transaksi`)
```

Total minimal

500000 − +

68 transaksi

Buka di Playground ↗

> **API Slak baru**
>
> - `select([kolom])`, `filter(syarat)`, `query("ekspresi")`, `sort_values(kolom, { ascending })`, `head(n)`, `tail(n)`, `sample(n)`.
> - Ekspresi `query` mendukung `and`, `or`, `not`, `in [...]`, `between ... and ...`, `is null`, serta fungsi `contains`, `startswith`, `lower`, `year`, `month`, `quarter`, `isna`, dan `len`.

14.5 Menambah kolom

```
const tk = await sl.read_csv("toko-nusantara.csv")
const lengkap = tk.assign({
  pajak: r => Math.round(r.total * 0.11),
  bulan: () => tk.tanggal.dt.month,
  hari: () => tk.tanggal.dt.day_name(),
  ukuran: () => sl.cut(tk.total, [0, 50000, 200000, Infinity], { labels: ["kecil", "sedang", "besar"] })
})
sl.dataframe(lengkap.select(["tanggal", "hari", "bulan", "total", "pajak", "ukuran"]).head(8))
```

	tanggal	hari	bulan	total	pajak	ukuran
0	1 Jan 2023	Minggu	1	170.000	18.700	sedang
1	1 Jan 2023	Minggu	1	170.000	18.700	sedang
2	1 Jan 2023	Minggu	1	350.000	38.500	besar
3	1 Jan 2023	Minggu	1	1.050.000	115.500	besar
4	2 Jan 2023	Senin	1	135.000	14.850	sedang
5	2 Jan 2023	Senin	1	36.000	3.960	kecil
6	2 Jan 2023	Senin	1	480.000	52.800	besar
7	3 Jan 2023	Selasa	1	55.000	6.050	sedang

8 baris × 6 kolom

Buka di Playground ↗

- `.dt` memberi akses ke bagian tanggal: `year`, `month`, `day`, `quarter`, `weekday` (0 = Senin), `day_name()` ("Senin"), `month_name()` ("Januari"), dan `strftime("%Y-%m")`.
- `sl.cut(series, batas, { labels })` memasukkan angka ke dalam kelompok.

14.6 Bisakah hasilnya dipercaya?

Pertanyaan wajar: Slak Data ditulis dalam JavaScript, bukan pandas. Apakah angkanya benar?

Slak Data diuji akurasinya terhadap pustaka Python yang dipakai di seluruh dunia: **pandas, scipy, statsmodels, dan scikit-learn**. Pengujiannya mencakup 150 kasus, dan hasilnya cocok, kecuali dua perbedaan yang memang disengaja: pembulatan 1,25 menjadi 1,3 seperti Excel, dan `str.title()` yang tidak membesarkan huruf setelah angka.

Kamu juga bisa memeriksanya sendiri. Ingat, kamu sudah menulis fungsi `rataRata` dan `median` di Bab 5:

```
const tc = await sl.read_csv("toko-nusantara.csv")
const nilai = tc.total.to_list()

const rataManual = nilai.reduce((a, b) => a + b, 0) / nilai.length
const urut = [...nilai].sort((a, b) => a - b)
const tengah = Math.floor(urut.length / 2)
const medianManual = urut.length % 2 ? urut[tengah] : (urut[tengah - 1] + urut[tengah]) / 2

const { test, expect } = sl.testing
await test("rata-rata sama", () => expect(tc.total.mean()).toBeCloseTo(rataManual, 6))
await test("median sama", () => expect(tc.total.median()).toBe(medianManual))
```

- ✓ rata-rata sama (0 md)
- ✓ median sama (1 md)

Buka di Playground ↗

14.7 Kapan JavaScript murni, kapan Slak Data?

Pakai JavaScript murni bila	Pakai Slak Data bila
Datanya kecil dan logikanya unik	Langkahnya umum: bersihkan, saring, kelompokkan, gabungkan
Kamu butuh kendali penuh atas setiap langkah	Kamu ingin kode yang singkat dan mudah dibaca orang lain
Membangun fungsi yang dipakai ulang di tempat lain	Butuh statistik, deret waktu, atau machine learning

Keduanya bisa dicampur bebas. `df.to_records()` mengubah DataFrame menjadi larik objek, dan `sl.DataFrame(larik)` mengembalikannya. `df.filter(r => ...)` bahkan menerima fungsi JavaScript biasa.

> **Coba di playground** Pada contoh 14.4, ganti `"total >= 300000"` dengan `"month(tanggal) == 12"`. Transaksi bulan apa saja yang tersisa?

Kesalahan umum

Gejala	Penyebab	Perbaikan
Pembersihan seolah tidak berpengaruh	Hasil tidak disimpan	`df = df.dropna()`, bukan hanya `df.dropna()`
`sl.groupby is not a function`	Awalan tertukar	Mengolah tabel memakai nama tabel: `df.groupby(...)`
`Kolom "Kota" tidak ada`	Nama kolom beda huruf besar-kecil atau ada spasi	Lihat `df.columns`, atau rapikan dengan `clean_names()`
Kolom angka terbaca teks	Ada sel berisi huruf	`df.to_numeric("kolom")`; sel yang gagal menjadi kosong

Latihan

Latihan 14.1 (mudah). Dari `toko-nusantara.csv`, hitung banyaknya transaksi kanal Online di variabel `nOnline`.

```
const df = await sl.read_csv("toko-nusantara.csv")
const nOnline = 0   // ganti

sl.metric("Transaksi online", nOnline)
await sl.testing.test("banyak transaksi online benar", () =>
  sl.testing.expect(nOnline).toBe(df.to_records().filter(r => r.kanal === "Online").length))
```

Transaksi online

0

✕ **banyak transaksi online benar** — Diharapkan: sama persis dengan 1029 — nilai sebenarnya: 0

Buka di Playground ↗

Latihan 14.2 (sedang). Buat DataFrame `murah` berisi transaksi kategori Makanan dengan harga di bawah 40.000, hanya kolom `produk` dan `harga`, diurutkan dari harga termurah.

```
const df = await sl.read_csv("toko-nusantara.csv")
const murah = df   // ganti

sl.dataframe(murah.head())
await sl.testing.test("dua kolom", () => sl.testing.expect(murah.columns).toEqual(["produk", "harga"]))
await sl.testing.test("semua di bawah 40 ribu", () => sl.testing.expect(murah.harga.max()).toBeLessThan(40000))
await sl.testing.test("urut naik", () => sl.testing.expect(murah.harga.to_list()[0]).toBe(murah.harga.min()))
```

	id_transaksi	tanggal	pelanggan_id	cabang	kota	provinsi	kanal	kategori	produk
0	TN000001	1 Jan 2023	P0622	Nusantara Ambon	Ambon	Maluku	Toko	Pakaian	Topi Rotan
1	TN000002	1 Jan 2023	P0039	Nusantara Surabaya	Surabaya	Jawa Timur	Online	Minuman	Kopi Arabika Wamena 2
2	TN000003	1 Jan 2023	P0399	Nusantara Surabaya	Surabaya	Jawa Timur	Online	Kerajinan	Tenun Ikat Sumba Seler
3	TN000004	1 Jan 2023	P0608	Nusantara Jayapura	Jayapura	Papua	Toko	Kerajinan	Tenun Ikat Sumba Seler
4	TN000005	2 Jan 2023	P0187	Nusantara Ambon	Ambon	Maluku	Online	Makanan	Sambal Roa 200g

5 baris × 13 kolom

✕ **dua kolom** — Diharapkan: setara dengan ["produk","harga"] — nilai sebenarnya: ["id_transaksi","tanggal","pelanggan_id","cabang","kota","provinsi","kanal","kategori","produk","jumlah","harga","diskon_persen","total"]

✕ **semua di bawah 40 ribu** — Diharapkan: lebih kecil dari 40000 — nilai sebenarnya: 367500

✕ **urut naik** — Diharapkan: sama persis dengan 18000 — nilai sebenarnya: 85000

Buka di Playground ↗

Latihan 14.3 (tantangan). Tambahkan kolom `margin` ke data bersih, yaitu `total` dikurangi 60% dari `total`, lalu hitung `marginRata` sebagai rata-rata margin transaksi kanal Toko.

```
const df = await sl.read_csv("toko-nusantara.csv")
const marginRata = 0   // ganti

sl.metric("Rata-rata margin kanal Toko", marginRata, null, { format: "rp" })
const harapan = df.filter(r => r.kanal === "Toko").total.mean() * 0.4
await sl.testing.test("margin rata-rata benar", () => sl.testing.expect(marginRata).toBeCloseTo(harapan, 2))
```

Rata-rata margin kanal Toko

Rp 0

✕ **margin rata-rata benar** — Diharapkan: mendekati 46182.11578947369 (2 desimal) — nilai sebenarnya: 0

Buka di Playground ↗

Mini-proyek: pembersih data serba guna

Pembersih ini menerima berkas CSV apa pun, menampilkan laporan kualitasnya, membersihkan dengan pilihan pengguna, lalu menawarkan hasilnya untuk diunduh.

```
sl.title("Pembersih Data :sparkles:")

const unggah = sl.file_uploader("CSV kotor", { type: ["csv"] })
let df = unggah ? await sl.read_csv(unggah) : await sl.read_csv("toko-nusantara-kotor.csv")
if (!unggah) sl.caption("Memakai contoh toko-nusantara-kotor.csv")

sl.subheader("Sebelum")
sl.dataframe(df.missing())

const rapikanTeks = sl.checkbox("Rapikan spasi dan nama kolom", { value: true })
const buangGanda = sl.checkbox("Buang baris ganda", { value: true })
const kosong = sl.radio("Baris dengan nilai kosong", ["Biarkan", "Buang"], { horizontal: true })

const n0 = df.shape[0]
if (rapikanTeks) df = df.strip_strings().clean_names()
if (buangGanda) df = df.drop_duplicates()
if (kosong === "Buang") df = df.dropna()
```

```
sl.subheader("Sesudah")
sl.write(`${n0} → ${df.shape[0]} baris`)
sl.dataframe(df.head(10))
sl.download_button("Unduh CSV bersih", df, { file_name: "bersih.csv" })
```

Pembersih Data ✨

CSV kotor

Seret & lepas berkas di sini, atau klik untuk memilih

Jenis: .csv

Memakai contoh toko-nusantara-kotor.csv

Sebelum

	kolom	kosong	persen
0	kanal	19	0,0482
1	jumlah	16	0,0406
2	id_transaksi	0	0
3	tanggal	0	0
4	kota	0	0
5	kategori	0	0
6	produk	0	0
7	harga	0	0

8 baris × 3 kolom

☑ Rapikan spasi dan nama kolom

☑ Buang baris ganda

Baris dengan nilai kosong

◉ Biarkan ○ Buang

Sesudah

394 → 380 baris

	id_transaksi	tanggal	kota	kanal	kategori	produk	jumlah	harga
0	TN001510	2024-02-14	Makassar	Online	Kerajinan	gelang manik papua	3	Rp 42.000
1	TN001785	2024-04-13	Jayapura	Toko	Kebutuhan Rumah	Minyak Kayu Putih 60ml	1	Rp 40.000
2	TN002217	2024-07-29	Jayapura	Toko	Makanan	Sagu Lempeng 1kg	1	31500
3	TN002245	2024-08-04	Surabaya	Toko	Minuman	Kopi Arabika Wamena 250g	1	89000
4	TN001425	2024-01-21	Makassar	Toko	Makanan	Sambal Roa 200g	5	47000
5	TN002331	2024-08-26	Makassar	Toko	Kerajinan	Gelang Manik Papua	1	42000
6	TN002173	2024-07-17	Surabaya	Toko	Pakaian	Topi Rotan	2	89000
7	TN001909	2024-05-14	Surabaya	Online	Pakaian	Kain Pantai Bali	1	100000
8	TN001999	05/06/2024	Surabaya	Online	Pakaian	Kaos Motif Cenderawasih	2	131000
9	TN001887	08/05/2024	Makassar	Online	Minuman	Kopi Arabika Wamena 250g	1	89000

10 baris × 8 kolom

Unduh CSV bersih

Buka di Playground ↗

> **API Slak baru**
>
> - `sl.radio(label, pilihan, { horizontal })` menampilkan pilihan tunggal.
> - `sl.download_button(label, data, { file_name })` mengunduh data. Akhiran nama berkas menentukan formatnya: `.csv`, `.xlsx`, `.json`, atau `.md`.

← 13. Membersihkan Data dengan JavaScript Murni

15. Meringkas Data →

Bab 15 · Meringkas Data

> **Tujuan belajar**
>
> - Mengelompokkan data dengan `groupby` dan meringkasnya dengan `agg`.
> - Menghitung frekuensi dengan `value_counts` dan tabel silang dengan `sl.crosstab`.
> - Membuat tabel pivot.
> - Menghitung porsi dan kontribusi terhadap total.
> - Menyusun peringkat dengan `nlargest` dan `nsmallest`.
> - Menampilkan angka utama dengan perubahan naik atau turun.

Data mentah berisi ribuan baris. Manajer tidak membaca ribuan baris; ia ingin tahu **cabang mana yang paling laku**, **produk apa yang menyumbang omzet terbesar**, dan **apakah bulan ini lebih baik dari bulan lalu**. Meringkas adalah jembatan dari data ke jawaban.

15.1 Group by: dari Bab 7 ke satu perintah

Di Latihan 7.3 kamu menghitung omzet per kota dengan perulangan dan objek. Itulah *group by*: membagi baris ke dalam kelompok, lalu meringkas setiap kelompok.

```
const rows = (await sl.read_csv("toko-nusantara.csv")).to_records()

// cara Bab 7: JavaScript murni
const perKotaManual = {}
for (const r of rows) perKotaManual[r.kota] = (perKotaManual[r.kota] || 0) + r.total
sl.json(perKotaManual)
```


```
{
  "Ambon": 33681000,
  "Surabaya": 44400600,
  "Jayapura": 41290150,
  "Manado": 30546500,
  "Makassar": 35906200,
  "Manokwari": 24455250,
  "Kupang": 22188500,
  "Denpasar": 33258200,
  "Jakarta": 39196450,
  "Sorong": 25393100
}
```


Buka di Playground ↗

Dengan Slak Data:

```
const df = await sl.read_csv("toko-nusantara.csv")
const perKota = df.groupby("kota").agg({
  omzet: ["total", "sum"],
  transaksi: ["id_transaksi", "nunique"],
  rata_rata: ["total", "mean"]
}).sort_values("omzet", { ascending: false })

sl.dataframe(perKota, { format: { omzet: "rupiah", rata_rata: "rupiah" } })
```

	kota	omzet	transaksi	rata_rata
0	Surabaya	Rp 44.400.600	387	Rp 114.730
1	Jayapura	Rp 41.290.150	340	Rp 121.442
2	Jakarta	Rp 39.196.450	384	Rp 102.074
3	Makassar	Rp 35.906.200	350	Rp 102.589
4	Ambon	Rp 33.681.000	301	Rp 111.897
5	Denpasar	Rp 33.258.200	273	Rp 121.825
6	Manado	Rp 30.546.500	245	Rp 124.680
7	Sorong	Rp 25.393.100	246	Rp 103.224
8	Manokwari	Rp 24.455.250	201	Rp 121.668
9	Kupang	Rp 22.188.500	202	Rp 109.844

10 baris × 4 kolom

Buka di Playground ↗

Cara membaca `agg` : setiap kolom hasil ditulis sebagai `nama_baru: [kolom_asal, fungsi]` . Jadi `omzet: ["total", "sum"]` artinya "kolom baru `omzet` berisi jumlah kolom `total` per kelompok".

Fungsi agregasi	Arti
`sum` , `mean` , `median`	Jumlah, rata-rata, nilai tengah
`min` , `max`	Terkecil, terbesar
`count`	Banyak nilai yang terisi
`size`	Banyak baris
`nunique`	Banyak nilai unik, misalnya pelanggan berbeda
`std` , `var`	Simpangan baku, varians
`p10` … `p99`	Persentil, misalnya `p90`

Mengelompokkan berdasarkan dua kolom sekaligus:

```
const df = await sl.read_csv("toko-nusantara.csv")
const kotaKanal = df.groupby(["kota", "kanal"]).agg({ omzet: ["total", "sum"] })
sl.dataframe(kotaKanal.head(10))
```

	kota	kanal	omzet
0	Ambon	Online	12.749.250
1	Ambon	Toko	20.931.750
2	Denpasar	Online	13.176.900
3	Denpasar	Toko	20.081.300
4	Jakarta	Online	13.318.400
5	Jakarta	Toko	25.878.050
6	Jayapura	Online	13.390.150
7	Jayapura	Toko	27.900.000
8	Kupang	Online	5.185.800
9	Kupang	Toko	17.002.700

10 baris × 3 kolom

Buka di Playground ↗

> **API Slak baru**
>
> - `df.groupby(kolom)` mengelompokkan; lanjutkan dengan `.agg({ nama: [kolom, fungsi] })`. Pintasan: `.sum()`, `.mean()`, `.size()`.
> - `sl.dataframe(df, { format })` merapikan angka per kolom: `"rupiah"`, `"percent"`, `"compact"`, `"decimal:2"`.

15.2 Frekuensi

Sering kali yang ditanyakan bukan "berapa jumlahnya" tetapi "berapa kali muncul".

```
const df = await sl.read_csv("toko-nusantara.csv")
sl.dataframe(df.value_counts("kategori"))
sl.dataframe(df.value_counts("kanal", { normalize: true }))
```

	kategori	jumlah
0	Makanan	1.044
1	Minuman	756
2	Kebutuhan Rumah	567
3	Kerajinan	314
4	Pakaian	248

5 baris × 2 kolom

	kanal	jumlah	proporsi
0	Toko	1.900	0,6487
1	Online	1.029	0,3513

2 baris × 3 kolom

Buka di Playground ↗

`normalize: true` mengubah hitungan menjadi proporsi.

Tabel silang menghitung frekuensi dua kolom sekaligus:

```
const df = await sl.read_csv("toko-nusantara.csv")
sl.dataframe(sl.crosstab(df.kategori, df.kanal))
sl.dataframe(sl.crosstab(df.kategori, df.kanal, { normalize: "index" }), { format: { Online:
"percent", Toko: "percent" } })
```

	kategori	Online	Toko
0	Kebutuhan Rumah	188	379
1	Kerajinan	114	200
2	Makanan	369	675
3	Minuman	277	479
4	Pakaian	81	167

5 baris × 3 kolom

	kategori	Online	Toko
0	Kebutuhan Rumah	33,2%	66,8%
1	Kerajinan	36,3%	63,7%
2	Makanan	35,3%	64,7%
3	Minuman	36,6%	63,4%
4	Pakaian	32,7%	67,3%

5 baris × 3 kolom

Buka di Playground ↗

Tabel kedua menjawab pertanyaan menarik: kategori mana yang paling banyak dibeli secara online?

15.3 Tabel pivot

Tabel pivot adalah ringkasan dua arah, persis seperti PivotTable di Excel: satu kolom menjadi baris, satu kolom menjadi kolom, dan sel berisi ringkasan.

```
const df = await sl.read_csv("toko-nusantara.csv")
const pivot = df
  .assign({ tahun: () => df.tanggal.dt.year })
  .pivot_table({ index: "kategori", columns: "tahun", values: "total", aggfunc: "sum",
margins: true })
sl.dataframe(pivot, { format: "compact" })
```

	kategori	2023	2024	Total
0	Kebutuhan Rumah	14.088.200	18.081.100	32.169.300
1	Kerajinan	36.904.000	51.749.250	88.653.250
2	Makanan	35.309.300	47.528.650	82.837.950
3	Minuman	36.402.300	47.499.350	83.901.650
4	Pakaian	22.426.000	20.327.800	42.753.800
5	Total	145.129.800	185.186.150	330.315.950

6 baris × 4 kolom

Buka di Playground ↗

`margins: true` menambah baris dan kolom total.

> **API Slak baru**
>
> - `df.value_counts(kolom, { normalize })` menghitung frekuensi.
> - `sl.crosstab(series1, series2, { normalize })` membuat tabel silang.

- `df.pivot_table({ index, columns, values, aggfunc, margins })` membuat tabel pivot.

15.4 Porsi dan kontribusi

Angka mutlak tanpa pembanding sulit dinilai. "Omzet Ambon 20 juta" belum berarti apa-apa sampai kamu tahu itu 6% dari total.

```
const df = await sl.read_csv("toko-nusantara.csv")
const perKategori = df.groupby("kategori").agg({ omzet: ["total", "sum"] })
const total = perKategori.omzet.sum()

const urut = perKategori
  .assign({ porsi: r => r.omzet / total })
  .sort_values("porsi", { ascending: false })
const kontribusi = urut.assign({ kumulatif: () => urut.porsi.cumsum() })

sl.dataframe(kontribusi, { format: { omzet: "rupiah", porsi: "percent", kumulatif: "percent" }
})
```

	kategori	omzet	porsi	kumulatif
0	Kerajinan	Rp 88.653.250	26,8%	26,8%
1	Minuman	Rp 83.901.650	25,4%	52,2%
2	Makanan	Rp 82.837.950	25,1%	77,3%
3	Pakaian	Rp 42.753.800	12,9%	90,3%
4	Kebutuhan Rumah	Rp 32.169.300	9,7%	100,0%

5 baris × 4 kolom

Buka di Playground ↗

Kolom `kumulatif` menjumlahkan porsi dari atas ke bawah. Dua kategori teratas menyumbang lebih dari separuh omzet. Pola "sebagian kecil menyumbang sebagian besar" ini sering ditemukan, dan Bab 22 membahasnya sebagai analisis Pareto.

Perhatikan bahwa `kumulatif` dihitung dari `urut`, DataFrame yang sudah diurutkan. Urutan penting: jumlah kumulatif selalu dihitung dari atas ke bawah.

15.5 Peringkat

```
const df = await sl.read_csv("toko-nusantara.csv")
const perProduk = df.groupby("produk").agg({ terjual: ["jumlah", "sum"], omzet: ["total",
"sum"] })

const [kiri, kanan] = sl.columns(2)
kiri.subheader("5 terlaris")
kiri.dataframe(perProduk.nlargest(5, "terjual"))
kanan.subheader("5 paling sepi")
kanan.dataframe(perProduk.nsmallest(5, "terjual"))
```

5 terlaris

	produk	terjual	om
0	Keripik Keladi 200g	467	10.469.1
1	Kopi Arabika Wamena 250g	423	36.749.5
2	Sabun Sereh Wangi	416	7.696.2
3	Sambal Roa 200g	372	17.077.7
4	Kopi Robusta Toraja 250g	365	26.970.2

5 baris × 3 kolom

5 paling sepi

	produk	terjual	or
0	Topi Rotan	82	7.098
1	Ukiran Asmat Mini	88	25.025
2	Tenun Ikat Sumba Selendang	95	33.841
3	Noken Anyaman Kecil	132	20.261
4	Kacang Kenari 250g	137	9.157

5 baris × 3 kolom

Buka di Playground ↗

> **API Slak baru**
>
> - `df.nlargest(n, kolom)` dan `df.nsmallest(n, kolom)` mengambil `n` baris dengan nilai terbesar atau terkecil.
> - `sl.subheader(teks)` menampilkan judul kecil.

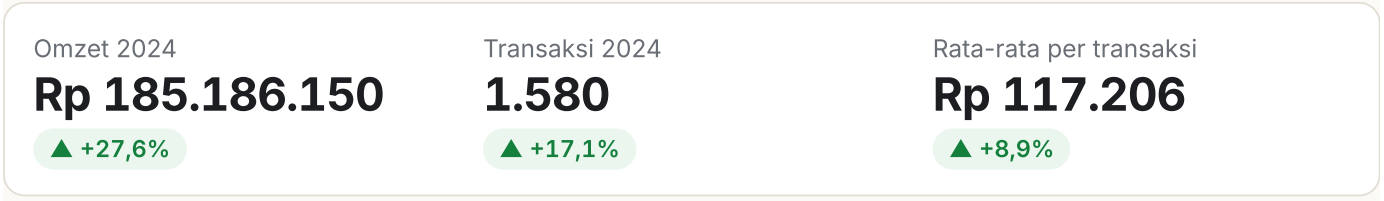
15.6 Angka utama dan perubahannya

Dasbor yang baik dibuka dengan beberapa angka utama, masing-masing dibandingkan dengan periode sebelumnya.

```
const df = await sl.read_csv("toko-nusantara.csv")
const thn = df.assign({ tahun: () => df.tanggal.dt.year })
const t23 = thn.query("tahun == 2023"), t24 = thn.query("tahun == 2024")

const persen = new Intl.NumberFormat("id-ID", { style: "percent", maximumFractionDigits: 1,
signDisplay: "always" })
const ubah = (a, b) => persen.format(b / a - 1)

const [a, b, c] = sl.columns(3)
a.metric("Omzet 2024", t24.total.sum(), ubah(t23.total.sum(), t24.total.sum()), { format: "rp"
})
b.metric("Transaksi 2024", t24.shape[0], ubah(t23.shape[0], t24.shape[0]))
c.metric("Rata-rata per transaksi", Math.round(t24.total.mean()), ubah(t23.total.mean(),
t24.total.mean()), { format: "rp" })
```



Buka di Playground ↗

Argumen ketiga `sl.metric` adalah perubahannya. Slak mewarnainya hijau bila naik dan merah bila turun. Untuk angka yang lebih baik bila turun, misalnya biaya atau keluhan, tambahkan `delta_color: "inverse"`.

> **Coba di playground** Ganti `"kategori"` di contoh 15.4 dengan `"kota"`. Berapa kota yang menyumbang separuh omzet?

Kesalahan umum

Gejala	Penyebab	Perbaikan
`Kolom "omzet" tidak ada` setelah `groupby`	Lupa memberi nama kolom hasil di `agg`	Tulis `omzet: ["total", "sum"]`
Banyak pelanggan terlalu besar	Memakai `count`	Pakai `nunique` untuk menghitung yang berbeda
Porsi berjumlah lebih dari 100%	Pembagi diambil dari data yang sudah disaring	Hitung total dari data yang sama
Tahun tampil `2.023`	Angka tahun diformat ribuan	Ubah ke teks, misalnya `String(tahun)`

Latihan

Latihan 15.1 (mudah). Hitung `nPelanggan`, banyaknya pelanggan berbeda di seluruh data.

```
const df = await sl.read_csv("toko-nusantara.csv")
const nPelanggan = 0   // ganti

sl.metric("Pelanggan", nPelanggan)
await sl.testing.test("banyak pelanggan benar", () => sl.testing.expect(nPelanggan).toBe(new
Set(df.pelanggan_id.to_list()).size))
```

Pelanggan

0

✕ **banyak pelanggan benar** — Diharapkan: sama persis dengan 631 — nilai sebenarnya: 0

Buka di Playground ↗

Latihan 15.2 (sedang). Buat `perCabang` berisi kolom `cabang`, `omzet` (jumlah total), dan `pelanggan` (banyak pelanggan berbeda), diurutkan dari omzet terbesar.

```
const df = await sl.read_csv("toko-nusantara.csv")
const perCabang = sl.DataFrame([])   // ganti: groupby + agg + sort_values

sl.dataframe(perCabang)
await sl.testing.test("10 cabang", () => sl.testing.expect(perCabang.shape[0]).toBe(10))
await sl.testing.test("kolom benar", () =>
sl.testing.expect(perCabang.columns).toEqual(["cabang", "omzet", "pelanggan"]))
await sl.testing.test("teratas Surabaya", () => sl.testing.expect(perCabang.to_records()
[0].cabang).toBe("Nusantara Surabaya"))
```

0 baris × 0 kolom

✕ **10 cabang** — Diharapkan: sama persis dengan 10 — nilai sebenarnya: 0

✕ **kolom benar** — Diharapkan: setara dengan ["cabang","omzet","pelanggan"] — nilai sebenarnya: []

✕ **teratas Surabaya** — Cannot read properties of undefined (reading 'cabang')

Buka di Playground ↗

Latihan 15.3 (tantangan). Berapa persen omzet kategori Minuman yang berasal dari kanal Online? Simpan sebagai angka 0 sampai 1 di `porsiOnline`.

```
const df = await sl.read_csv("toko-nusantara.csv")
const porsiOnline = 0   // ganti

sl.metric("Porsi online kategori Minuman", (porsiOnline * 100).toFixed(1) + "%")
const m = df.filter(r => r.kategori === "Minuman")
await sl.testing.test("porsi benar", () =>
sl.testing.expect(porsiOnline).toBeCloseTo(m.filter(r => r.kanal === "Online").total.sum() /
m.total.sum(), 6))
```

Porsi online kategori Minuman

0.0%

✕ **porsi benar** — Diharapkan: mendekati 0.35402343100523054 (6 desimal) — nilai sebenarnya: 0

Buka di Playground ↗

Mini-proyek: ringkasan penjualan per cabang dan kategori

```
sl.title("Ringkasan Penjualan :bar_chart:")

const df = await sl.read_csv("toko-nusantara.csv")
const tahun = sl.radio("Tahun", ["2023", "2024", "Semua"], { horizontal: true, index: 1 })
const data = tahun === "Semua" ? df : df.filter(r => r.tanggal.getFullYear() ===
Number(tahun))

const [a, b, c] = sl.columns(3)
a.metric("Omzet", data.total.sum(), null, { format: "rp" })
b.metric("Transaksi", data.shape[0])
c.metric("Pelanggan", data.pelanggan_id.nunique())

const dimensi = sl.selectbox("Ringkas menurut", ["cabang", "kategori", "kanal", "produk"])
const ringkas = data.groupby(dimensi).agg({
  omzet: ["total", "sum"],
  transaksi: ["id_transaksi", "nunique"],
  pelanggan: ["pelanggan_id", "nunique"]
}).sort_values("omzet", { ascending: false })

const total = ringkas.omzet.sum()
sl.dataframe(ringkas.assign({ porsi: r => r.omzet / total }),
  { format: { omzet: "rupiah", porsi: "percent" }, gradient: ["omzet"] })

const juara = ringkas.to_records()[0]
sl.success(`Teratas: **${juara[dimensi]}**, ${(juara.omzet / total * 100).toFixed(1)}% dari
omzet.`)
```

Ringkasan Penjualan 📊

Tahun

○ 2023 ◉ 2024 ○ Semua

Omzet
Rp 185.186.150

Transaksi
1.580

Pelanggan
559

Ringkas menurut

cabang

	cabang	omzet	transaksi	pelanggan	porsi
0	Nusantara Jayapura	Rp 24.341.550	188	88	13,1%
1	Nusantara Ambon	Rp 22.277.100	175	71	12,0%
2	Nusantara Surabaya	Rp 21.563.000	192	88	11,6%
3	Nusantara Denpasar	Rp 20.284.800	153	72	11,0%
4	Nusantara Jakarta	Rp 19.908.350	207	111	10,8%
5	Nusantara Makassar	Rp 18.659.000	181	88	10,1%
6	Nusantara Manokwari	Rp 15.522.250	110	57	8,4%
7	Nusantara Manado	Rp 15.208.500	135	72	8,2%
8	Nusantara Sorong	Rp 14.206.900	129	64	7,7%
9	Nusantara Kupang	Rp 13.214.700	110	44	7,1%

10 baris × 5 kolom

✓ Teratas: **Nusantara Jayapura**, 13.1% dari omzet.

Buka di Playground ↗

Opsi `gradient` mewarnai sel dari muda ke tua sesuai nilainya, sehingga yang terbesar langsung terlihat.

Bab 16 · Visualisasi Dasar

> **Tujuan belajar**
>
> - Memilih grafik yang tepat untuk pertanyaan yang tepat.
> - Menyiapkan data agar siap digambar.
> - Memberi judul, label, teks pengganti, dan warna yang jujur.
> - Mengenali grafik yang menyesatkan.

16.1 Mulai dari pertanyaan, bukan dari grafik

Grafik yang baik menjawab satu pertanyaan dengan sekali lihat. Sebelum memilih grafik, tulis dulu pertanyaannya.

Pertanyaan	Grafik	Perintah
Bagaimana perubahannya dari waktu ke waktu?	Garis	`sl.line_chart`
Kategori mana yang paling besar?	Batang, diurutkan	`sl.bar_chart`
Bagaimana total menumpuk dari beberapa bagian sepanjang waktu?	Area	`sl.area_chart`
Apakah X berhubungan dengan Y?	Sebar	`sl.scatter_chart`
Bagaimana sebaran nilainya?	Histogram	`sl.histogram`
Berapa porsi setiap bagian (sedikit bagian)?	Donat	`sl.pie_chart`

Semua grafik Slak digambar di browser tanpa internet, bisa diarahkan kursor untuk melihat nilainya, dan bisa diunduh sebagai PNG atau SVG lewat tombol di pojok kanan atas.

16.2 Menyiapkan data

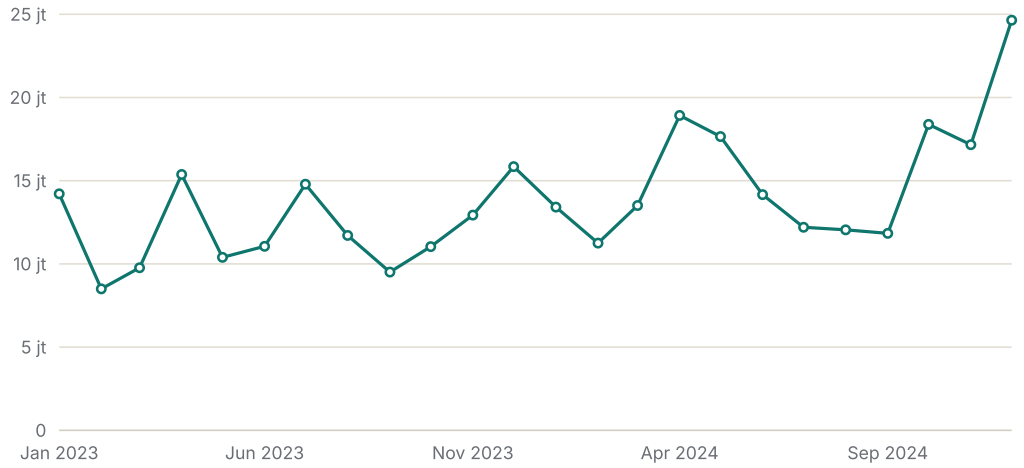
Grafik membutuhkan data yang **sudah diringkas**. Menggambar 2.929 transaksi mentah sebagai garis hanya menghasilkan benang kusut. Kelompokkan dulu:

```
const df = await sl.read_csv("toko-nusantara.csv")
const bulanan = df.resample("tanggal", "M").agg({ omzet: ["total", "sum"] })
sl.dataframe(bulanan.head(4))
sl.line_chart(bulanan, { x: "tanggal", y: "omzet", title: "Omzet bulanan Toko
Nusantara" })
```

	tanggal	omzet
0	1 Jan 2023	14.215.000
1	1 Feb 2023	8.497.000
2	1 Mar 2023	9.765.000
3	1 Apr 2023	15.370.800

4 baris × 2 kolom

Omzet bulanan Toko Nusantara



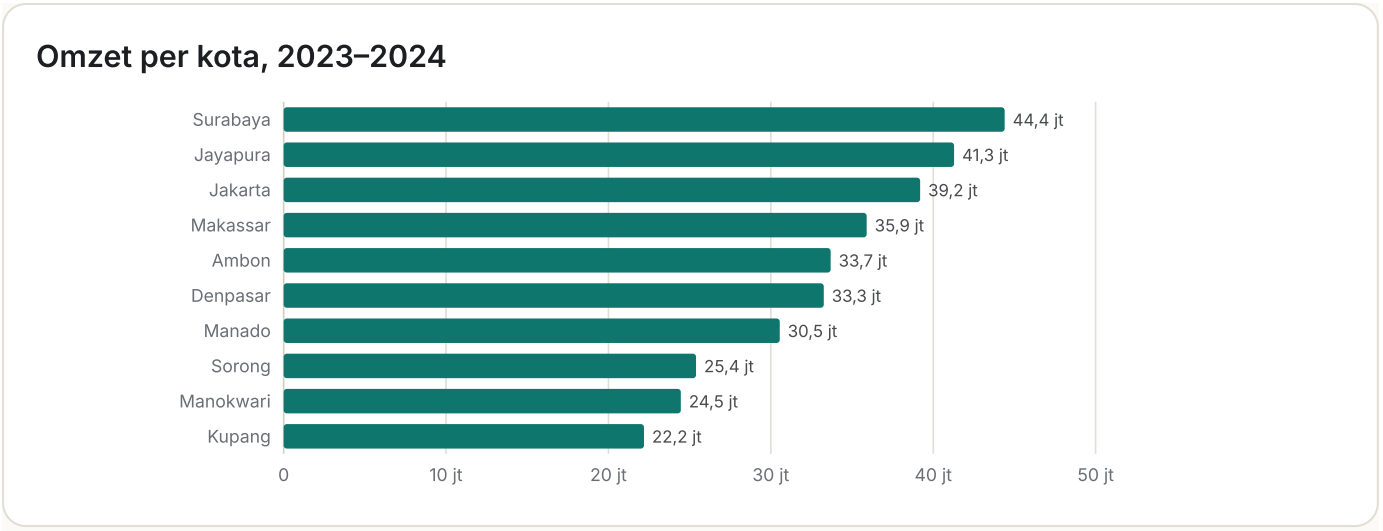
Buka di Playground ↗

`resample("tanggal", "M")` mengelompokkan per bulan. Bab 20 membahasnya lebih jauh. Karena kolom `x` berisi tanggal, sumbu mendatar otomatis menjadi sumbu waktu.

Garis ini sudah bercerita: omzet naik dari 2023 ke 2024, dengan lonjakan menjelang Lebaran dan akhir tahun.

16.3 Batang: membandingkan kategori

```
const df = await sl.read_csv("toko-nusantara.csv")
const perKota = df.groupby("kota").agg({ omzet: ["total", "sum"] })
sl.bar_chart(perKota, {
  x: "kota", y: "omzet",
  horizontal: true, sort: true, labels: true, format: "compact",
  title: "Omzet per kota, 2023–2024"
})
```



Buka di Playground ↗

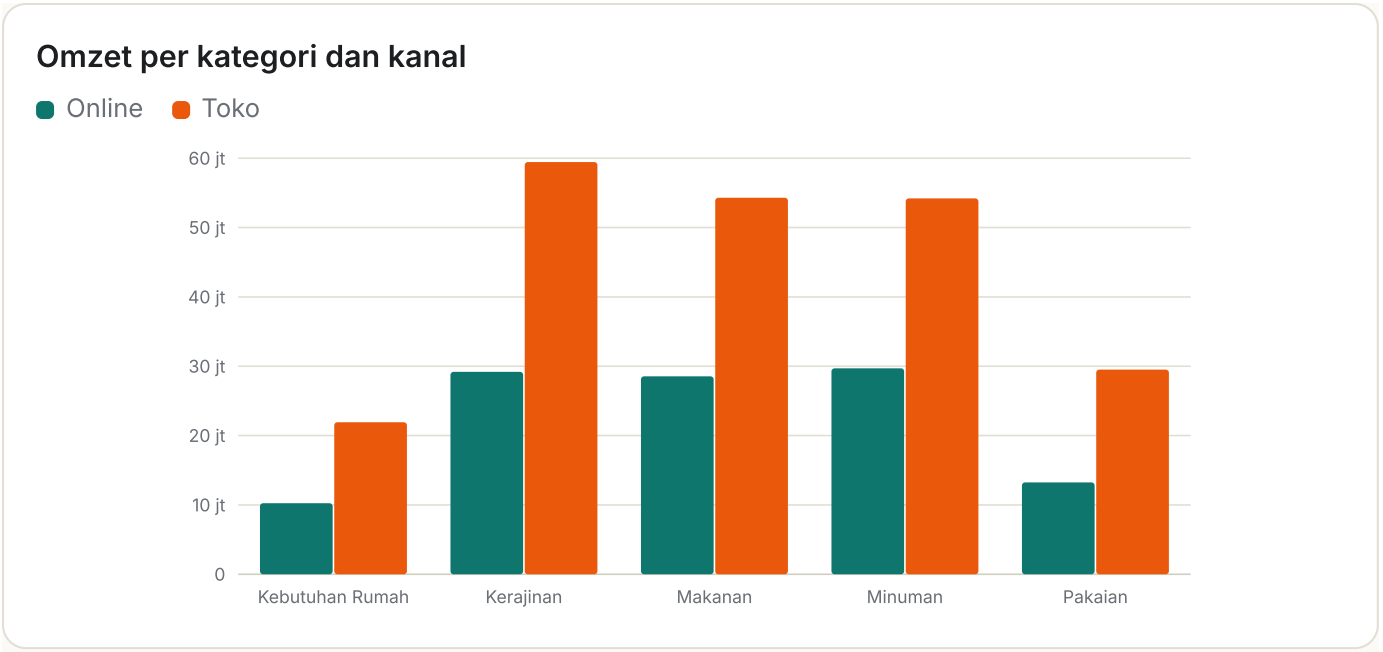
Tiga kebiasaan untuk grafik batang:

1. **Urutkan** (`sort: true`). Pembaca langsung melihat juara dan yang tertinggal.
2. **Mendatar** (`horizontal: true`) bila nama kategorinya panjang, supaya label tidak miring.
3. **Tampilkan nilainya** (`labels: true`) bila batangnya sedikit, supaya pembaca tidak menebak.

Grafik batang selalu dimulai dari nol. Panjang batang adalah nilainya; memotong sumbu membuat selisih kecil tampak besar.

Beberapa seri sekaligus

```
const df = await sl.read_csv("toko-nusantara.csv")
const kk = df.groupby(["kategori", "kanal"]).agg({ omzet: ["total", "sum"] })
sl.bar_chart(kk, { x: "kategori", y: "omzet", color: "kanal", format: "compact",
title: "Omzet per kategori dan kanal" })
```

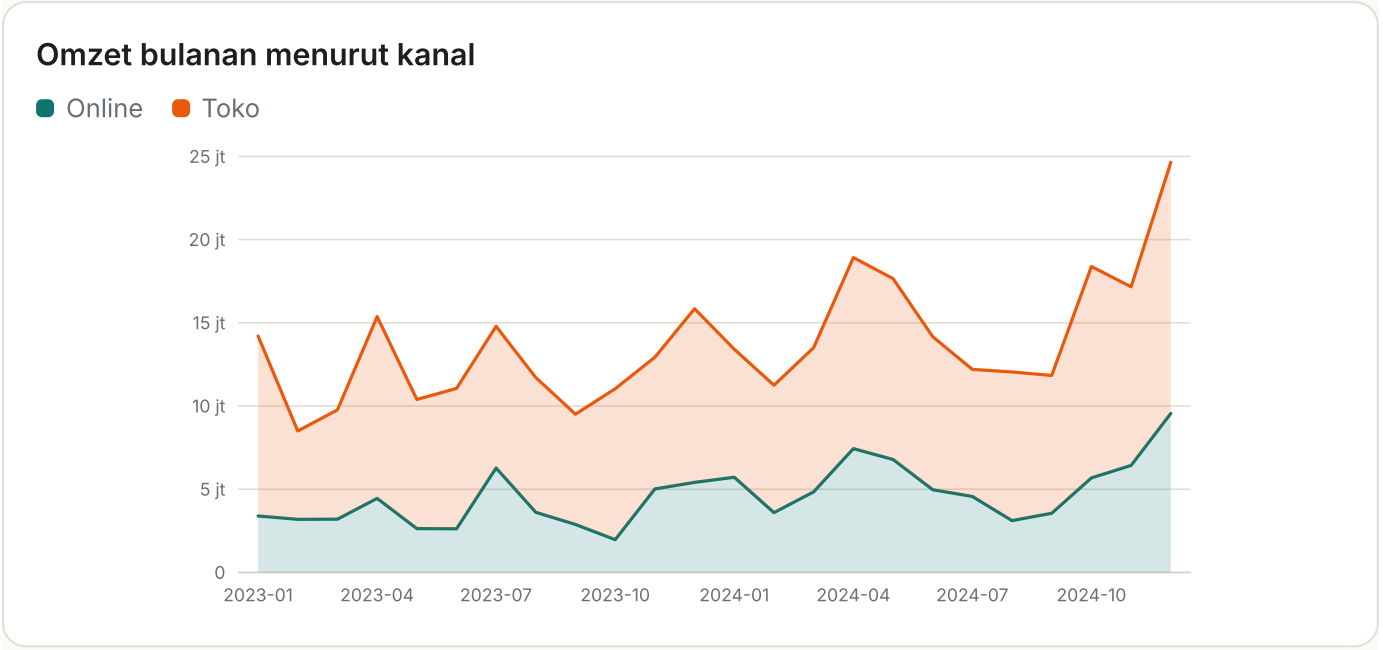


Buka di Playground ↗

`color: "kanal"` memberi satu warna untuk setiap nilai kolom `kanal`. Tambahkan `stack: true` untuk menumpuk batangnya.

16.4 Area: total dan bagiannya

```
const df = await sl.read_csv("toko-nusantara.csv")
const bulanKanal = df.assign({ bulan: () => df.tanggal.dt.strftime("%Y-%m") })
  .groupby(["bulan", "kanal"]).agg({ omzet: ["total", "sum"] })
sl.area_chart(bulanKanal, { x: "bulan", y: "omzet", color: "kanal", stack: true,
format: "compact", title: "Omzet bulanan menurut kanal" })
```

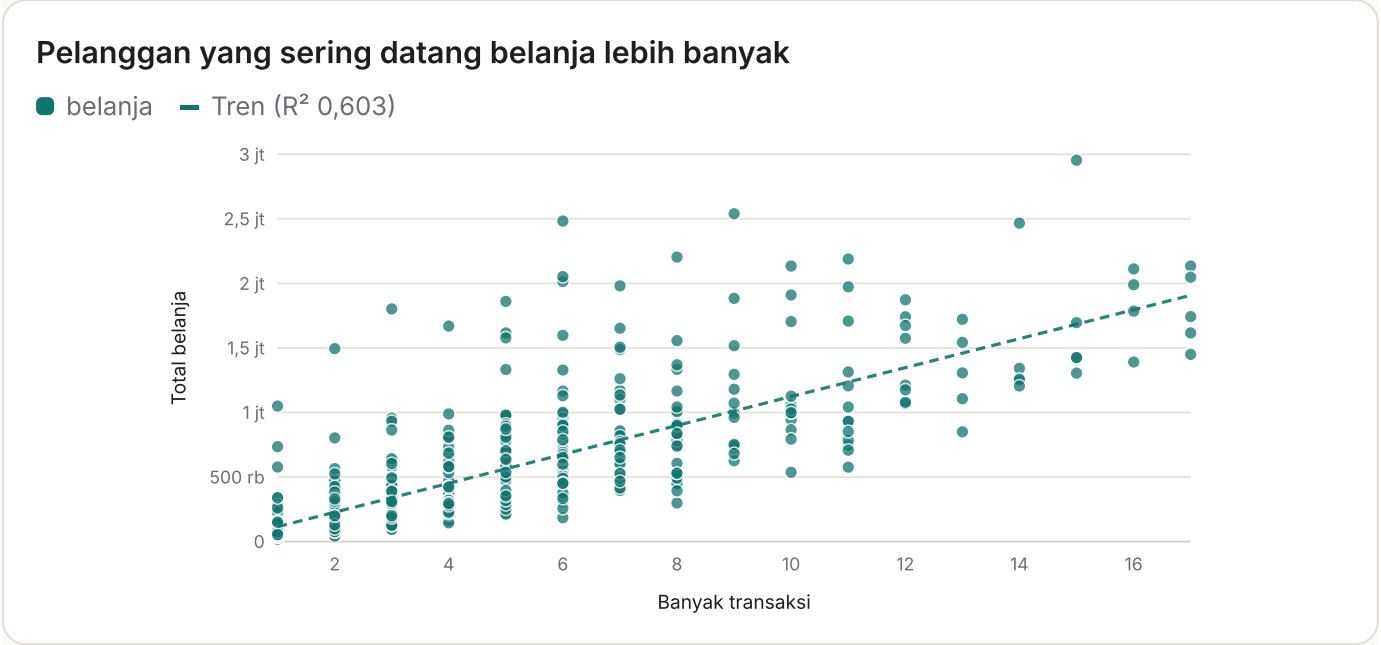


Buka di Playground ↗

Grafik area bertumpuk menunjukkan total (tinggi seluruhnya) sekaligus bagiannya. Terlihat porsi kanal Online semakin tebal di 2024.

16.5 Sebar: mencari hubungan

```
const df = await sl.read_csv("toko-nusantara.csv")
const perPelanggan = df.groupby("pelanggan_id").agg({
  kunjungan: ["id_transaksi", "nunique"],
  belanja: ["total", "sum"]
})
sl.scatter_chart(perPelanggan, {
  x: "kunjungan", y: "belanja", trendline: true, format: "compact",
  x_label: "Banyak transaksi", y_label: "Total belanja", title: "Pelanggan yang
sering datang belanja lebih banyak"
})
```



Buka di Playground ↗

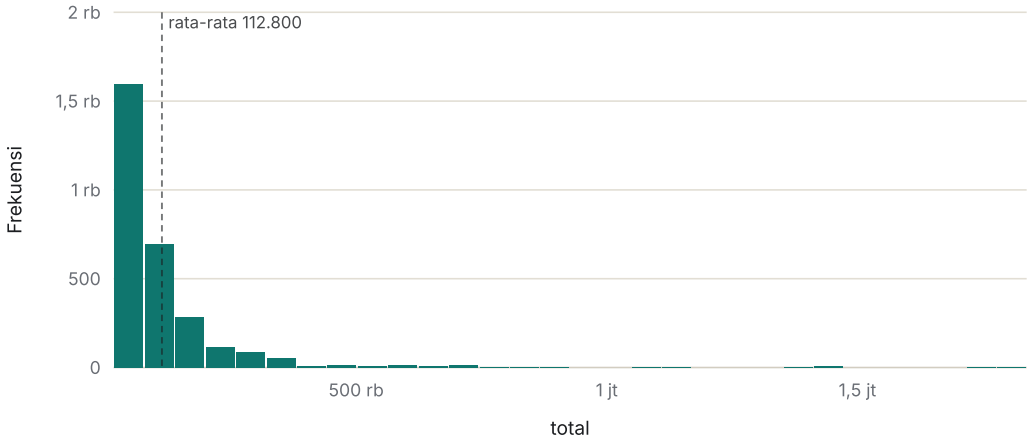
Setiap titik adalah satu pelanggan. `trendline: true` menambahkan garis tren beserta nilai R^2 di legenda, yang mengukur seberapa rapat titik mengikuti garis. Bab 19 dan 22 membahasnya.

Perhatikan judulnya: bukan "Kunjungan vs belanja", melainkan **temuannya**. Judul yang menyatakan temuan membuat pembaca langsung paham maksud grafik.

16.6 Histogram: melihat sebaran

```
const df = await sl.read_csv("toko-nusantara.csv")
sl.histogram(df, { x: "total", bins: 30, title: "Kebanyakan transaksi di bawah 200 ribu" })
sl.write(`Rata-rata ${Math.round(df.total.mean()).toLocaleString("id-ID")}, median ${df.total.median().toLocaleString("id-ID")}`)
```

Kebanyakan transaksi di bawah 200 ribu



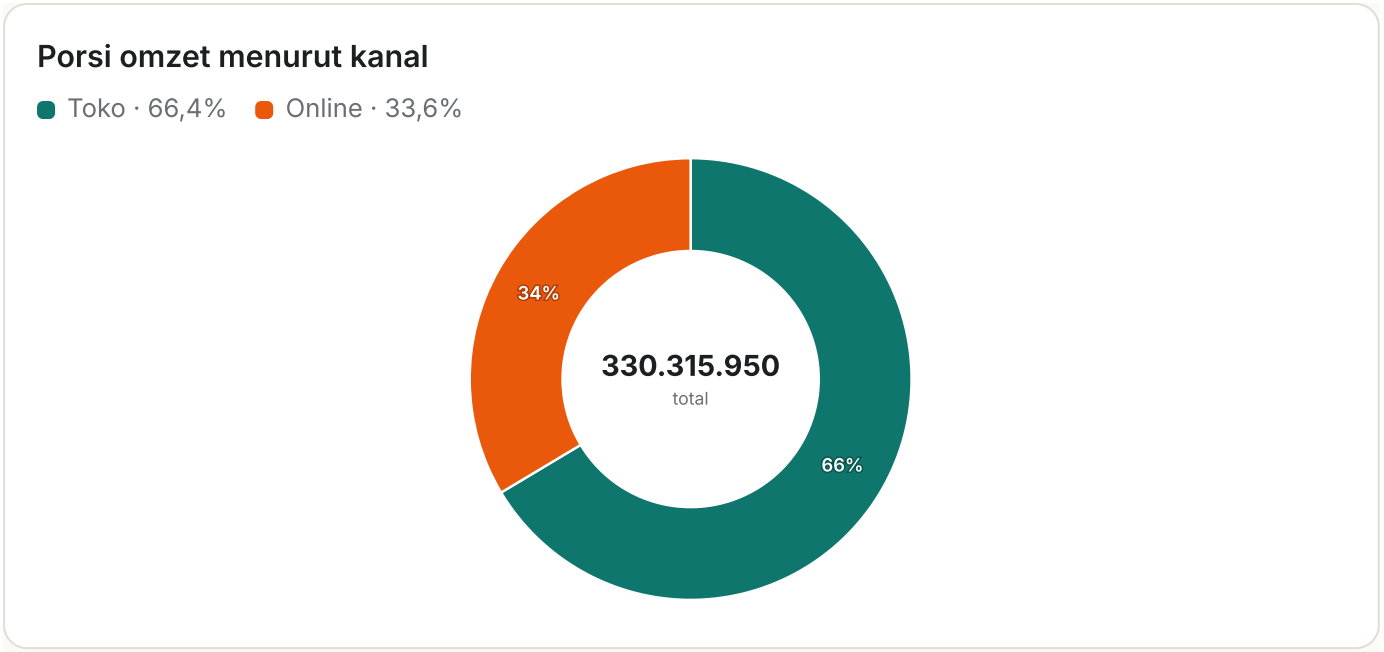
Rata-rata 112.774, median 72.000

Buka di Playground ↗

Histogram membagi rentang nilai ke dalam beberapa "keranjang" (`bins`) dan menghitung isi setiap keranjang. Sebaran transaksi kita **condong ke kanan**: banyak transaksi kecil, sedikit transaksi besar. Itulah sebabnya rata-ratanya lebih besar dari mediannya (ingat Bab 5).

16.7 Donat: porsi

```
const df = await sl.read_csv("toko-nusantara.csv")
const perKanal = df.groupby("kanal").agg({ omzet: ["total", "sum"] })
sl.pie_chart(perKanal, { names: "kanal", values: "omzet", donut: true, title: "Porsi omzet menurut kanal" })
```



Buka di Playground ↗

Pakai donat hanya bila bagiannya **sedikit** (dua sampai lima) dan jumlahnya 100%. Mata manusia sulit membandingkan sudut; untuk lebih dari lima bagian, grafik batang yang diurutkan hampir selalu lebih jelas.

> **API Slak baru**
>
> - `sl.line_chart`, `sl.bar_chart`, `sl.area_chart`, `sl.scatter_chart(data, { x, y, color, title, x_label, y_label, format })`.
> - Opsi khusus: `horizontal`, `sort`, `labels`, `stack` (batang dan area), `trendline` (sebar).
> - `sl.histogram(data, { x, bins })` dan `sl.pie_chart(data, { names, values, donut })`.
> - `format`: `"rupiah"`, `"percent"`, `"compact"` (ditulis singkat: rb, jt, M), `"decimal:2"`.

16.8 Grafik yang jujur dan ramah semua orang

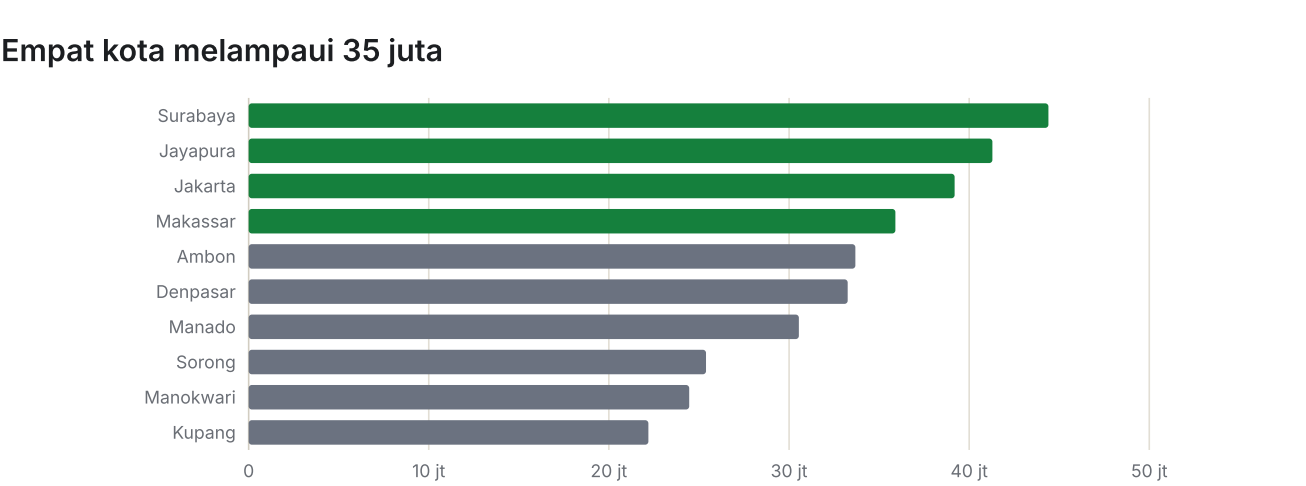
Hindari grafik yang menyesatkan:

Jebakan	Akibat	Perbaikan
Sumbu batang tidak dimulai dari nol	Selisih 2% tampak dua kali lipat	Batang selalu dari nol (bawaan Slak)
Donat dengan sepuluh potong	Tidak ada yang bisa dibandingkan	Batang yang diurutkan
Garis untuk kategori yang tidak berurutan	Tersirat ada "tren" antar-kota	Batang
Warna pelangi tanpa arti	Pembaca mencari makna yang tidak ada	Satu warna, sorot yang penting saja

Jebakan	Akibat	Perbaikan
Membandingkan bulan tidak lengkap	Desember yang baru berjalan seminggu tampak anjlok	Beri catatan, atau buang periode yang belum selesai

Warna yang bermakna. Sorot hanya yang penting, misalnya cabang di atas target:

```
const df = await sl.read_csv("toko-nusantara.csv")
const perCabang = df.groupby("kota").agg({ omzet: ["total", "sum"] })
const target = 35000000
sl.bar_chart(perCabang, {
  x: "kota", y: "omzet", horizontal: true, sort: true, format: "compact",
  color: r => r.omzet >= target ? "var(--slak-green)" : "var(--slak-gray)",
  title: "Empat kota melampaui 35 juta"
})
```

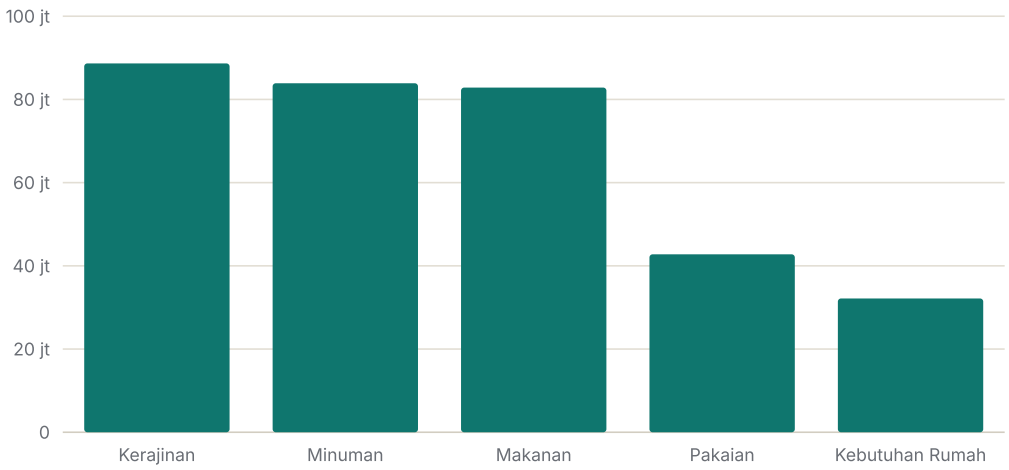


Buka di Playground ↗

Warna `var(--slak-green)`, `var(--slak-red)`, `var(--slak-gray)`, dan `var(--slak-c1)` sampai `var(--slak-c8)` otomatis menyesuaikan tema terang dan gelap.

Teks pengganti. Pembaca layar membacakan grafik untuk pengguna tunanetra. Slak membuat teks penggantinya otomatis, misalnya "Grafik omzet menurut kota, 10 titik". Opsi `alt` memberi teks yang lebih bermakna:

```
const df = await sl.read_csv("toko-nusantara.csv")
const perKategori = df.groupby("kategori").agg({ omzet: ["total", "sum"] })
sl.bar_chart(perKategori, { x: "kategori", y: "omzet", sort: true, alt: "Kerajinan
menyumbang omzet terbesar, diikuti Minuman dan Makanan" })
```



Buka di Playground ↗

> **Coba di playground** Di contoh 16.3, ganti `y: "omzet"` menjadi rata-rata nilai transaksi dengan `rata: ["total", "mean"]` di `agg`. Apakah urutan kotanya berubah?

Kesalahan umum

Gejala	Penyebab	Perbaikan
Grafik garis kusut berisi ribuan titik	Data belum diringkas	`groupby` atau `resample` dulu
Sumbu waktu tidak urut	Kolom tanggal masih berupa teks	`to_datetime`, atau urutkan dulu
`Kolom "omzet" tidak ada`	Nama `y` tidak sesuai nama kolom hasil `agg`	Samakan namanya
Angka sumbu terlalu panjang	Format bawaan	`format: "compact"`

Latihan

Latihan 16.1 (mudah). Siapkan `perKategori` berisi kolom `kategori` dan `omzet`, lalu gambar sebagai batang yang diurutkan.

```
const df = await sl.read_csv("toko-nusantara.csv")
const perKategori = sl.DataFrame([])   // ganti dengan groupby + agg

sl.bar_chart(perKategori, { x: "kategori", y: "omzet", sort: true })
await sl.testing.test("lima kategori", () =>
sl.testing.expect(perKategori.shape[0]).toBe(5))
await sl.testing.test("kolom omzet ada", () =>
sl.testing.expect(perKategori.columns).toContain("omzet"))
```

Tidak ada data numerik untuk digambar.

✕ **lima kategori** — Diharapkan: sama persis dengan 5 — nilai sebenarnya: 0

✕ **kolom omzet ada** — Diharapkan: memuat "omzet" — nilai sebenarnya: []

Buka di Playground ↗

Latihan 16.2 (sedang). Siapkan `perHari` berisi rata-rata omzet harian menurut nama hari (`Senin` sampai `Minggu`). Petunjuk: kelompokkan per tanggal dulu, lalu tambahkan nama hari dengan `.dt.day_name()`, lalu kelompokkan lagi.

```
const df = await sl.read_csv("toko-nusantara.csv")
const perHari = sl.DataFrame([])   // ganti

sl.bar_chart(perHari, { x: "hari", y: "rata_omzet" })
await sl.testing.test("tujuh hari", () =>
sl.testing.expect(perHari.shape[0]).toBe(7))
await sl.testing.test("Sabtu lebih ramai dari Selasa", () => {
  const r = Object.fromEntries(perHari.to_records().map(x => [x.hari,
x.rata_omzet]))
  sl.testing.expect(r.Sabtu).toBeGreaterThan(r.Selasa)
})
```

Tidak ada data numerik untuk digambar.

✕ **tujuh hari** — Diharapkan: sama persis dengan 7 — nilai sebenarnya: 0

✕ **Sabtu lebih ramai dari Selasa** — Diharapkan: lebih besar dari undefined — nilai sebenarnya: undefined

Buka di Playground ↗

Latihan 16.3 (tantangan). Gambar grafik garis omzet bulanan dengan satu garis per tahun, sehingga 2023 dan 2024 bisa dibandingkan bulan demi bulan. Siapkan `perBulanTahun` dengan kolom `bulan` (1–12), `tahun` (teks), dan `omzet`.

```
const df = await sl.read_csv("toko-nusantara.csv")
const perBulanTahun = sl.DataFrame([])   // ganti

sl.line_chart(perBulanTahun, { x: "bulan", y: "omzet", color: "tahun" })
await sl.testing.test("24 baris", () =>
sl.testing.expect(perBulanTahun.shape[0]).toBe(24))
await sl.testing.test("tahun berupa teks", () => sl.testing.expect(typeof
perBulanTahun.to_records()[0].tahun).toBe("string"))
```

Tidak ada data numerik untuk digambar.

✕ **24 baris** — Diharapkan: sama persis dengan 24 — nilai sebenarnya: 0

✕ **tahun berupa teks** — Cannot read properties of undefined (reading 'tahun')

Buka di Playground ↗

Tahun dijadikan teks agar Slak memperlakukannya sebagai kategori, bukan angka.

Mini-proyek: empat grafik utama Toko Nusantara

```
sl.title("Toko Nusantara dalam Empat Grafik :chart_with_upwards_trend:")

const df = await sl.read_csv("toko-nusantara.csv")
const [kiri, kanan] = sl.columns(2)
```

```
const bulanan = df.resample("tanggal", "M").agg({ omzet: ["total", "sum"] })
kiri.line_chart(bulanan, { x: "tanggal", y: "omzet", format: "compact", height:
260,
  title: "Omzet naik, dengan puncak Lebaran dan Desember" })

const perKota = df.groupby("kota").agg({ omzet: ["total", "sum"] })
kanan.bar_chart(perKota, { x: "kota", y: "omzet", horizontal: true, sort: true,
format: "compact", height: 260,
  title: "Surabaya dan Jayapura memimpin" })

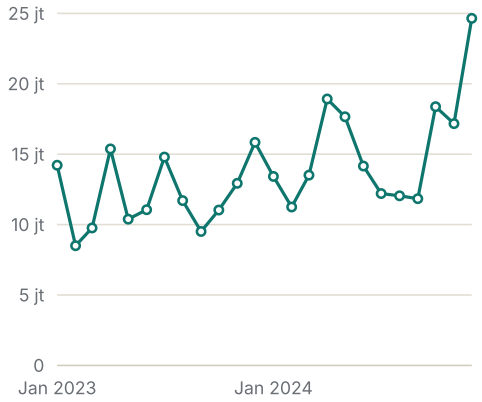
const [kiri2, kanan2] = sl.columns(2)
const perKategori = df.groupby("kategori").agg({ omzet: ["total", "sum"] })
kiri2.bar_chart(perKategori, { x: "kategori", y: "omzet", sort: true, format:
"compact", height: 260,
  title: "Kerajinan menyumbang omzet terbesar" })

kanan2.histogram(df, { x: "total", bins: 25, height: 260, title: "Kebanyakan
transaksi di bawah 200 ribu" })

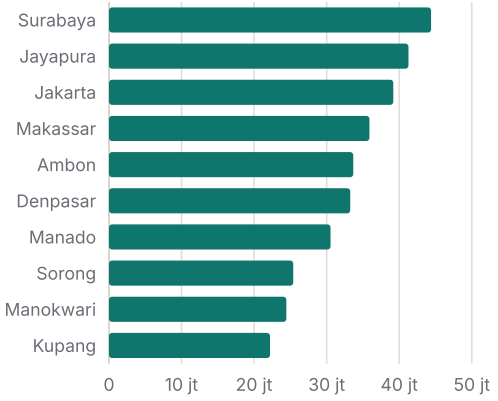
sl.caption("Sumber: data fiktif Toko Nusantara 2023–2024.")
```

Toko Nusantara dalam Empat Grafik 📈

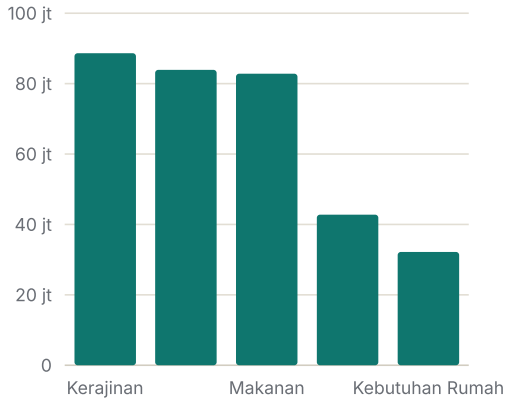
Omzet naik, dengan puncak Lebaran dan Desember



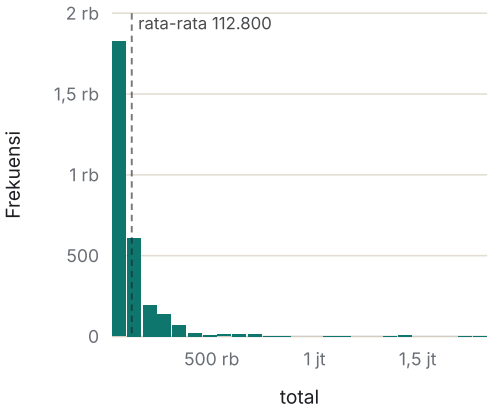
Surabaya dan Jayapura memimpin



Kerajinan menyumbang omzet terbesar



Kebanyakan transaksi di bawah 200 ribu



Sumber: data fiktif Toko Nusantara 2023–2024.

Buka di Playground ↗

Empat grafik, empat temuan, masing-masing ditulis sebagai judul. Itulah laporan visual yang baik: pembaca tidak perlu menebak apa yang harus dilihat.

← 15. Meringkas Data

17. Proyek: Laporan Interaktif Pertama →

Bab 17 · Proyek: Laporan Interaktif Pertama

> **Tujuan belajar**
>
> - Merancang laporan dari pertanyaan bisnis.
> - Menyusun tata letak: sidebar berisi filter, angka utama, grafik, dan tabel.
> - Memakai tab dan templat tata letak `sl.layout`.
> - Membuat laporan yang ramah untuk semua pembaca, termasuk pengguna pembaca layar.
> - Menerbitkan laporan supaya bisa dibuka siapa saja lewat tautan.

Inilah penutup Tahap 2. Semua yang kamu pelajari sejak Bab 1 dipakai di sini untuk membuat satu karya utuh: laporan penjualan interaktif Toko Nusantara yang bisa kamu tunjukkan kepada siapa saja.

17.1 Merancang sebelum menulis kode

Laporan yang baik dimulai dengan **pertanyaan**, bukan dengan grafik. Bayangkan kamu diminta manajer wilayah Toko Nusantara. Tuliskan pertanyaannya:

1. Berapa omzet, transaksi, dan pelanggan pada periode yang dipilih, dan bagaimana dibanding periode sebelumnya?
2. Bagaimana tren omzet bulanannya?
3. Cabang dan kategori mana yang memimpin?
4. Produk apa saja yang paling laku?

Lalu tentukan **filter** yang dibutuhkan manajer: tahun, cabang, dan kanal. Setiap pertanyaan menjadi satu bagian laporan:

Bagian	Isi	Fungsi Slak
Filter	Tahun, cabang, kanal	`sl.sidebar`, `selectbox`, `multiselect`, `radio`
Angka utama	Omzet, transaksi, pelanggan, beserta perubahannya	`sl.metric` dalam `sl.columns`
Tren	Omzet bulanan	`sl.line_chart`
Perbandingan	Cabang dan kategori	`sl.bar_chart` dalam `sl.tabs`
Rincian	Produk terlaris dan unduhan	`sl.dataframe`, `sl.download_button`

17.2 Sidebar dan tab

Filter ditaruh di **sidebar**, panel di sisi kiri, supaya tidak menghalangi isi laporan. Semua fungsi Slak bisa dipakai di sidebar dengan awalan `sl.sidebar.`.

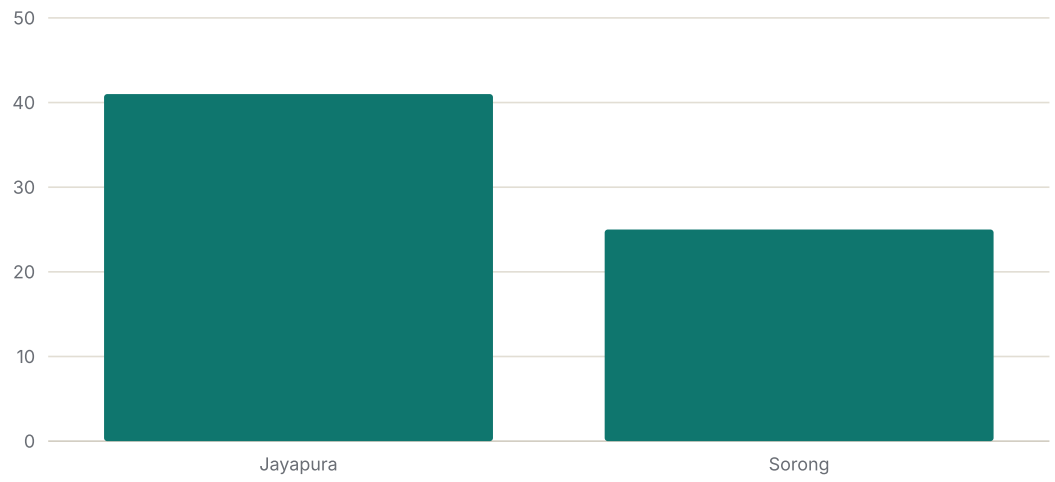
```
const tahun = sl.sidebar.selectbox("Tahun", ["2024", "2023"])
sl.write(`Laporan tahun ${tahun}`)
```

Contoh ini ditandai *tidak dijalankan di halaman buku*, karena halaman buku sendiri memakai sidebar untuk daftar bab. Coba di playground.

Tab memisahkan isi ke beberapa halaman kecil yang bisa dipilih pengguna:

```
const [tGrafik, tTabel] = sl.tabs(["Grafik", "Tabel"])
tGrafik.bar_chart([{ kota: "Jayapura", omzet: 41 }, { kota: "Sorong", omzet: 25 }], { x: "kota", y: "omzet" })
tTabel.table([{ kota: "Jayapura", omzet: 41 }, { kota: "Sorong", omzet: 25 }])
```

Grafik Tabel



Buka di Playground ↗

> **API Slak baru**
>
> - `sl.sidebar.namaFungsi(...)` menaruh apa saja di sidebar.
> - `sl.tabs(["A", "B"])` mengembalikan larik tab; setiap tab punya semua fungsi tampilan.

17.3 Templat tata letak

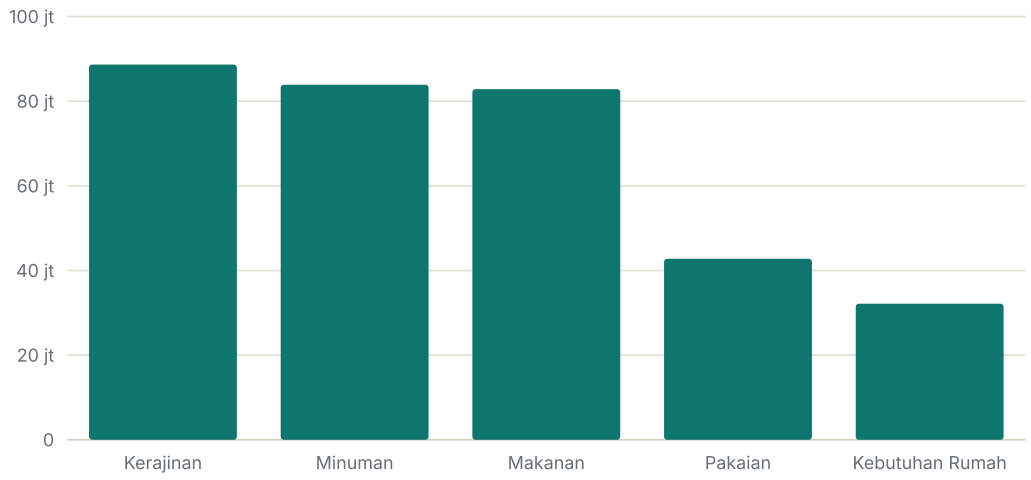
Menyusun kolom dan sidebar sendiri tidak sulit, tetapi Slak menyediakan empat kerangka siap pakai lewat `sl.layout`:

Kerangka	Area yang tersedia
`"kpi-atas"`	`kpi`, `grafik`, `tabel`
`"sidebar-filter"`	`filter` (di sidebar), `kpi`, `grafik`, `tabel`
`"grid-2x2"`	`kpi`, `kiri_atas`, `kanan_atas`, `kiri_bawah`, `kanan_bawah`
`"laporan"`	`judul`, `ringkasan`, `isi`, `catatan`, satu kolom yang nyaman dicetak

```
const df = await sl.read_csv("toko-nusantara.csv")
const area = sl.layout("kpi-atas", { title: "Ringkasan Toko Nusantara" })
area.kpi.metric("Omzet", df.total.sum(), null, { format: "rp" })
area.kpi.metric("Transaksi", df.shape[0])
area.kpi.metric("Pelanggan", df.pelanggan_id.nunique())
area.grafik.bar_chart(df.groupby("kategori").agg({ omzet: ["total", "sum"] }), { x: "kategori", y: "omzet", sort: true, format:
"compact" })
area.tabel.dataframe(df.head(5))
```

Ringkasan Toko Nusantara

Omzet	Transaksi	Pelanggan
Rp 330.315.950	2.929	631



	id_transaksi	tanggal	pelanggan_id	cabang	kota	provinsi	kanal	kategori	produk
0	TN000001	1 Jan 2023	P0622	Nusantara Ambon	Ambon	Maluku	Toko	Pakaian	Topi Rotan
1	TN000002	1 Jan 2023	P0039	Nusantara Surabaya	Surabaya	Jawa Timur	Online	Minuman	Kopi Arabika Wamena 2
2	TN000003	1 Jan 2023	P0399	Nusantara Surabaya	Surabaya	Jawa Timur	Online	Kerajinan	Tenun Ikat Sumba Seler
3	TN000004	1 Jan 2023	P0608	Nusantara Jayapura	Jayapura	Papua	Toko	Kerajinan	Tenun Ikat Sumba Seler
4	TN000005	2 Jan 2023	P0187	Nusantara Ambon	Ambon	Maluku	Online	Makanan	Sambal Roa 200g

5 baris × 13 kolom

Buka di Playground ↗

Metrik yang ditulis di area `kpi` otomatis tampil sebagai kartu yang sejajar. Di layar sempit seperti ponsel, kerangka ini menyesuaikan diri menjadi satu kolom.

> **API Slak baru**
>
> - `sl.layout(nama, { title })` membuat kerangka tata letak dan mengembalikan objek berisi area-areanya.

17.4 Laporan untuk semua orang

Laporanmu mungkin dibaca orang yang memakai pembaca layar, hanya memakai papan ketik, atau membuka di ponsel dengan layar kecil. Slak 1.9 sudah mengurus banyak hal secara otomatis:

- Setiap widget punya nama untuk pembaca layar, diambil dari labelnya.
- Semua kontrol bisa dicapai dengan tombol **Tab** dan punya garis fokus yang terlihat.
- Grafik punya teks pengganti otomatis.
- Warna teks dan menu memenuhi standar kontras WCAG AA, di tema terang maupun gelap.

Bagianmu sebagai penulis laporan:

1. **Beri label yang jelas** pada setiap widget, juga saat labelnya disembunyikan dengan `label_visibility: "collapsed"`. Pembaca layar tetap membacakannya.
2. **Tulis temuan sebagai teks**, bukan hanya sebagai grafik. Kalimat "Omzet 2024 naik 27,6%" bisa dibaca semua orang.
3. **Beri teks pengganti** yang bermakna pada grafik penting dengan opsi `alt`.
4. **Jangan mengandalkan warna saja.** Tambahkan tanda atau teks, misalnya ▲ dan ▼ yang sudah dipakai `sl.metric`.
5. **Periksa di layar ponsel.** Di Slak Studio, bilah pratinjau punya tombol lebar layar ponsel (390 px).

17.5 Laporan lengkap

Inilah laporan utuhnya. Kode ini memakai sidebar, jadi bacalah di sini dan jalankan di playground atau Slak Studio.

```
sl.set_page_config({ page_title: "Laporan Toko Nusantara", page_icon: "📊", layout: "wide" })

const semua = await sl.read_csv("toko-nusantara.csv")
const data = semua.assign({ tahun: () => semua.tanggal.dt.year })

// ---------- Filter ----------
```

```
sl.sidebar.header("Filter")
const tahun = Number(sl.sidebar.selectbox("Tahun", ["2024", "2023"]))
const daftarCabang = data.cabang.unique().sort()
const cabang = sl.sidebar.multiselect("Cabang", daftarCabang, { placeholder: "Semua cabang" })
const kanal = sl.sidebar.radio("Kanal", ["Semua", "Toko", "Online"])

const saring = (df) => df
  .filter(r => cabang.length === 0 || cabang.includes(r.cabang))
  .filter(r => kanal === "Semua" || r.kanal === kanal)

const ini = saring(data.filter(r => r.tahun === tahun))
const lalu = saring(data.filter(r => r.tahun === tahun - 1))

// ---------- Judul ----------
sl.title(`Laporan Penjualan ${tahun}`)
sl.caption(`${cabang.length ? cabang.join(", ") : "Semua cabang"} · kanal ${kanal.toLowerCase()}`)

// ---------- Angka utama ----------
const persen = new Intl.NumberFormat("id-ID", { style: "percent", maximumFractionDigits: 1, signDisplay: "always" })
const banding = (a, b) => lalu.shape[0] ? persen.format(a / b - 1) : null

const [k1, k2, k3, k4] = sl.columns(4)
k1.metric("Omzet", ini.total.sum(), banding(ini.total.sum(), lalu.total.sum()), { format: "rp" })
k2.metric("Transaksi", ini.shape[0], banding(ini.shape[0], lalu.shape[0]))
k3.metric("Pelanggan", ini.pelanggan_id.nunique(), banding(ini.pelanggan_id.nunique(), lalu.pelanggan_id.nunique()))
k4.metric("Rata-rata transaksi", Math.round(ini.total.mean()), banding(ini.total.mean(), lalu.total.mean()), { format: "rp" })

// ---------- Tren ----------
const bulanan = ini.resample("tanggal", "M").agg({ omzet: ["total", "sum"] })
const puncak = bulanan.nlargest(1, "omzet").to_records()[0]
const namaBulan = new Intl.DateTimeFormat("id-ID", { month: "long" })
sl.subheader("Tren omzet bulanan")
sl.line_chart(bulanan, { x: "tanggal", y: "omzet", format: "compact",
  alt: `Omzet bulanan ${tahun}, tertinggi di bulan ${namaBulan.format(puncak.tanggal)}` })
sl.write(`Omzet tertinggi terjadi di bulan **${namaBulan.format(puncak.tanggal)}**.`)

// ---------- Perbandingan ----------
const [tCabang, tKategori] = sl.tabs(["Per cabang", "Per kategori"])
tCabang.bar_chart(ini.groupby("cabang").agg({ omzet: ["total", "sum"] }),
  { x: "cabang", y: "omzet", horizontal: true, sort: true, format: "compact" })
tKategori.bar_chart(ini.groupby("kategori").agg({ omzet: ["total", "sum"] }),
  { x: "kategori", y: "omzet", sort: true, labels: true, format: "compact" })

// ---------- Rincian ----------
sl.subheader("Produk terlaris")
const produk = ini.groupby("produk").agg({ terjual: ["jumlah", "sum"], omzet: ["total", "sum"] })
  .sort_values("omzet", { ascending: false })
sl.dataframe(produk.head(10), { format: { omzet: "rupiah" }, gradient: ["omzet"] })
sl.download_button("Unduh data terpilih (Excel)", { Transaksi: ini, Produk: produk }, { file_name: `toko-
nusantara-${tahun}.xlsx` })
```

Beberapa hal yang patut diperhatikan:

- **Fungsi** `saring` dipakai dua kali, untuk tahun ini dan tahun lalu, sehingga perbandingan selalu adil: filternya sama persis.
- `banding` mengembalikan `null` bila tidak ada data tahun lalu (misalnya untuk 2023), sehingga perubahannya tidak ditampilkan.
- **Temuan ditulis sebagai teks** di bawah grafik tren, dan dipakai juga sebagai teks pengganti grafiknya.
- **Unduhan Excel** berisi dua lembar sekaligus: objek `{ Transaksi: ..., Produk: ... }` menjadi dua lembar bernama sama.

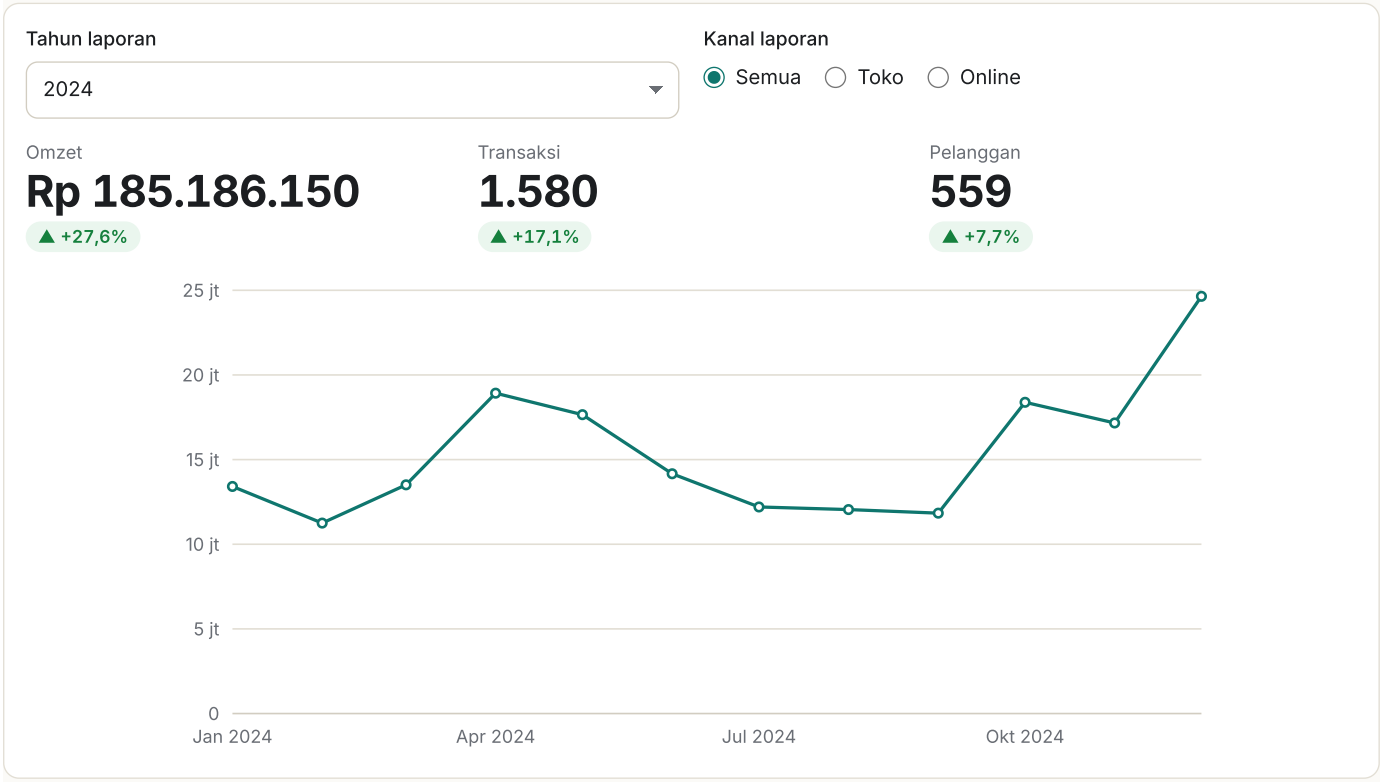
Berikut bagian intinya yang bisa dijalankan langsung di halaman ini, dengan filter di atas, bukan di sidebar:

```
const semua = await sl.read_csv("toko-nusantara.csv")
const data = semua.assign({ tahun: () => semua.tanggal.dt.year })
const [f1, f2] = sl.columns(2)
const tahun = Number(f1.selectbox("Tahun laporan", ["2024", "2023"]))
const kanal = f2.radio("Kanal laporan", ["Semua", "Toko", "Online"], { horizontal: true })

const saring = (df) => df.filter(r => kanal === "Semua" || r.kanal === kanal)
const ini = saring(data.filter(r => r.tahun === tahun))
```

```
const lalu = saring(data.filter(r => r.tahun === tahun - 1))
const persen = new Intl.NumberFormat("id-ID", { style: "percent", maximumFractionDigits: 1, signDisplay: "always" })
const banding = (a, b) => lalu.shape[0] ? persen.format(a / b - 1) : null

const [k1, k2, k3] = sl.columns(3)
k1.metric("Omzet", ini.total.sum(), banding(ini.total.sum(), lalu.total.sum()), { format: "rp" })
k2.metric("Transaksi", ini.shape[0], banding(ini.shape[0], lalu.shape[0]))
k3.metric("Pelanggan", ini.pelanggan_id.nunique(), banding(ini.pelanggan_id.nunique(), lalu.pelanggan_id.nunique()))
sl.line_chart(ini.resample("tanggal", "M").agg({ omzet: ["total", "sum"] }), { x: "tanggal", y: "omzet", format: "compact" })
```



Buka di Playground ↗

17.6 Menerbitkan

Laporan yang hanya ada di laptopmu belum bisa dipakai orang lain. Ada empat cara membagikannya, dari yang paling sederhana:

Satu berkas HTML

Di Slak Studio, tekan **Ctrl+E** (Ekspor ke HTML). Hasilnya satu berkas `.html` yang berisi `slak.js`, kodemu, dan data CSV yang dibaca dengan nama berkas. Berkas ini bisa dikirim lewat email atau WhatsApp dan dibuka dengan klik dua kali, tanpa internet.

Folder situs

Pilih **Berkas › Ekspor folder situs** (**Ctrl+Shift+E**). Hasilnya folder berisi `index.html`, `slak.js`, berkas tambahan yang dipakai (misalnya `slak-data.js`), dan `PANDUAN-UNGGAH.txt`. Folder ini siap diunggah ke hosting apa pun.

Unggah sendiri

Tempat	Langkah	Alamat hasilnya
cPanel	File Manager › `public_html` (atau subfolder) › Upload semua berkas	`https://domainmu.com/`
GitHub Pages	Repositori baru › Add file › Upload files › Settings › Pages › Deploy from a branch	`https://namamu.github.io/nama-repo/`
Netlify	Buka `app.netlify.com/drop`, seret folder situs	Alamat acak yang bisa diganti

Terbit sekali klik

Pilih **Berkas › Terbitkan…** (**Ctrl+Shift+U**), lalu pilih tujuannya:

Tujuan	Yang perlu disiapkan
GitHub Pages	Akun GitHub dan token akses pribadi dengan izin Contents dan Pages

Tujuan	Yang perlu disiapkan
Netlify	Akun Netlify dan token akses pribadi
FTP (cPanel)	Server FTP, nama pengguna, dan password dari menu FTP Accounts di cPanel

Tekan **Terbitkan**, dan tautan situsmu muncul setelah prosesnya selesai. Untuk FTP, Slak Studio mencoba FTPS (terenkripsi) lebih dulu, dan hanya mengunggah berkas yang berubah sejak terbit terakhir.

Token dan password disimpan terenkripsi oleh sistem operasi di komputermu dan **tidak pernah ikut ke situs**. Bila token ditemukan di kodemu, terbit dibatalkan. Centang **Bisa dipasang dan jalan offline (PWA)** agar pengunjung bisa memasang laporanmu seperti aplikasi dan membukanya tanpa internet setelah kunjungan pertama.

Yang perlu diingat saat online

- Setiap pengunjung menjalankan laporan **di browsernya sendiri**, jadi hosting termurah pun cukup.
- Data yang disematkan bisa dilihat siapa saja lewat "View source". **Jangan menerbitkan data pribadi atau rahasia**; pakai data fiktif seperti Toko Nusantara, atau data yang sudah dianonimkan.

> **Coba di playground** Salin laporan lengkap 17.5 ke playground, lalu tekan **Bagikan**. Kirim tautannya ke temanmu. Ingat, data CSV tidak ikut di tautan; temanmu perlu memilih berkasnya, atau terbitkan lewat Slak Studio supaya datanya ikut.

Kesalahan umum

Gejala	Penyebab	Perbaikan
Halaman hasil unggah kosong	`slak.js` tidak ikut diunggah	Unggah seluruh isi folder situs
Data tidak terbaca di hosting	Nama berkas CSV beda huruf besar-kecil	Hosting Linux membedakan `Data.csv` dan `data.csv`
Perbandingan tahun lalu aneh	Filter tahun lalu berbeda dengan tahun ini	Pakai satu fungsi `saring` untuk keduanya
Tampilan sempit di ponsel	Terlalu banyak kolom berdampingan	Pakai `sl.layout`, atau kurangi kolom

Latihan

Latihan 17.1 (mudah). Lengkapi fungsi `saring(df, kanal)` agar mengembalikan semua baris bila `kanal` bernilai `"Semua"`, dan hanya kanal itu bila tidak.

```
const df = await sl.read_csv("toko-nusantara.csv")
function saring(data, kanal) {
  return data
}

await sl.testing.test("Semua tidak menyaring", () => sl.testing.expect(saring(df, "Semua").shape[0]).toBe(df.shape[0]))
await sl.testing.test("Online saja", () => sl.testing.expect(saring(df, "Online").kanal.unique()).toEqual(["Online"]))
```

✓ Semua tidak menyaring (0 md)

✗ **Online saja** — Diharapkan: setara dengan ["Online"] — nilai sebenarnya: ["Toko","Online"]

Buka di Playground ↗

Latihan 17.2 (sedang). Uji di bawah menjalankan aplikasi kecil lalu memilih tahun 2023. Lengkapi kode aplikasinya agar metrik "Omzet" menampilkan omzet tahun yang dipilih.

```
const aplikasi = `
const df = await sl.read_csv("toko-nusantara.csv")
const tahun = Number(sl.selectbox("Tahun", ["2024", "2023"]))
const data = df   // saring berdasarkan tahun
sl.metric("Omzet", data.total.sum())
`
sl.code(aplikasi)

const df = await sl.read_csv("toko-nusantara.csv")
await sl.testing.test("pilih 2023", async () => {
  const at = await sl.testing.run(aplikasi)
  await at.selectbox("Tahun").set_value("2023").run()
  sl.testing.expect(at.metric("Omzet").value).toBe(df.filter(r => r.tanggal.getFullYear() === 2023).total.sum())
})
```

```
const df = await sl.read_csv("toko-nusantara.csv")
const tahun = Number(sl.selectbox("Tahun", ["2024", "2023"]))
const data = df   // saring berdasarkan tahun
sl.metric("Omzet", data.total.sum())
```

✕ **pilih 2023** — Diharapkan: sama persis dengan 145129800 — nilai sebenarnya: 330315950

Buka di Playground ↗

Latihan 17.3 (tantangan). Lengkapi `temuan(df)` yang mengembalikan satu kalimat berisi cabang dengan omzet tertinggi dan porsinya, misalnya `"Nusantara Surabaya memimpin dengan 13,4% omzet."`, dengan persen bergaya Indonesia satu desimal.

```
const df = await sl.read_csv("toko-nusantara.csv")
function temuan(data) {
  return ""
}

sl.write(temuan(df))
await sl.testing.test("kalimat temuan benar", () => sl.testing.expect(temuan(df)).toBe("Nusantara Surabaya memimpin dengan
13,4% omzet."))
await sl.testing.test("berlaku untuk data lain", () =>
  sl.testing.expect(temuan(df.filter(r => r.kategori === "Pakaian"))).toMatch(/^Nusantara .+ memimpin dengan \d+,\d%
omzet\.$/))
```

✕ **kalimat temuan benar** — Diharapkan: sama persis dengan "Nusantara Surabaya memimpin dengan 13,4% omzet." — nilai sebenarnya: ""

✕ **berlaku untuk data lain** — Diharapkan: cocok dengan /^Nusantara .+ memimpin dengan \d+,\d% omzet\.$/ — nilai sebenarnya: ""

Buka di Playground ↗

Mini-proyek: terbitkan laporanmu

Tugas penutup Tahap 2:

1. Salin laporan lengkap 17.5 ke Slak Studio dan simpan sebagai `laporan.slak` di folder yang sama dengan `toko-nusantara.csv`.
2. Tambahkan satu bagian baru yang menjawab pertanyaanmu sendiri, misalnya "Hari apa toko paling ramai?" (Latihan 16.2).
3. Tulis tiga temuan utamamu sebagai teks dengan `sl.markdown` di akhir laporan.
4. Periksa tampilannya di lebar ponsel.
5. Terbitkan ke GitHub Pages atau Netlify, lalu bagikan tautannya.

Selamat! Kamu baru saja menerbitkan karya analisis data pertamamu. Di Tahap 3, laporan ini akan tumbuh menjadi dasbor portofolio multi-halaman.

← 16. Visualisasi Dasar

18. Menggabungkan dan Membentuk Ulang Data →

Bab 18 · Menggabungkan dan Membentuk Ulang Data

> **Tujuan belajar**
>
> - Memahami inner, left, dan outer join.
> - Menggabungkan tabel dengan `Map` di JavaScript murni, lalu dengan `merge`.
> - Menyambung tabel dengan `sl.concat`.
> - Mengubah bentuk tabel dari panjang ke lebar dengan `pivot`, dan sebaliknya dengan `sl.melt`.

Selamat datang di Tahap 3. Data di dunia kerja jarang tersimpan dalam satu tabel. Transaksi ada di sistem kasir, harga pokok ada di bagian pembelian, target ada di berkas Excel manajer. Analis data menggabungkannya.

Bahan buku menyediakan tiga tabel pendamping transaksi:

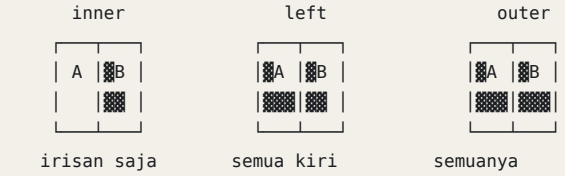
Berkas	Kunci	Isi
`produk.csv`	`produk`	Kategori, harga jual, **harga pokok**, pemasok
`cabang.csv`	`cabang`	Kode, kota, provinsi, **koordinat**, tanggal buka
`target.csv`	`cabang` + `bulan`	**Target omzet** bulanan setiap cabang

18.1 Kunci dan jenis join

Dua tabel digabung lewat **kunci**: kolom yang isinya sama di kedua tabel. Transaksi punya kolom `produk`, dan `produk.csv` juga punya kolom `produk`. Dengan kunci itu, setiap transaksi bisa diberi harga pokoknya.

Pertanyaannya: bagaimana dengan baris yang kuncinya hanya ada di salah satu tabel?

Jenis	Hasil	Mirip di Excel
inner (bawaan)	Hanya baris yang kuncinya ada di kedua tabel	
left	Semua baris tabel kiri; kolom kanan kosong bila tidak cocok	VLOOKUP
right	Semua baris tabel kanan	
outer	Semua baris dari kedua tabel	



Untuk analisis transaksi, **left join** paling sering dipakai: kita ingin semua transaksi tetap ada, apa pun yang terjadi di tabel pendamping.

18.2 Join dengan JavaScript murni

Cara paling sederhana adalah mencari dengan `find` untuk setiap transaksi. Tetapi untuk 3.000 transaksi dan banyak produk, itu berarti jutaan perbandingan. Cara yang benar: buat `Map`, kamus yang mencari dalam sekejap.

```
const trx = (await sl.read_csv("toko-nusantara.csv")).to_records()
const produk = (await sl.read_csv("produk.csv")).to_records()

const kamusProduk = new Map(produk.map(p => [p.produk, p]))   // produk → baris produk

const gabung = trx.map(t => {
  const p = kamusProduk.get(t.produk)
  return { ...t, harga_pokok: p ? p.harga_pokok : null, pemasok: p ? p.pemasok : null }
})

sl.write(`Transaksi tanpa pasangan: ${gabung.filter(r => r.harga_pokok == null).length}`)
sl.dataframe(gabung.slice(0, 5).map(r => ({ produk: r.produk, harga: r.harga, harga_pokok: r.harga_pokok, pemasok: r.pemasok
}))
```

Transaksi tanpa pasangan: 0

	produk	harga	harga_pokok	pemasok
0	Topi Rotan	85.000	46.000	Sanggar Noken Abepura
1	Kopi Arabika Wamena 250g	85.000	52.000	Koperasi Baliem
2	Tenun Ikat Sumba Selendang	350.000	230.000	Koperasi Timor Lestari
3	Tenun Ikat Sumba Selendang	350.000	230.000	Koperasi Timor Lestari
4	Sambal Roa 200g	45.000	25.000	CV Bitung Bahari

5 baris × 4 kolom

Buka di Playground ↗

> **`Map` di JavaScript**
>
> - `new Map(larikPasangan)` membuat kamus dari larik `[kunci, nilai]`.
> - `peta.get(kunci)` mengambil nilai, `peta.has(kunci)` memeriksa ada tidaknya, `peta.set(kunci, nilai)` menambah.
> - Bedanya dengan objek biasa: kunci `Map` boleh bertipe apa saja, dan urutan penambahannya terjaga.

Ini **left join**: semua transaksi dipertahankan, dan yang tidak punya pasangan diberi `null`.

18.3 merge

Slak Data melakukan hal yang sama dalam satu baris:

```
const trx = await sl.read_csv("toko-nusantara.csv")
const produk = await sl.read_csv("produk.csv")

const lengkap = trx.merge(produk.select(["produk", "harga_pokok", "pemasok"]), { on: "produk", how: "left" })
  .assign({ laba_kotor: r => r.total - r.jumlah * r.harga_pokok })

const perPemasok = lengkap.groupby("pemasok").agg({
  omzet: ["total", "sum"],
  laba: ["laba_kotor", "sum"]
}).assign({ margin: r => r.laba / r.omzet }).sort_values("laba", { ascending: false })

sl.dataframe(perPemasok, { format: { omzet: "rupiah", laba: "rupiah", margin: "percent" } })
```

	pemasok	omzet	laba	margin
0	Sanggar Noken Abepura	Rp 61.909.750	Rp 27.121.750	43,8%
1	Koperasi Timor Lestari	Rp 53.400.300	Rp 19.148.300	35,9%
2	UD Rempah Maluku	Rp 43.177.350	Rp 18.877.350	43,7%
3	UKM Wangi Bali	Rp 32.131.900	Rp 15.883.900	49,4%
4	CV Bitung Bahari	Rp 34.830.200	Rp 15.201.200	43,6%
5	Koperasi Baliem	Rp 36.749.500	Rp 14.753.500	40,1%
6	CV Tana Toraja	Rp 26.970.200	Rp 10.545.200	39,1%
7	Konveksi Jayapura	Rp 21.854.800	Rp 9.814.800	44,9%
8	UKM Sagu Sentani	Rp 19.291.950	Rp 9.546.950	49,5%

9 baris × 4 kolom

Buka di Playground ↗

Kini kita bisa menjawab pertanyaan yang tidak mungkin dijawab dari satu tabel: **pemasok mana yang paling menguntungkan?**

> **API Slak baru**

- `df.merge(lain, { on, how })` menggabungkan dua DataFrame. `how`: `"inner"`, `"left"`, `"right"`, `"outer"`. Bila nama kuncinya berbeda: `{ left_on: "nama_produk", right_on: "produk" }`.
- `sl.merge(kiri, kanan, opsi)` adalah bentuk lain yang sama.
- `df.join(lain, { on })` adalah singkatan untuk left join.

Memeriksa hasil join

Join adalah sumber kesalahan analisis yang paling licin. Dua pemeriksaan wajib:

```
const trx = await sl.read_csv("toko-nusantara.csv")
const cabang = await sl.read_csv("cabang.csv")
const hasil = trx.merge(cabang.select(["cabang", "kode", "tanggal_buka"]), { on: "cabang", how: "left", indicator: true })

sl.write(`Baris sebelum: ${trx.shape[0]}, sesudah: ${hasil.shape[0]}`)
sl.dataframe(hasil.value_counts("_merge"))
```

Baris sebelum: 2929, sesudah: 2929

	_merge	jumlah
0	both	2.929

1 baris × 2 kolom

Buka di Playground ↗

1. **Banyak baris tidak boleh bertambah** pada left join. Bila bertambah, berarti kunci di tabel kanan ada yang ganda, dan transaksi ikut tergandakan. Omzetmu akan terhitung lebih besar dari kenyataan.
2. **Periksa baris tanpa pasangan.** `indicator: true` menambahkan kolom `_merge` berisi `both`, `left_only`, atau `right_only`.

18.4 Join dua kunci: omzet dibanding target

Target disimpan per cabang **per bulan**. Kuncinya dua kolom sekaligus.

```
const trx = await sl.read_csv("toko-nusantara.csv")
const target = await sl.read_csv("target.csv")

const omzetBulanan = trx
  .assign({ bulan: () => trx.tanggal.dt.strftime("%Y-%m") })
  .groupby(["cabang", "bulan"]).agg({ omzet: ["total", "sum"] })

const capaian = omzetBulanan.merge(target, { on: ["cabang", "bulan"], how: "left" })
  .assign({ capai: r => r.omzet / r.target })

const ringkas = capaian.groupby("cabang").agg({
  bulan_tercapai: ["capai", x => x.filter(v => v >= 1).length],
  rata_capaian: ["capai", "mean"]
}).sort_values("rata_capaian", { ascending: false })

sl.dataframe(ringkas, { format: { rata_capaian: "percent" } })
```

	cabang	bulan_tercapai	rata_capaian
0	Nusantara Ambon	17	104,2%
1	Nusantara Manado	13	101,8%
2	Nusantara Manokwari	14	101,4%
3	Nusantara Kupang	12	101,3%
4	Nusantara Sorong	13	101,1%
5	Nusantara Makassar	13	100,6%
6	Nusantara Jakarta	11	100,4%
7	Nusantara Jayapura	11	98,8%
8	Nusantara Denpasar	10	98,6%
9	Nusantara Surabaya	6	95,0%

10 baris × 3 kolom

Buka di Playground ↗

Perhatikan `["capai", x => x.filter(v => v >= 1).length]`. Fungsi agregasi boleh berupa fungsi buatanmu sendiri; ia menerima larik nilai setiap kelompok.

18.5 Menyambung tabel

Kadang datanya datang terpisah per periode atau per cabang, dengan kolom yang sama. `sl.concat` menumpuknya menjadi satu.

```
const df = await sl.read_csv("toko-nusantara.csv")
const data2023 = df.filter(r => r.tanggal.getFullYear() === 2023)
const data2024 = df.filter(r => r.tanggal.getFullYear() === 2024)
sl.write(`${data2023.shape[0]} + ${data2024.shape[0]} baris`)

const satu = sl.concat([data2023, data2024])
sl.write(`Disambung: ${satu.shape[0]} baris`)
```

1349 + 1580 baris

Disambung: 2929 baris

Buka di Playground ↗

18.6 Panjang dan lebar

Tabel yang sama bisa disusun dua cara:

Bentuk panjang				Bentuk lebar		
kategori	**tahun**	**omzet**		**kategori**	**2023**	**2024**
Minuman	2023	36,4 jt		Minuman	36,4 jt	47,5 jt
Minuman	2024	47,5 jt		Makanan	35,3 jt	47,5 jt
Makanan	2023	35,3 jt				

Bentuk **lebar** enak dibaca manusia. Bentuk **panjang** disukai komputer dan grafik, karena setiap baris satu pengamatan.

```
const df = await sl.read_csv("toko-nusantara.csv")
const panjang = df.assign({ tahun: () => df.tanggal.dt.year.astype("str") })
  .groupby(["kategori", "tahun"]).agg({ omzet: ["total", "sum"] })

const lebar = panjang.pivot({ index: "kategori", columns: "tahun", values: "omzet" })
sl.dataframe(lebar, { format: "compact" })

const kembali = sl.melt(lebar, { id_vars: ["kategori"], var_name: "tahun", value_name: "omzet" })
sl.dataframe(kembali.head(4), { format: "compact" })
```

	kategori	2023	2024
0	Kebutuhan Rumah	14.088.200	18.081.100
1	Kerajinan	36.904.000	51.749.250
2	Makanan	35.309.300	47.528.650
3	Minuman	36.402.300	47.499.350
4	Pakaian	22.426.000	20.327.800

5 baris × 3 kolom

	kategori	tahun	omzet
0	Kebutuhan Rumah	2023	14.088.200
1	Kerajinan	2023	36.904.000
2	Makanan	2023	35.309.300
3	Minuman	2023	36.402.300

4 baris × 3 kolom

Buka di Playground ↗

- `pivot` mengubah panjang menjadi lebar: nilai kolom `tahun` menjadi nama kolom.
- `sl.melt` mengubah lebar menjadi panjang: nama kolom kembali menjadi nilai.

Kapan butuh `melt`? Saat datamu datang dalam bentuk lebar dari Excel, misalnya satu kolom per bulan, dan kamu ingin menggambarnya dengan `color: "bulan"` atau mengelompokkannya.

> **API Slak baru**
>
> - `sl.concat([df1, df2])` menyambung tabel ke bawah.
> - `df.pivot({ index, columns, values })` panjang ke lebar; `pivot_table` (Bab 15) melakukan hal yang sama sambil meringkas.
> - `sl.melt(df, { id_vars, var_name, value_name })` lebar ke panjang.

> **Coba di playground** Di contoh 18.3, ganti `"pemasok"` dengan `"kategori"`. Kategori mana yang marginnya paling tipis?

Kesalahan umum

Gejala	Penyebab	Perbaikan
Omzet setelah join lebih besar dari sebelumnya	Kunci ganda di tabel kanan	Periksa `drop_duplicates` pada tabel kanan dan banyak baris sebelum-sesudah
Banyak nilai kosong setelah left join	Ejaan kunci berbeda, misalnya spasi atau huruf besar	Bersihkan kunci di kedua tabel (Bab 13–14)
Kolom `harga_x` dan `harga_y` muncul	Kolom bernama sama di kedua tabel	Pilih kolom dengan `select` sebelum `merge`, atau beri `suffixes`
Baris hilang setelah join	Memakai inner join padahal butuh left	`how: "left"`

Latihan

Latihan 18.1 (mudah). Gabungkan `cabang.csv` ke transaksi dengan left join agar setiap transaksi punya kolom `kode`. Pastikan banyak barisnya tidak berubah.

```
const trx = await sl.read_csv("toko-nusantara.csv")
const cabang = await sl.read_csv("cabang.csv")
const hasil = trx   // ganti

await sl.testing.test("banyak baris tetap", () => sl.testing.expect(hasil.shape[0]).toBe(trx.shape[0]))
await sl.testing.test("kolom kode ada", () => sl.testing.expect(hasil.columns).toContain("kode"))
await sl.testing.test("tidak ada kode kosong", () => sl.testing.expect(hasil.kode.isna().sum()).toBe(0))
```

- ✓ banyak baris tetap (0 md)
- ✗ **kolom kode ada** — Diharapkan: memuat "kode" — nilai sebenarnya: ["id_transaksi","tanggal","pelanggan_id","cabang","kota","provinsi","kanal","kategori","produk","jumlah","harga","diskon_persen","total"]
- ✗ **tidak ada kode kosong** — Cannot read properties of undefined (reading 'isna')

Buka di Playground ↗

Latihan 18.2 (sedang). Dengan JavaScript murni dan `Map`, lengkapi `tambahKategori(trx, produk)` yang menambahkan kolom `kategori_produk` dari `produk.csv` ke setiap transaksi.

```
function tambahKategori(trx, produk) {
  return trx
}

const trx = [{ produk: "Topi Rotan", total: 85000 }, { produk: "Sambal Roa 200g", total: 45000 }, { produk: "Tidak Ada",
total: 1 }]
const produk = (await sl.read_csv("produk.csv")).to_records()
const h = tambahKategori(trx, produk)
await sl.testing.test("Topi Rotan → Pakaian", () => sl.testing.expect(h[0].kategori_produk).toBe("Pakaian"))
await sl.testing.test("produk tidak dikenal → null", () => sl.testing.expect(h[2].kategori_produk).toBeNull())
await sl.testing.test("data asli tidak berubah", () => sl.testing.expect("kategori_produk" in trx[0]).toBe(false))
```

- ✗ **Topi Rotan → Pakaian** — Diharapkan: sama persis dengan "Pakaian" — nilai sebenarnya: undefined
- ✓ produk tidak dikenal → null (0 md)
- ✓ data asli tidak berubah (0 md)

Buka di Playground ↗

Latihan 18.3 (tantangan). Hitung `persenTercapai`: persentase pasangan cabang-bulan yang omzetnya mencapai atau melampaui target, sebagai angka 0 sampai 1.

```
const trx = await sl.read_csv("toko-nusantara.csv")
const target = await sl.read_csv("target.csv")
const persenTercapai = 0   // ganti

sl.metric("Bulan yang tercapai", (persenTercapai * 100).toFixed(1) + "%")
await sl.testing.test("antara 0 dan 1", () => { sl.testing.expect(persenTercapai).toBeGreaterThan(0);
sl.testing.expect(persenTercapai).toBeLessThan(1) })
await sl.testing.test("nilai benar", () => {
  const ob = trx.assign({ bulan: () => trx.tanggal.dt.strftime("%Y-%m") }).groupby(["cabang", "bulan"]).agg({ omzet:
["total", "sum"] })
  const g = ob.merge(target, { on: ["cabang", "bulan"] }).to_records()
  sl.testing.expect(persenTercapai).toBeCloseTo(g.filter(r => r.omzet >= r.target).length / g.length, 6)
})
```

Bulan yang tercapai

0.0%

✕ **antara 0 dan 1** — Diharapkan: lebih besar dari 0 — nilai sebenarnya: 0

✕ **nilai benar** — Diharapkan: mendekati 0.5 (6 desimal) — nilai sebenarnya: 0

Buka di Playground ↗

Mini-proyek: laba per cabang dan capaian target

```
sl.title("Laba dan Target per Cabang :dart:")

const [trx, produk, cabang, target] = await Promise.all([
  sl.read_csv("toko-nusantara.csv"), sl.read_csv("produk.csv"), sl.read_csv("cabang.csv"), sl.read_csv("target.csv")
])

const tahun = sl.radio("Tahun", [2023, 2024], { horizontal: true, index: 1 })

const lengkap = trx
  .filter(r => r.tanggal.getFullYear() === tahun)
  .merge(produk.select(["produk", "harga_pokok"]), { on: "produk", how: "left" })
  .assign({ laba: r => r.total - r.jumlah * r.harga_pokok, bulan: r => sl.strftime(r.tanggal, "%Y-%m") })

const perCabang = lengkap.groupby("cabang").agg({ omzet: ["total", "sum"], laba: ["laba", "sum"] })
const targetTahun = target.filter(r => r.bulan.startsWith(String(tahun))).groupby("cabang").agg({ target: ["target", "sum"]
})

const hasil = perCabang
  .merge(targetTahun, { on: "cabang", how: "left" })
  .merge(cabang.select(["cabang", "provinsi"]), { on: "cabang", how: "left" })
  .assign({ capaian: r => r.omzet / r.target, margin: r => r.laba / r.omzet })
  .sort_values("capaian", { ascending: false })

sl.dataframe(hasil.select(["cabang", "provinsi", "omzet", "target", "capaian", "laba", "margin"]),
  { format: { omzet: "rupiah", target: "rupiah", laba: "rupiah", capaian: "percent", margin: "percent" }, gradient:
["capaian"] })

const lewat = hasil.filter(r => r.capaian >= 1).shape[0]
sl.write(`**${lewat} dari ${hasil.shape[0]} cabang** mencapai target ${tahun}.`)
```

Laba dan Target per Cabang 🎯

Tahun

◯ 2023 ◉ 2024

	cabang	provinsi	omzet	target	capaian	laba	margin
0	Nusantara Ambon	Maluku	Rp 22.277.100	Rp 21.500.000	103,6%	Rp 9.632.100	43,2%
1	Nusantara Manokwari	Papua Barat	Rp 15.522.250	Rp 15.000.000	103,5%	Rp 6.533.250	42,1%
2	Nusantara Makassar	Sulawesi Selatan	Rp 18.659.000	Rp 18.400.000	101,4%	Rp 8.420.000	45,1%
3	Nusantara Kupang	Nusa Tenggara Timur	Rp 13.214.700	Rp 13.100.000	100,9%	Rp 5.870.700	44,4%
4	Nusantara Manado	Sulawesi Utara	Rp 15.208.500	Rp 15.100.000	100,7%	Rp 6.767.500	44,5%
5	Nusantara Jakarta	DKI Jakarta	Rp 19.908.350	Rp 19.900.000	100,0%	Rp 8.957.350	45,0%
6	Nusantara Sorong	Papua Barat Daya	Rp 14.206.900	Rp 14.400.000	98,7%	Rp 6.295.900	44,3%
7	Nusantara Jayapura	Papua	Rp 24.341.550	Rp 24.700.000	98,5%	Rp 10.490.550	43,1%
8	Nusantara Denpasar	Bali	Rp 20.284.800	Rp 21.000.000	96,6%	Rp 8.762.800	43,2%
9	Nusantara Surabaya	Jawa Timur	Rp 21.563.000	Rp 22.800.000	94,6%	Rp 9.375.000	43,5%

10 baris × 7 kolom

6 dari 10 cabang mencapai target 2024.

Buka di Playground ↗

Empat tabel, satu jawaban. `Promise.all` (Bab 12) membaca keempat berkas bersamaan.

← 17. Proyek: Laporan Interaktif Pertama

19. Statistik Deskriptif Lanjutan →

Bab 19 · Statistik Deskriptif Lanjutan

> **Tujuan belajar**
>
> - Mengukur sebaran data: rentang, varians, dan simpangan baku.
> - Memakai persentil, kuartil, dan box plot.
> - Mendeteksi pencilan dengan metode IQR, dan memutuskan apa yang harus dilakukan terhadapnya.
> - Mengukur hubungan dua variabel dengan korelasi, dan memahami bedanya dengan sebab-akibat.
> - Membandingkan dua kelompok dengan uji t dan uji A/B.

Rata-rata saja tidak cukup. Dua cabang bisa punya rata-rata transaksi yang sama, tetapi yang satu stabil dan yang lain naik turun tak menentu. Bab ini memberimu alat untuk melihat **bentuk** data, bukan hanya pusatnya.

19.1 Sebaran: seberapa jauh nilai-nilai menyebar

Mari mulai dengan JavaScript murni, seperti di Bab 5:

```
const nilaiA = [100, 102, 98, 101, 99]     // stabil
const nilaiB = [60, 140, 80, 130, 90]      // naik turun

function rata(x) { return x.reduce((a, b) => a + b, 0) / x.length }
function varians(x) {
  const m = rata(x)
  return x.reduce((a, v) => a + (v - m) ** 2, 0) / (x.length - 1)
}
const simpangan = x => Math.sqrt(varians(x))

sl.write(`A: rata-rata ${rata(nilaiA)}, simpangan baku
${simpangan(nilaiA).toFixed(1)}`)
sl.write(`B: rata-rata ${rata(nilaiB)}, simpangan baku
${simpangan(nilaiB).toFixed(1)}`)
```

A: rata-rata 100, simpangan baku 1.6

B: rata-rata 100, simpangan baku 33.9

Buka di Playground ↗

Rata-ratanya sama-sama 100, tetapi simpangan bakunya sangat berbeda.

- **Varians** adalah rata-rata kuadrat jarak setiap nilai ke rata-ratanya. Kuadrat dipakai supaya jarak ke bawah dan ke atas tidak saling meniadakan.
- **Simpangan baku** (*standard deviation*) adalah akar varians, sehingga satuannya kembali sama dengan data. Kira-kira: "biasanya nilai menyimpang sejauh ini dari rata-rata".
- Pembagi `x.length - 1` dipakai untuk **sampel**. Ini juga bawaan pandas dan Excel `STDEV.S`.

Dengan Slak Data:

```
const df = await sl.read_csv("toko-nusantara.csv")
const t = df.total
sl.json(sl.stats.describe(t))
```


```
{
  "count": 2929,
  "mean": 112774.30863776033,
  "std": 149395.26518938204,
  "min": 16200,
  "25%": 40000,
  "50%": 72000,
  "75%": 126000,
  "max": 1837500
}
```


Buka di Playground ↗

> **API Slak baru**
>
> - `sl.stats.describe(x)` meringkas satu kolom: banyak data, rata-rata, simpangan baku, minimum, kuartil, dan maksimum.
> - `sl.stats.var(x)`, `sl.stats.std(x)`, `sl.stats.mean(x)`, `sl.stats.median(x)`, juga tersedia sebagai metode Series: `df.total.std()`.

19.2 Persentil dan kuartil

Persentil ke-90 adalah nilai yang di bawahnya terdapat 90% data. **Kuartil** adalah persentil ke-25, 50, dan 75: tiga titik yang membagi data menjadi empat bagian sama banyak.

```
const df = await sl.read_csv("toko-nusantara.csv")
const t = df.total
const [q1, q2, q3] = sl.stats.quantile(t, [0.25, 0.5, 0.75])

const rp = new Intl.NumberFormat("id-ID")
sl.markdown(`
- 25% transaksi bernilai di bawah **Rp ${rp.format(q1)}** (kuartil 1)
- separuh transaksi di bawah **Rp ${rp.format(q2)}** (median)
- 75% transaksi di bawah **Rp ${rp.format(q3)}** (kuartil 3)
- 10% transaksi teratas di atas **Rp ${rp.format(sl.stats.percentile(t, 90))}**
(persentil 90)
`)
```

- 25% transaksi bernilai di bawah **Rp 40.000** (kuartil 1)
- separuh transaksi di bawah **Rp 72.000** (median)
- 75% transaksi di bawah **Rp 126.000** (kuartil 3)
- 10% transaksi teratas di atas **Rp 240.000** (persentil 90)

Buka di Playground ↗

Kalimat seperti ini jauh lebih mudah dipahami manajer daripada "simpangan baku 149 ribu".

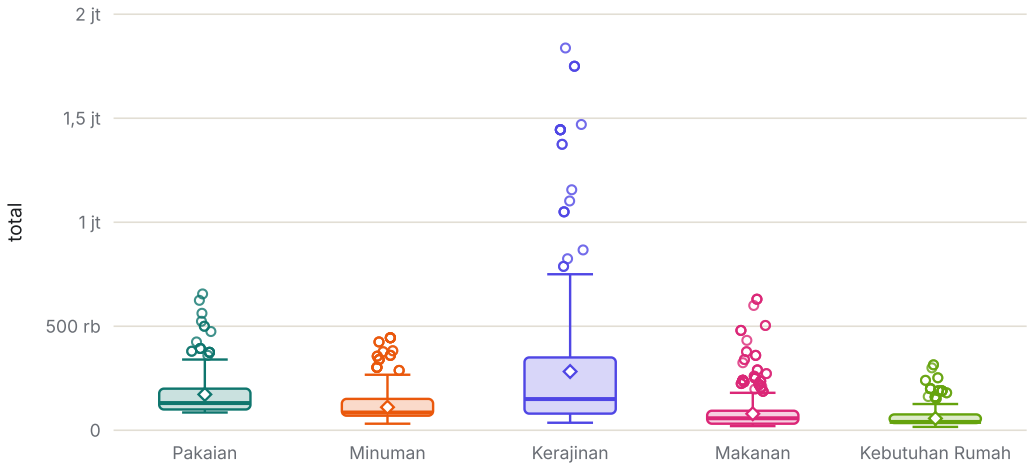
Rentang antarkuartil (IQR) adalah `Q3 − Q1`, lebar separuh data yang di tengah. IQR tidak terpengaruh nilai ekstrem, sehingga lebih bisa diandalkan daripada rentang `max − min`.

19.3 Box plot

Box plot menggambar lima angka sekaligus: minimum, Q1, median, Q3, dan maksimum, ditambah titik-titik pencilan.

```
const df = await sl.read_csv("toko-nusantara.csv")
sl.box_plot(df, { y: "total", by: "kategori", title: "Nilai transaksi per kategori"
})
```

Nilai transaksi per kategori



Buka di Playground ↗

Cara membacanya:

- **Kotak** membentang dari Q1 sampai Q3; garis di dalamnya adalah median.
- **Kumis** memanjang sampai nilai terjauh yang masih dalam 1,5 × IQR dari kotak.
- **Titik** di luar kumis adalah pencilan.

Satu grafik ini memperlihatkan bahwa transaksi Kerajinan jauh lebih besar dan lebih beragam daripada kategori lain.

> **API Slak baru**
>
> - `sl.box_plot(data, { y, by })` menggambar box plot, satu kotak per kelompok.

19.4 Pencilan: buang, potong, atau biarkan?

Metode IQR menganggap nilai sebagai pencilan bila berada di bawah `Q1 − 1,5 × IQR` atau di atas `Q3 + 1,5 × IQR`.

```
const df = await sl.read_csv("toko-nusantara.csv")
const masker = df.outliers("total")
sl.write(`Batas normal: ${masker.bounds.map(x => x.toLocaleString("id-ID")).join("
sampai ")}`)
sl.write(`${masker.values.filter(Boolean).length} transaksi di luar batas`)

const pencilan = df.filter(masker)
sl.dataframe(pencilan.value_counts("kategori"))
```

Batas normal: -89.000 sampai 255.000

235 transaksi di luar batas

	kategori	jumlah
0	Kerajinan	118
1	Minuman	47
2	Pakaian	42
3	Makanan	25
4	Kebutuhan Rumah	3

5 baris × 2 kolom

Buka di Playground ↗

Ada 235 transaksi "pencilan", hampir semuanya Kerajinan. Apakah mereka salah? **Tidak.** Tenun ikat dan ukiran Asmat memang mahal. Inilah pelajaran terpenting bagian ini:

> **Pencilan bukan berarti salah.** Sebelum membuang, tanyakan: apakah nilai ini mungkin terjadi? Di Bab 13, jumlah 250 barang dalam satu transaksi oleh-oleh hampir pasti salah ketik, jadi dibuang. Di sini, transaksi 1,8 juta untuk tenun ikat adalah penjualan sungguhan. Membuangnya berarti menghilangkan pelanggan terbaikmu.

Tiga pilihan yang tersedia:

Pilihan	Perintah	Kapan
Buang	`df.remove_outliers("total")`	Nilainya pasti salah
Potong ke batas	`df.cap_outliers("total")`	Nilainya benar, tetapi mengganggu model atau rata-rata
Biarkan, pakai median	`df.total.median()`	Nilainya benar dan penting; laporkan median di samping rata-rata

19.5 Korelasi

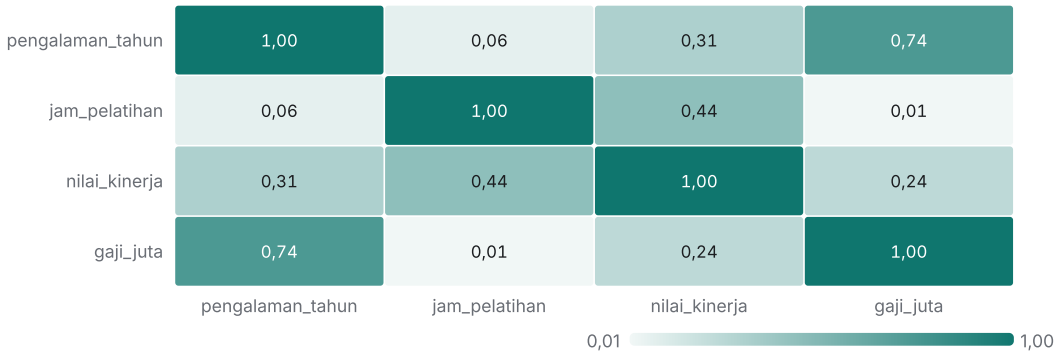
Korelasi mengukur seberapa kuat dua variabel bergerak bersama, dari −1 sampai +1:

Nilai	Arti
mendekati +1	Bila satu naik, yang lain cenderung naik
sekitar 0	Tidak ada hubungan garis lurus
mendekati −1	Bila satu naik, yang lain cenderung turun

```
const kar = sl.dataset("karyawan")
const angka = kar.select_dtypes("number")
sl.heatmap(angka.corr(), { title: "Korelasi antarvariabel karyawan" })

const r = sl.stats.pearsonr(kar.pengalaman_tahun, kar.gaji_juta)
sl.write(`Korelasi pengalaman dan gaji: ${r.statistic.toFixed(2)}`)
sl.write(r.conclusion)
```

Korelasi antarvariabel karyawan



Korelasi pengalaman dan gaji: 0.74

p = < 0,0001 < 0,05, jadi H_o ditolak. Ada hubungan positif yang signifikan (sangat kuat).

Buka di Playground ↗

Dataset bawaan `karyawan` berisi 300 karyawan fiktif. Pengalaman dan gaji berkorelasi 0,74: kuat dan positif.

> **API Slak baru**
>
> - `df.corr()` menghitung korelasi semua pasangan kolom angka. `df.select_dtypes("number")` memilih kolom angka saja.
> - `sl.heatmap(data)` menggambar tabel angka sebagai kotak berwarna.
> - `sl.stats.pearsonr(x, y)` dan `sl.stats.spearmanr(x, y)` menguji korelasi, lengkap dengan kesimpulan berbahasa Indonesia.

Korelasi bukan sebab-akibat

Penjualan es krim dan kasus tenggelam berkorelasi positif. Apakah es krim menyebabkan orang tenggelam? Tentu tidak. Keduanya naik di musim panas, saat orang lebih banyak berenang. Variabel ketiga seperti "musim" disebut **variabel pengganggu**.

Saat menemukan korelasi, tuliskan "berhubungan dengan", bukan "menyebabkan". Membuktikan sebab-akibat butuh eksperimen, seperti uji A/B di bagian berikut.

19.6 Uji hipotesis: apakah perbedaannya nyata?

Rata-rata transaksi kanal Toko sedikit lebih besar daripada Online. Apakah perbedaan itu nyata, atau kebetulan saja?

Uji hipotesis menjawabnya dengan cara berikut:

1. Anggap dulu **tidak ada perbedaan** (hipotesis nol, H_o).
2. Hitung **p-value**: peluang melihat perbedaan sebesar ini bila H_o benar.
3. Bila p-value lebih kecil dari 0,05, perbedaannya dianggap **signifikan**: terlalu besar untuk dianggap kebetulan.

```
const df = await sl.read_csv("toko-nusantara.csv")
const hasil = sl.stats.ttest_ind(df, { value: "total", group: "kanal" })
sl.write(hasil)
```

Uji-t dua sampel independen (Welch)

t	p-value	Cohen's d
1,3361	**0,1816**	**0,051** ▲ sangat kecil

ⓘ p = 0,1816 ≥ 0,05, jadi H_o tidak ditolak. Tidak ada bukti rata-rata Toko berbeda dari Online.

› Rincian

Buka di Playground ↗

Kesimpulannya: tidak ada bukti bahwa rata-rata transaksi kedua kanal berbeda. Perbedaan kecil yang terlihat bisa saja kebetulan.

`sl.stats.ttest_ind` menampilkan statistik uji, p-value, ukuran efek, dan kesimpulan dalam bahasa Indonesia. Uji t ini memakai versi Welch, yang tidak menganggap kedua kelompok punya sebaran sama.

Uji A/B

Uji A/B adalah eksperimen: pengunjung dibagi acak ke dua versi (A dan B), lalu hasilnya dibandingkan. Karena pembagiannya acak, perbedaan hasil bisa dianggap **disebabkan** oleh versinya. Dataset bawaan `ab_test` berisi 4.000 pengunjung halaman checkout.

```
const ab = sl.dataset("ab_test")
const uji = sl.stats.ab_test(ab, { group: "grup", metric: "beli", control: "A" })
sl.write(uji)
```

Uji A/B: Uji-z dua proporsi

z
2,2463

p-value
0,0247

p = 0,0247 < 0,05, jadi H_0 ditolak. B lebih baik daripada A (lift 21,4%).

› Rincian

Buka di Playground ↗

Versi B meningkatkan pembelian sekitar 21%, dan p-value-nya di bawah 0,05. Kini kita boleh mengatakan versi B **menyebabkan** lebih banyak pembelian.

> **API Slak baru**
>
> - `sl.stats.ttest_ind(df, { value, group })` membandingkan rata-rata dua kelompok.
> - `sl.stats.ab_test(df, { group, metric, control })` menganalisis uji A/B. Metrik 0/1 diuji sebagai proporsi, metrik lain sebagai rata-rata.
> - Setiap hasil uji punya `statistic`, `p_value`, `significant`, `conclusion`, dan bisa ditampilkan langsung dengan `sl.write`.

Hasil uji statistik Slak sudah dicocokkan dengan scipy dan statsmodels, pustaka statistik baku di Python.

Kesalahan umum

Gejala	Penyebab	Perbaikan
Simpangan baku berbeda sedikit dengan kalkulator	Pembagi `n` dan `n - 1`	Slak dan pandas memakai `n - 1` untuk sampel
Membuang ratusan "pencilan" yang sebenarnya pelanggan terbaik	Menganggap pencilan pasti salah	Periksa dulu apakah nilainya mungkin terjadi
Menulis "X menyebabkan Y" dari korelasi	Lupa variabel pengganggu	Tulis "berhubungan dengan"

Gejala	Penyebab	Perbaikan
"Tidak signifikan" dibaca "tidak ada perbedaan"	Salah tafsir	Artinya "tidak ada bukti cukup", bukan "pasti sama"

Latihan

Latihan 19.1 (mudah). Hitung `iqr` kolom `total` dengan `sl.stats.quantile`, tanpa memakai `sl.stats.iqr`.

```
const df = await sl.read_csv("toko-nusantara.csv")
const iqr = 0   // ganti

sl.write(iqr)
await sl.testing.test("IQR benar", () =>
sl.testing.expect(iqr).toBe(sl.stats.iqr(df.total)))
```

0

✕ **IQR benar** — Diharapkan: sama persis dengan 86000 — nilai sebenarnya: 0

Buka di Playground ↗

Latihan 19.2 (sedang). Lengkapi fungsi `batasPencilan(x)` dengan JavaScript murni (boleh memakai `sl.stats.quantile`), yang mengembalikan larik `[bawah, atas]` menurut metode IQR.

```
function batasPencilan(x) {
  return [0, 0]
}

const df = await sl.read_csv("toko-nusantara.csv")
await sl.testing.test("batas sama dengan Slak", () =>
sl.testing.expect(batasPencilan(df.total)).toEqual(df.outliers("total").bounds))
```

✕ **batas sama dengan Slak** — Diharapkan: setara dengan [-89000,255000] — nilai sebenarnya: [0,0]

Buka di Playground ↗

Latihan 19.3 (tantangan). Apakah rata-rata transaksi di cabang **Jayapura** berbeda nyata dengan **Jakarta**? Simpan hasil `ttest_ind` di `hasil`, dan simpan jawaban `true` atau `false` di `berbeda`.

```
const df = await sl.read_csv("toko-nusantara.csv")
const hasil = null       // ganti
const berbeda = null     // ganti: true bila signifikan

sl.write(hasil)
await sl.testing.test("hasil uji ada", () => sl.testing.expect(hasil && typeof
hasil.p_value).toBe("number"))
await sl.testing.test("kesimpulan sesuai p-value", () =>
sl.testing.expect(berbeda).toBe(hasil.p_value < 0.05))
```

```
null
```

✕ **hasil uji ada** — Diharapkan: sama persis dengan "number" — nilai sebenarnya: null

✕ **kesimpulan sesuai p-value** — Cannot read properties of null (reading 'p_value')

Buka di Playground ↗

Mini-proyek: profil statistik otomatis

Aplikasi ini membuat profil statistik untuk kolom angka mana pun dari berkas CSV apa pun.

```
sl.title("Profil Statistik :mag:")

const unggah = sl.file_uploader("CSV (kosongkan untuk contoh)", { type: ["csv"] })
const df = unggah ? await sl.read_csv(unggah) : await sl.read_csv("toko-
nusantara.csv")
const kolomAngka = df.select_dtypes("number").columns
const kolomTeks = df.select_dtypes("string").columns

const [a, b] = sl.columns(2)
const pilih = a.selectbox("Kolom angka", kolomAngka, { index:
kolomAngka.indexOf("total") >= 0 ? kolomAngka.indexOf("total") : 0 })
const kelompok = b.selectbox("Bandingkan menurut", ["(tidak ada)", ...kolomTeks])

const x = df[pilih].dropna()
const [q1, med, q3] = sl.stats.quantile(x, [0.25, 0.5, 0.75])
const rata = x.mean()
```

```
const m = df.outliers(pilih)

const [k1, k2, k3, k4] = sl.columns(4)
k1.metric("Rata-rata", Math.round(rata))
k2.metric("Median", med)
k3.metric("Simpangan baku", Math.round(x.std()))
k4.metric("Pencilan", m.values.filter(Boolean).length)

sl.histogram(df, { x: pilih, bins: 30 })
if (kelompok !== "(tidak ada)") sl.box_plot(df, { y: pilih, by: kelompok })

const condong = rata > med * 1.2 ? "condong ke kanan: ada sebagian kecil nilai yang
sangat besar"
  : rata < med * 0.8 ? "condong ke kiri" : "cukup simetris"
sl.info(`Sebaran **${pilih}** ${condong}. Separuh data berada antara
${q1.toLocaleString("id-ID")} dan ${q3.toLocaleString("id-ID")}.`)
```

Profil Statistik 🔍

CSV (kosongkan untuk contoh)

Seret & lepas berkas di sini, atau klik untuk memilih

Jenis: .csv

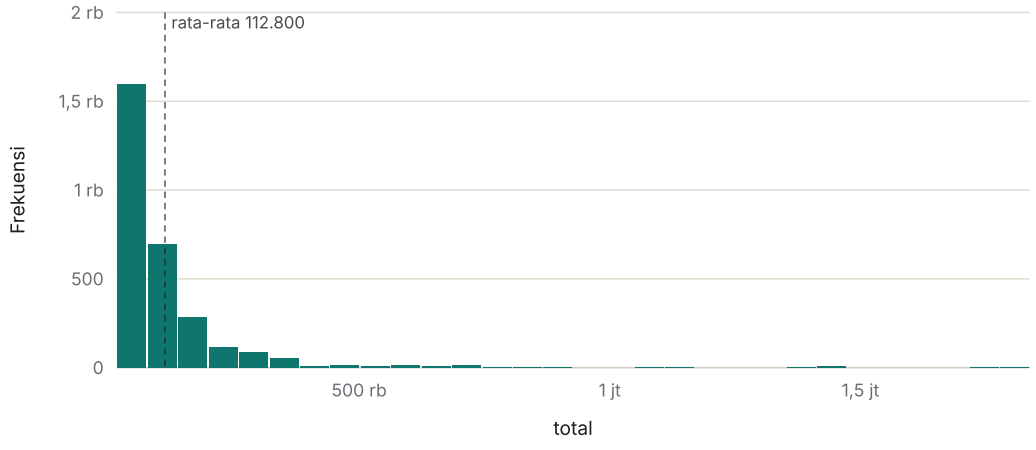
Kolom angka

total

Bandingkan menurut

(tidak ada)

Rata-rata	Median	Simpangan baku	Pencilan
112.774	**72.000**	**149.395**	**235**



Sebaran **total** condong ke kanan: ada sebagian kecil nilai yang sangat besar. Separuh data berada antara 40.000 dan 126.000.

Buka di Playground ↗

Kalimat di akhir ditulis otomatis dari angka-angkanya. Aplikasi yang **menjelaskan** temuannya sendiri jauh lebih berguna daripada tabel angka saja.

← 18. Menggabungkan dan Membentuk Ulang Data

20. Data Deret Waktu →

Bab 20 · Data Deret Waktu

> **Tujuan belajar**
>
> - Mengubah teks menjadi tanggal dan memformatnya.
> - Mengelompokkan per hari, minggu, bulan, kuartal, dan tahun dengan `resample`.
> - Menghitung pertumbuhan bulan ke bulan dan tahun ke tahun.
> - Menghaluskan data dengan rata-rata bergerak.
> - Menguraikan tren dan pola musiman.

Deret waktu adalah data yang diurutkan menurut waktu: omzet harian, pengunjung mingguan, harga bulanan. Hampir semua laporan bisnis memuat deret waktu, karena pertanyaan terpenting manajer adalah "**bagaimana perkembangannya?**"

20.1 Tanggal di Slak Data

`sl.read_csv` mengubah kolom tanggal ISO menjadi `Date` secara otomatis. Untuk format lain, termasuk format Indonesia, pakai `to_datetime`:

```
const tgl = sl.to_datetime(["2024-04-10", "10/04/2024", "10 April 2024"])
sl.write(tgl.map(d => sl.strftime(d, "%A, %d %B %Y")))
```

```
[
  "Rabu, 10 April 2024",
  "Rabu, 10 April 2024",
  "Rabu, 10 April 2024"
]
```

Buka di Playground ↗

Ketiga format menghasilkan tanggal yang sama: Rabu, 10 April 2024.

Membuat deretan tanggal, misalnya untuk memastikan setiap bulan ada:

```
const bulan = sl.date_range("2024-01-01", { end: "2024-06-01", freq: "MS" })
sl.write(bulan.to_list().map(d => sl.strftime(d, "%B %Y")))
```

```
[
  "Januari 2024",
  "Februari 2024",
  "Maret 2024",
  "April 2024",
  "Mei 2024",
  "Juni 2024"
]
```

Buka di Playground ↗

> **API Slak baru**
>
> - `sl.to_datetime(nilai)` mengubah teks atau larik teks menjadi tanggal; `df.to_datetime(kolom)` mengubah satu kolom DataFrame.
> - `sl.date_range(awal, { end, periods, freq })` membuat deretan tanggal. `freq`: `"D"` hari, `"W"` minggu, `"MS"` awal bulan, `"ME"` akhir bulan, `"QS"` awal kuartal, `"YS"` awal tahun.

20.2 resample: mengelompokkan menurut waktu

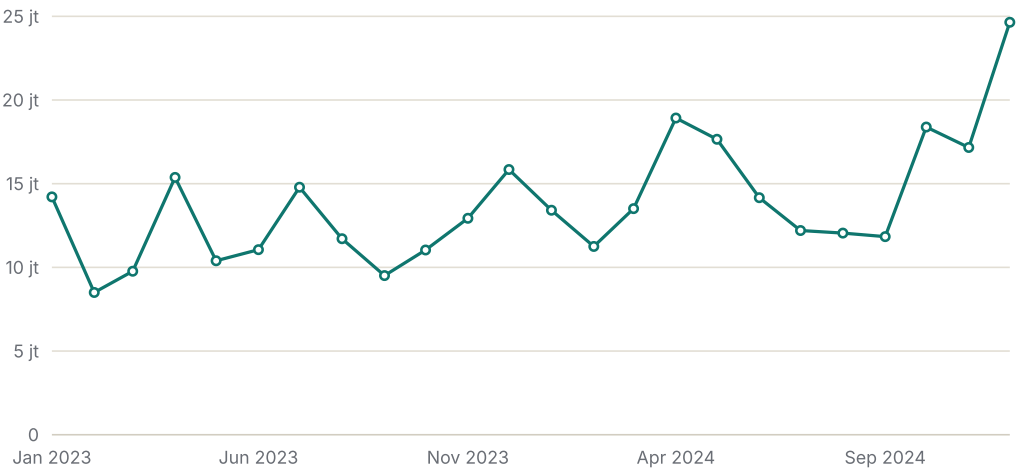
`resample` adalah `groupby` khusus tanggal. Ia juga memastikan periode tanpa data tetap muncul, sehingga grafik tidak melompat.

```
const df = await sl.read_csv("toko-nusantara.csv")
const freq = sl.segmented_control("Periode", ["D", "W", "M", "Q"], { default: "M" })
const per = df.resample("tanggal", freq).agg({ omzet: ["total", "sum"], transaksi:
["id_transaksi", "nunique"] })
sl.write(`${per.shape[0]} periode`)
sl.line_chart(per, { x: "tanggal", y: "omzet", format: "compact" })
```

Periode

D | W | M | Q

24 periode



Buka di Playground ↗

Geser pilihan dari hari (`D`) ke kuartal (`Q`). Data harian penuh gerigi; data bulanan dan kuartalan memperlihatkan polanya.

Kode	Periode
`"D"`	Hari
`"W"`	Minggu (mulai Senin)
`"M"`	Bulan
`"Q"`	Kuartal
`"Y"`	Tahun

> **API Slak baru**
>
> - `df.resample(kolomTanggal, freq).agg({...})` meringkas per periode.
> - `sl.segmented_control(label, pilihan, { default })` menampilkan tombol pilihan yang berdampingan.

20.3 Pertumbuhan

Dua ukuran pertumbuhan yang paling sering diminta:

- **MoM** (*month over month*): dibanding bulan sebelumnya.
- **YoY** (*year over year*): dibanding bulan yang sama tahun lalu.

```
const df = await sl.read_csv("toko-nusantara.csv")
const bulanan = df.resample("tanggal", "M").agg({ omzet: ["total", "sum"] })
const tumbuh = bulanan.assign({
  mom: () => bulanan.omzet.pct_change(),
  yoy: () => bulanan.omzet.pct_change(12)
})
sl.dataframe(tumbuh.tail(6), { format: { omzet: "rupiah", mom: "percent", yoy: "percent" }
})
```

	tanggal	omzet	mom	yoy
0	1 Jul 2024	Rp 12.203.500	-13,8%	-17,5%
1	1 Agu 2024	Rp 12.044.000	-1,3%	2,9%
2	1 Sep 2024	Rp 11.839.500	-1,7%	24,5%
3	1 Okt 2024	Rp 18.380.000	55,2%	66,5%
4	1 Nov 2024	Rp 17.165.000	-6,6%	32,7%
5	1 Des 2024	Rp 24.647.500	43,6%	55,6%

6 baris × 4 kolom

Buka di Playground ↗

`pct_change()` membandingkan setiap baris dengan satu baris sebelumnya; `pct_change(12)` dengan dua belas baris sebelumnya, yaitu bulan yang sama tahun lalu. Tahun pertama tidak punya YoY, sehingga kolomnya kosong.

Mengapa YoY penting? Omzet Desember naik tajam dibanding November setiap tahun karena musim liburan. MoM Desember yang tinggi bukan prestasi, melainkan musim. YoY membandingkan Desember dengan Desember, sehingga pengaruh musim hilang.

Pintasan untuk laporan pertumbuhan:

```
const df = await sl.read_csv("toko-nusantara.csv")
const g = sl.analysis.growth(df, { date: "tanggal", value: "total" })
sl.dataframe(g.tail(4), { format: { total: "rupiah", pertumbuhan: "percent",
pertumbuhan_yoy: "percent", kumulatif: "rupiah" } })
```

	periode	tanggal	total	pertumbuhan	pertumbuhan_yoy	kumulatif
0	2024-09	1 Sep 2024	Rp 11.839.500	-1,7%	24,5%	Rp 270.123.450
1	2024-10	1 Okt 2024	Rp 18.380.000	55,2%	66,5%	Rp 288.503.450
2	2024-11	1 Nov 2024	Rp 17.165.000	-6,6%	32,7%	Rp 305.668.450
3	2024-12	1 Des 2024	Rp 24.647.500	43,6%	55,6%	Rp 330.315.950

4 baris × 6 kolom

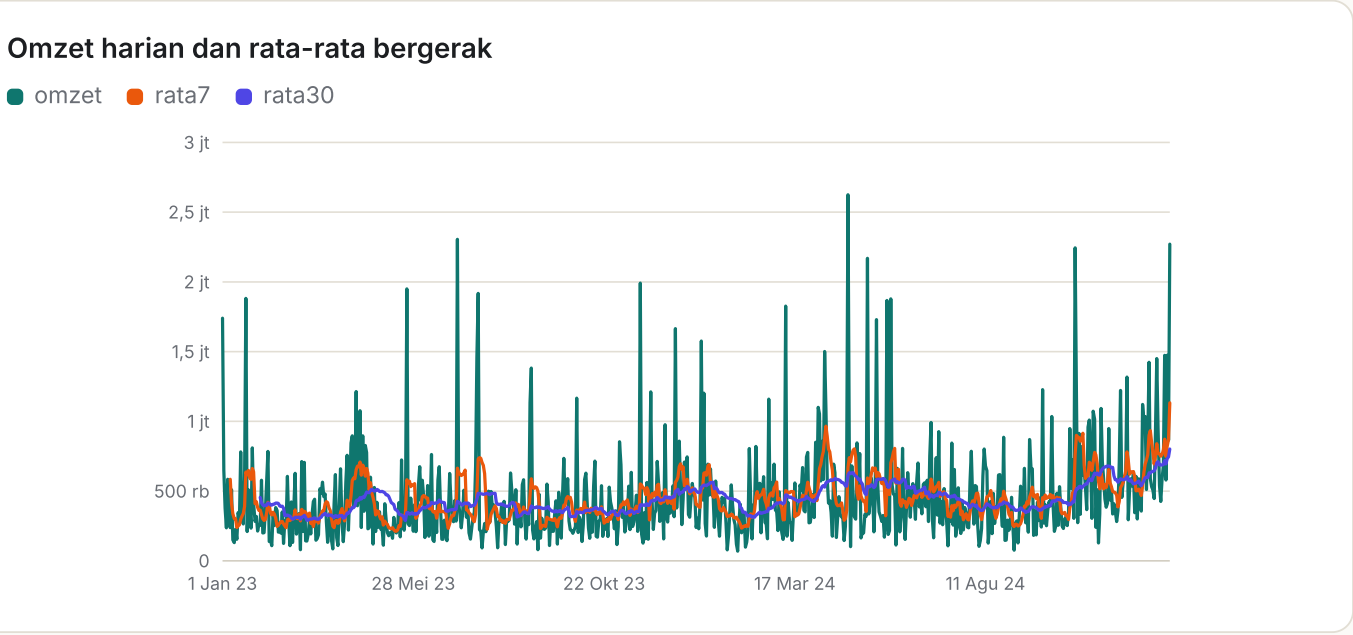
Buka di Playground ↗

> **API Slak baru**
>
> - `series.pct_change(n)` menghitung persen perubahan dibanding `n` baris sebelumnya.
> - `sl.analysis.growth(df, { date, value })` menghasilkan kolom `periode`, `pertumbuhan` (MoM), `pertumbuhan_yoy`, dan `kumulatif`.

20.4 Rata-rata bergerak

Data harian naik turun karena banyak hal kecil: hujan, hari libur, pembeli rombongan. **Rata-rata bergerak** menghaluskannya dengan merata-ratakan beberapa periode terakhir.

```
const df = await sl.read_csv("toko-nusantara.csv")
const harian = df.resample("tanggal", "D").agg({ omzet: ["total", "sum"] })
const halus = harian.assign({
  rata7: () => harian.omzet.rolling(7).mean(),
  rata30: () => harian.omzet.rolling(30).mean()
})
sl.line_chart(halus, { x: "tanggal", y: ["omzet", "rata7", "rata30"], format: "compact",
title: "Omzet harian dan rata-rata bergerak" })
```



Buka di Playground ↗

- `rolling(7).mean()` merata-ratakan 7 hari terakhir: pola mingguan hilang.
- `rolling(30).mean()` merata-ratakan 30 hari: hanya tren besarnya yang tersisa.

Mengirim larik ke opsi `y` menggambar beberapa garis sekaligus.

Cara kerjanya bisa kamu tulis sendiri dengan JavaScript murni:

```
function rataBergerak(x, n) {
  return x.map((_, i) => {
    if (i < n - 1) return null
    const jendela = x.slice(i - n + 1, i + 1)
    return jendela.reduce((a, b) => a + b, 0) / n
  })
}
sl.write(rataBergerak([10, 20, 30, 40, 50], 3))
```

```
[
  null,
  null,
  20,
  30,
  40
]
```

Buka di Playground ↗

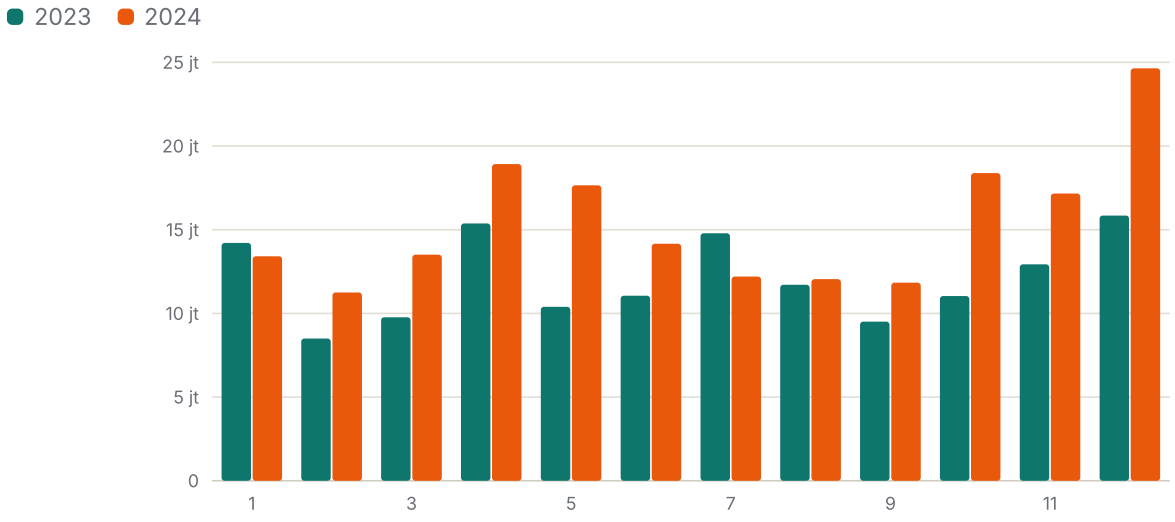
> **API Slak baru**
>
> - `series.rolling(n).mean()` (juga `.sum()`, `.max()`) menghitung nilai bergerak.
> - `series.cumsum()` menjumlahkan terus-menerus; `series.shift(n)` menggeser nilai `n` baris; `series.diff()` selisih dengan baris sebelumnya.

20.5 Musim: Lebaran dan akhir tahun

Toko oleh-oleh punya dua musim ramai: menjelang **Lebaran** dan **akhir tahun**. Lebaran bergeser sekitar 11 hari lebih awal setiap tahun (22 April 2023, 10 April 2024), jadi puncaknya pun bergeser.

```
const df = await sl.read_csv("toko-nusantara.csv")
const t = df.assign({ tahun: () => df.tanggal.dt.year.astype("str"), bulan: () =>
df.tanggal.dt.month })
const perBulan = t.groupby(["tahun", "bulan"]).agg({ omzet: ["total", "sum"] })
sl.bar_chart(perBulan, { x: "bulan", y: "omzet", color: "tahun", format: "compact", title:
"Dua puncak setiap tahun: Lebaran dan Desember" })
```

Dua puncak setiap tahun: Lebaran dan Desember



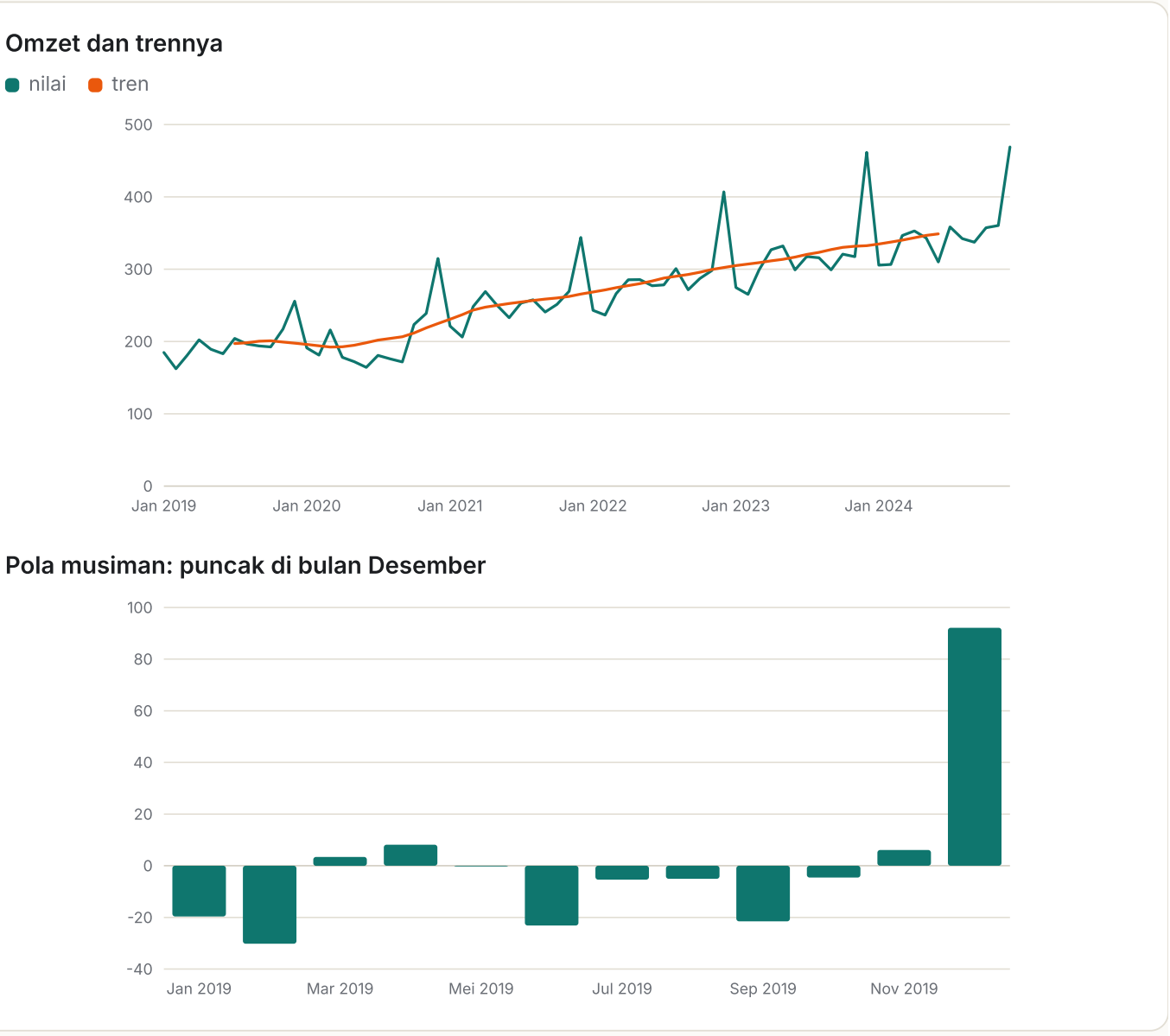
Buka di Playground ↗

Menempatkan dua tahun berdampingan per bulan memperlihatkan pola yang berulang.

Menguraikan tren dan musim

Dekomposisi memisahkan deret waktu menjadi tiga bagian: **tren** (arah jangka panjang), **musiman** (pola yang berulang setiap periode), dan **residu** (sisa yang tidak bisa dijelaskan). Butuh beberapa tahun data supaya polanya terlihat jelas. Dataset bawaan `penjualan_bulanan` berisi omzet bulanan selama enam tahun:

```
const pb = sl.dataset("penjualan_bulanan")
const d = sl.ts.decompose(pb, { period: 12 })
sl.line_chart(d, { x: "bulan", y: ["nilai", "tren"], title: "Omzet dan trennya" })
sl.bar_chart(d.head(12), { x: "bulan", y: "musiman", title: "Pola musiman: puncak di bulan Desember" })
```



Buka di Playground ↗

> **API Slak baru**
>
> - `sl.ts.decompose(df, { period })` menghasilkan kolom `nilai`, `tren`, `musiman`, dan `residu`. `period: 12` untuk data bulanan dengan pola tahunan, `7` untuk data harian dengan pola mingguan.

> **Coba di playground** Ganti `rolling(30)` menjadi `rolling(90)`. Apa yang terjadi pada garisnya, dan berapa hari pertama yang kosong?

Kesalahan umum

Gejala	Penyebab	Perbaikan
Grafik waktu melompat-lompat	Data belum diurutkan atau ada periode kosong	Pakai `resample`, yang mengurutkan dan mengisi periode kosong
Pertumbuhan Desember selalu tampak luar biasa	Membandingkan dengan November (MoM)	Lihat juga YoY
Bulan terakhir tampak anjlok	Bulannya belum selesai	Buang periode yang belum lengkap, atau beri catatan
`rolling` menghasilkan kosong di awal	Belum cukup periode untuk jendela	Wajar; jendela 7 butuh 7 data

Latihan

Latihan 20.1 (mudah). Buat `kuartalan`: omzet per kuartal, dengan kolom `omzet`.

```
const df = await sl.read_csv("toko-nusantara.csv")
const kuartalan = sl.DataFrame([])   // ganti

sl.dataframe(kuartalan)
await sl.testing.test("8 kuartal", () => sl.testing.expect(kuartalan.shape[0]).toBe(8))
await sl.testing.test("jumlahnya sama dengan total omzet", () =>
sl.testing.expect(kuartalan.omzet.sum()).toBe(df.total.sum()))
```

0 baris × 0 kolom

✗ **8 kuartal** — Diharapkan: sama persis dengan 8 — nilai sebenarnya: 0

✗ **jumlahnya sama dengan total omzet** — Cannot read properties of undefined (reading 'sum')

Buka di Playground ↗

Latihan 20.2 (sedang). Lengkapi `pertumbuhan(x)` dengan JavaScript murni: untuk larik angka `x`, kembalikan larik persen perubahan dibanding nilai sebelumnya (angka 0 sampai 1), dengan isi pertama `null`.

```
function pertumbuhan(x) {
  return []
}

await sl.testing.test("[100, 110, 99]", () => {
  const p = pertumbuhan([100, 110, 99])
  sl.testing.expect(p[0]).toBeNull()
  sl.testing.expect(p[1]).toBeCloseTo(0.1, 6)
  sl.testing.expect(p[2]).toBeCloseTo(-0.1, 6)
})
```

✗ **[100, 110, 99]** — Diharapkan: mendekati 0.1 (6 desimal) — nilai sebenarnya: undefined

Buka di Playground ↗

Latihan 20.3 (tantangan). Cari bulan dengan omzet tertinggi di setiap tahun. Simpan sebagai objek `puncak`, misalnya `{ 2023: 12, 2024: 12 }`, dengan bulan berupa angka 1–12.

```
const df = await sl.read_csv("toko-nusantara.csv")
const puncak = {}   // isi

sl.json(puncak)
await sl.testing.test("ada dua tahun", () =>
sl.testing.expect(Object.keys(puncak).length).toBe(2))
await sl.testing.test("puncak 2024 benar", () => {
  const b = df.filter(r => r.tanggal.getFullYear() === 2024).resample("tanggal", "M").agg({
omzet: ["total", "sum"] }).to_records()
  const maks = b.reduce((a, r) => r.omzet > a.omzet ? r : a)
  sl.testing.expect(puncak[2024]).toBe(maks.tanggal.getMonth() + 1)
})
```

```
{}
```

✗ **ada dua tahun** — Diharapkan: sama persis dengan 2 — nilai sebenarnya: 0

✗ **puncak 2024 benar** — Diharapkan: sama persis dengan 12 — nilai sebenarnya: undefined

Buka di Playground ↗

Mini-proyek: tren penjualan per cabang

```
sl.title("Tren Penjualan Cabang :calendar:")

const df = await sl.read_csv("toko-nusantara.csv")
const cabang = sl.selectbox("Cabang", ["Semua", ...df.cabang.unique().sort()])
const data = cabang === "Semua" ? df : df.filter(r => r.cabang === cabang)

const bulanan = data.resample("tanggal", "M").agg({ omzet: ["total", "sum"] })
const lengkap = bulanan.assign({
  tren: () => bulanan.omzet.rolling(3).mean(),
  yoy: () => bulanan.omzet.pct_change(12)
})

const akhir = lengkap.to_records().at(-1)
const persen = new Intl.NumberFormat("id-ID", { style: "percent", maximumFractionDigits: 1,
signDisplay: "always" })
const [a, b] = sl.columns(2)
a.metric("Omzet Desember 2024", akhir.omzet, akhir.yoy != null ? persen.format(akhir.yoy) +
" YoY" : null, { format: "rp" })
const tahunan = data.assign({ th: () => data.tanggal.dt.year }).groupby("th").agg({ omzet:
["total", "sum"] }).to_records()
b.metric("Pertumbuhan 2024", persen.format(tahunan[1].omzet / tahunan[0].omzet - 1))

sl.line_chart(lengkap, { x: "tanggal", y: ["omzet", "tren"], format: "compact" })

const yoy = lengkap.filter(r => r.yoy != null)
const turun = yoy.filter(r => r.yoy < 0).shape[0]
sl.write(`Pada 2024, **${12 - turun} dari 12 bulan** lebih tinggi daripada bulan yang sama
di 2023.`)
```

Tren Penjualan Cabang 📅

Cabang

Semua

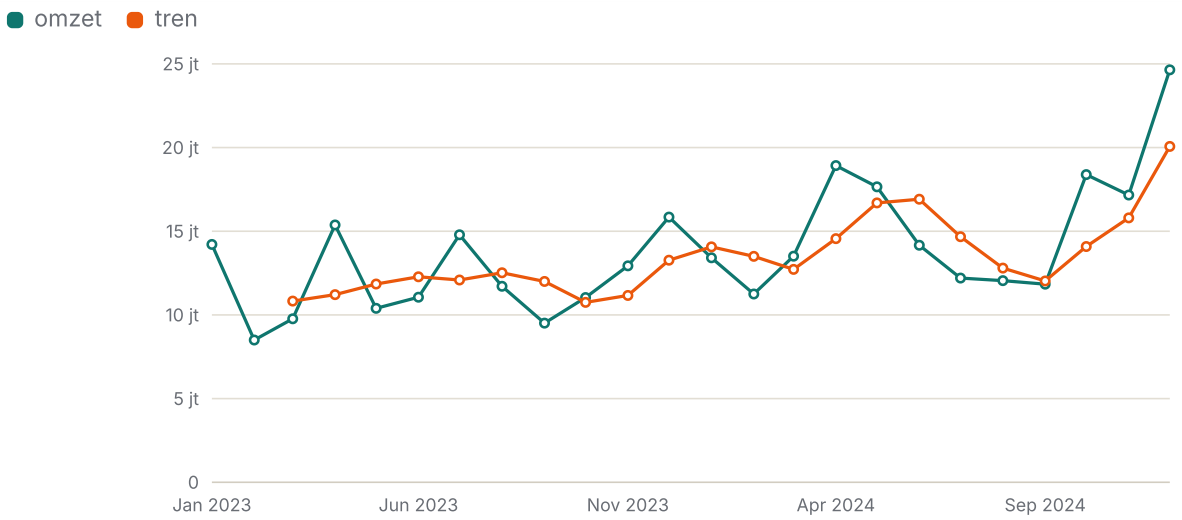
Omzet Desember 2024

Rp 24.647.500

▲ +55,6% YoY

Pertumbuhan 2024

+27,6%



Pada 2024, **10 dari 12 bulan** lebih tinggi daripada bulan yang sama di 2023.

Buka di Playground ↗

`larik.at(-1)` mengambil isi terakhir larik; lebih ringkas daripada `larik[larik.length - 1]`.

← 19. Statistik Deskriptif Lanjutan

21. SQL di Browser →

Bab 21 · SQL di Browser

> **Tujuan belajar**
>
> - Memahami mengapa analis data wajib bisa SQL.
> - Menulis `SELECT`, `WHERE`, `GROUP BY`, `ORDER BY`, dan `JOIN`.
> - Memakai CTE (`WITH`) dan window function untuk peringkat dan total berjalan.
> - Menulis SQL yang juga jalan di PostgreSQL.
> - Mengolah data besar dan berkas Parquet dengan DuckDB.
> - Menyimpan data di browser dengan `sl.sqlite` dan `sl.db`.
> - Membuat laporan dari SQL dengan `sl.report`.

21.1 Mengapa SQL?

SQL (*Structured Query Language*) adalah bahasa untuk bertanya kepada database. Hampir setiap lowongan analis data mencantumkan SQL, karena data perusahaan tersimpan di database seperti PostgreSQL, MySQL, atau BigQuery. Kabar baiknya: SQL jauh lebih sederhana daripada JavaScript, dan cara berpikirnya sudah kamu kuasai di Bab 14–18.

Yang sudah kamu pelajari	Di SQL
`select([...])`	`SELECT kolom1, kolom2`
`filter(...)` / `query(...)`	`WHERE syarat`
`groupby(...).agg(...)`	`GROUP BY ...` dengan `SUM`, `COUNT`, `AVG`
`sort_values(...)`	`ORDER BY ... DESC`
`head(10)`	`LIMIT 10`
`merge(...)`	`JOIN ... ON ...`

Slak menyertakan **SQLite** sungguhan, database yang sama yang ada di setiap ponsel Android dan iPhone. Mesinnya ada di dalam `slak-sql.js`, sehingga SQL tetap jalan tanpa server dan tanpa internet.

```
<script src="slak.js"></script>
<script src="slak-data.js"></script>
<script src="slak-sql.js"></script>
```

21.2 Kueri pertama

```
const df = await sl.read_csv("toko-nusantara.csv")
const hasil = await sl.sql(`
  SELECT kota, COUNT(*) AS transaksi, SUM(total) AS omzet
  FROM jual
  GROUP BY kota
  ORDER BY omzet DESC
`, { jual: df })
sl.dataframe(hasil, { format: { omzet: "rupiah" } })
```

	kota	transaksi	omzet
0	Surabaya	387	Rp 44.400.600
1	Jayapura	340	Rp 41.290.150
2	Jakarta	384	Rp 39.196.450
3	Makassar	350	Rp 35.906.200
4	Ambon	301	Rp 33.681.000
5	Denpasar	273	Rp 33.258.200
6	Manado	245	Rp 30.546.500
7	Sorong	246	Rp 25.393.100
8	Manokwari	201	Rp 24.455.250
9	Kupang	202	Rp 22.188.500

10 baris × 3 kolom

Buka di Playground ↗

- Argumen kedua memberi **nama tabel** untuk setiap DataFrame: `{ jual: df }` membuat tabel bernama `jual`.
- Hasilnya DataFrame, jadi bisa langsung ditampilkan, digambar, atau diolah lagi.
- Kueri ditulis di antara backtick supaya boleh beberapa baris.

> **API Slak baru**
>
> - `await sl.sql(kueri, { nama: df }, { params })` menjalankan SQL atas DataFrame atau larik objek. Butuh `slak-sql.js`.

Bagian-bagian SELECT

```
const df = await sl.read_csv("toko-nusantara.csv")
const hasil = await sl.sql(`
  SELECT produk,
         SUM(jumlah)           AS terjual,
         SUM(total)            AS omzet,
         ROUND(AVG(total), 0)  AS rata_transaksi
  FROM jual
  WHERE kategori = 'Minuman'
    AND tanggal >= '2024-01-01'
  GROUP BY produk
  HAVING SUM(jumlah) > 50
  ORDER BY omzet DESC
`, { jual: df })
sl.dataframe(hasil)
```

	produk	terjual	omzet	rata_transaksi
0	Kopi Arabika Wamena 250g	231	20.514.500	162.813
1	Kopi Robusta Toraja 250g	212	15.975.800	124.811
2	Sirup Pala 500ml	116	5.832.750	79.901
3	Teh Kayu Manis 100g	140	5.176.300	59.498

4 baris × 4 kolom

Buka di Playground ↗

Urutan **penulisan**: `SELECT` → `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `ORDER BY` → `LIMIT`.

Urutan **pengerjaannya** berbeda: ambil tabel (`FROM`), saring baris (`WHERE`), kelompokkan (`GROUP BY`), saring kelompok (`HAVING`), pilih kolom (`SELECT`), lalu urutkan (`ORDER BY`). Karena itu `WHERE` tidak bisa memakai hasil `SUM`; untuk menyaring hasil ringkasan, pakai `HAVING`.

Teks di SQL diapit **tanda kutip tunggal**: `'Minuman'`. Tanggal dikirim ke SQLite sebagai teks `YYYY-MM-DD`, sehingga perbandingan `tanggal >= '2024-01-01'` langsung bekerja.

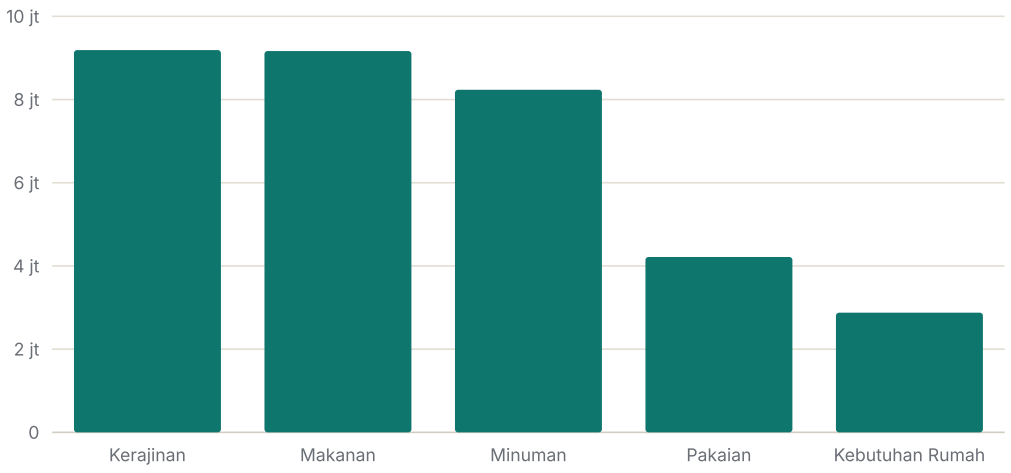
21.3 Parameter dari widget

Jangan pernah menyambung isian pengguna langsung ke teks SQL. Pakai **parameter**:

```
const df = await sl.read_csv("toko-nusantara.csv")
const kota = sl.selectbox("Kota", df.kota.unique().sort())
const hasil = await sl.sql(
  "SELECT kategori, SUM(total) AS omzet FROM jual WHERE kota = :kota GROUP BY
kategori ORDER BY omzet DESC",
  { jual: df }, { params: { kota } })
sl.bar_chart(hasil, { x: "kategori", y: "omzet", format: "compact" })
```

Kota

Ambon



Buka di Playground ↗

`:kota` di kueri diisi dari `params`. Parameter mencegah kesalahan kutip (misalnya nama "Jum'at") dan serangan yang disebut *SQL injection*. Kebiasaan ini wajib di tempat kerja.

21.4 JOIN

```
const [trx, produk] = await Promise.all([sl.read_csv("toko-nusantara.csv"),
sl.read_csv("produk.csv")])
const hasil = await sl.sql(`
  SELECT p.pemasok,
         SUM(t.total)                          AS omzet,
         SUM(t.total - t.jumlah * p.harga_pokok) AS laba
  FROM jual AS t
  LEFT JOIN produk AS p ON p.produk = t.produk
  GROUP BY p.pemasok
  ORDER BY laba DESC
`, { jual: trx, produk })
sl.dataframe(hasil, { format: { omzet: "rupiah", laba: "rupiah" } })
```

	pemasok	omzet	laba
0	Sanggar Noken Abepura	Rp 61.909.750	Rp 27.121.750
1	Koperasi Timor Lestari	Rp 53.400.300	Rp 19.148.300
2	UD Rempah Maluku	Rp 43.177.350	Rp 18.877.350
3	UKM Wangi Bali	Rp 32.131.900	Rp 15.883.900
4	CV Bitung Bahari	Rp 34.830.200	Rp 15.201.200
5	Koperasi Baliem	Rp 36.749.500	Rp 14.753.500
6	CV Tana Toraja	Rp 26.970.200	Rp 10.545.200
7	Konveksi Jayapura	Rp 21.854.800	Rp 9.814.800
8	UKM Sagu Sentani	Rp 19.291.950	Rp 9.546.950

9 baris × 3 kolom

Buka di Playground ↗

Hasilnya sama dengan `merge` di Bab 18.3. `AS t` dan `AS p` memberi nama pendek (alias) untuk tabel, supaya kolom bisa ditulis `t.total` dan `p.harga_pokok` tanpa tertukar.

21.5 CTE dan window function

CTE (*Common Table Expression*) memecah kueri panjang menjadi langkah-langkah bernama dengan `WITH`. Mirip menyimpan hasil antara di variabel.

Window function menghitung sesuatu **di sekitar setiap baris** tanpa meringkasnya menjadi satu: peringkat, total berjalan, nilai bulan sebelumnya.

```
const df = await sl.read_csv("toko-nusantara.csv")
const hasil = await sl.sql(`
  WITH bulanan AS (
    SELECT to_char(tanggal, 'YYYY-MM') AS bulan, kota, SUM(total) AS omzet
    FROM jual
    GROUP BY 1, 2
  )
  SELECT bulan, kota, omzet,
         RANK() OVER (PARTITION BY bulan ORDER BY omzet DESC)           AS
peringkat,
         omzet - LAG(omzet) OVER (PARTITION BY kota ORDER BY bulan)       AS
selisih,
         SUM(omzet) OVER (PARTITION BY kota ORDER BY bulan)               AS
kumulatif
  FROM bulanan
  WHERE kota IN ('Jayapura', 'Sorong')
  ORDER BY kota, bulan
`, { jual: df })
sl.dataframe(hasil.head(8), { format: { omzet: "rupiah", selisih: "rupiah",
kumulatif: "rupiah" } })
```

	bulan	kota	omzet	peringkat	selisih	kumulatif
0	2023-01	Jayapura	Rp 1.363.000	1	—	Rp 1.363.000
1	2023-02	Jayapura	Rp 674.000	2	Rp -689.000	Rp 2.037.000
2	2023-03	Jayapura	Rp 1.066.000	2	Rp 392.000	Rp 3.103.000
3	2023-04	Jayapura	Rp 2.167.600	1	Rp 1.101.600	Rp 5.270.600
4	2023-05	Jayapura	Rp 2.309.000	1	Rp 141.400	Rp 7.579.600
5	2023-06	Jayapura	Rp 570.000	2	Rp -1.739.000	Rp 8.149.600
6	2023-07	Jayapura	Rp 2.098.000	1	Rp 1.528.000	Rp 10.247.600
7	2023-08	Jayapura	Rp 1.870.000	1	Rp -228.000	Rp 12.117.600

8 baris × 6 kolom

Buka di Playground ↗

Fungsi jendela	Arti
`RANK() OVER (PARTITION BY a ORDER BY b DESC)`	Peringkat `b` di dalam setiap kelompok `a`
`LAG(x) OVER (PARTITION BY a ORDER BY t)`	Nilai `x` pada baris sebelumnya di kelompok yang sama
`SUM(x) OVER (PARTITION BY a ORDER BY t)`	Total berjalan
`ROW_NUMBER()`, `LEAD(x)`, `AVG(x) OVER (...)`	Nomor urut, nilai sesudahnya, rata-rata bergerak

`PARTITION BY` membagi baris ke dalam jendela-jendela (mirip `groupby`), dan `ORDER BY` di dalam `OVER` menentukan urutan di setiap jendela.

21.6 SQL yang juga jalan di PostgreSQL

Setiap database punya logat SQL sendiri. Kueri di bab ini sengaja ditulis dengan gaya yang jalan di **SQLite dan PostgreSQL** sekaligus, karena PostgreSQL paling sering dipakai di tempat kerja.

Hindari (khusus SQLite)	Pakai (jalan di keduanya)
`strftime('%Y-%m', tanggal)`	`to_char(tanggal, 'YYYY-MM')`
`strftime('%Y', tanggal)`	`EXTRACT(YEAR FROM tanggal)`
	`date_trunc('month', tanggal)`
`IFNULL(x, 0)`	`COALESCE(x, 0)`
`GROUP_CONCAT(x)`	`string_agg(x, ', ')`

Slak bisa memeriksanya untukmu:

```
sl.dataframe(sl.sql.check("SELECT strftime('%Y', tanggal), IFNULL(kanal, '-') FROM jual"))
```

	masalah	gunakan
0	strftime()	date_trunc('month', tgl), to_char(tgl, 'YYYY-MM'), atau EXTRACT(YEA…
1	IFNULL()	COALESCE()

2 baris × 2 kolom

Buka di Playground ↗

> **API Slak baru**

> - `sl.sql.check(kueri)` mendaftar sintaks khusus SQLite beserta penggantinya yang portabel.
> - Opsi `{ portable: true }` pada `sl.sql` memberi peringatan langsung saat kueri memakai sintaks khusus SQLite.

21.7 Data besar dengan DuckDB

SQLite cocok untuk data harian. Untuk ratusan ribu sampai jutaan baris, atau berkas **Parquet**, Slak menyediakan **DuckDB**, database analitik yang dirancang untuk meringkas data besar dengan cepat. DuckDB ada di berkas tambahan:

```
<script src="slak.js"></script>
<script src="slak-data.js"></script>
<script src="slak-duckdb.js"></script>
```

> **Butuh internet:** mesin DuckDB (sekitar 35 MB) diunduh sekali saat pertama dipakai, lalu disimpan browser. Di Slak Studio, DuckDB tersedia tanpa internet. Playground tidak memuat `slak-duckdb.js`, jadi contoh DuckDB dijalankan di halaman buku ini atau di Slak Studio.

```
if (sl.button("Rangkum 250 ribu transaksi dengan DuckDB")) {
  const t0 = performance.now()
  const hasil = await sl.spinner("DuckDB bekerja…", () => sl.duckdb(`
    SELECT kanal, EXTRACT(YEAR FROM CAST(tanggal AS DATE)) AS tahun,
           COUNT(*) AS transaksi, SUM(total) AS omzet
    FROM 'toko-nusantara-besar.csv'
    GROUP BY 1, 2
    ORDER BY 2, 1
  `))
  sl.success(`Selesai dalam ${((performance.now() - t0) / 1000).toFixed(1)} detik`)
  sl.dataframe(hasil, { format: { omzet: "compact" } })
}
```

Rangkum 250 ribu transaksi dengan DuckDB

Buka di Playground ↗

DuckDB bisa membaca berkas CSV dan Parquet langsung dari kueri, cukup dengan menyebut namanya dalam tanda kutip tunggal. Berkasnya tidak perlu dimuat ke DataFrame lebih dulu, sehingga data yang terlalu besar untuk Slak Data pun masih bisa diringkas.

DuckDB juga menerima DataFrame, sama seperti `sl.sql`:

```
const df = await sl.read_csv("toko-nusantara.csv")
const r = await sl.duckdb("SELECT kota, SUM(total) AS omzet FROM jual GROUP BY kota
ORDER BY omzet DESC LIMIT 5", { jual: df })
sl.dataframe(r)
```

	kota	omzet
0	Surabaya	44.400.600
1	Jayapura	41.290.150
2	Jakarta	39.196.450
3	Makassar	35.906.200
4	Ambon	33.681.000

5 baris × 2 kolom

Buka di Playground ↗

Parquet

Parquet menyimpan data per kolom dan terkompresi, biasanya 5 sampai 10 kali lebih kecil daripada CSV dan jauh lebih cepat dibaca. Banyak dataset terbuka kini tersedia dalam Parquet.

```
const df = await sl.read_csv("toko-nusantara.csv")
const pq = await sl.duckdb.to_file("SELECT * FROM jual", "toko-nusantara.parquet",
{ jual: df })
sl.download_button("Unduh sebagai Parquet", pq, { file_name: "toko-
nusantara.parquet" })
```

Unduh sebagai Parquet

Buka di Playground ↗

Berkas Parquet dibaca kembali dengan `await sl.read_parquet("toko-nusantara.parquet")`.

> **API Slak baru** (dari `slak-duckdb.js`)
>
> - `await sl.duckdb(kueri, { nama: df })` menjalankan SQL dengan DuckDB.
> - `await sl.read_parquet(nama)` membaca berkas Parquet menjadi DataFrame.

> - `await sl.duckdb.to_file(kueri, namaBerkas, tabel)` menyimpan hasil kueri sebagai Parquet untuk diunduh.

Kapan SQLite, kapan DuckDB?

	`sl.sql` / `sl.sqlite` **(SQLite)**	`sl.duckdb` **(DuckDB)**
Cocok untuk	Data kecil sampai menengah, tabel yang sering diubah	Analisis data besar, berkas Parquet
Tanpa internet	Selalu	Di Slak Studio; di halaman lain diunduh sekali
Gaya SQL	SQLite ditambah sebagian gaya PostgreSQL	Sangat mirip PostgreSQL

Aturan sederhananya: **mulai dengan SQLite.** Pindah ke DuckDB bila data lebih dari sekitar 500 ribu baris, bila kamu memakai Parquet, atau bila ringkasan terasa lambat.

21.8 Menyimpan data di browser

Sejauh ini SQL hanya membaca DataFrame. Untuk data yang ditambah dan diubah pengguna, misalnya catatan stok, simpan di database yang tinggal di browser.

SQLite yang tersimpan:

```
const db = await sl.sqlite({ name: "stok-toko" })
db.query("CREATE TABLE IF NOT EXISTS stok (produk TEXT PRIMARY KEY, jumlah
INTEGER)")

const f = sl.form("tambah-stok", { clear_on_submit: true })
const produk = f.text_input("Produk")
const jumlah = f.number_input("Jumlah", { min: 0, value: 1 })
if (f.form_submit_button("Simpan") && produk) {
  db.query("INSERT INTO stok (produk, jumlah) VALUES (?, ?) ON CONFLICT(produk) DO
UPDATE SET jumlah = excluded.jumlah", [produk, jumlah])
}
sl.dataframe(db.query("SELECT * FROM stok ORDER BY produk"))
```

Produk

Jumlah

1 − +

Simpan

produk	jumlah

0 baris × 2 kolom

Buka di Playground ↗

IndexedDB lewat `sl.db`, tanpa SQL sama sekali:

```
const db = sl.db("buku-kunjungan", { kunjungan: "++id, kota" })
if (sl.button("Catat kunjungan dari Ambon")) await db.kunjungan.insert({ kota:
"Ambon", waktu: new Date().toISOString() })
sl.metric("Kunjungan dari Ambon", await
db.kunjungan.where("kota").equals("Ambon").count())
sl.write(await sl.db_list())
```

Catat kunjungan dari Ambon

Kunjungan dari Ambon

0

```
[
  "buku-kunjungan",
  "slak-sqlite",
  "slak-unduhan"
]
```

Buka di Playground ↗

> **API Slak baru**

- `await sl.sqlite({ name })` membuka database SQLite yang tersimpan permanen di browser. `db.query(sql, params)` menjalankan SQL; `SELECT` mengembalikan DataFrame.
- `sl.db(nama, skema)` membuka database IndexedDB dengan tabel dan perintah `insert`, `update`, `delete`, `where`, `all`, `count`.
- `await sl.db_list()` mendaftar database `sl.db` yang ada di browser ini.

Seperti `local_state` (Bab 10), kedua database ini tinggal **di browser setiap pengunjung**, tidak dibagi antar-pengunjung.

21.9 Laporan dari SQL

`sl.report` menulis laporan dalam Markdown dengan blok SQL di dalamnya. Blok SQL dijalankan lebih dulu, lalu hasilnya disisipkan ke teks atau digambar sebagai grafik.

```
const df = await sl.read_csv("toko-nusantara.csv")
await sl.report(`
# Laporan Singkat Toko Nusantara

Selama 2023–2024, omzet mencapai **{{ ringkas.omzet | rp }}** dari {{
ringkas.transaksi }} transaksi.
Cabang teratas adalah **{{ teratas.kota }}**.

\`\`\`sql name=ringkas hide
SELECT SUM(total) AS omzet, COUNT(*) AS transaksi FROM jual
\`\`\`

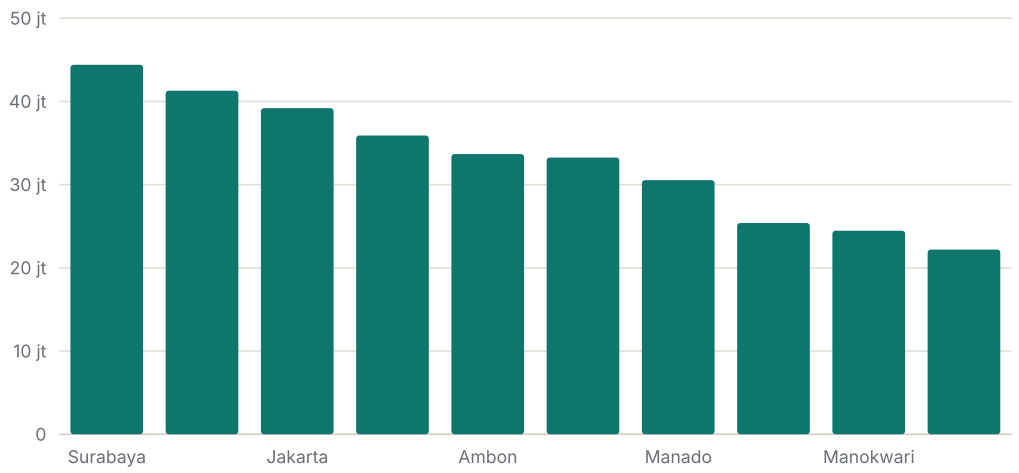
\`\`\`sql name=teratas hide
SELECT kota FROM jual GROUP BY kota ORDER BY SUM(total) DESC LIMIT 1
\`\`\`

\`\`\`sql chart=bar x=kota y=omzet format=rp title="Omzet per kota"
SELECT kota, SUM(total) AS omzet FROM jual GROUP BY kota ORDER BY omzet DESC
\`\`\`
`, { tables: { jual: df } })
```

Laporan Singkat Toko Nusantara

Selama 2023–2024, omzet mencapai **Rp330.315.950** dari 2.929 transaksi. Cabang teratas adalah **Surabaya**.

Omzet per kota



Buka di Playground ↗

Opsi blok `sql`	Fungsi
`name=nama`	Memberi nama hasil, supaya bisa dipakai di teks dengan `{{ nama.kolom }}`
`hide`	Tidak menampilkan tabelnya
`chart=bar` (`line`, `area`, `scatter`, `pie`)	Menggambar grafik
`x=`, `y=`, `format=`, `title=`	Opsi grafik

Di dalam teks, `{{ nama.kolom | rp }}` diisi nilai baris pertama hasil kueri `nama`, dengan format `rp`, `persen`, angka desimal `0`–`9`, atau `tanggal`.

> **API Slak baru**
>
> - `await sl.report(teks, { tables, db, params })` membuat laporan dari Markdown dan SQL.

> **Coba di playground** Di contoh 21.5, ganti `RANK()` dengan `ROW_NUMBER()`. Apa bedanya bila dua kota punya omzet yang sama?

Kesalahan umum

Gejala	Penyebab	Perbaikan
`no such column: Minuman`	Teks ditulis dengan kutip ganda	Teks memakai kutip tunggal: `'Minuman'`
`misuse of aggregate`	Memakai `SUM` di `WHERE`	Pindahkan ke `HAVING`
`no such table`	Nama tabel di kueri tidak sama dengan kunci objek kedua	`FROM jual` butuh `{ jual: df }`

Gejala	Penyebab	Perbaikan
Kueri lambat pada tabel lebar	`SELECT *` memuat semua kolom	Sebutkan kolom yang dibutuhkan saja
`sl.duckdb is not a function`	`slak-duckdb.js` belum dimuat	Tambahkan `<script src="slak-duckdb.js">`

Latihan

Latihan 21.1 (mudah). Tulis kueri yang menghitung omzet per kanal, dengan kolom `kanal` dan `omzet`, diurutkan dari terbesar.

```
const df = await sl.read_csv("toko-nusantara.csv")
const hasil = await sl.sql(`SELECT 1 AS ganti_saya`, { jual: df })

sl.dataframe(hasil)
const r = hasil.to_records()
await sl.testing.test("dua kanal", () => sl.testing.expect(r).toHaveLength(2))
await sl.testing.test("Toko teratas", () =>
sl.testing.expect(r[0].kanal).toBe("Toko"))
await sl.testing.test("omzet benar", () => sl.testing.expect(r[0].omzet +
r[1].omzet).toBe(df.total.sum()))
```

	ganti_saya
0	1

1 baris × 1 kolom

✕ **dua kanal** — Diharapkan: panjangnya 2 — nilai sebenarnya: [{"ganti_saya":1}]

✕ **Toko teratas** — Diharapkan: sama persis dengan "Toko" — nilai sebenarnya: undefined

✕ **omzet benar** — Cannot read properties of undefined (reading 'omzet')

Buka di Playground ↗

Latihan 21.2 (sedang). Cari produk terlaris (berdasarkan jumlah terjual) di setiap kota. Hasilnya satu baris per kota dengan kolom `kota`, `produk`, dan `terjual`. Petunjuk: CTE dan `RANK()`.

```
const df = await sl.read_csv("toko-nusantara.csv")
const hasil = await sl.sql(`SELECT 1 AS ganti_saya`, { jual: df })

sl.dataframe(hasil)
await sl.testing.test("sepuluh kota", () =>
sl.testing.expect(hasil.shape[0]).toBe(10))
await sl.testing.test("kolom benar", () =>
sl.testing.expect(hasil.columns).toEqual(["kota", "produk", "terjual"]))
```

	ganti_saya
0	1

1 baris × 1 kolom

✕ **sepuluh kota** — Diharapkan: sama persis dengan 10 — nilai sebenarnya: 1

✕ **kolom benar** — Diharapkan: setara dengan ["kota","produk","terjual"] — nilai sebenarnya: ["ganti_saya"]

Buka di Playground ↗

Latihan 21.3 (tantangan). Hitung omzet bulanan 2024 beserta total berjalannya (`kumulatif`). Kueri harus portabel: `sl.sql.check` tidak boleh menemukan masalah.

```
const df = await sl.read_csv("toko-nusantara.csv")
const kueri = `SELECT 1 AS ganti_saya`
const hasil = await sl.sql(kueri, { jual: df })

sl.dataframe(hasil)
const r = hasil.to_records()
await sl.testing.test("12 bulan", () => sl.testing.expect(r).toHaveLength(12))
await sl.testing.test("kumulatif akhir = omzet 2024", () =>
  sl.testing.expect(r.at(-1).kumulatif).toBe(df.filter(x => x.tanggal.getFullYear()
=== 2024).total.sum()))
await sl.testing.test("portabel", () =>
sl.testing.expect(sl.sql.check(kueri)).toHaveLength(0))
```

	ganti_saya
0	1

1 baris × 1 kolom

12 bulan — Diharapkan: panjangnya 12 — nilai sebenarnya: [{"ganti_saya":1}]

kumulatif akhir = omzet 2024 — Diharapkan: sama persis dengan 185186150 — nilai sebenarnya: undefined

portabel (0 md)

Buka di Playground ↗

Mini-proyek: sepuluh pertanyaan bisnis

Jawab sepuluh pertanyaan bisnis Toko Nusantara hanya dengan SQL. Pengguna memilih pertanyaan, aplikasi menampilkan kueri dan jawabannya.

```
sl.title("Sepuluh Pertanyaan untuk Toko Nusantara :question:")

const [jual, produk, cabang] = await Promise.all([
  sl.read_csv("toko-nusantara.csv"), sl.read_csv("produk.csv"),
sl.read_csv("cabang.csv")])

const SOAL = {
  "1. Berapa omzet setiap tahun?":
    "SELECT EXTRACT(YEAR FROM tanggal) AS tahun, SUM(total) AS omzet FROM jual
GROUP BY 1 ORDER BY 1",
  "2. Lima produk dengan omzet terbesar?":
    "SELECT produk, SUM(total) AS omzet FROM jual GROUP BY produk ORDER BY omzet
DESC LIMIT 5",
  "3. Berapa porsi kanal Online per tahun?":
    "SELECT EXTRACT(YEAR FROM tanggal) AS tahun, SUM(CASE WHEN kanal = 'Online'
THEN total ELSE 0 END) * 1.0 / SUM(total) AS porsi_online FROM jual GROUP BY 1
ORDER BY 1",
  "4. Pemasok mana yang paling menguntungkan?":
    "SELECT p.pemasok, SUM(j.total - j.jumlah * p.harga_pokok) AS laba FROM jual j
JOIN produk p ON p.produk = j.produk GROUP BY 1 ORDER BY 2 DESC",
  "5. Provinsi mana dengan omzet terbesar?":
    "SELECT c.provinsi, SUM(j.total) AS omzet FROM jual j JOIN cabang c ON c.cabang
```

```
= j.cabang GROUP BY 1 ORDER BY 2 DESC",
  "6. Hari apa yang paling ramai?":
    "SELECT EXTRACT(DOW FROM tanggal) AS hari_ke, COUNT(*) AS transaksi FROM jual
GROUP BY 1 ORDER BY 2 DESC",
  "7. Berapa pelanggan yang belanja lebih dari 5 kali?":
    "WITH p AS (SELECT pelanggan_id, COUNT(*) AS n FROM jual GROUP BY 1) SELECT
COUNT(*) AS pelanggan FROM p WHERE n > 5",
  "8. Siapa 5 pelanggan dengan belanja terbesar?":
    "SELECT pelanggan_id, COUNT(*) AS transaksi, SUM(total) AS belanja FROM jual
GROUP BY 1 ORDER BY 3 DESC LIMIT 5",
  "9. Kategori apa yang tumbuh paling cepat dari 2023 ke 2024?":
    "WITH t AS (SELECT kategori, SUM(CASE WHEN EXTRACT(YEAR FROM tanggal) = 2023
THEN total END) AS t23, SUM(CASE WHEN EXTRACT(YEAR FROM tanggal) = 2024 THEN total
END) AS t24 FROM jual GROUP BY 1) SELECT kategori, t23, t24, t24 * 1.0 / t23 - 1 AS
tumbuh FROM t ORDER BY tumbuh DESC",
  "10. Berapa rata-rata nilai transaksi saat diskon Lebaran dibanding tanpa
diskon?":
    "SELECT CASE WHEN diskon_persen > 0 THEN 'Diskon' ELSE 'Tanpa diskon' END AS
jenis, COUNT(*) AS transaksi, ROUND(AVG(total), 0) AS rata FROM jual GROUP BY 1"
}

const soal = sl.selectbox("Pertanyaan", Object.keys(SOAL))
sl.code(SOAL[soal], { language: "sql" })
const hasil = await sl.sql(SOAL[soal], { jual, produk, cabang })
sl.dataframe(hasil)

const masalah = sl.sql.check(SOAL[soal])
if (masalah.length === 0) sl.caption("Kueri ini portabel: jalan juga di
PostgreSQL.")
```

Sepuluh Pertanyaan untuk Toko Nusantara ❓

Pertanyaan

1. Berapa omzet setiap tahun?

```
SELECT EXTRACT(YEAR FROM tanggal) AS tahun, SUM(total) AS omzet FROM jual GROUP
BY 1 ORDER BY 1
```

	tahun	omzet
0	2023	145.129.800
1	2024	185.186.150

2 baris × 2 kolom

Kueri ini portabel: jalan juga di PostgreSQL.

Buka di Playground ↗

`EXTRACT(DOW FROM tanggal)` memberi nomor hari: 0 Minggu sampai 6 Sabtu. Coba ubah pertanyaan 6 agar menampilkan nama hari dengan `CASE`.

← 20. Data Deret Waktu

22. Tren, Prediksi, dan Machine Learning Dasar →

Bab 22 · Tren, Prediksi, dan Machine Learning Dasar

> **Tujuan belajar**
>
> - Membuat garis tren dengan regresi linear sederhana, dan memahami rumusnya.
> - Membaca hasil regresi: kemiringan, R^2, dan batas kepercayaan sebuah prediksi.
> - Meramal penjualan beberapa bulan ke depan.
> - Memakai analisis bisnis siap pakai: Pareto, RFM, kohort, dan funnel.
> - Mengenal alur machine learning: data latih dan uji, model, dan ukuran kebaikan model.
> - Membuat demo model interaktif yang bisa dicoba orang lain.

Sejauh ini kamu menjawab pertanyaan **"apa yang sudah terjadi?"**. Bab ini melangkah ke pertanyaan **"apa yang mungkin terjadi?"** dan **"siapa pelanggan kita?"**.

22.1 Garis tren dengan JavaScript murni

Regresi linear sederhana mencari garis lurus `y = a + b × x` yang paling dekat dengan semua titik data. `b` adalah **kemiringan** (berapa `y` bertambah setiap `x` naik satu), dan `a` adalah **titik potong** (nilai `y` saat `x` nol).

Rumusnya bisa ditulis dengan JavaScript yang sudah kamu kuasai:

```
function regresi(x, y) {
  const n = x.length
  const rx = x.reduce((a, v) => a + v, 0) / n
  const ry = y.reduce((a, v) => a + v, 0) / n
  let atas = 0, bawah = 0
  for (let i = 0; i < n; i++) {
    atas += (x[i] - rx) * (y[i] - ry)
    bawah += (x[i] - rx) ** 2
  }
  const b = atas / bawah
  return { a: ry - b * rx, b }
}

const df = await sl.read_csv("toko-nusantara.csv")
const bulanan = df.resample("tanggal", "M").agg({ omzet: ["total", "sum"] }).to_records()
const x = bulanan.map((_, i) => i + 1)          // bulan ke-1, ke-2, ...
const y = bulanan.map(r => r.omzet)

const { a, b } = regresi(x, y)
sl.write(`Garis tren: omzet = ${Math.round(a).toLocaleString("id-ID")} +
${Math.round(b).toLocaleString("id-ID")} × bulan`)
sl.write(`Artinya omzet cenderung naik sekitar Rp ${Math.round(b).toLocaleString("id-ID")} setiap
bulan.`)
```

Garis tren: omzet = 10.053.712 + 296.756 × bulan

Artinya omzet cenderung naik sekitar Rp 296.756 setiap bulan.

Buka di Playground ↗

Ini disebut metode **kuadrat terkecil**: garis yang dipilih adalah garis dengan jumlah kuadrat jarak titik ke garis paling kecil.

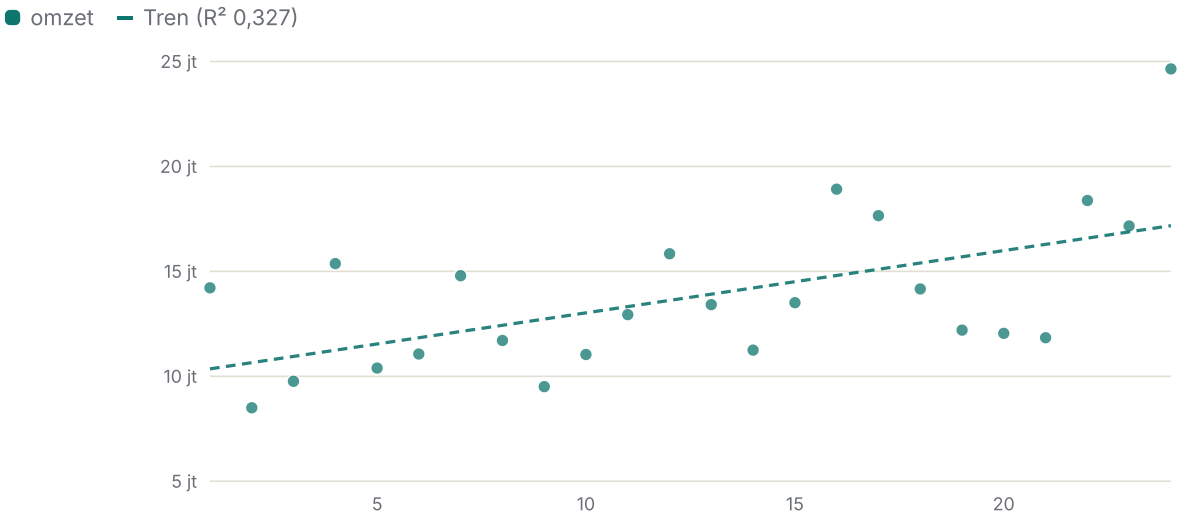
22.2 Regresi dengan Slak Data

```
const df = await sl.read_csv("toko-nusantara.csv")
const b = df.resample("tanggal", "M").agg({ omzet: ["total", "sum"] })
const data = b.assign({ bulan_ke: () => sl.Series(b.omzet.to_list().map((_, i) => i + 1)) })

const lr = sl.stats.linregress(data.bulan_ke, data.omzet)
sl.write(`Kemiringan: ${Math.round(lr.slope).toLocaleString("id-ID")} per bulan`)
sl.write(`R²: ${lr.rsquared.toLocaleString("id-ID", { maximumFractionDigits: 2 })}, p-value:
${lr.pvalue.toLocaleString("id-ID", { maximumFractionDigits: 4 })}`)
sl.scatter_chart(data, { x: "bulan_ke", y: "omzet", trendline: true, format: "compact" })
```

Kemiringan: 296.756 per bulan

R²: 0,33, p-value: 0,0035



Buka di Playground ↗

Angkanya sama dengan hasil fungsi buatanmu. Dua angka baru:

- **R²** (0 sampai 1) mengukur berapa bagian naik-turunnya omzet yang dijelaskan garis tren. R² 0,33 artinya garis lurus menjelaskan sekitar sepertiganya; sisanya karena musim dan hal lain.
- **p-value** di bawah 0,05 artinya kecenderungan naik itu nyata, bukan kebetulan.

Untuk **beberapa** variabel penjelas sekaligus, pakai `sl.stats.ols` dengan rumus gaya statsmodels:

```
const model = sl.stats.ols(sl.dataset("karyawan"), "gaji_juta ~ pengalaman_tahun + C(pendidikan)")
sl.write(model)
```

Regresi linear (OLS): `gaji_juta ~ pengalaman_tahun + C(pendidikan)`

Data	R^2	R^2 terkoreksi	p (uji F)
300	**0,819**	**0,817**	**< 0,0001**

variabel	koefisien	std_error	t	p_value	batas_bawah	batas_atas	signifikan
Intercept	5,7702	0,2082	27,713	< 0,0001	5,3604	6,1799	✓
pengalaman_tahun	0,4606	0,0161	28,642	< 0,0001	0,4289	0,4922	✓
pendidikan[S1]	1,3568	0,1983	6,842	< 0,0001	0,9666	1,7470	✓
pendidikan[S2]	3,0363	0,2136	14,217	< 0,0001	2,6160	3,4566	✓
pendidikan[SMA]	-0,5954	0,2107	-2,826	0,0050	-1,0099	-0,1808	✓

5 baris × 8 kolom

Variabel yang berpengaruh signifikan (α = 0.05): pengalaman_tahun, pendidikan[S1], pendidikan[S2], pendidikan[SMA].

› Diagnostik model

Buka di Playground ↗

`C(pendidikan)` memberi tahu bahwa `pendidikan` adalah kategori. Tabel koefisien menjawab: dengan pendidikan yang sama, setiap tahun pengalaman menambah gaji sekian juta.

> **API Slak baru**
>
> - `sl.stats.linregress(x, y)` menghasilkan `slope`, `intercept`, `rvalue`, `rsquared`, dan `pvalue`.
> - `sl.stats.ols(df, "y ~ x1 + x2 + C(kategori)")` membuat regresi berganda lengkap dengan tabel koefisien, R^2, dan diagnostik.

22.3 Meramal penjualan

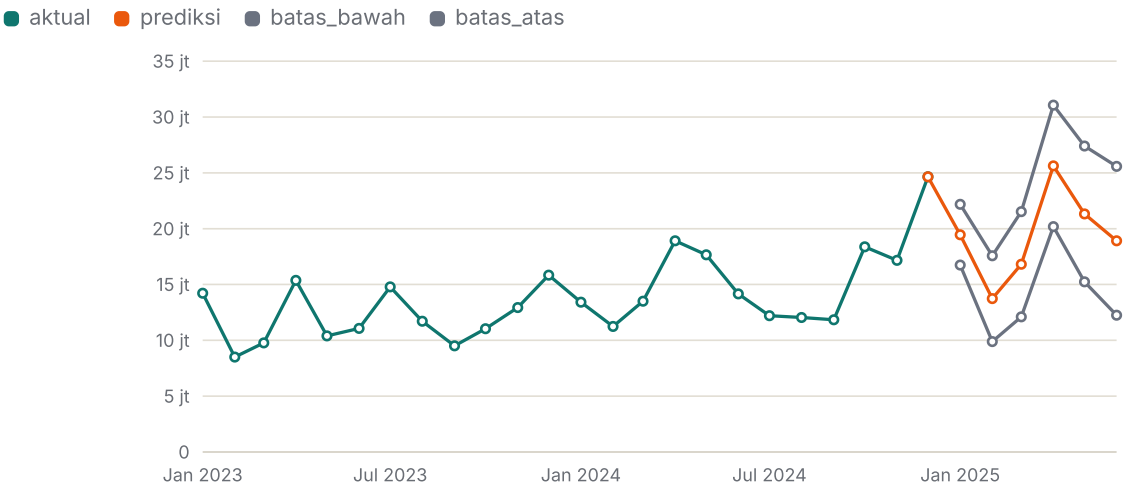
Garis lurus mengabaikan musim. Untuk meramal, Slak menyediakan `sl.ts.forecast`, yang mencoba beberapa metode (termasuk **Holt-Winters**, yang memperhitungkan tren dan musim sekaligus) lalu memilih yang paling cocok dengan data.

```
const df = await sl.read_csv("toko-nusantara.csv")
const bulanan = df.resample("tanggal", "M").agg({ omzet: ["total", "sum"] })
const n = sl.slider("Ramal berapa bulan ke depan?", 3, 12, 6)
const ramalan = sl.ts.forecast(bulanan, { periods: n })
sl.write(ramalan)
```

Ramal berapa bulan ke depan? **6**

3 12

Peramalan omzet · metode Holt-Winters (tren + musiman multiplikatif) · 6 periode ke depan



MAE	RMSE	MAPE
1,18e+6	1,39e+6	8%

Batas bawah/atas adalah perkiraan interval 95%. Galat dihitung dari data historis (in-sample).

> Tabel ramalan

Buka di Playground ↗

Hasilnya mencakup grafik, tabel ramalan, dan ukuran kesalahan. Yang terpenting adalah **batas bawah dan batas atas**: rentang yang kemungkinan besar (sekitar 95%) memuat nilai sebenarnya.

> **Ramalan bukan janji.** Selalu laporkan rentangnya, bukan hanya satu angka. "Omzet Januari diperkirakan 10–18 juta, paling mungkin 14 juta" jauh lebih jujur daripada "omzet Januari 14 juta". Dua tahun data juga terbilang pendek untuk meramal musim; semakin panjang datanya, semakin bisa diandalkan ramalannya.

Ukuran	Arti
MAE	Rata-rata selisih ramalan dengan kenyataan, dalam rupiah
RMSE	Mirip MAE, tetapi menghukum selisih besar lebih berat
MAPE	Rata-rata selisih dalam persen

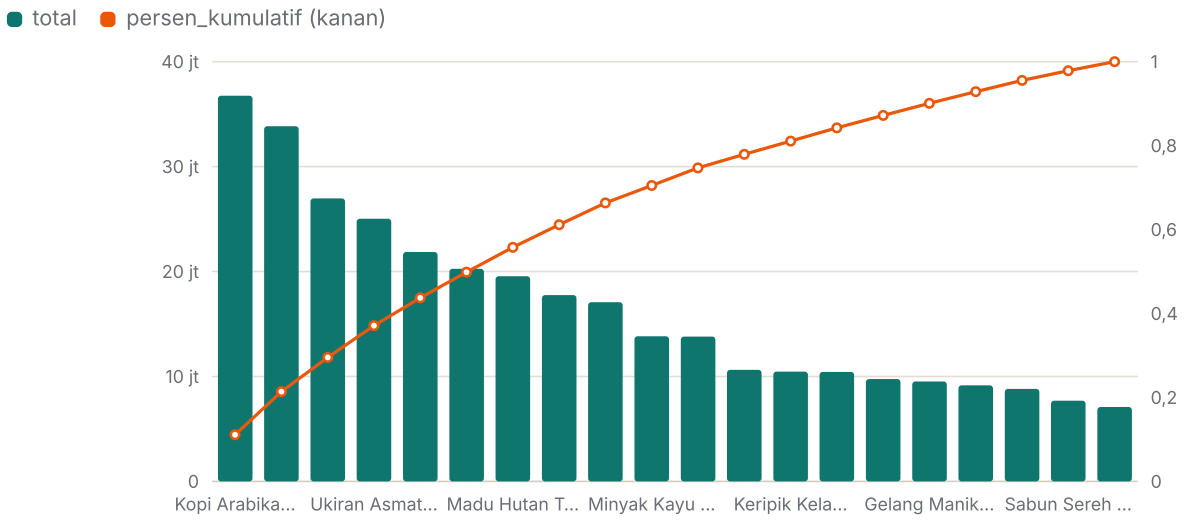
> **API Slak baru**
>
> - `sl.ts.forecast(df, { periods, method })` meramal deret waktu. Hasilnya bisa ditampilkan dengan `sl.write`, dan tabelnya ada di `.forecast` (kolom `prediksi`, `batas_bawah`, `batas_atas`).

22.4 Analisis bisnis siap pakai

Beberapa analisis begitu sering diminta sehingga Slak menyediakannya dalam satu perintah.

Pareto: produk mana yang paling penting?

```
const df = await sl.read_csv("toko-nusantara.csv")
const par = sl.analysis.pareto(df, { category: "produk", value: "total" })
sl.combo_chart(par, { x: "produk", bar: "total", line: "persen_kumulatif" })
const a = par.filter(r => r.kelas === "A")
sl.write(`${a.shape[0]} dari ${par.shape[0]} produk (kelas A) bersama-sama menyumbang sekitar 80% omzet.`)
```



13 dari 20 produk (kelas A) bersama-sama menyumbang sekitar 80% omzet.

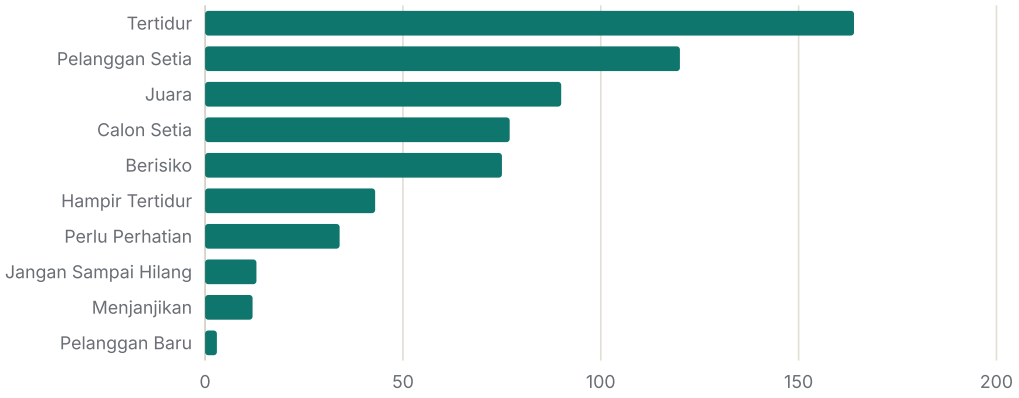
Buka di Playground ↗

Analisis Pareto mengurutkan produk dari omzet terbesar dan menjumlahkan porsinya. Produk yang bersama-sama mencapai 80% omzet masuk kelas **A**: wajib selalu tersedia di rak. Di banyak toko, kelas A hanya sekitar 20% produk (inilah "aturan 80/20"). Di Toko Nusantara omzetnya lebih merata, sehingga kelas A lebih banyak; temuan seperti ini pun berharga, karena artinya toko tidak bergantung pada segelintir produk.

RFM: mengenal pelanggan

RFM menilai setiap pelanggan dari tiga sisi: **R**ecency (kapan terakhir belanja), **F**requency (seberapa sering), dan **M**onetary (berapa banyak). Pelanggan lalu dikelompokkan ke segmen berbahasa Indonesia seperti *Juara*, *Pelanggan Setia*, *Berisiko*, dan *Tertidur*.

```
const df = await sl.read_csv("toko-nusantara.csv")
const rfm = sl.analysis.rfm(df, { customer: "pelanggan_id", date: "tanggal", amount: "total" })
sl.bar_chart(rfm.value_counts("segmen"), { x: "segmen", y: "jumlah", horizontal: true, sort: true })
sl.dataframe(rfm.sort_values("skor_rfm", { ascending: false }).head(5))
```



	pelanggan_id	recency	frequency	monetary	R	F	M	skor_rfm	segmen
0	P0395	19	14	2.467.500	5	5	5	555	Juara
1	P0615	19	8	2.204.500	5	5	5	555	Juara
2	P0101	6	17	2.135.000	5	5	5	555	Juara
3	P0390	5	16	2.113.000	5	5	5	555	Juara
4	P0488	1	17	2.047.500	5	5	5	555	Juara

5 baris × 9 kolom

Buka di Playground ↗

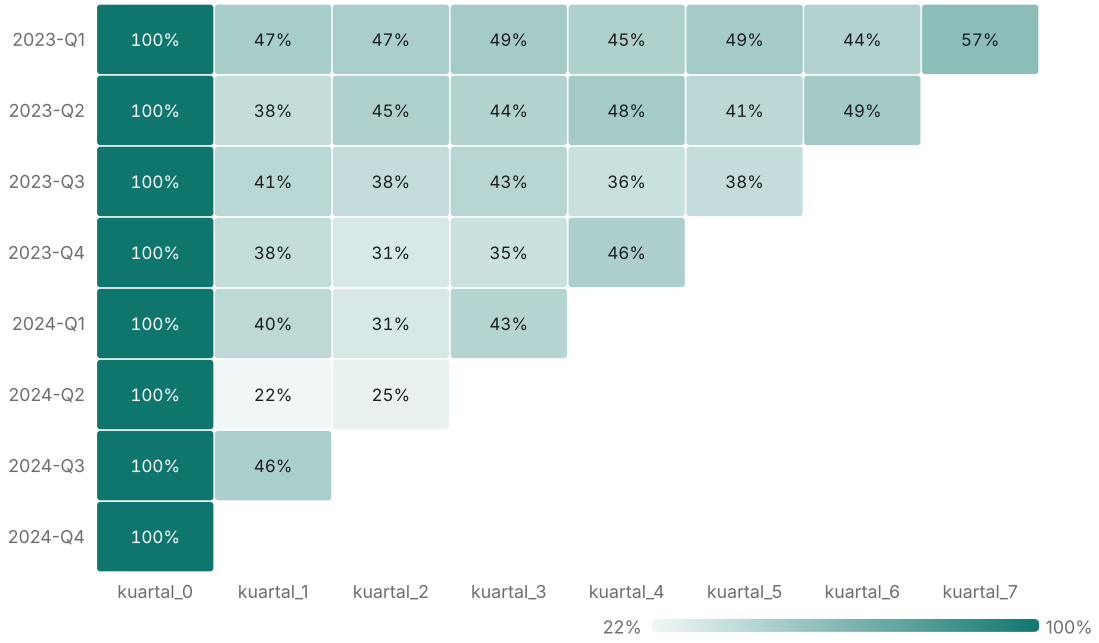
Segmen *Berisiko* dan *Jangan Sampai Hilang* adalah pelanggan bernilai tinggi yang lama tidak datang: sasaran utama untuk ditawari promo.

Kohort dan funnel

```
const df = await sl.read_csv("toko-nusantara.csv")
const ko = sl.analysis.cohort(df, { customer: "pelanggan_id", date: "tanggal", period: "Q" })
sl.heatmap(ko.drop("ukuran"), { format: "percent:0", title: "Berapa persen pelanggan kembali di
kuartal berikutnya?" })

sl.dataframe(sl.analysis.funnel({ "Lihat produk": 10000, "Masuk keranjang": 1200, "Bayar": 300 }),
{ format: { konversi_dari_awal: "percent", konversi_dari_sebelumnya: "percent" } })
```

Berapa persen pelanggan kembali di kuartal berikutnya?



	tahap	jumlah	konversi_dari_awal	konversi_dari_sebelumnya	hilang
0	Lihat produk	10.000	100,0%	100,0%	0
1	Masuk keranjang	1.200	12,0%	12,0%	8.800
2	Bayar	300	3,0%	25,0%	900

3 baris × 5 kolom

Buka di Playground ↗

Kohort mengelompokkan pelanggan menurut kapan mereka pertama datang, lalu melacak berapa persen yang kembali. **Funnel** menghitung berapa yang lolos di setiap tahap, sehingga tahap yang paling banyak "bocor" terlihat.

> **API Slak baru**
>
> - `sl.analysis.pareto(df, { category, value })`, `sl.analysis.rfm(df, { customer, date, amount })`, `sl.analysis.cohort(df, { customer, date, period })`, `sl.analysis.funnel(tahap)`.
> - `sl.combo_chart(df, { x, bar, line })` menggambar batang dan garis sekaligus.

22.5 Machine learning: belajar dari contoh

Machine learning adalah cara membuat program yang belajar aturan sendiri dari contoh-contoh, bukan dari aturan yang kita tulis dengan `if`. Ada tiga jenis tugas yang paling sering:

Tugas	Pertanyaan	Contoh
Regresi	Berapa nilainya?	Berapa gaji yang wajar untuk pengalaman 5 tahun?
Klasifikasi	Masuk kelompok mana?	Apakah pelanggan ini akan berhenti berlangganan?
Pengelompokan (klaster)	Kelompok apa saja yang ada?	Pelanggan kita terbagi ke tipe apa saja?

Data latih dan data uji

Aturan emas machine learning: **jangan menilai model dengan data yang dipakai untuk melatihnya.** Itu seperti murid yang dinilai dengan soal latihan yang sudah ia hafal. Bagi datanya dua: sebagian untuk **latih**, sebagian lagi disimpan untuk **uji**.

```
const kar = sl.dataset("karyawan")
const [latih, uji] = sl.ml.train_test_split(kar, { test_size: 0.25, random_state: 7 })
sl.write(`${latih.shape[0]} baris latih, ${uji.shape[0]} baris uji`)

const model = sl.ml.LinearRegression().fit(latih.select(["pengalaman_tahun", "jam_pelatihan",
"gaji_juta"]), "gaji_juta")
const tebakan = model.predict(uji.select(["pengalaman_tahun", "jam_pelatihan"]))

const [a, b] = sl.columns(2)
a.metric("R² pada data uji", Math.round(sl.ml.r2_score(uji.gaji_juta, tebakan) * 100) / 100)
b.metric("Rata-rata meleset", `${sl.ml.mae(uji.gaji_juta, tebakan).toLocaleString("id-ID", {
maximumFractionDigits: 1 })} juta`)
```

225 baris latih, 75 baris uji

R² pada data uji
0,5

Rata-rata meleset
1,5 juta

Buka di Playground ↗

Alurnya selalu sama, apa pun modelnya: **bagi → latih (`fit`) → tebak (`predict`) → ukur**. Nama-namanya sengaja dibuat sama dengan scikit-learn, pustaka machine learning paling terkenal di Python.

Klasifikasi: siapa yang akan berhenti?

Dataset bawaan `pelanggan` berisi 1.000 pelanggan layanan internet fiktif dan kolom `churn` (1 = berhenti berlangganan).

```
const data = sl.dataset("pelanggan").drop("pelanggan_id")
const [latih, uji] = sl.ml.train_test_split(data, { test_size: 0.25, stratify: "churn",
random_state: 42 })

const pohon = sl.ml.DecisionTreeClassifier({ max_depth: 4 }).fit(latih, "churn")
const tebak = pohon.predict(uji.drop("churn"))

sl.metric("Akurasi pada data uji", sl.ml.accuracy_score(uji.churn, tebak), null, { format:
"percent" })
sl.dataframe(sl.ml.confusion_matrix(uji.churn, tebak))
sl.code(pohon.rules())
```

Akurasi pada data uji
79,6%

	aktual	prediksi_0	prediksi_1
0	0	150	28
1	1	23	49

2 baris × 3 kolom

```
jika lama_berlangganan ≤ 19,5:
   jika jumlah_keluhan ≤ 0,5:
      jika pakai_aplikasi ≤ 0,5:
         jika lama_berlangganan ≤ 14,5:
            → 1 (63%)  [n=68]
         selain itu:
            → 0 (72%)  [n=32]
      selain itu:
         jika tagihan_bulanan ≤ 174.500:
            → 0 (74%)  [n=27]
         selain itu:
            → 0 (90%)  [n=94]
   selain itu:
      jika pakai_aplikasi ≤ 0,5:
         jika umur ≤ 34,5:
            → 1 (94%)  [n=33]
         selain itu:
            → 1 (65%)  [n=20]
      selain itu:
         jika lama_berlangganan ≤ 12,5:
            → 1 (74%)  [n=46]
         selain itu:
            → 0 (61%)  [n=31]
selain itu:
   jika jumlah_keluhan ≤ 1,5:
      jika lama_berlangganan ≤ 23,5:
         jika umur ≤ 49:
            → 0 (84%)  [n=67]
         selain itu:
            → 1 (100%)  [n=2]
      selain itu:
         jika jumlah_keluhan ≤ 0,5:
            → 0 (98%)  [n=185]
         selain itu:
            → 0 (88%)  [n=73]
   selain itu:
      jika lama_berlangganan ≤ 32,5:
         jika jumlah_keluhan ≤ 2,5:
            → 1 (54%)  [n=24]
         selain itu:
            → 1 (85%)  [n=13]
      selain itu:
```

```
          jika kota_Manokwari ≤ 0,5:
             → 0 (88%)  [n=26]
          selain itu:
             → 1 (56%)  [n=9]
```

Buka di Playground ↗

- **Akurasi** adalah persentase tebakan yang benar.
- **Matriks kebingungan** (*confusion matrix*) memperlihatkan jenis kesalahannya: berapa pelanggan yang sebenarnya berhenti tetapi ditebak bertahan, dan sebaliknya. Untuk churn, kesalahan jenis pertama biasanya lebih mahal.
- **Pohon keputusan** bisa dibaca sebagai aturan "jika … maka …" lewat `rules()`. Ini sangat berguna untuk menjelaskan model kepada orang bisnis.

Pengelompokan: tipe-tipe pelanggan

KMeans membagi data ke `k` kelompok sehingga anggota satu kelompok saling mirip. Tidak ada jawaban benar yang diberikan; model menemukan kelompoknya sendiri.

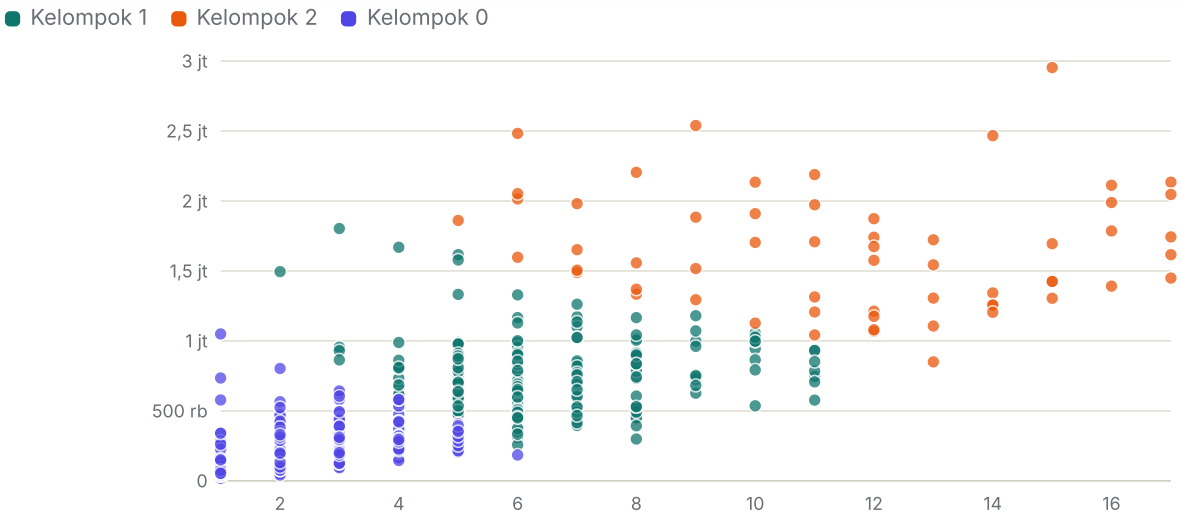
```
const df = await sl.read_csv("toko-nusantara.csv")
const pel = df.groupby("pelanggan_id").agg({ transaksi: ["id_transaksi", "nunique"], belanja:
["total", "sum"] })

const X = sl.ml.StandardScaler().fit_transform(pel.select(["transaksi", "belanja"]))
const km = sl.ml.KMeans({ n_clusters: 3, random_state: 1 }).fit(X)
const dikelompokkan = pel.assign({ klaster: () => km.predict(X) })

const profil = dikelompokkan.groupby("klaster").agg({
  pelanggan: ["pelanggan_id", "size"], rata_transaksi: ["transaksi", "mean"], rata_belanja:
["belanja", "mean"]
}).sort_values("rata_belanja")
sl.dataframe(profil, { format: { rata_transaksi: "decimal:1", rata_belanja: "rupiah" } })
sl.scatter_chart(dikelompokkan.assign({ klaster: r => `Kelompok ${r.klaster}` }), { x: "transaksi",
y: "belanja", color: "klaster", format: "compact" })
```

	klaster	pelanggan	rata_transaksi	rata_belanja
0	0	380	2,6	Rp 241.266
1	1	192	6,6	Rp 736.724
2	2	59	11,6	Rp 1.647.185

3 baris × 4 kolom



Buka di Playground ↗

`StandardScaler` menyamakan skala kolom lebih dulu. Tanpanya, kolom `belanja` (jutaan) akan mendominasi kolom `transaksi` (satuan), dan pengelompokan hanya melihat belanja.

Hasilnya biasanya tiga tipe yang mudah diberi nama: **pembeli sesekali** (sebagian besar), **pelanggan tetap**, dan **pelanggan setia** yang sedikit jumlahnya tetapi besar belanjanya.

> **API Slak baru**
>
> - `sl.ml.train_test_split(df, { test_size, stratify, random_state })` membagi data latih dan uji.
> - Model: `sl.ml.LinearRegression()`, `sl.ml.DecisionTreeClassifier({ max_depth })`, `sl.ml.KMeans({ n_clusters })`, dan lainnya. Semuanya memakai `.fit(df, "target")` dan `.predict(df)`.
> - Ukuran: `sl.ml.accuracy_score`, `sl.ml.confusion_matrix`, `sl.ml.r2_score`, `sl.ml.mae`, `sl.ml.rmse`.
> - `sl.ml.StandardScaler().fit_transform(df)` menyamakan skala kolom.

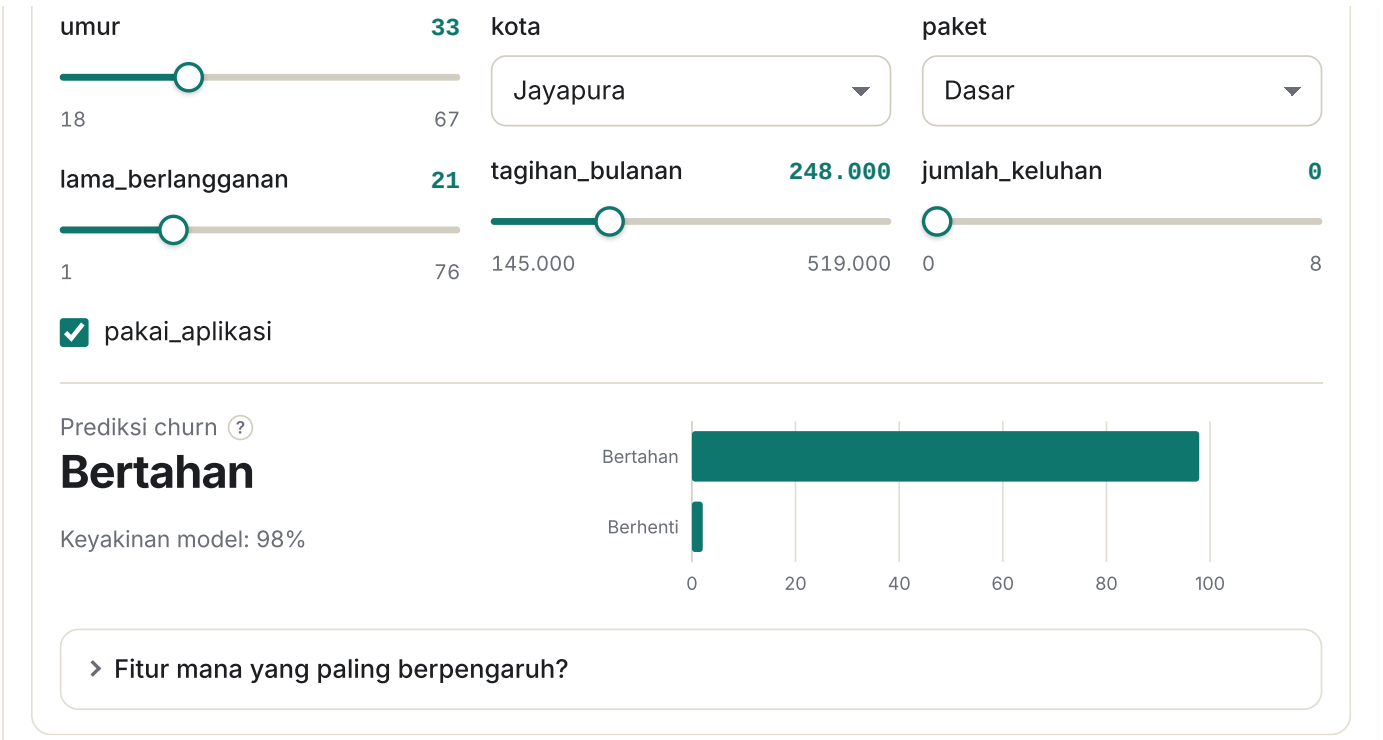
22.6 Demo model untuk orang lain

Model yang hanya kamu pahami sendiri belum berguna. `sl.ml_demo` membuat formulir interaktif: setiap kolom mendapat widget yang sesuai, dan tebakan model tampil seketika.

```
sl.ml_demo(
  sl.ml.DecisionTreeClassifier({ max_depth: 4 }),
  sl.dataset("pelanggan").drop("pelanggan_id"),
  { target: "churn", labels: { 0: "Bertahan", 1: "Berhenti" } }
)
```

Coba model: tebak churn

Ubah nilai di bawah. Prediksi dihitung ulang langsung di browser.



Buka di Playground ↗

Ubah jumlah keluhan atau lama berlangganan, lalu perhatikan tebakan dan keyakinan model berubah. Bagian "Fitur mana yang paling berpengaruh?" menunjukkan apa yang paling diperhatikan model.

> **API Slak baru**
>
> - `sl.ml_demo(model, df, { target, labels })` membuat formulir coba model. Model yang belum dilatih dilatih otomatis.

Hasil model Slak sudah dicocokkan dengan scikit-learn. Untuk model yang lebih rumit, seperti gradient boosting atau deep learning, analis biasanya beralih ke Python (Bab 26).

> **Coba di playground** Di contoh klasifikasi, ganti `max_depth: 4` menjadi `max_depth: 2` lalu `max_depth: 12`. Apa yang terjadi pada akurasi data uji? Pohon yang terlalu dalam menghafal data latih, sehingga justru buruk pada data baru. Gejala ini disebut *overfitting*.

Kesalahan umum

Gejala	Penyebab	Perbaikan
Akurasi 100%	Dinilai dengan data latih, atau target ikut sebagai fitur	Nilai dengan data uji; buang kolom target dari fitur
Akurasi tinggi, tetapi semua ditebak "bertahan"	Kelas tidak seimbang	Lihat matriks kebingungan, bukan akurasi saja
Ramalan meyakinkan padahal datanya sedikit	Satu angka tanpa rentang	Laporkan batas bawah dan atas
Klaster hanya mengikuti satu kolom	Skala kolom berbeda jauh	Pakai `StandardScaler`
Galat nilai kosong saat `fit`	Model butuh data lengkap	`dropna` atau `fillna` dulu

Latihan

Latihan 22.1 (mudah). Pakai fungsi `regresi` buatanmu untuk data kecil di bawah. Hasil `b` (kemiringan) harus 2 dan `a` harus 1.

```
function regresi(x, y) {
  // salin dan lengkapi dari 22.1
  return { a: 0, b: 0 }
}

const hasil = regresi([1, 2, 3, 4], [3, 5, 7, 9])
await sl.testing.test("kemiringan 2", () => sl.testing.expect(hasil.b).toBeCloseTo(2, 6))
await sl.testing.test("titik potong 1", () => sl.testing.expect(hasil.a).toBeCloseTo(1, 6))
```

✕ **kemiringan 2** — Diharapkan: mendekati 2 (6 desimal) — nilai sebenarnya: 0

✕ **titik potong 1** — Diharapkan: mendekati 1 (6 desimal) — nilai sebenarnya: 0

Buka di Playground ↗

Latihan 22.2 (sedang). Hitung `porsiBerisiko`: proporsi pelanggan Toko Nusantara yang masuk segmen RFM "Berisiko" atau "Jangan Sampai Hilang".

```
const df = await sl.read_csv("toko-nusantara.csv")
const porsiBerisiko = 0   // ganti

sl.metric("Pelanggan berisiko", (porsiBerisiko * 100).toFixed(1) + "%")
const rfm = sl.analysis.rfm(df, { customer: "pelanggan_id", date: "tanggal", amount: "total" })
await sl.testing.test("porsi benar", () => sl.testing.expect(porsiBerisiko).toBeCloseTo(
  rfm.filter(r => ["Berisiko", "Jangan Sampai Hilang"].includes(r.segmen)).shape[0] / rfm.shape[0],
6))
```

Pelanggan berisiko

0.0%

✕ **porsi benar** — Diharapkan: mendekati 0.13946117274167988 (6 desimal) — nilai sebenarnya: 0

Buka di Playground ↗

Latihan 22.3 (tantangan). Latih `RandomForestClassifier` untuk memprediksi churn, lalu simpan akurasinya pada data uji di `akurasi`. Akurasinya harus lebih baik daripada menebak "bertahan" untuk semua orang.

```
const data = sl.dataset("pelanggan").drop("pelanggan_id")
const [latih, uji] = sl.ml.train_test_split(data, { test_size: 0.25, stratify: "churn",
random_state: 42 })
const akurasi = 0   // ganti

sl.metric("Akurasi", (akurasi * 100).toFixed(1) + "%")
const tebakBertahan = uji.churn.to_list().filter(v => v === 0).length / uji.shape[0]
await sl.testing.test("lebih baik dari tebakan sederhana", () =>
sl.testing.expect(akurasi).toBeGreaterThan(tebakBertahan))
```

Akurasi

0.0%

✕ **lebih baik dari tebakan sederhana** — Diharapkan: lebih besar dari 0.712 — nilai sebenarnya: 0

Buka di Playground ↗

Mini-proyek: prakiraan dan segmen pelanggan

```
sl.title("Prakiraan dan Pelanggan Toko Nusantara :crystal_ball:")

const df = await sl.read_csv("toko-nusantara.csv")
const [tRamal, tSegmen] = sl.tabs(["Prakiraan omzet", "Segmen pelanggan"])

// ---------- Prakiraan ----------
const bulanan = df.resample("tanggal", "M").agg({ omzet: ["total", "sum"] })
const fc = sl.ts.forecast(bulanan, { periods: 6 })
const tabel = fc.forecast.to_records()
const total6 = tabel.reduce((a, r) => a + r.prediksi, 0)
const bawah6 = tabel.reduce((a, r) => a + r.batas_bawah, 0)
const atas6 = tabel.reduce((a, r) => a + r.batas_atas, 0)
const jt = x => (x / 1e6).toFixed(0) + " jt"
tRamal.metric("Perkiraan omzet 6 bulan ke depan", jt(total6))
tRamal.caption(`Rentang yang mungkin: ${jt(bawah6)} sampai ${jt(atas6)}`)
tRamal.write(fc)

// ---------- Segmen ----------
const rfm = sl.analysis.rfm(df, { customer: "pelanggan_id", date: "tanggal", amount: "total" })
const perSegmen = rfm.groupby("segmen").agg({ pelanggan: ["pelanggan_id", "size"], belanja:
["monetary", "sum"] })
  .sort_values("belanja", { ascending: false })
tSegmen.bar_chart(perSegmen, { x: "segmen", y: "belanja", horizontal: true, format: "compact" })
const juara = perSegmen.filter(r => r.segmen === "Juara").to_records()[0]
if (juara) tSegmen.success(`${juara.pelanggan} pelanggan "Juara" menyumbang ${(juara.belanja /
rfm.monetary.sum() * 100).toFixed(0)}% omzet. Jaga mereka baik-baik.`)
tSegmen.dataframe(rfm.filter(r => r.segmen === "Berisiko").sort_values("monetary", { ascending:
false }).head(10))
```

Prakiraan dan Pelanggan Toko Nusantara 🔮

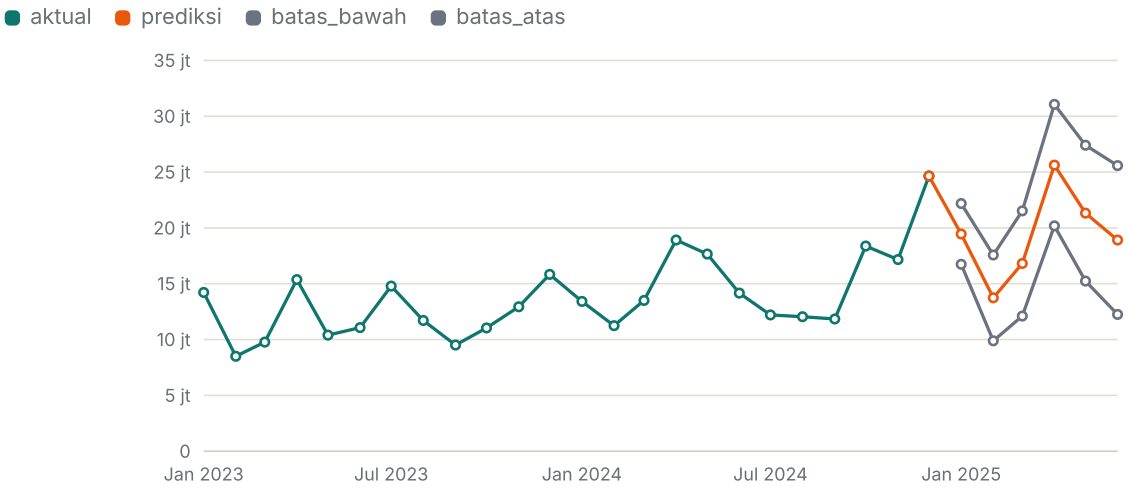
Prakiraan omzet | Segmen pelanggan

Perkiraan omzet 6 bulan ke depan

116 jt

Rentang yang mungkin: 86 jt sampai 145 jt

Peramalan omzet · metode Holt-Winters (tren + musiman multiplikatif) · 6 periode ke depan



MAE	RMSE	MAPE
1,18e+6	1,39e+6	8%

Batas bawah/atas adalah perkiraan interval 95%. Galat dihitung dari data historis (in-sample).

› Tabel ramalan

Buka di Playground ↗

Daftar pelanggan *Berisiko* di bawahnya adalah hasil kerja yang bisa langsung dipakai tim pemasaran: siapa yang perlu dihubungi minggu ini.

Bab 23 · Peta dan Data Wilayah

> **Tujuan belajar**
>
> - Mengenal data berbasis lokasi: kota, provinsi, dan koordinat.
> - Menggambar peta titik, peta gelembung, dan rute.
> - Membuat peta warna wilayah (choropleth) untuk 38 provinsi dan 514 kabupaten/kota, tanpa internet.
> - Mencocokkan nama wilayah yang ditulis berbeda-beda, dan menghitung jarak antartitik.

Indonesia membentang lebih dari 5.000 km dari Sabang sampai Merauke. Bagi Toko Nusantara yang cabangnya tersebar dari Jakarta sampai Jayapura, peta bukan hiasan: peta memperlihatkan pola yang tidak terlihat di tabel.

23.1 Memasang peta

Peta ada di berkas tambahan:

```
<script src="slak.js"></script>
<script src="slak-data.js"></script>
<script src="slak-peta.js"></script>      <!-- batas 38 provinsi dan titik 514
kab/kota -->
<script src="slak-peta-kab.js"></script>  <!-- batas poligon kab/kota (opsional) --
>
```

Batas 38 provinsi, termasuk pemekaran Papua tahun 2022, sudah ada di dalam `slak-peta.js`. Peta bekerja **tanpa internet**. Playground, Slak Studio, dan halaman buku ini sudah memuatnya.

23.2 Tiga cara menyebut lokasi

Cara	Contoh	Ketelitian
Koordinat	`lat: -2.5337, lon: 140.7181`	Paling tepat
Nama kota atau kabupaten	`"Jayapura"`, `"Kab. Mimika"`	Titik tengah wilayah
Nama provinsi	`"Papua Barat Daya"`	Untuk peta warna wilayah

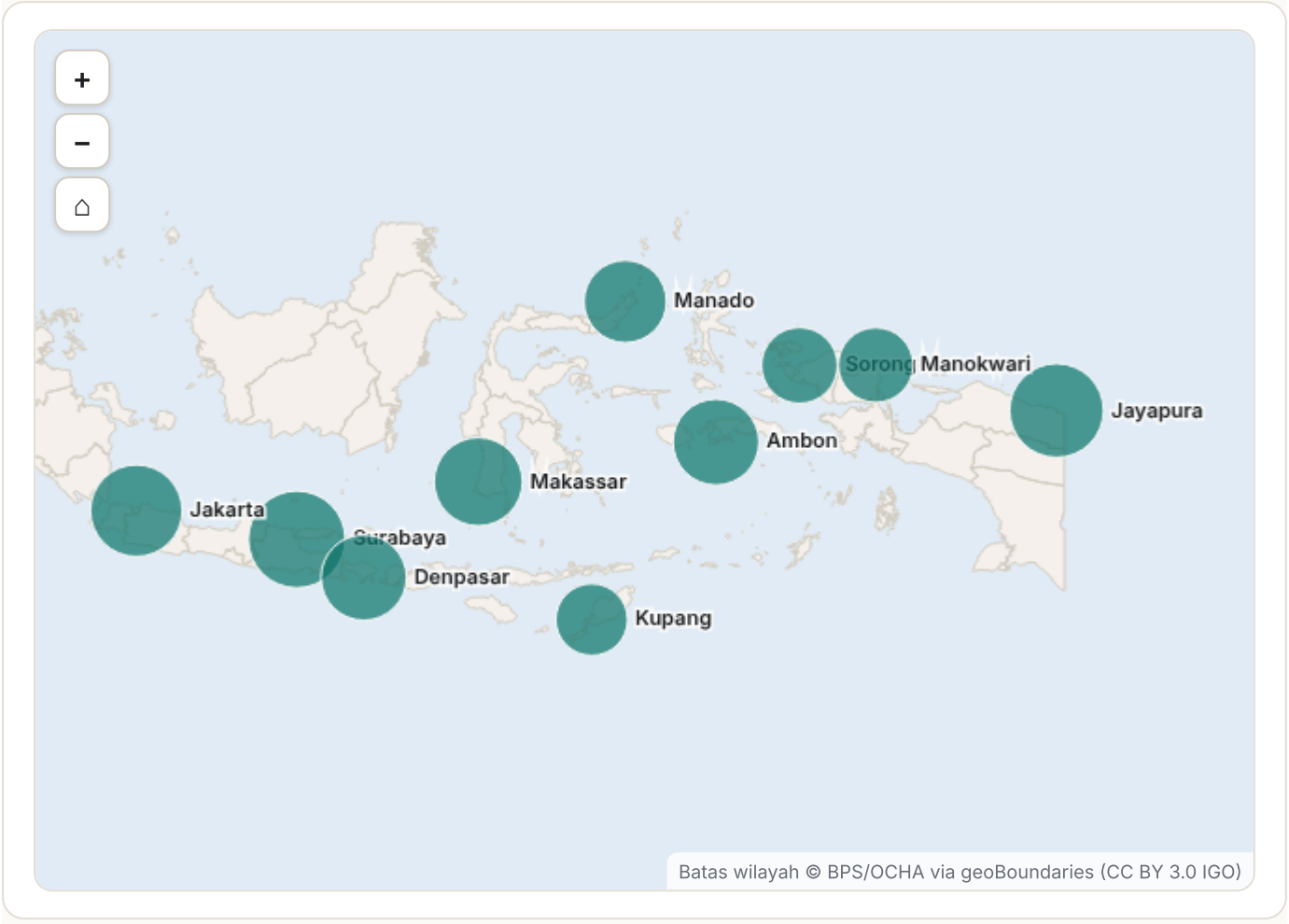
Lintang (*latitude*, `lat`) adalah posisi utara-selatan: negatif di selatan khatulistiwa. **Bujur** (*longitude*, `lon`) adalah posisi barat-timur: Indonesia berada sekitar 95 sampai 141 derajat bujur timur.

23.3 Peta titik dan gelembung

`cabang.csv` menyimpan koordinat setiap cabang. Gabungkan dengan omzet (Bab 18), lalu gambar:

```
const [trx, cabang] = await Promise.all([sl.read_csv("toko-nusantara.csv"),
sl.read_csv("cabang.csv")])
const perCabang = trx.groupby("cabang").agg({ omzet: ["total", "sum"]
}).merge(cabang, { on: "cabang" })

sl.map(perCabang, { lat: "lat", lon: "lon", size: "omzet", label: "kota" })
```

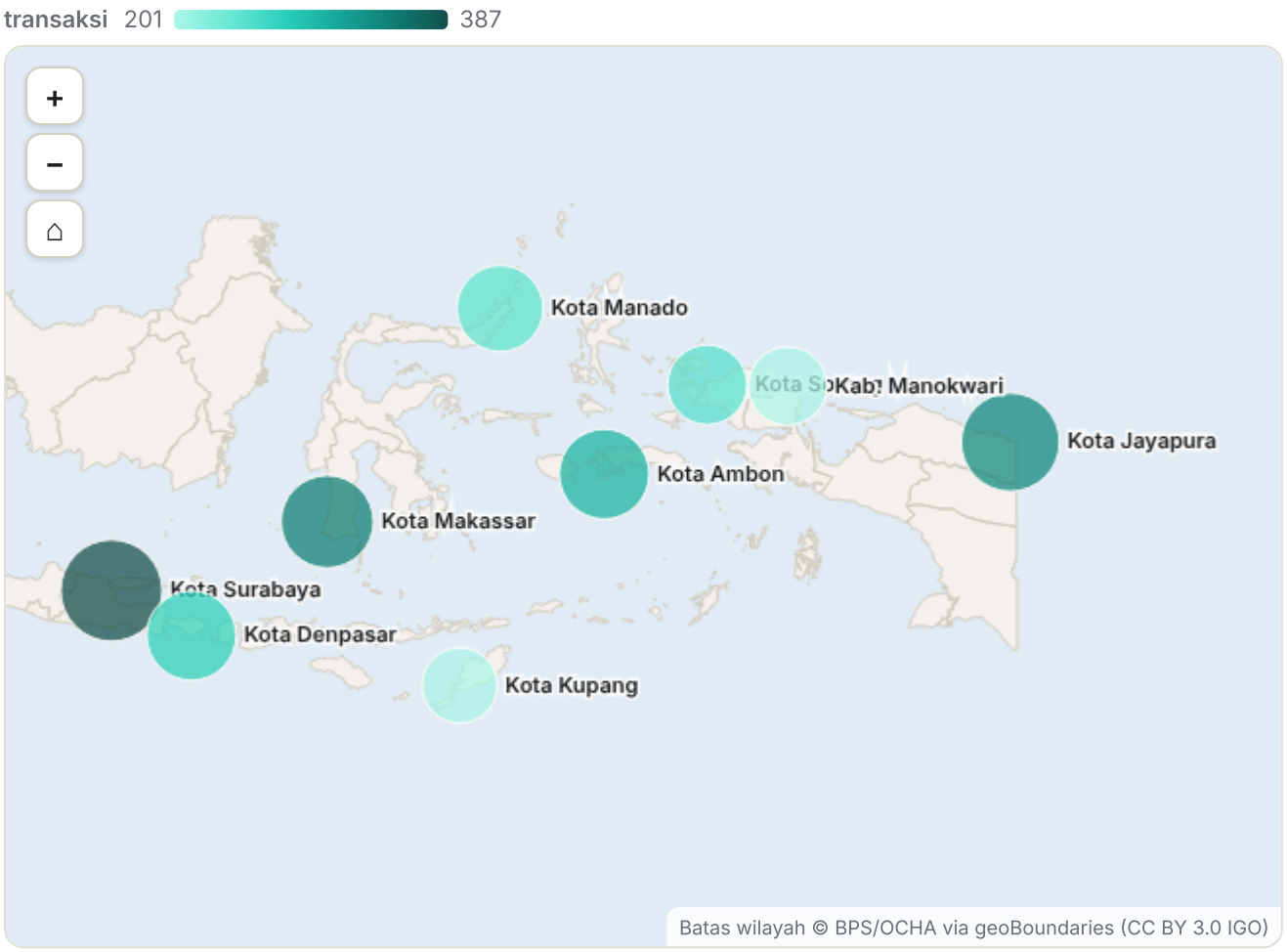


Buka di Playground ↗

Ukuran gelembung sebanding dengan omzet. Arahkan kursor ke gelembung untuk melihat nilainya; gulir atau tekan **+** dan **−** untuk memperbesar.

Bila datamu tidak punya koordinat, sebut saja nama wilayahnya:

```
const trx = await sl.read_csv("toko-nusantara.csv")
const perKota = trx.groupby("kota").agg({ omzet: ["total", "sum"], transaksi:
["id_transaksi", "nunique"] })
sl.map(perKota, { region: "kota", size: "omzet", color: "transaksi", label: "kota"
})
```



1 wilayah tidak dikenali: Jakarta

Buka di Playground ↗

Slak mencari titik tengah setiap kota dari daftar 514 kabupaten/kota yang dibawanya.

Perhatikan catatan di bawah peta: **1 wilayah tidak dikenali: Jakarta**. Bukan salah Slak. Jakarta adalah provinsi yang terdiri dari lima kota administrasi (Jakarta Pusat, Jakarta Utara, dan seterusnya), jadi "Jakarta" tidak bisa dicocokkan dengan satu kota. Perbaikannya: tulis nama kotanya, misalnya "Jakarta Pusat", atau pakai koordinat seperti contoh sebelumnya. Inilah alasan analis lebih suka menyimpan koordinat atau kode wilayah daripada nama.

> **API Slak baru**
>
> - `sl.map(df, { lat, lon, size, color, label })` menggambar titik atau gelembung.

- `sl.map(df, { region: "kolom" })` mencari lokasi dari nama wilayah.
- Opsi lain: `heatmap: true` (peta kepadatan), `cluster: true` (titik yang berdekatan dikumpulkan), `route: { arrows: true }` (garis sesuai urutan baris), `tiles: "osm"` (latar OpenStreetMap, **butuh internet**).

23.4 Rute

Bayangkan rute pengiriman barang dari gudang di Surabaya ke cabang-cabang di timur. Baris data digambar berurutan sebagai garis:

```
const cabang = await sl.read_csv("cabang.csv")
const urutan = ["Surabaya", "Denpasar", "Kupang", "Makassar", "Ambon", "Sorong",
"Manokwari", "Jayapura"]
const rute = urutan.map(k => cabang.filter(r => r.kota === k).to_records()[0])
sl.map(rute, { lat: "lat", lon: "lon", label: "kota", route: { arrows: true } })
```

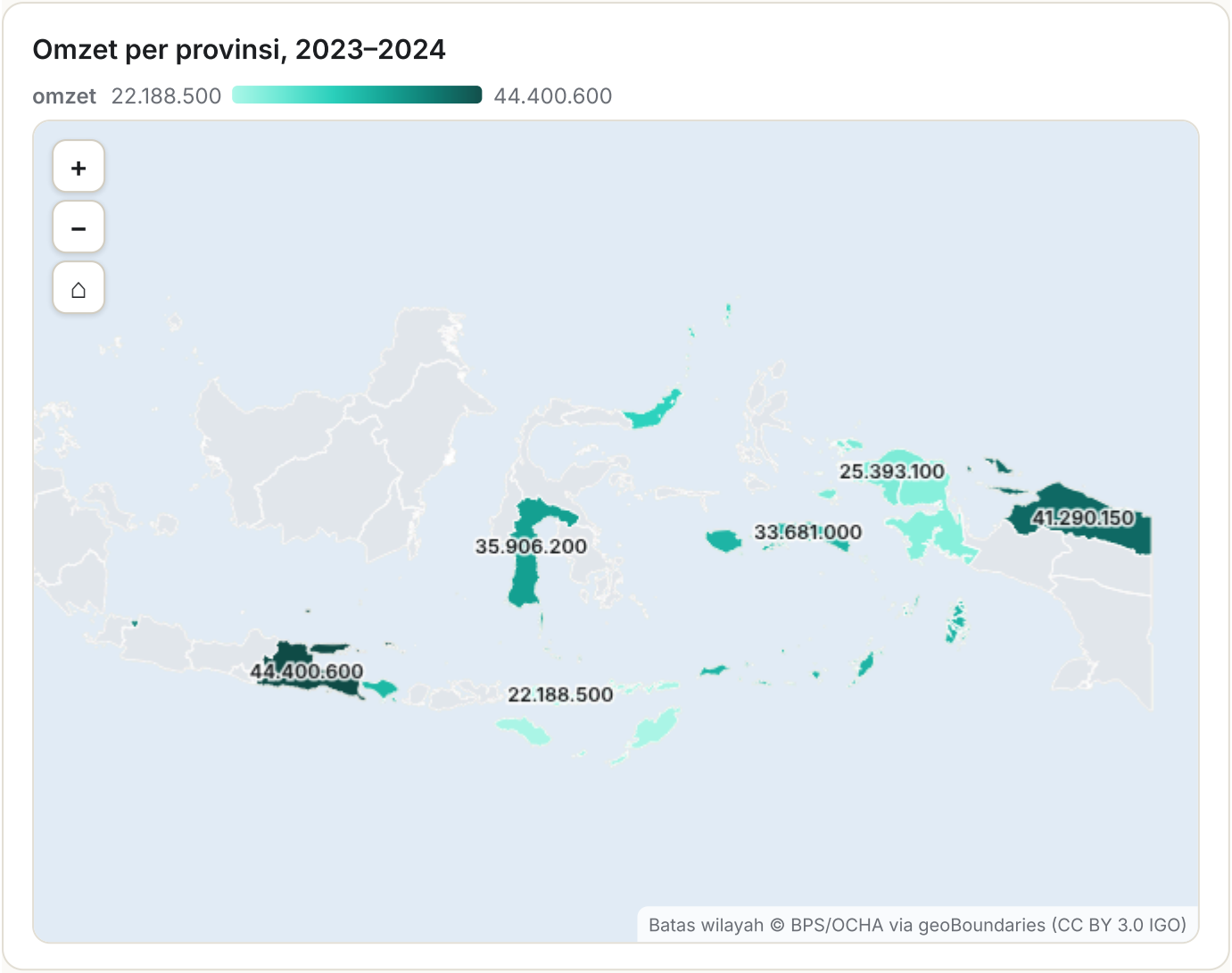


Buka di Playground ↗

23.5 Peta warna wilayah (choropleth)

Choropleth mewarnai setiap wilayah sesuai nilainya. Cocok untuk membandingkan provinsi.

```
const trx = await sl.read_csv("toko-nusantara.csv")
const perProvinsi = trx.groupby("provinsi").agg({ omzet: ["total", "sum"] })
sl.choropleth(perProvinsi, { region: "provinsi", value: "omzet", labels: true,
title: "Omzet per provinsi, 2023–2024" })
```

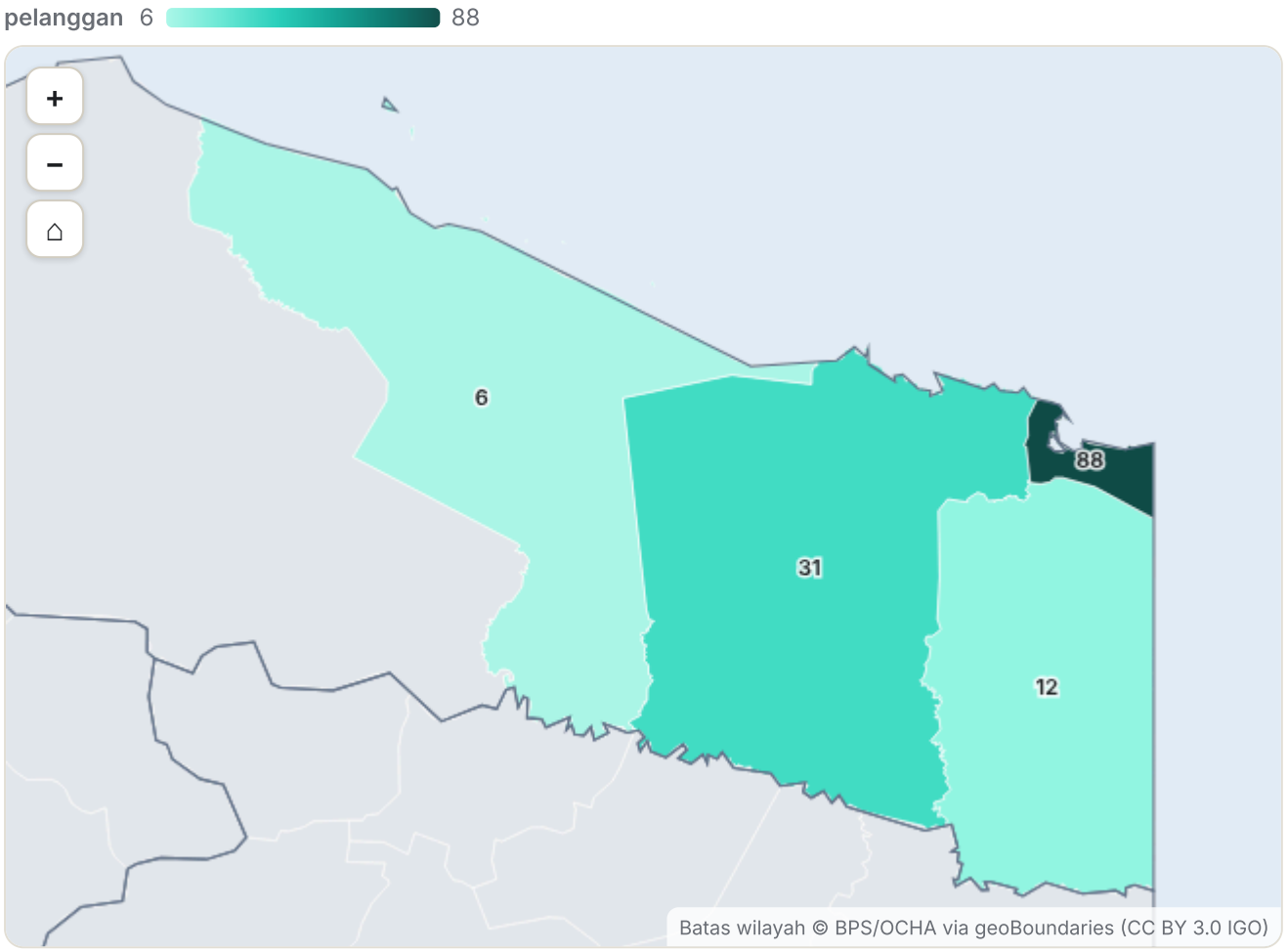


Buka di Playground ↗

Provinsi tanpa cabang tampil abu-abu. Peta seperti ini langsung menjawab pertanyaan strategis: provinsi mana yang belum terjangkau?

Untuk tingkat kabupaten/kota, tambahkan `level: "kabkota"`. Batas poligonnya dimuat otomatis dari `slak-peta-kab.js`:

```
const contoh = sl.DataFrame([
  { wilayah: "Kota Jayapura", pelanggan: 88 }, { wilayah: "Kab. Jayapura",
pelanggan: 31 },
  { wilayah: "Kab. Keerom", pelanggan: 12 }, { wilayah: "Kab. Sarmi", pelanggan: 6
}
])
sl.choropleth(contoh, { region: "wilayah", value: "pelanggan", level: "kabkota",
labels: true })
```



Buka di Playground ↗

> **API Slak baru**
>
> - `sl.choropleth(df, { region, value, labels, level, bins })` mewarnai wilayah. `level: "kabkota"` untuk kabupaten/kota.

> **Peta dan kejujuran.** Wilayah yang luas, seperti provinsi-provinsi di Papua, tampak dominan di peta walau penduduknya sedikit. Untuk data seperti omzet atau jumlah pelanggan,

> pertimbangkan nilai **per penduduk** atau tampilkan juga grafik batang di sampingnya.

23.6 Nama wilayah yang berantakan

Nama wilayah di data nyata jarang seragam: "Jaypura", "Kab. Mimika", "Timika", "Jabar". Slak mengenalinya dengan toleran: singkatan, awalan (Prov., Kab., Kota), nama lama, ibu kota kabupaten, dan salah ketik ringan.

```
for (const nama of ["Jaypura", "Timika", "Jabar", "Kota Sorong", "Manokwari"]) {
  const m = sl.geo.match(nama)
  sl.write(`${nama} → ${m ? `${m.nama} (${m.provinsi || "provinsi"})` : "tidak
dikenali"}`)
}
```

Jaypura → Kota Jayapura (Papua)

Timika → Kab. Mimika (Papua Tengah)

Jabar → Jawa Barat (provinsi)

Kota Sorong → Kota Sorong (Papua Barat Daya)

Manokwari → Kab. Manokwari (Papua Barat)

Buka di Playground ↗

Perhatikan "Timika" dikenali sebagai Kabupaten Mimika (Timika adalah ibu kotanya), dan "Manokwari" sebagai Kabupaten Manokwari. Selalu periksa hasil pencocokan untuk nama yang bisa berarti dua wilayah, misalnya kota dan kabupaten bernama sama. Wilayah yang tidak dikenali dilaporkan di bawah peta.

Untuk satu kolom sekaligus, `sl.geo.geocode` menambahkan nama baku dan koordinatnya:

```
const alamat = sl.DataFrame([{ kota: "Manokwari" }, { kota: "Ambon" }, { kota:
"Kupang" }])
sl.dataframe(sl.geo.geocode(alamat, "kota"))
```

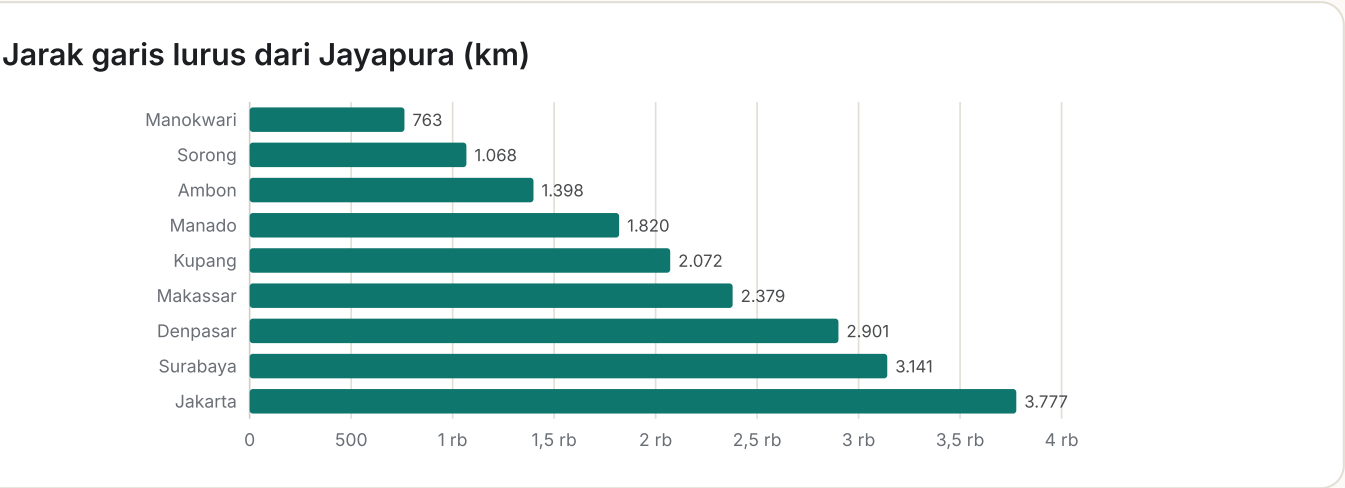
	kota	wilayah_baku	lat	lon
0	Manokwari	Kab. Manokwari	-0,9083	133,8505
1	Ambon	Kota Ambon	-3,656	128,1186
2	Kupang	Kota Kupang	-10,2042	123,6072

3 baris × 4 kolom

Buka di Playground ↗

23.7 Jarak

```
const cabang = (await sl.read_csv("cabang.csv")).to_records()
const jyp = cabang.find(c => c.kota === "Jayapura")
const jarak = cabang
  .filter(c => c.kota !== "Jayapura")
  .map(c => ({ kota: c.kota, km: Math.round(sl.geo.distance([jyp.lat, jyp.lon],
[c.lat, c.lon])) }))
  .sort((a, b) => a.km - b.km)
sl.bar_chart(jarak, { x: "kota", y: "km", horizontal: true, labels: true, title:
"Jarak garis lurus dari Jayapura (km)" })
```



Buka di Playground ↗

`sl.geo.distance` menghitung jarak garis lurus di permukaan bumi (rumus *haversine*), bukan jarak tempuh jalan atau laut.

> **API Slak baru**
>
> - `sl.geo.match(nama)` mencocokkan satu nama wilayah dan mengembalikan nama baku, provinsi, dan koordinatnya.
> - `sl.geo.geocode(df, kolom)` menambah kolom `wilayah_baku`, `lat`, dan `lon`.

> - `sl.geo.distance([lat1, lon1], [lat2, lon2])` menghitung jarak dalam km.
> - `sl.geo.province_of(lat, lon)` dan `sl.geo.kabkota_of(lat, lon)` menebak wilayah dari koordinat.

Peta tiga dimensi

Untuk peta data besar dan tampilan tiga dimensi, Slak menyediakan `sl.pydeck_chart`, yang memakai pustaka deck.gl dengan spesifikasi yang sama seperti pydeck di Python. Pustakanya diunduh saat dipakai, jadi fitur ini **butuh internet**.

> **Coba di playground** Di contoh 23.3, tambahkan `tiles: "osm"` ke opsi peta (butuh internet). Bandingkan dengan peta luring bawaan.

Kesalahan umum

Gejala	Penyebab	Perbaikan
Titik muncul di laut atau di luar negeri	`lat` dan `lon` tertukar	Lintang Indonesia sekitar −11 sampai 6, bujur 95 sampai 141
Ada wilayah tidak dikenali di bawah peta	Ejaan terlalu jauh dari nama baku	Perbaiki dengan kamus (Bab 13) atau `sl.geo.match`
Peta kosong	`slak-peta.js` belum dimuat	Tambahkan `<script src="slak-peta.js">`
Provinsi baru Papua tidak berwarna	Data memakai nama sebelum 2022	Pakai nama 38 provinsi terbaru, misalnya Papua Barat Daya

Latihan

Latihan 23.1 (mudah). Buat `perProvinsi` berisi banyak transaksi per provinsi dengan kolom `provinsi` dan `transaksi`, lalu gambar choropleth-nya.

```
const trx = await sl.read_csv("toko-nusantara.csv")
const perProvinsi = sl.DataFrame([])   // ganti

sl.choropleth(perProvinsi, { region: "provinsi", value: "transaksi" })
await sl.testing.test("10 provinsi", () =>
sl.testing.expect(perProvinsi.shape[0]).toBe(10))
await sl.testing.test("jumlahnya benar", () =>
sl.testing.expect(perProvinsi.transaksi.sum()).toBe(trx.shape[0]))
```



✕ **10 provinsi** — Diharapkan: sama persis dengan 10 — nilai sebenarnya: 0

✕ **jumlahnya benar** — Cannot read properties of undefined (reading 'sum')

Buka di Playground ↗

Latihan 23.2 (sedang). Cari cabang yang **paling jauh** dari cabang Jakarta. Simpan nama kotanya di `terjauh`.

```
const cabang = (await sl.read_csv("cabang.csv")).to_records()
const terjauh = ""   // ganti

sl.write(terjauh)
await sl.testing.test("terjauh dari Jakarta", () =>
sl.testing.expect(terjauh).toBe("Jayapura"))
```

✕ **terjauh dari Jakarta** — Diharapkan: sama persis dengan "Jayapura" — nilai sebenarnya: ""

Buka di Playground ↗

Latihan 23.3 (tantangan). Lengkapi `bakukan(daftar)` yang mengubah larik nama kota berantakan menjadi nama baku dari `sl.geo.match`, atau `null` bila tidak dikenali.

```
function bakukan(daftar) {
  return daftar
}

const h = bakukan(["Jaypura", "Timika", "Kota Sorong", "Planet Mars"])
await sl.testing.test("Jaypura dibakukan", () => sl.testing.expect(h[0]).toBe("Kota Jayapura"))
await sl.testing.test("Timika → Kab. Mimika", () =>
sl.testing.expect(h[1]).toBe("Kab. Mimika"))
await sl.testing.test("tidak dikenali → null", () =>
sl.testing.expect(h[3]).toBeNull())
```

✕ **Jaypura dibakukan** — Diharapkan: sama persis dengan "Kota Jayapura" — nilai sebenarnya: "Jaypura"

✕ **Timika → Kab. Mimika** — Diharapkan: sama persis dengan "Kab. Mimika" — nilai sebenarnya: "Timika"

✕ **tidak dikenali → null** — Diharapkan: kosong (null) — nilai sebenarnya: "Planet Mars"

Buka di Playground ↗

Mini-proyek: peta penjualan Toko Nusantara

```
sl.title("Peta Penjualan Toko Nusantara :world_map:")

const [trx, cabang] = await Promise.all([sl.read_csv("toko-nusantara.csv"),
sl.read_csv("cabang.csv")])
const ukuran = sl.segmented_control("Ukuran", ["Omzet", "Transaksi", "Pelanggan"],
{ default: "Omzet" })
```

```
const kategori = sl.selectbox("Kategori", ["Semua",
...trx.kategori.unique().sort()])

const data = kategori === "Semua" ? trx : trx.filter(r => r.kategori === kategori)
const kolom = { Omzet: "omzet", Transaksi: "transaksi", Pelanggan: "pelanggan" }
[ukuran]

const perCabang = data.groupby("cabang").agg({
  omzet: ["total", "sum"], transaksi: ["id_transaksi", "nunique"], pelanggan:
["pelanggan_id", "nunique"]
}).merge(cabang, { on: "cabang" })

const [kiri, kanan] = sl.columns([3, 2])
kiri.map(perCabang, { lat: "lat", lon: "lon", size: kolom, label: "kota" })
kanan.bar_chart(perCabang, { x: "kota", y: kolom, horizontal: true, sort: true,
format: "compact", height: 380 })

const perProv = data.groupby("provinsi").agg({ nilai: [kolom === "omzet" ? "total"
: kolom === "transaksi" ? "id_transaksi" : "pelanggan_id", kolom === "omzet" ?
"sum" : "nunique"] })
sl.choropleth(perProv, { region: "provinsi", value: "nilai", labels: true, title:
`${ukuran} per provinsi${kategori === "Semua" ? "" : " · " + kategori}` })

const teratas = perCabang.sort_values(kolom, { ascending: false }).to_records()[0]
sl.info(`${ukuran} ${kategori === "Semua" ? "" : kategori.toLowerCase() + "
"}tertinggi ada di **${teratas.kota}**, ${teratas.provinsi}.`)
```

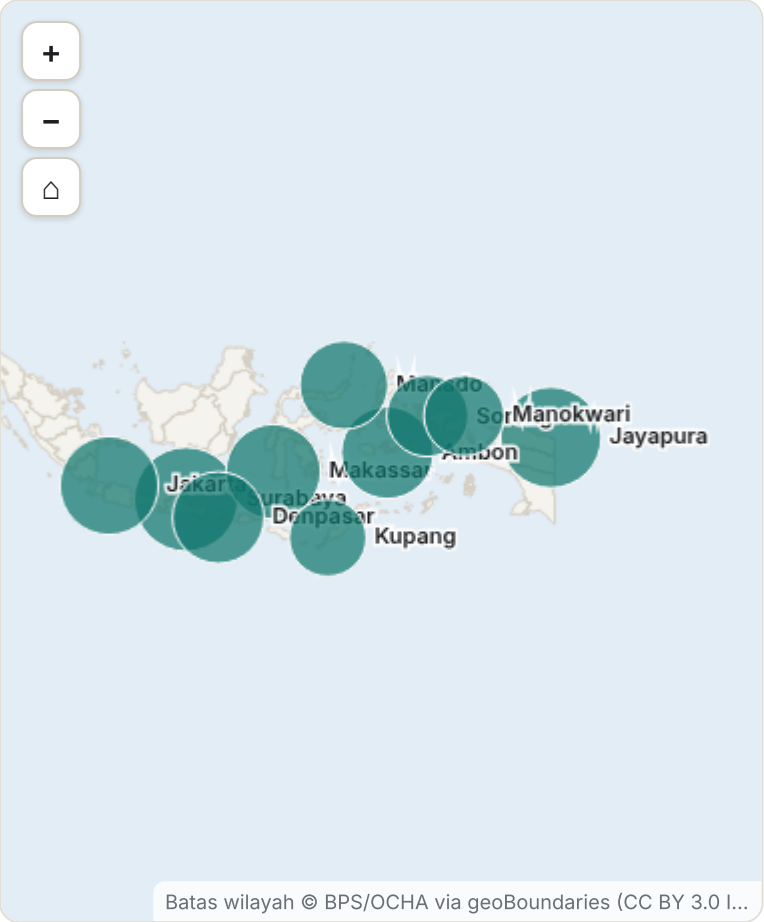
Peta Penjualan Toko Nusantara 🗺

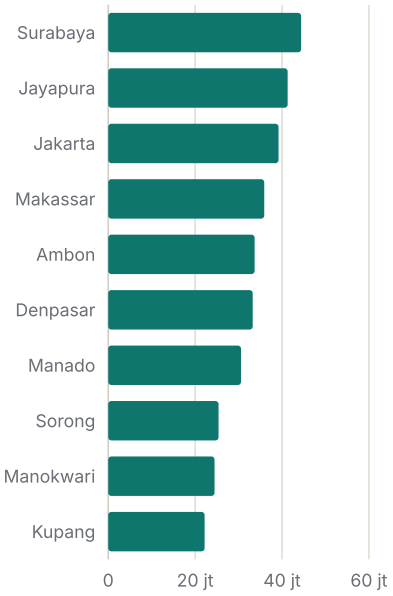
Ukuran

Omzet | Transaksi | Pelanggan

Kategori

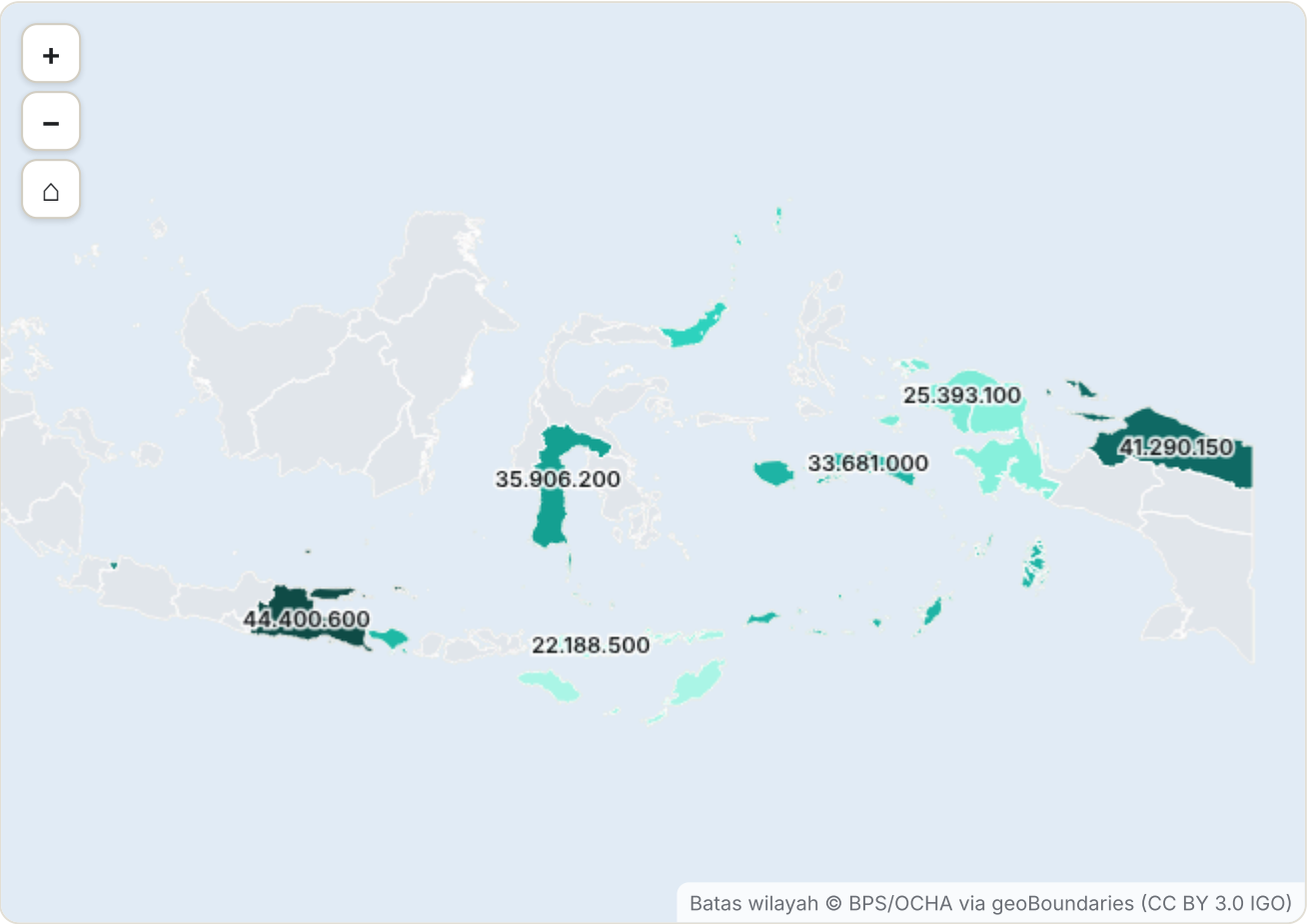
Semua





Omzet per provinsi

nilai 22.188.500 44.400.600



Omzet tertinggi ada di **Surabaya**, Jawa Timur.

Buka di Playground ↗

`sl.columns([3, 2])` membagi baris dengan perbandingan lebar 3 : 2. Peta di kiri dan grafik batang di kanan saling melengkapi: peta memperlihatkan **di mana**, grafik memperlihatkan **berapa** dengan tepat.

← 22. Tren, Prediksi, dan Machine Learning Dasar

24. Merapikan dan Menguji Kode →

Bab 24 · Merapikan dan Menguji Kode

> **Tujuan belajar**
>
> - Memahami mengapa satu berkas besar menyulitkan, dan cara merapikannya.
> - Memisahkan fungsi pembersihan, analisis, dan tampilan.
> - Memakai berkas JavaScript tambahan, dan mengenal `import` / `export` ES Modules, tanpa build.
> - Membuat class sederhana untuk membungkus logika yang dipakai ulang.
> - Menguji aplikasi dengan `sl.testing` supaya angka laporan tetap benar setiap kali kode diubah.
> - Menulis komentar dan nama yang membantu dirimu sendiri enam bulan lagi.

Laporan di Bab 17 sekitar 50 baris. Dasbor di Bab 25 akan jauh lebih panjang. Kode yang panjang tanpa susunan seperti gudang tanpa rak: semuanya ada, tetapi tidak ada yang bisa ditemukan. Bab ini tentang menyusun rak.

24.1 Tanda-tanda kode perlu dirapikan

- Kamu menyalin-tempel potongan kode yang sama di dua tempat.
- Kamu takut mengubah satu baris karena tidak tahu apa lagi yang ikut rusak.
- Kamu harus menggulir jauh untuk menemukan sesuatu.
- Nama variabel seperti `x2`, `data3`, atau `tmp` bertebaran.

Obatnya satu prinsip: **satu bagian, satu tugas.** Pisahkan kode yang **menyiapkan data**, yang **menghitung**, dan yang **menampilkan**.

24.2 Rapi di dalam satu berkas

Langkah pertama tidak butuh berkas baru. Kelompokkan fungsi ke dalam objek bernama, sehingga jelas fungsi mana untuk apa:

```
// ---------- Data ----------
const Data = {
  async muat() {
    const df = await sl.read_csv("toko-nusantara.csv")
    return df.assign({ tahun: () => df.tanggal.dt.year })
  }
}

// ---------- Analisis ----------
const Analisis = {
```

```
  ringkas(df) {
    return { omzet: df.total.sum(), transaksi: df.shape[0], pelanggan:
df.pelanggan_id.nunique() }
  },
  perKategori(df) {
    return df.groupby("kategori").agg({ omzet: ["total", "sum"]
}).sort_values("omzet", { ascending: false })
  }
}

// ---------- Tampilan ----------
const Tampilan = {
  kartu(ring) {
    const [a, b, c] = sl.columns(3)
    a.metric("Omzet", ring.omzet, null, { format: "rp" })
    b.metric("Transaksi", ring.transaksi)
    c.metric("Pelanggan", ring.pelanggan)
  }
}

// ---------- Aplikasi ----------
const df = await Data.muat()
const tahun = sl.segmented_control("Tahun", [2023, 2024], { default: 2024 })
const terpilih = df.filter(r => r.tahun === tahun)
Tampilan.kartu(Analisis.ringkas(terpilih))
sl.bar_chart(Analisis.perKategori(terpilih), { x: "kategori", y: "omzet", format:
"compact" })
```

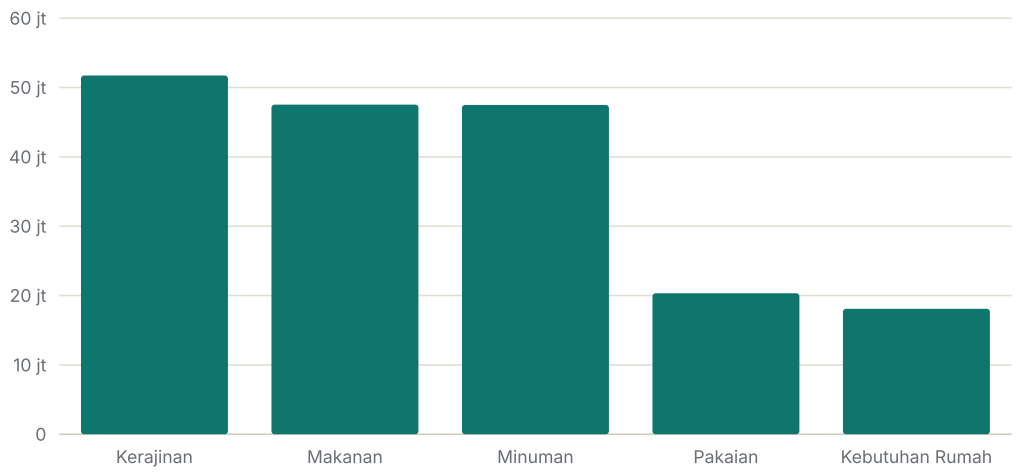
Tahun

2023 | 2024

Omzet
Rp 185.186.150

Transaksi
1.580

Pelanggan
559



Buka di Playground ↗

Bagian **Aplikasi** di bawah kini hanya lima baris dan bisa dibaca seperti kalimat. Kalau angka omzet salah, kamu tahu harus mencari di `Analisis`. Kalau tampilan kartu kurang bagus, cari di `Tampilan`.

Di Slak Studio, ganti garis pemisah `// ----------` dengan `// %% Data`, `// %% Analisis`, dan seterusnya. Kode yang sama sekaligus menjadi sel-sel di **mode sel** (Bab 1): data cukup dimuat sekali, lalu bagian tampilan bisa diubah berkali-kali dengan cepat.

24.3 Berkas JavaScript tambahan

Fungsi yang dipakai di beberapa aplikasi sebaiknya tinggal di berkas sendiri. Cara paling sederhana, dan yang paling setia pada **Write and Run**, adalah berkas JavaScript biasa yang dimuat dengan tag `<script>`:

```
// toko-analisis.js
window.TokoAnalisis = {
  saring(df, { tahun, cabang = [], kanal = "Semua" } = {}) {
    return df
      .filter(r => tahun == null || r.tanggal.getFullYear() === tahun)
      .filter(r => cabang.length === 0 || cabang.includes(r.cabang))
      .filter(r => kanal === "Semua" || r.kanal === kanal)
  },
  ringkas(df) {
    return { omzet: df.total.sum(), transaksi: df.shape[0], pelanggan:
df.pelanggan_id.nunique(),
             rata: df.shape[0] ? df.total.sum() / df.shape[0] : 0 }
  },
  banding(a, b) {
    if (!a) return null
```

```
    return new Intl.NumberFormat("id-ID", { style: "percent",
maximumFractionDigits: 1, signDisplay: "always" }).format(b / a - 1)
  }
}
```

```
<script src="slak.js"></script>
<script src="slak-data.js"></script>
<script src="toko-analisis.js"></script>
<script type="text/slak">
  const df = await sl.read_csv("toko-nusantara.csv")
  const r = TokoAnalisis.ringkas(TokoAnalisis.saring(df, { tahun: 2024 }))
  sl.metric("Omzet 2024", r.omzet, null, { format: "rp" })
</script>
```

- Semua fungsi dikumpulkan di **satu objek** `TokoAnalisis`, supaya namanya tidak bertabrakan dengan nama lain.
- `window.TokoAnalisis = ...` membuat objek itu bisa dipakai di mana saja di halaman.
- Cara ini jalan di semua tempat: klik dua kali, Slak Studio, dan hosting.
- Parameter `{ tahun, cabang = [], kanal = "Semua" } = {}` adalah destructuring dengan nilai bawaan (Bab 5 dan 7): fungsi boleh dipanggil hanya dengan sebagian pilihan.

`toko-analisis.js` ada di folder bahan buku, dan halaman buku ini sudah memuatnya, jadi contoh berikut bisa dijalankan di sini:

```
const df = await sl.read_csv("toko-nusantara.csv")
const ini = TokoAnalisis.ringkas(TokoAnalisis.saring(df, { tahun: 2024, kanal:
"Online" }))
const lalu = TokoAnalisis.ringkas(TokoAnalisis.saring(df, { tahun: 2023, kanal:
"Online" }))
sl.metric("Omzet online 2024", ini.omzet, TokoAnalisis.banding(lalu.omzet,
ini.omzet), { format: "rp" })
```

Omzet online 2024

Rp 66.275.500

▲ +48,3%

Buka di Playground ↗

24.4 ES Modules: import dan export

JavaScript modern punya cara resmi untuk berbagi kode antarberkas: **ES Modules**. Sebuah berkas menandai apa yang dibagikan dengan `export`, dan berkas lain mengambilnya dengan `import`. Tidak perlu build: browser memahaminya langsung.

```
// toko-analisis.mjs
export function ringkas(df) {
  return { omzet: df.total.sum(), transaksi: df.shape[0], pelanggan:
df.pelanggan_id.nunique() }
}
export const rupiah = (x) =>
  new Intl.NumberFormat("id-ID", { style: "currency", currency: "IDR",
maximumFractionDigits: 0 }).format(x)
```

Di skrip Slak, ambil modul dengan `await import(...)`:

```
try {
  const { ringkas, rupiah } = await import(new URL("toko-analisis.mjs",
location.href))
  const df = await sl.read_csv("toko-nusantara.csv")
  sl.write(`Omzet seluruhnya: ${rupiah(ringkas(df).omzet)}`)
} catch (e) {
  sl.warning("Modul tidak bisa dimuat. ES Modules hanya jalan lewat hosting atau
server lokal, tidak saat halaman dibuka dengan klik dua kali. " + e.message)
}
```

Omzet seluruhnya: Rp 330.315.950

Buka di Playground ↗

- `new URL("toko-analisis.mjs", location.href)` membuat alamat lengkap berkas modul di folder halaman. Skrip Slak dijalankan dari dalam Slak, jadi alamat relatif saja tidak cukup.
- `const { ringkas, rupiah } = ...` mengambil dua fungsi yang di-`export` sekaligus.

Batasnya: browser tidak mengizinkan modul dimuat dari halaman yang dibuka dengan klik dua kali (alamat `file://`). Karena itu buku ini menganjurkan:

Cara	Klik dua kali	Hosting	Kapan dipakai
Objek di berkas yang sama	✓	✓	Aplikasi kecil sampai menengah

Cara	Klik dua kali	Hosting	Kapan dipakai
Berkas `.js` biasa dengan `<script src>`	✓	✓	Fungsi yang dipakai di beberapa aplikasi
ES Modules `import` / `export`	×	✓	Proyek yang pasti diterbitkan ke hosting, atau saat bekerja dengan pustaka modern

Mengenal `import` dan `export` tetap penting: hampir semua kode JavaScript modern di internet ditulis dengan cara ini, dan kamu akan sering membacanya.

24.5 Class

Kadang data dan fungsi yang mengolahnya selalu berpasangan. **Class** membungkus keduanya menjadi satu "cetakan" yang bisa dibuat berkali-kali.

```
class Cabang {
  constructor(nama, df) {
    this.nama = nama
    this.data = df.filter(r => r.cabang === nama)
  }
  get omzet() { return this.data.total.sum() }
  omzetTahun(t) { return this.data.filter(r => r.tanggal.getFullYear() ===
t).total.sum() }
  pertumbuhan() { return this.omzetTahun(2024) / this.omzetTahun(2023) - 1 }
  toString() { return `${this.nama}: tumbuh ${(this.pertumbuhan() *
100).toLocaleString("id-ID", { maximumFractionDigits: 1 })}%` }
}

const df = await sl.read_csv("toko-nusantara.csv")
const semua = df.cabang.unique().map(n => new Cabang(n, df))
const tercepat = semua.reduce((a, c) => c.pertumbuhan() > a.pertumbuhan() ? c : a)
sl.write(`Tumbuh paling cepat: ${tercepat}`)
sl.dataframe(semua.map(c => ({ cabang: c.nama, omzet: c.omzet, tumbuh:
c.pertumbuhan() })), { format: { omzet: "rupiah", tumbuh: "percent" } })
```

```
Tumbuh paling cepat: Nusantara Ambon: tumbuh 95,3%
```

	cabang	omzet	tumbuh
0	Nusantara Ambon	Rp 33.681.000	95,3%
1	Nusantara Surabaya	Rp 44.400.600	-5,6%
2	Nusantara Jayapura	Rp 41.290.150	43,6%
3	Nusantara Manado	Rp 30.546.500	-0,8%
4	Nusantara Makassar	Rp 35.906.200	8,2%
5	Nusantara Manokwari	Rp 24.455.250	73,8%
6	Nusantara Kupang	Rp 22.188.500	47,3%
7	Nusantara Denpasar	Rp 33.258.200	56,4%
8	Nusantara Jakarta	Rp 39.196.450	3,2%
9	Nusantara Sorong	Rp 25.393.100	27,0%

10 baris × 3 kolom

Buka di Playground ↗

Bagian	Arti
`class Cabang { ... }`	Cetakan bernama `Cabang`
`constructor(...)`	Dijalankan saat objek dibuat dengan `new Cabang(...)`
`this.nama`	Properti milik objek ini
`get omzet()`	Properti yang dihitung; dibaca tanpa kurung: `c.omzet`
`omzetTahun(t)`	Metode: fungsi milik objek

Kamu sudah memakai class sejak Bab 14: `sl.DataFrame` dan `sl.ml.KMeans` adalah class. Untuk aplikasi analisis, fungsi dan objek biasa sering sudah cukup; pakai class bila kamu membuat **banyak objek sejenis** yang masing-masing punya data dan perilaku.

24.6 Menguji aplikasi

Di Bab 9 kamu menguji fungsi. Kini uji **aplikasinya**: jalankan aplikasi di balik layar, ubah widgetnya dari kode, lalu periksa apa yang tampil.

```
const aplikasi = `
const df = await sl.read_csv("toko-nusantara.csv")
const tahun = Number(sl.selectbox("Tahun", ["2024", "2023"]))
const kanal = sl.radio("Kanal", ["Semua", "Toko", "Online"])
```

```
const data = df.filter(r => r.tanggal.getFullYear() === tahun && (kanal === "Semua"
|| r.kanal === kanal))
sl.metric("Omzet", data.total.sum())
sl.metric("Transaksi", data.shape[0])
`


const { test, expect } = sl.testing
const df = await sl.read_csv("toko-nusantara.csv")

await test("bawaan menampilkan 2024", async () => {
  const at = await sl.testing.run(aplikasi)
  expect(at.metric("Transaksi").value).toBe(df.filter(r => r.tanggal.getFullYear()
=== 2024).shape[0])
})

await test("filter Online 2023 benar", async () => {
  const at = await sl.testing.run(aplikasi)
  await at.selectbox("Tahun").set_value("2023").run()
  await at.radio("Kanal").set_value("Online").run()
  const harapan = df.filter(r => r.tanggal.getFullYear() === 2023 && r.kanal ===
"Online").total.sum()
  expect(at.metric("Omzet").value).toBe(harapan)
})

await test("Toko + Online = Semua", async () => {
  const at = await sl.testing.run(aplikasi)
  const semua = at.metric("Omzet").value
  await at.radio("Kanal").set_value("Toko").run()
  const toko = at.metric("Omzet").value
  await at.radio("Kanal").set_value("Online").run()
  expect(toko + at.metric("Omzet").value).toBe(semua)
})
```

- ✓ bawaan menampilkan 2024 (8 md)
- ✓ filter Online 2023 benar (17 md)
- ✓ Toko + Online = Semua (14 md)

Buka di Playground ↗

Uji ketiga menarik: ia tidak tahu angka pastinya, tetapi tahu **hubungan** yang harus selalu benar. Uji seperti ini menangkap kesalahan filter yang paling sering terjadi.

> **API Slak baru**
>
> - `await sl.testing.run(kode)` menjalankan aplikasi tersembunyi dan mengembalikan objek uji `at`.
> - `at.selectbox("Label").set_value(...)`, `at.button("Label").click()`, lalu `await ... .run()` untuk menjalankan ulang.
> - `at.metric("Label").value`, `at.dataframe[0].value`, `at.exception` untuk memeriksa hasil.
> - `sl.testing.from_string(kode)`, `from_file("app.slak")`, dan `from_function(fn)` membuat objek uji tanpa langsung menjalankannya.

Berkas uji di Slak Studio

Di Slak Studio, simpan uji di berkas `nama.uji.slak` di samping aplikasinya, misalnya `laporan.uji.slak` untuk `laporan.slak`:

```
const { test, expect } = sl.testing

await test("laporan jalan tanpa galat", async () => {
  const at = await sl.testing.run("laporan.slak")
  expect(at.exception).toHaveLength(0)
})
```

Panel **Uji** di bilah kiri Slak Studio menjalankan semua berkas uji di folder itu. **Uji cepat** memastikan berkas yang sedang dibuka jalan tanpa galat, dan **Buat berkas uji** membuat kerangkanya. Di luar Slak Studio, buka `slak-test.html` dari paket Slak.

Jalankan uji setiap kali selesai mengubah kode. Bila semuanya hijau, kamu boleh yakin perubahanmu tidak merusak yang lama.

24.7 Komentar dan nama untuk dirimu enam bulan lagi

Kode dibaca jauh lebih sering daripada ditulis, dan pembaca yang paling sering adalah dirimu sendiri, beberapa bulan kemudian, yang sudah lupa semuanya.

Nama yang baik menjelaskan isinya:

Kurang jelas	Lebih jelas
`d`, `data2`, `tmp`	`transaksi2024`, `omzetPerKota`
`hitung()`	`hitungMarginKotor()`

Kurang jelas	Lebih jelas
`f` (boolean)	`sudahLunas`, `adaStok`
`x > 35000000`	`omzet >= TARGET_OMZET_BULANAN`

Komentar menjelaskan mengapa, bukan apa. Kode sudah mengatakan *apa* yang dikerjakan.

```
// Kurang berguna: mengulang yang sudah jelas dari kode
// kurangi 1 dari umur
umur--

// Berguna: menjelaskan alasan yang tidak terlihat dari kode
// Belum berulang tahun tahun ini, jadi umurnya belum bertambah
if (!sudahUlangTahun) umur--
```

Tulis juga satu komentar di atas setiap fungsi penting: apa yang diterima, apa yang dikembalikan. Lihat lagi `toko-analisis.js` di bahan buku sebagai contoh.

Kesalahan umum

Gejala	Penyebab	Perbaikan
`TokoAnalisis is not defined`	Berkas `.js` belum dimuat, atau dimuat sesudah skrip Slak	`<script src>` di `<head>`, sebelum `<script type="text/slak">`
`Failed to fetch dynamically imported module`	Modul dimuat dari halaman klik dua kali	Pakai berkas `.js` biasa, atau buka lewat hosting
`Failed to resolve module specifier`	Alamat modul relatif	`import(new URL("x.mjs", location.href))`
Lupa `new` saat membuat objek class	`Cabang("x", df)`	`new Cabang("x", df)`
Uji lulus padahal aplikasi salah	Uji hanya memeriksa ada tidaknya, bukan nilainya	Periksa nilai, atau hubungan antarnilai

Latihan

Latihan 24.1 (mudah). Lengkapi objek `Format` dengan fungsi `rupiah(x)` dan `persen(x)` bergaya Indonesia.

```
const Format = {
  rupiah(x) { return "" },
  persen(x) { return "" }
}

await sl.testing.test("rupiah", () =>
sl.testing.expect(Format.rupiah(1250000)).toBe(new Intl.NumberFormat("id-ID", {
style: "currency", currency: "IDR", maximumFractionDigits: 0 }).format(1250000)))
await sl.testing.test("persen", () =>
sl.testing.expect(Format.persen(0.125)).toBe("12,5%"))
```

✕ **rupiah** — Diharapkan: sama persis dengan "Rp 1.250.000" — nilai sebenarnya: ""

✕ **persen** — Diharapkan: sama persis dengan "12,5%" — nilai sebenarnya: ""

Buka di Playground ↗

Latihan 24.2 (sedang). Lengkapi class `Produk` dengan getter `margin` (persen laba kotor dari harga jual) dan metode `laku(n)` yang mengembalikan laba kotor bila terjual `n` buah.

```
class Produk {
  constructor(nama, hargaJual, hargaPokok) {
    this.nama = nama
    this.hargaJual = hargaJual
    this.hargaPokok = hargaPokok
  }
  // tambahkan get margin() dan laku(n)
}

const kopi = new Produk("Kopi Wamena", 85000, 52000)
await sl.testing.test("margin kopi", () =>
sl.testing.expect(kopi.margin).toBeCloseTo(0.388, 3))
await sl.testing.test("laba 10 buah", () =>
sl.testing.expect(kopi.laku(10)).toBe(330000))
```

✕ **margin kopi** — Diharapkan: mendekati 0.388 (3 desimal) — nilai sebenarnya: undefined

✕ **laba 10 buah** — kopi.laku is not a function

Buka di Playground ↗

Latihan 24.3 (tantangan). Tulis uji aplikasi yang memastikan aplikasi `kalkulator` di bawah menghitung dengan benar saat diskon diubah menjadi 25.

```
const kalkulator = `
const harga = sl.number_input("Harga", { value: 100000 })
const diskon = sl.slider("Diskon (%)", 0, 90, 10)
sl.metric("Bayar", harga * (1 - diskon / 100))
`

await sl.testing.test("diskon 25% dari 100 ribu = 75 ribu", async () => {
  const at = await sl.testing.run(kalkulator)
  // ubah slider, jalankan ulang, lalu periksa metric "Bayar"
  sl.testing.expect(true).toBe(false)
})
```

✕ **diskon 25% dari 100 ribu = 75 ribu** — Diharapkan: sama persis dengan false — nilai sebenarnya: true

Buka di Playground ↗

Mini-proyek: laporan Bab 17 yang rapi dan teruji

Laporan Bab 17 disusun ulang: logika di `toko-analisis.js`, tampilan di aplikasi, dan uji yang memastikan angkanya benar.

```
// %% Data
const df = await sl.read_csv("toko-nusantara.csv")

// %% Pilihan
const [f1, f2] = sl.columns(2)
const tahun = Number(f1.selectbox("Tahun", ["2024", "2023"], { key: "rapi-tahun"
}))
const kanal = f2.radio("Kanal", ["Semua", "Toko", "Online"], { horizontal: true,
key: "rapi-kanal" })

// %% Hitung
const ini = TokoAnalisis.saring(df, { tahun, kanal })
```

```
const lalu = TokoAnalisis.saring(df, { tahun: tahun - 1, kanal })
const r = TokoAnalisis.ringkas(ini), r0 = TokoAnalisis.ringkas(lalu)

// %% Tampilkan
const [k1, k2, k3] = sl.columns(3)
k1.metric("Omzet", r.omzet, TokoAnalisis.banding(r0.omzet, r.omzet), { format: "rp"
})
k2.metric("Transaksi", r.transaksi, TokoAnalisis.banding(r0.transaksi,
r.transaksi))
k3.metric("Pelanggan", r.pelanggan, TokoAnalisis.banding(r0.pelanggan,
r.pelanggan))

// %% Uji
const { test, expect } = sl.testing
await test("ringkas() menjumlah dengan benar", () => {
  const kecil = sl.DataFrame([{ total: 10, pelanggan_id: "A" }, { total: 30,
pelanggan_id: "A" }])
  expect(TokoAnalisis.ringkas(kecil)).toEqual({ omzet: 40, transaksi: 2, pelanggan:
1, rata: 20 })
})
await test("Toko + Online = Semua", () => {
  const s = TokoAnalisis.ringkas(TokoAnalisis.saring(df, { tahun })).omzet
  const t = TokoAnalisis.ringkas(TokoAnalisis.saring(df, { tahun, kanal: "Toko"
})).omzet
  const o = TokoAnalisis.ringkas(TokoAnalisis.saring(df, { tahun, kanal: "Online"
})).omzet
  expect(t + o).toBe(s)
})
await test("tanpa data tahun lalu, banding() kosong", () =>
expect(TokoAnalisis.banding(0, 100)).toBeNull())
```

Tahun

2024

Kanal

(•) Semua ( ) Toko ( ) Online

Omzet	Transaksi	Pelanggan
Rp 185.186.150	**1.580**	**559**
▲ +27,6%	▲ +17,1%	▲ +7,7%

✓ ringkas() menjumlah dengan benar (0 md)

✓ Toko + Online = Semua (38 md)

✓ tanpa data tahun lalu, banding() kosong (0 md)

Buka di Playground ↗

Bagian **Uji** bisa dipindah ke `laporan.uji.slak` supaya tidak tampil di laporan. Contoh ini memakai `toko-analisis.js`, jadi jalankan di halaman buku ini atau di Slak Studio dengan berkas itu di folder yang sama.

← 23. Peta dan Data Wilayah

25. Proyek Akhir: Dasbor Portofolio Multi-Halaman →

Bab 25 · Proyek Akhir: Dasbor Portofolio Multi-Halaman

> **Tujuan belajar**
>
> - Merancang dasbor dari pertanyaan bisnis.
> - Menyusun aplikasi multi-halaman dengan `sl.navigation` dan `sl.page`.
> - Membuat filter global yang berlaku di semua halaman.
> - Melindungi satu halaman dengan login dan peran.
> - Menambahkan halaman "Tentang Saya" untuk portofolio.
> - Menerbitkan dasbor sebagai aplikasi yang bisa dipasang di ponsel.

Inilah puncak perjalananmu. Dasbor ini menggabungkan hampir semua yang kamu pelajari: membaca dan menggabungkan empat tabel (Bab 11, 18), meringkas (Bab 15), grafik (Bab 16), deret waktu (Bab 20), Pareto dan prakiraan (Bab 22), peta (Bab 23), dan kode yang rapi (Bab 24). Hasilnya adalah karya portofolio yang bisa kamu lampirkan saat melamar kerja.

25.1 Dari pertanyaan ke halaman

Tulis pertanyaan bisnisnya dulu, lalu jadikan setiap kelompok pertanyaan satu halaman:

Halaman	Pertanyaan yang dijawab	Bab rujukan
Ringkasan	Bagaimana kinerja tahun ini dibanding tahun lalu? Apa tiga temuan utamanya?	15, 17
Penjualan	Cabang, kategori, dan hari apa yang paling ramai?	15, 16
Produk	Produk mana yang paling penting dan paling menguntungkan?	18, 22
Wilayah	Di mana penjualan terkonsentrasi?	23
Prakiraan	Berapa omzet beberapa bulan ke depan?	20, 22
Target	Cabang mana yang mencapai target? (khusus manajer)	18
Tentang Saya	Siapa pembuatnya, apa keahliannya, dan metode apa yang dipakai?	

25.2 Aplikasi multi-halaman

Setiap halaman ditulis sebagai **fungsi** yang menerima `sl`. `sl.page` mendaftarkannya, dan `sl.navigation` membuat menunya:

```
const beranda = (sl) => { sl.title("Beranda"); sl.write("Selamat datang!") }
const data = (sl) => { sl.title("Data"); sl.dataframe([{ kota: "Jayapura", omzet:
41 }]) }

await sl.navigation([
  sl.page("Beranda", beranda, { icon: "🏠" }),
  sl.page("Data", data, { icon: "📊" })
]).run()
```

- `sl.navigation([...])` menampilkan menu di sidebar dan memilih halaman yang sedang dibuka; `.run()` menjalankan fungsinya.
- Setiap halaman punya alamat sendiri, misalnya `#/data`, sehingga tombol Back browser bekerja dan halaman bisa dibagikan langsung.
- Menu bisa dikelompokkan: `sl.navigation({ "Analisis": [...], "Profil": [...] })`.
- Berpindah halaman dari kode: `sl.switch_page("Data")`. Tautan ke halaman lain: `sl.page_link("Data")`.

Contoh di bab ini tidak dijalankan di halaman buku, karena halaman buku sendiri sudah memakai navigasi untuk daftar bab. Jalankan di playground atau Slak Studio.

> **API Slak baru**
>
> - `sl.navigation(daftarHalaman, { position })` dan `sl.page(judul, fungsi, { icon, url, default })` membuat aplikasi multi-halaman. `position`: `"sidebar"` (bawaan), `"top"`, atau `"hidden"`.
> - `sl.switch_page(judul)` dan `sl.page_link(judul)` berpindah halaman.

25.3 Filter global

Kode di luar fungsi halaman berjalan di **semua** halaman. Itulah tempat yang tepat untuk membaca data sekali dan menaruh filter yang berlaku di mana-mana:

```
sl.sidebar.header("Filter")
const tahun = Number(sl.sidebar.selectbox("Tahun", ["2024", "2023"], { key:
"f_tahun" }))
const kanal = sl.sidebar.radio("Kanal", ["Semua", "Toko", "Online"], { key:
"f_kanal" })

const saring = (df) => df.filter(r => kanal === "Semua" || r.kanal === kanal)
const ini = saring(semua.filter(r => r.tahun === tahun))
```

Variabel `ini` lalu dipakai oleh setiap fungsi halaman. Karena filter ditulis sebelum `sl.navigation`, ia tampil di sidebar semua halaman, dan `key` menjaga pilihannya saat pengguna berpindah halaman.

25.4 Satu halaman khusus manajer

Halaman **Target** berisi angka internal yang tidak untuk semua orang. `sl.login` di dalam fungsi halaman itu menampilkan formulir masuk hanya saat halaman itu dibuka; halaman lain tetap terbuka untuk umum.

```
async function halTarget(sl) {
  await sl.login({
    users: {
      manajer: { password:
"$2y$10$h9g8XzzwmQwTu8Biy2kxzur5ioPnQbi.AzHsPpTxCIVMQ7GQN65T6", name: "Manajer
Wilayah", role: "manajer" },
      staf: { password:
"$2y$10$dIcMwFfIEjs3i5STBTCIO.k88fjSkemiaIxaT1b20blp4rYaqmeKK", name: "Staf Toko",
role: "staf" }
    },
    title: "Halaman ini khusus manajer"
  })
  sl.require_role("manajer")
  sl.title("Capaian Target")
  sl.caption(`Masuk sebagai ${sl.user.name}`)
  // ...
}
```

- Password tidak pernah ditulis apa adanya, melainkan sebagai **hash bcrypt**. Buat hash-nya dengan `await sl.crypto.hash_password("passwordmu")`, atau di Slak Studio lewat **Alat › Lindungi password untuk sl.login**. Di contoh ini, password `manajer` adalah `manajer123` dan password `staf` adalah `staf123`.
- `sl.require_role("manajer")` menghentikan halaman dengan pesan bila pengguna yang masuk bukan manajer. Coba masuk sebagai `staf`.
- `sl.user` berisi `name`, `role`, dan `has_role()`. `sl.logout()` keluar.

> **API Slak baru**
>
> - `await sl.login({ users, title })`, `sl.require_role(peran)`, `sl.user`, `sl.logout()`.

> **Batas login di browser.** Login ini berjalan sepenuhnya di browser. Ia cocok untuk memisahkan tampilan manajer dan staf, tetapi **tidak menyembunyikan data** dari orang yang membuka kode halaman: data target tetap ikut terkirim ke browser pengunjung. Untuk data yang benar-benar rahasia dibutuhkan server, misalnya `slak-bridge.php` (Bab 26). Di portofolio, pakai data fiktif atau yang sudah dianonimkan.

25.5 Halaman Tentang Saya

Dasbor portofolio harus menjawab pertanyaan perekrut: **siapa kamu, apa yang bisa kamu lakukan, dan bagaimana kamu bekerja?** Halaman *Tentang Saya* berisi nama, peran yang dituju, keahlian, data yang dipakai, metode, dan tautan ke GitHub atau LinkedIn. Ganti teksnya dengan dirimu sendiri.

Tips menulis halaman ini:

- Sebut **pertanyaan bisnis** dan **temuanmu**, bukan hanya alat yang dipakai.
- Jujur tentang data: sebutkan bahwa Toko Nusantara adalah data fiktif.
- Sebut **metode**: pembersihan, penggabungan tabel, Pareto, Holt-Winters.

25.6 Kode lengkap

Kode lengkap dasbor ada di bawah, dan juga di folder bahan buku sebagai `proyek-akhir/dasbor.slak` (untuk Slak Studio) dan `proyek-akhir/index.html` (langsung dibuka dengan klik dua kali). Bacalah per bagian; semuanya sudah kamu pelajari.

```
// Dasbor Portofolio Toko Nusantara · proyek akhir buku Write and Run
// Slak 1.9.1 · butuh slak-data.js dan slak-peta.js · data: toko-nusantara.csv,
produk.csv, cabang.csv, target.csv

sl.set_page_config({ page_title: "Dasbor Toko Nusantara", page_icon: "📊", layout:
"wide" })
```

```
// ========================================================================
// 1. Data: dibaca sekali, dipakai semua halaman
// ========================================================================
const [trx, produk, cabang, target] = await Promise.all([
  sl.read_csv("toko-nusantara.csv"), sl.read_csv("produk.csv"),
sl.read_csv("cabang.csv"), sl.read_csv("target.csv")
])
const semua = trx
  .merge(produk.select(["produk", "harga_pokok"]), { on: "produk", how: "left" })
  .assign({
    laba: r => r.total - r.jumlah * r.harga_pokok,
    tahun: r => r.tanggal.getFullYear(),
    bulan: r => sl.strftime(r.tanggal, "%Y-%m")
  })

// ========================================================================
// 2. Alat bantu: hitungan dan format yang dipakai berulang
// ========================================================================
const persen = new Intl.NumberFormat("id-ID", { style: "percent",
maximumFractionDigits: 1, signDisplay: "always" })
const banding = (a, b) => a ? persen.format(b / a - 1) : null
const ringkas = (df) => ({
  omzet: df.total.sum(), laba: df.laba.sum(),
  transaksi: df.shape[0], pelanggan: df.pelanggan_id.nunique()
})

// ========================================================================
// 3. Filter global di sidebar: berlaku di semua halaman
// ========================================================================
sl.sidebar.header("Filter")
const tahun = Number(sl.sidebar.selectbox("Tahun", ["2024", "2023"], { key:
"f_tahun" }))
const kanal = sl.sidebar.radio("Kanal", ["Semua", "Toko", "Online"], { key:
"f_kanal" })
const pilihCabang = sl.sidebar.multiselect("Cabang", semua.cabang.unique().sort(),
{ key: "f_cabang", placeholder: "Semua cabang" })

const saring = (df) => df
  .filter(r => kanal === "Semua" || r.kanal === kanal)
  .filter(r => pilihCabang.length === 0 || pilihCabang.includes(r.cabang))
const ini = saring(semua.filter(r => r.tahun === tahun))
const lalu = saring(semua.filter(r => r.tahun === tahun - 1))
const keterangan = `${tahun} · ${pilihCabang.length ? pilihCabang.length + "
cabang" : "semua cabang"} · kanal ${kanal.toLowerCase()}`
```

```
// ========================================================================
// 4. Halaman
// ========================================================================
function halRingkasan(sl) {
  sl.title("Ringkasan")
  sl.caption(keterangan)
  const r = ringkas(ini), r0 = ringkas(lalu)
  const [a, b, c, d] = sl.columns(4)
  a.metric("Omzet", r.omzet, banding(r0.omzet, r.omzet), { format: "rp" })
  b.metric("Laba kotor", r.laba, banding(r0.laba, r.laba), { format: "rp" })
  c.metric("Transaksi", r.transaksi, banding(r0.transaksi, r.transaksi))
  d.metric("Pelanggan", r.pelanggan, banding(r0.pelanggan, r.pelanggan))

  const bulanan = ini.resample("tanggal", "M").agg({ omzet: ["total", "sum"] })
  sl.line_chart(bulanan, { x: "tanggal", y: "omzet", format: "compact", title:
"Omzet bulanan" })

  sl.subheader("Temuan utama")
  const kat = ini.groupby("kategori").agg({ omzet: ["total", "sum"]
}).sort_values("omzet", { ascending: false }).to_records()
  const cab = ini.groupby("cabang").agg({ omzet: ["total", "sum"]
}).sort_values("omzet", { ascending: false }).to_records()
  const online = ini.filter(x => x.kanal === "Online").total.sum() / (r.omzet || 1)
  sl.markdown(`
1. Omzet ${tahun} ${r0.omzet ? (r.omzet >= r0.omzet ? "naik" : "turun") + " " +
banding(r0.omzet, r.omzet).replace(/^[+-]/, "") + " dibanding " + (tahun - 1) :
"tercatat"} menjadi **Rp ${r.omzet.toLocaleString("id-ID")}**.
2. Kategori **${kat[0] ? kat[0].kategori : "-"}** menyumbang omzet terbesar, dan
cabang **${cab[0] ? cab[0].cabang : "-"}** memimpin.
3. Kanal online menyumbang **${(online * 100).toLocaleString("id-ID", {
maximumFractionDigits: 1 })}%** omzet.
`)
}

function halPenjualan(sl) {
  sl.title("Penjualan")
  sl.caption(keterangan)
  const [t1, t2, t3] = sl.tabs(["Per cabang", "Per kategori", "Per hari"])
  t1.bar_chart(ini.groupby("cabang").agg({ omzet: ["total", "sum"] }), { x:
"cabang", y: "omzet", horizontal: true, sort: true, format: "compact" })
  t2.bar_chart(ini.groupby(["kategori", "kanal"]).agg({ omzet: ["total", "sum"] }),
{ x: "kategori", y: "omzet", color: "kanal", format: "compact" })
  const harian = ini.resample("tanggal", "D").agg({ omzet: ["total", "sum"] })
  const perHari = harian.assign({ hari: () => harian.tanggal.dt.day_name(), urut:
```

```
() => harian.tanggal.dt.weekday })
    .groupby(["urut", "hari"]).agg({ rata: ["omzet", "mean"] })
  t3.bar_chart(perHari, { x: "hari", y: "rata", format: "compact", title: "Rata-
rata omzet per hari" })
}

function halProduk(sl) {
  sl.title("Produk")
  sl.caption(keterangan)
  const per = ini.groupby("produk").agg({ terjual: ["jumlah", "sum"], omzet:
["total", "sum"], laba: ["laba", "sum"] })
    .assign({ margin: r => r.laba / r.omzet }).sort_values("omzet", { ascending:
false })
  const par = sl.analysis.pareto(ini, { category: "produk", value: "total" })
  sl.combo_chart(par, { x: "produk", bar: "total", line: "persen_kumulatif" })
  sl.dataframe(per, { format: { omzet: "rupiah", laba: "rupiah", margin: "percent"
}, gradient: ["laba"] })
  sl.download_button("Unduh tabel produk (Excel)", { Produk: per }, { file_name:
`produk-${tahun}.xlsx` })
}

function halWilayah(sl) {
  sl.title("Wilayah")
  sl.caption(keterangan)
  const per = ini.groupby("cabang").agg({ omzet: ["total", "sum"], pelanggan:
["pelanggan_id", "nunique"] }).merge(cabang, { on: "cabang" })
  sl.map(per, { lat: "lat", lon: "lon", size: "omzet", color: "pelanggan", label:
"kota" })
  sl.choropleth(ini.groupby("provinsi").agg({ omzet: ["total", "sum"] }), { region:
"provinsi", value: "omzet", labels: true })
}

function halPrakiraan(sl) {
  sl.title("Prakiraan")
  sl.caption("Berdasarkan seluruh data 2023–2024 dengan filter kanal dan cabang
yang dipilih")
  const n = sl.slider("Bulan ke depan", 3, 12, 6)
  const bulanan = saring(semua).resample("tanggal", "M").agg({ omzet: ["total",
"sum"] })
  sl.write(sl.ts.forecast(bulanan, { periods: n }))
  sl.info("Ramalan memakai dua tahun data. Laporkan selalu rentangnya, bukan hanya
satu angka.")
}

async function halTarget(sl) {
```

```
  await sl.login({
    users: {
      manajer: { password:
"$2y$10$h9g8XzzwmQwTu8Biy2kxzur5ioPnQbi.AzHsPpTxCIVMQ7GQN65T6", name: "Manajer
Wilayah", role: "manajer" },
      staf: { password:
"$2y$10$dIcMwFfIEjs3i5STBTCIO.k88fjSkemiaIxaT1b20blp4rYaqmeKK", name: "Staf Toko",
role: "staf" }
    },
    title: "Halaman ini khusus manajer"
  })
  sl.require_role("manajer")
  sl.title("Capaian Target")
  sl.caption(`Masuk sebagai ${sl.user.name}`)
  const perBulan = ini.groupby(["cabang", "bulan"]).agg({ omzet: ["total", "sum"]
})
    .merge(target, { on: ["cabang", "bulan"], how: "left" })
    .assign({ capaian: r => r.omzet / r.target })
  const perCab = perBulan.groupby("cabang").agg({ omzet: ["omzet", "sum"], target:
["target", "sum"], bulan_tercapai: ["capaian", x => x.filter(v => v >= 1).length]
})
    .assign({ capaian: r => r.omzet / r.target }).sort_values("capaian", {
ascending: false })
  sl.dataframe(perCab, { format: { omzet: "rupiah", target: "rupiah", capaian:
"percent" }, gradient: ["capaian"] })
  sl.bar_chart(perCab, { x: "cabang", y: "capaian", horizontal: true, format:
"percent",
    color: r => r.capaian >= 1 ? "var(--slak-green)" : "var(--slak-red)" })
  sl.button("Keluar", { on_click: () => sl.logout() })
}

function halTentang(sl) {
  sl.title("Tentang Saya")
  sl.markdown(`
**Nama Kamu** · Calon Analis Data · Jayapura

Saya membangun dasbor ini sebagai proyek akhir buku *Write and Run: Belajar
JavaScript untuk Analisis Data dengan Slak*.

**Keahlian:** JavaScript, Slak, SQL (SQLite, DuckDB, gaya PostgreSQL), statistik
deskriptif, uji hipotesis, deret waktu, visualisasi, dan peta.

**Data:** Toko Nusantara adalah data fiktif 2.929 transaksi dari sepuluh cabang,
2023–2024.
```

```
**Metode:** pembersihan data, penggabungan empat tabel, analisis Pareto, prakiraan
Holt-Winters, dan capaian target per cabang.
`)
  const [a, b] = sl.columns(2)
  a.link_button("GitHub", "https://github.com/")
  b.link_button("LinkedIn", "https://www.linkedin.com/")
}

// ============================================================================
// 5. Navigasi
// ============================================================================
await sl.navigation({
  "Analisis": [
    sl.page("Ringkasan", halRingkasan, { icon: "📊", default: true }),
    sl.page("Penjualan", halPenjualan, { icon: "🧾" }),
    sl.page("Produk", halProduk, { icon: "📦" }),
    sl.page("Wilayah", halWilayah, { icon: "🗺️" }),
    sl.page("Prakiraan", halPrakiraan, { icon: "🔮" })
  ],
  "Internal": [sl.page("Target", halTarget, { icon: "🎯" })],
  "Profil": [sl.page("Tentang Saya", halTentang, { icon: "👋" })]
}).run()
```

Hal-hal yang patut diperhatikan:

1. **Data dibaca sekali** di bagian 1 dengan `Promise.all`, lalu digabung dengan harga pokok dan diberi kolom `laba`, `tahun`, dan `bulan`. Semua halaman memakai hasil yang sama.
2. **Alat bantu** di bagian 2 (`banding`, `ringkas`) mencegah kode yang sama ditulis berulang di setiap halaman.
3. **Temuan utama** di halaman Ringkasan ditulis otomatis dari angka. Saat filter diubah, kalimatnya ikut berubah dan tetap benar.
4. **Halaman Prakiraan** sengaja memakai seluruh data dua tahun, bukan hanya tahun terpilih, karena ramalan butuh data sepanjang mungkin.
5. **Halaman Target** memakai fungsi agregasi buatan sendiri untuk menghitung bulan yang tercapai (Bab 18.4).

25.7 Menguji dasbor

Dasbor sebesar ini wajib diuji. Uji berikut membuka setiap halaman dan memastikan tidak ada galat. Simpan sebagai `dasbor.uji.slak` di samping `dasbor.slak`, lalu jalankan dari panel **Uji** di Slak Studio:

```
const { test, expect } = sl.testing
for (const hal of ["Ringkasan", "Penjualan", "Produk", "Wilayah", "Prakiraan",
"Target", "Tentang Saya"]) {
  await test(`halaman ${hal} tanpa galat`, async () => {
    const at = await sl.testing.run("dasbor.slak", { timeout: 20000 })
    if (hal !== "Ringkasan") { at.switch_page(hal); await at.run({ timeout: 20000
}) }
    expect(at.exception).toHaveLength(0)
  })
}
await test("filter 2023 Online tanpa galat", async () => {
  const at = await sl.testing.run("dasbor.slak", { timeout: 20000 })
  await at.selectbox("Tahun").set_value("2023").run()
  await at.radio("Kanal").set_value("Online").run()
  expect(at.exception).toHaveLength(0)
})
```

25.8 Menerbitkan sebagai aplikasi

Terbitkan dasbormu dengan cara di Bab 17.6. Dua hal tambahan untuk proyek akhir:

1. **Bisa dipasang dan jalan tanpa internet.** Di jendela **Terbitkan** Slak Studio, centang **Bisa dipasang dan jalan offline (PWA)**. Setelah diterbitkan di alamat https, Chrome dan Edge menampilkan tombol **Pasang**, dan di ponsel Android ada pilihan **Tambahkan ke layar utama**. Ikon aplikasinya diambil dari `page_icon` di `sl.set_page_config`. Setelah dibuka sekali, dasbor tetap terbuka tanpa internet.
2. **Bagikan tautannya** di CV, profil LinkedIn, dan README GitHub-mu. Tuliskan satu kalimat temuan di samping tautan, misalnya: "Dasbor penjualan 10 cabang: omzet 2024 naik 27,6%, dengan kerajinan sebagai penyumbang terbesar."

Latihan

Latihan 25.1 (mudah). Tambahkan halaman **Pelanggan** yang menampilkan segmen RFM (Bab 22.4) untuk data terpilih, lalu daftarkan di kelompok "Analisis".

Latihan 25.2 (sedang). Tambahkan pilihan **"Bandingkan dengan"** di sidebar berisi "Tahun lalu" dan "Target", lalu ubah metrik Omzet di halaman Ringkasan agar perubahannya dihitung terhadap pilihan itu.

Latihan 25.3 (tantangan). Tambahkan ke `dasbor.uji.slak` satu uji yang memastikan **jumlah omzet semua cabang di halaman Penjualan sama dengan metrik Omzet di halaman Ringkasan**, untuk setiap kombinasi tahun dan kanal.

Ketiga latihan ini sengaja tidak punya satu jawaban benar. Contoh solusinya ada di Lampiran D, tetapi dasbormu boleh berbeda. Justru di situlah nilainya sebagai karya portofolio: dasbormu menunjukkan caramu berpikir.

Selamat!

Kamu memulai buku ini dengan `sl.title("Halo, dunia!")`. Kini kamu punya dasbor multi-halaman yang membaca empat tabel, menggabungkan, meringkas, menggambar, meramal, memetakan, melindungi satu halaman dengan login, dan teruji otomatis. Semuanya ditulis dengan JavaScript, berjalan di browser, tanpa build.

← 24. Merapikan dan Menguji Kode

26. Langkah Berikutnya →

Bab 26 · Langkah Berikutnya

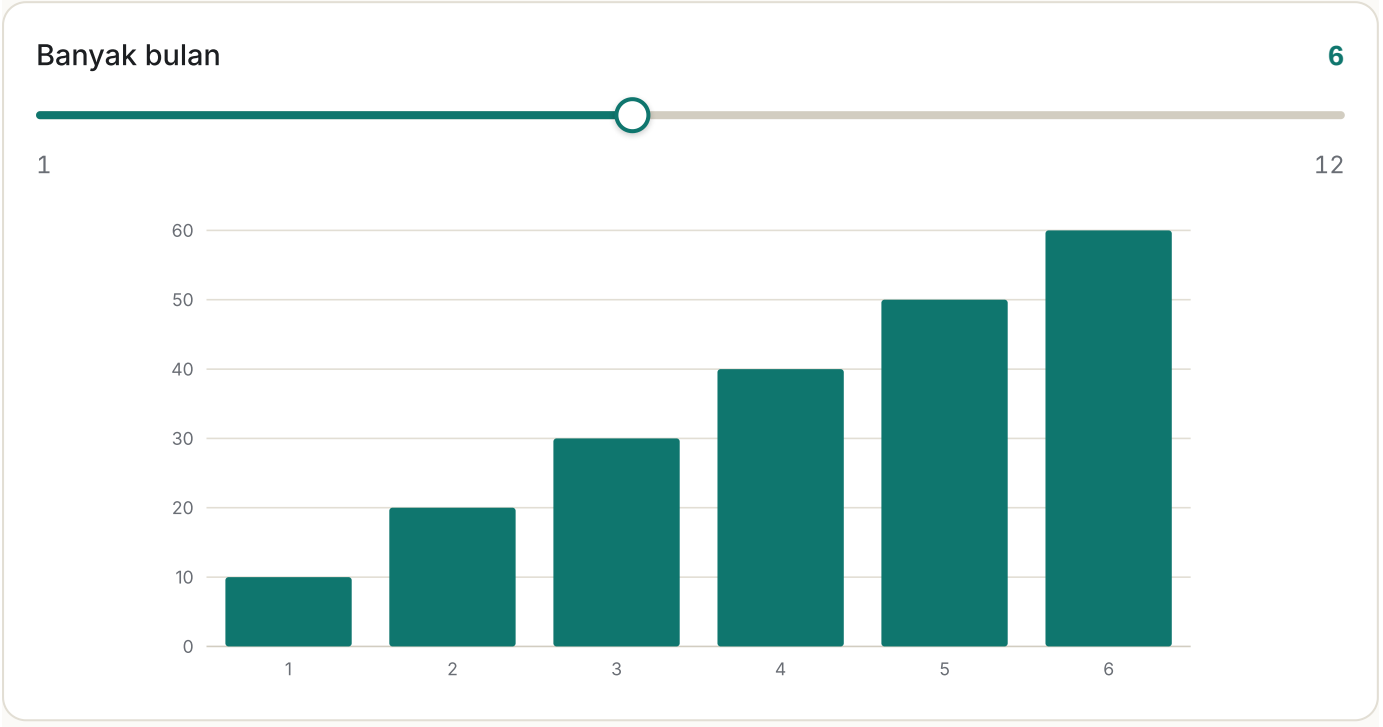
> **Tujuan belajar**
>
> - Memahami apa yang dikerjakan Slak untukmu, dan apa batas JavaScript untuk analisis data.
> - Mengenal jembatan ke Python di browser dengan `sl.python`.
> - Mengetahui cara terhubung ke database server.
> - Mengenal komponen kustom dan dokumentasi proyek dengan `Slak.docs`.
> - Menyusun peta jalan belajar dan portofolio berikutnya.

26.1 Apa yang dikerjakan Slak untukmu

Di Bab 10 kamu melihat penghitung klik dengan JavaScript biasa. Kini bandingkan sesuatu yang lebih besar: penggeser angka yang mengubah grafik batang.

Dengan Slak:

```
const n = sl.slider("Banyak bulan", 1, 12, 6)
sl.bar_chart(Array.from({ length: n }, (_, i) => ({ bulan: i + 1, omzet: (i + 1) *
10 })), { x: "bulan", y: "omzet" })
```



Buka di Playground ↗

Dengan JavaScript biasa, tanpa pustaka apa pun:

```
<label for="n">Banyak bulan: <span id="nilai">6</span></label>
<input id="n" type="range" min="1" max="12" value="6">
<svg id="grafik" width="400" height="200"></svg>
<script>
  const masukan = document.getElementById("n")
  const nilai = document.getElementById("nilai")
  const svg = document.getElementById("grafik")

  function gambar() {
    const n = Number(masukan.value)
    nilai.textContent = n
    const data = Array.from({ length: n }, (_, i) => (i + 1) * 10)
    const maks = Math.max(...data)
    const lebar = 400 / n
    svg.innerHTML = ""
    data.forEach((v, i) => {
      const tinggi = v / maks * 180
      const r = document.createElementNS("http://www.w3.org/2000/svg", "rect")
      r.setAttribute("x", i * lebar + 4)
      r.setAttribute("y", 200 - tinggi)
      r.setAttribute("width", lebar - 8)
      r.setAttribute("height", tinggi)
      r.setAttribute("fill", "teal")
      svg.append(r)
    })
  }
  masukan.addEventListener("input", gambar)
  gambar()
</script>
```

Versi kedua masih belum punya sumbu, label, warna tema gelap, tooltip, tombol unduh PNG, teks untuk pembaca layar, atau tampilan ponsel. Semua itu dikerjakan Slak di balik `sl.bar_chart`. Kini kamu bisa **membaca** kode seperti ini: `document.getElementById` mencari elemen halaman, `addEventListener` memasang fungsi yang menunggu kejadian, dan `createElementNS` membuat elemen gambar. Semuanya JavaScript yang sudah kamu pelajari, ditambah sedikit pengetahuan tentang **DOM**, struktur halaman web.

Kamu tidak terkunci pada Slak. Bila suatu hari kamu ingin membuat situs web biasa, JavaScript yang kamu kuasai tetap berlaku sepenuhnya. Langkah berikutnya untuk jalur itu adalah mempelajari HTML, CSS, dan DOM.

26.2 Batas JavaScript untuk analisis data

Slak Data membawa JavaScript sangat jauh. Tetapi ada hal yang memang lebih baik dikerjakan di tempat lain:

Kebutuhan	Di Slak	Bila melewati batas
Data di atas 1 juta baris	DuckDB (Bab 21)	Database server, BigQuery
Data puluhan GB	Di luar kemampuan browser	Python di komputer, database server
Gradient boosting, deep learning, pengolahan teks	Tidak tersedia	Python: scikit-learn, XGBoost, PyTorch
Model statistik lanjut (model campuran, ARIMA lengkap)	Sebagian lewat `sl.python`	Python atau R
Laporan terjadwal yang dikirim otomatis	Tidak ada server	Penjadwal di server
Login sungguhan dan data pribadi banyak pengguna	Login di browser saja	slak-bridge di hosting

Batas ini bukan kekurangan yang memalukan, melainkan keputusan desain: Slak memilih **tanpa server, tanpa instalasi, tanpa build**. Untuk portofolio, laporan internal, alat bantu kecil, dan belajar, pilihan itu tepat. Untuk data perusahaan besar, analis memakai alat lain, dan kamu kini punya dasar untuk mempelajarinya.

26.3 Python di browser

Slak menyediakan jembatan ke **Python** lewat Pyodide, Python yang berjalan di dalam browser. Kamu mengirim DataFrame Slak, menulis kode pandas, dan menerima hasilnya kembali.

> **Butuh internet:** Python (puluhan MB) diunduh saat pertama dipakai. Python tidak tersedia di halaman yang dibuka dengan klik dua kali; jalankan lewat Slak Studio, playground, atau hosting.

```
const df = await sl.read_csv("toko-nusantara.csv")
try {
  const hasil = await sl.spinner("Memuat Python…", () => sl.python(`
import pandas as pd
df.groupby("kategori")["total"].sum().sort_values(ascending=False)
`, { data: { df } }))
  sl.write(hasil)
} catch (e) {
  sl.warning("Python belum bisa dimuat: " + e.message)
}
```

Memuat Python...

Lampiran A · Ringkasan API Slak di Buku Ini

Daftar ini merangkum setiap fungsi Slak yang dikenalkan di buku, beserta bab tempat ia pertama dijelaskan. Semua nama `sl.*` dan `Slak.*` di sini tercantum di daftar API stabil Slak 1.9 (API-STABIL.md) dan dijamin tetap ada sampai Slak 2.0, kecuali yang ditandai berasal dari berkas tambahan.

Berkas	Isi	Dimuat oleh playground
`slak.js`	Inti: teks, widget, tata letak, grafik, state, form, navigasi, login, `sl.testing`, `sl.crypto`	✓
`slak-data.js`	Slak Data: DataFrame, statistik, ML, deret waktu, analisis bisnis, dataset	✓
`slak-sql.js`	SQLite: `sl.sql`, `sl.sqlite`	✓
`slak-peta.js` (+ `slak-peta-kab.js`)	Peta: `sl.map`, `sl.choropleth`, `sl.geo`	✓
`slak-komponen.js`	Galeri 10 komponen untuk `sl.component`	✓
`slak-duckdb.js`	DuckDB: `sl.duckdb`, `sl.read_parquet` (mesin diunduh saat pertama dipakai)	×
`slak-docs.js`	`Slak.docs` untuk situs dokumentasi (versi 0.1, eksperimental)	×

Bab 1

Fungsi	Kegunaan
`sl.title(teks)`	Menampilkan judul besar di bagian atas.
`sl.write(nilai)`	Menampilkan apa saja: teks, angka, tabel, atau objek. Teks boleh memakai Markdown, misalnya `**tebal**`.
`sl.set_page_config({ page_title, page_icon, layout })`	Mengatur judul tab, ikon, dan tata letak. `layout: "wide"` membuat konten lebih lebar.
`Slak.share.link(kode, alamat)`	Membuat tautan playground yang berisi kode. Perhatikan awalannya `Slak.`, bukan `sl.`: ini fungsi milik Slak sebagai alat, bukan perintah untuk menampilkan sesuatu.
`sl.code(teks)`	Menampilkan teks sebagai kode.

Bab 2

Fungsi	Kegunaan
`sl.markdown(teks)`	Menampilkan teks berformat Markdown: tebal, miring, daftar, tabel, tautan, dan warna `:green[...]`, `:red[...]`, `:blue[...]`, `:orange[...]`, `:violet[...]`, `:gray[...]`.
`sl.text(teks)`	Menampilkan teks persis seperti ditulis.

Bab 3

Fungsi	Kegunaan
`sl.success(teks)`, `sl.info(teks)`, `sl.warning(teks)`, `sl.error(teks)`	Menampilkan kotak pesan hijau, biru, kuning, dan merah.
`sl.text_input(label, nilaiAwal)`	Mengembalikan teks. Nilainya dipakai setelah kamu menekan Enter atau pindah ke kotak lain.
`sl.number_input(label, { min, max, value, step })`	Mengembalikan angka.
`sl.button(label)`	Mengembalikan `true` pada satu jalan ulang setelah diklik, lalu `false` lagi.

Bab 4

Fungsi	Kegunaan
`sl.slider(label, min, max, nilaiAwal, langkah)`	Mengembalikan angka yang dipilih. Urutan argumennya: label, nilai terkecil, nilai terbesar, nilai awal, lalu langkah (boleh dilewati).

Bab 5

Fungsi	Kegunaan
`sl.selectbox(label, pilihan)`	Menampilkan daftar pilihan dan mengembalikan pilihan yang dipilih.
`sl.columns(n)`	Membagi baris menjadi `n` kolom dan mengembalikan larik kolom. Setiap kolom punya semua fungsi tampilan, misalnya `kiri.write(...)`.
`sl.metric(label, nilai)`	Menampilkan angka besar dengan label. Kamu akan sering memakainya untuk dasbor.

Bab 6

Fungsi	Kegunaan
`sl.multiselect(label, pilihan, { default })`	Mengembalikan **larik** berisi pilihan yang dipilih.

Bab 7

Fungsi	Kegunaan
`sl.table(data)`	Menampilkan tabel statis.
`sl.dataframe(data)`	Menampilkan tabel interaktif: kolomnya bisa diurutkan dengan mengeklik judul, dan ada tombol unduh CSV.
`sl.metric(label, nilai, delta)`	Menampilkan angka besar. Argumen ketiga (opsional) adalah perubahan, misalnya `"+12%"`, yang tampil hijau atau merah.
`sl.dataset(nama)`	Mengembalikan dataset contoh sebagai DataFrame, yaitu tabel milik Slak Data (Bab 14).
`sl.checkbox(label)`	Mengembalikan `true` atau `false`.

Bab 8

Fungsi	Kegunaan
`sl.date_input(label, nilaiAwal)`	Mengembalikan tanggal sebagai **teks** `"YYYY-MM-DD"`. Ubah dengan `new Date(tgl)` bila ingin menghitung.
`sl.strftime(tanggal, pola)`	Memformat tanggal dengan pola seperti `%d`, `%B`, `%Y`, `%A`.

Bab 9

Fungsi	Kegunaan
`sl.warning(teks)`	Menampilkan kotak peringatan kuning.
`sl.exception(galat)`	Menampilkan galat lengkap dengan tombol **Detail** berisi stack trace, tanpa menghentikan aplikasi.
`sl.testing.expect(nilai)`	Membuat pernyataan yang diperiksa, misalnya `.toBe(...)`.

Bab 10

Fungsi	Kegunaan
`sl.session_state`	Menyimpan nilai selama halaman terbuka. Isinya hilang bila halaman dimuat ulang.
`sl.local_state`	Seperti `session_state`, tetapi permanen. Data disimpan per browser dan hanya cocok untuk data JSON: angka, teks, boolean, larik, dan objek biasa.
`sl.clear_local_state()`	Mengosongkannya.
`sl.form(key, { clear_on_submit })`	Membuat formulir. Widget di dalamnya ditulis dengan awalan formulir, misalnya `f.text_input(...)`, dan tidak memicu jalan ulang sampai tombol kirim ditekan.
`sl.crypto.quiz_hash(jawaban)`	Membuat kode pelindung. Bisa juga larik beberapa jawaban yang sama-sama benar.
`sl.crypto.quiz_check(jawaban, kode)`	Mengembalikan `true` bila jawaban cocok.

Bab 11

Fungsi	Kegunaan
`sl.json(objek)`	Menampilkan objek atau larik dengan rapi dan bisa dilipat.
`sl.file_uploader(label, { type })`	Mengembalikan objek berkas (`name`, `size`, `text`, `bytes`) atau `null` bila belum ada berkas. Berkasnya tidak pernah dikirim ke server mana pun; semuanya diproses di browser.

Bab 12

Fungsi	Kegunaan
`sl.open_data(id)`, `sl.open_data.list()`, `sl.github(jalur)`, `sl.worldbank(indikator, { negara, tahun })`, `sl.huggingface(dataset, { file })`, `sl.kaggle(dataset, { file })`	Mengambil dataset terbuka sebagai DataFrame. Semuanya ada di `slak-data.js`.
`sl.datasets()`	Menampilkan daftar dataset bawaan.
`sl.line_chart(data, { x, y, title })`	Menggambar grafik garis. Grafik dibahas di Bab 16.

Fungsi	Kegunaan
`sl.progress(nilai, teks)`	Menampilkan bilah kemajuan berskala 0–100. Perbarui dengan `.progress(nilai, teks)`.

Bab 13

Fungsi	Kegunaan
`sl.data_editor(data, { num_rows })`	Menampilkan tabel yang bisa diedit, lalu mengembalikan larik hasil editan. `num_rows: "dynamic"` mengizinkan pengguna menambah dan menghapus baris.

Bab 14

Fungsi	Kegunaan
`sl.DataFrame(larikObjek)`	Membuat DataFrame dari larik objek. Kebalikannya: `df.to_records()`.
`sl.radio(label, pilihan, { horizontal })`	Menampilkan pilihan tunggal.
`sl.download_button(label, data, { file_name })`	Mengunduh data. Akhiran nama berkas menentukan formatnya: `.csv`, `.xlsx`, `.json`, atau `.md`.

Bab 15

Fungsi	Kegunaan
`sl.dataframe(df, { format })`	Merapikan angka per kolom: `"rupiah"`, `"percent"`, `"compact"`, `"decimal:2"`.
`sl.crosstab(series1, series2, { normalize })`	Membuat tabel silang.
`sl.subheader(teks)`	Menampilkan judul kecil.

Bab 16

Fungsi	Kegunaan
`sl.line_chart`, `sl.bar_chart`, `sl.area_chart`, `sl.scatter_chart`	`sl.line_chart`, `sl.bar_chart`, `sl.area_chart`, `sl.scatter_chart(data, { x, y, color, title, x_label, y_label, format })`.

Fungsi	Kegunaan
`sl.histogram(data, { x, bins })`	Dan `sl.pie_chart(data, { names, values, donut })`.

Bab 17

Fungsi	Kegunaan
`sl.sidebar.namaFungsi(...)`	Menaruh apa saja di sidebar.
`sl.tabs(["A", "B"])`	Mengembalikan larik tab; setiap tab punya semua fungsi tampilan.
`sl.layout(nama, { title })`	Membuat kerangka tata letak dan mengembalikan objek berisi area-areanya.

Bab 18

Fungsi	Kegunaan
`sl.merge(kiri, kanan, opsi)`	Adalah bentuk lain yang sama.
`sl.concat([df1, df2])`	Menyambung tabel ke bawah.
`sl.melt(df, { id_vars, var_name, value_name })`	Lebar ke panjang.

Bab 19

Fungsi	Kegunaan
`sl.stats.describe(x)`	Meringkas satu kolom: banyak data, rata-rata, simpangan baku, minimum, kuartil, dan maksimum.
`sl.stats.var`, `sl.stats.std`, `sl.stats.mean`, `sl.stats.median`	`sl.stats.var(x)`, `sl.stats.std(x)`, `sl.stats.mean(x)`, `sl.stats.median(x)`, juga tersedia sebagai metode Series: `df.total.std()`.
`sl.box_plot(data, { y, by })`	Menggambar box plot, satu kotak per kelompok.
`sl.heatmap(data)`	Menggambar tabel angka sebagai kotak berwarna.
`sl.stats.pearsonr(x, y)`	Dan `sl.stats.spearmanr(x, y)` menguji korelasi, lengkap dengan kesimpulan berbahasa Indonesia.
`sl.stats.ttest_ind(df, { value, group })`	Membandingkan rata-rata dua kelompok.
`sl.stats.ab_test(df, { group, metric, control })`	Menganalisis uji A/B. Metrik 0/1 diuji sebagai proporsi, metrik lain sebagai rata-rata.

Fungsi	Kegunaan
`sl.write`	Setiap hasil uji punya `statistic`, `p_value`, `significant`, `conclusion`, dan bisa ditampilkan langsung dengan `sl.write`.

Bab 20

Fungsi	Kegunaan
`sl.to_datetime(nilai)`	Mengubah teks atau larik teks menjadi tanggal; `df.to_datetime(kolom)` mengubah satu kolom DataFrame.
`sl.date_range(awal, { end, periods, freq })`	Membuat deretan tanggal. `freq`: `"D"` hari, `"W"` minggu, `"MS"` awal bulan, `"ME"` akhir bulan, `"QS"` awal kuartal, `"YS"` awal tahun.
`sl.segmented_control(label, pilihan, { default })`	Menampilkan tombol pilihan yang berdampingan.
`sl.analysis.growth(df, { date, value })`	Menghasilkan kolom `periode`, `pertumbuhan` (MoM), `pertumbuhan_yoy`, dan `kumulatif`.
`sl.ts.decompose(df, { period })`	Menghasilkan kolom `nilai`, `tren`, `musiman`, dan `residu`. `period: 12` untuk data bulanan dengan pola tahunan, `7` untuk data harian dengan pola mingguan.

Bab 21

Fungsi	Kegunaan
`sl.sql.check(kueri)`	Mendaftar sintaks khusus SQLite beserta penggantinya yang portabel.
Opsi `{ portable: true }`	Pada `sl.sql` memberi peringatan langsung saat kueri memakai sintaks khusus SQLite.
`sl.db(nama, skema)`	Membuka database IndexedDB dengan tabel dan perintah `insert`, `update`, `delete`, `where`, `all`, `count`.
`await sl.db_list()`	Mendaftar database `sl.db` yang ada di browser ini.

Bab 22

Fungsi	Kegunaan
`sl.stats.linregress(x, y)`	Menghasilkan `slope`, `intercept`, `rvalue`, `rsquared`, dan `pvalue`.

Fungsi	Kegunaan
`sl.stats.ols(df, "y ~ x1 + x2 + C(kategori)")`	Membuat regresi berganda lengkap dengan tabel koefisien, R^2, dan diagnostik.
`sl.ts.forecast(df, { periods, method })`	Meramal deret waktu. Hasilnya bisa ditampilkan dengan `sl.write`, dan tabelnya ada di `.forecast` (kolom `prediksi`, `batas_bawah`, `batas_atas`).
`sl.analysis.pareto`, `sl.analysis.rfm`, `sl.analysis.cohort`, `sl.analysis.funnel`	`sl.analysis.pareto(df, { category, value })`, `sl.analysis.rfm(df, { customer, date, amount })`, `sl.analysis.cohort(df, { customer, date, period })`, `sl.analysis.funnel(tahap)`.
`sl.combo_chart(df, { x, bar, line })`	Menggambar batang dan garis sekaligus.
`sl.ml.train_test_split(df, { test_size, stratify, random_state })`	Membagi data latih dan uji.
Model: `sl.ml.LinearRegression()`, `sl.ml.DecisionTreeClassifier({ max_depth })`, `sl.ml.KMeans({ n_clusters })`, dan lainnya. Semuanya memakai `.fit(df, "target")`	Dan `.predict(df)`.
`sl.ml.accuracy_score`, `sl.ml.confusion_matrix`, `sl.ml.r`, `sl.ml.mae`, `sl.ml.rmse`	Ukuran: `sl.ml.accuracy_score`, `sl.ml.confusion_matrix`, `sl.ml.r2_score`, `sl.ml.mae`, `sl.ml.rmse`.
`sl.ml.StandardScaler().fit_transform(df)`	Menyamakan skala kolom.
`sl.ml_demo(model, df, { target, labels })`	Membuat formulir coba model. Model yang belum dilatih dilatih otomatis.

Bab 23

Fungsi	Kegunaan
`sl.map(df, { lat, lon, size, color, label })`	Menggambar titik atau gelembung.
`sl.map(df, { region: "kolom" })`	Mencari lokasi dari nama wilayah.
`sl.choropleth(df, { region, value, labels, level, bins })`	Mewarnai wilayah. `level: "kabkota"` untuk kabupaten/kota.

Fungsi	Kegunaan
`sl.geo.match(nama)`	Mencocokkan satu nama wilayah dan mengembalikan nama baku, provinsi, dan koordinatnya.
`sl.geo.geocode(df, kolom)`	Menambah kolom `wilayah_baku`, `lat`, dan `lon`.
`sl.geo.distance([lat1, lon1], [lat2, lon2])`	Menghitung jarak dalam km.
`sl.geo.province_of(lat, lon)`	Dan `sl.geo.kabkota_of(lat, lon)` menebak wilayah dari koordinat.

Bab 24

Fungsi	Kegunaan
`sl.testing.from_string(kode)`, `from_file("app.slak")`, dan `from_function(fn)`	Membuat objek uji tanpa langsung menjalankannya.

Bab 25

Fungsi	Kegunaan
`sl.navigation(daftarHalaman, { position })`	Dan `sl.page(judul, fungsi, { icon, url, default })` membuat aplikasi multi-halaman. `position`: `"sidebar"` (bawaan), `"top"`, atau `"hidden"`.
`sl.switch_page(judul)`	Dan `sl.page_link(judul)` berpindah halaman.
`sl.require_role`, `sl.user`, `sl.logout`	`await sl.login({ users, title })`, `sl.require_role(peran)`, `sl.user`, `sl.logout()`.

Bab 26

Fungsi	Kegunaan
`Slak.docs.site(sl, { judul, bab })` *(berkas tambahan)*	Membuat situs dokumentasi lengkap.
`Slak.docs.markdown(sl, teks)` *(berkas tambahan)*	Menampilkan satu bab Markdown dengan contoh yang dijalankan.
Perhatikan awalannya: `Slak.docs`, bukan `sl.docs`, dan `sl` *(berkas tambahan)*	Dikirim sebagai argumen pertama. `slak-docs.js` tidak menambah nama apa pun ke `sl`.

Daftar lengkap 938 nama API stabil ada di berkas `API-STABIL.md` di paket Slak 1.9.1, dan setiap fungsinya punya contoh di panel **Referensi** Slak Studio (tekan **F1**).

← 26. Langkah Berikutnya

B · Cheat Sheet JavaScript untuk Data →

Lampiran B · Cheat Sheet JavaScript untuk Data

Ringkasan JavaScript yang dipakai di buku ini, dalam urutan yang sama dengan Tahap 1.

Variabel dan tipe

```
const nama = "Kopi Wamena"   // tidak bisa diganti
let stok = 24                // bisa diganti
typeof 25000                 // "number"   (juga "string", "boolean", "object",
"undefined")
Number("1250")  String(1250)  parseFloat("3.75 kg")  parseInt("12 buah")
Number.isNaN(x)              // memeriksa NaN
null  undefined  NaN         // "sengaja kosong", "belum diisi", "hitungan gagal"
```

Operator

Jenis	Operator
Aritmetika	`+ - * / % **`
Penugasan	`= += -= *= /= ++ --`
Perbandingan	`=== !== < > <= >=` (hindari `==`)
Logika	`&& \
Pilihan singkat	`syarat ? a : b`
Aman untuk nilai kosong	`obj?.prop`, `nilai ?? cadangan`

Percabangan dan perulangan

```
if (a) { ... } else if (b) { ... } else { ... }
switch (kode) { case "JYP": ...; break; default: ... }

for (let i = 0; i < n; i++) { ... }
for (const x of larik) { ... }
while (syarat) { ... }
break    continue
```

Fungsi

```
function luas(p, l = 1) { return p * l }
const pajak = harga => harga * 0.11
const ongkir = (kg, kota) => { const t = 8000; return Math.ceil(kg) * t }
async function ambil() { const r = await fetch(url); return r.json() }
```

Larik

Metode	Hasil	Contoh
`push`, `pop`	Tambah/ambil di akhir (mengubah larik)	`a.push(5)`
`slice(a, b)`	Potongan baru	`a.slice(0, 3)`
`includes`, `indexOf`	Ada? Di mana?	`a.includes("Ambon")`
`map(fn)`	Larik baru, setiap isi diubah	`a.map(x => x * 2)`
`filter(fn)`	Larik baru, yang memenuhi syarat	`a.filter(x => x > 10)`
`reduce(fn, awal)`	Satu nilai ringkasan	`a.reduce((s, x) => s + x, 0)`
`sort(fn)`	Urut (mengubah larik)	`[...a].sort((x, y) => x - y)`
`find`, `findIndex`	Isi/indeks pertama yang cocok	`a.find(x => x.id === 3)`
`some`, `every`	Ada yang / semua memenuhi?	`a.some(x => x === 0)`
`join(sep)`	Teks	`a.join(", ")`
`at(-1)`	Isi terakhir	`a.at(-1)`
`[...new Set(a)]`	Nilai unik	
`Math.max(...a)`	Terbesar	

Objek

```
const p = { nama: "Sagu", harga: 30000 }
p.nama   p["harga"]   p.stok = 40   delete p.stok   "stok" in p
Object.keys(p)   Object.values(p)   Object.entries(p)
Object.fromEntries(pasangan)
const { nama, harga } = p              // destructuring
const salinan = { ...p, harga: 32000 } // spread
const peta = new Map([["JYP", "Jayapura"]]); peta.get("JYP"); peta.has("SOQ")
```

Teks

```
s.trim()  s.toLowerCase()  s.toUpperCase()  s.includes("x")  s.startsWith("Papua")
s.split(",")  s.replace("a", "b")  s.replaceAll("-", "/")  s.padStart(4, "0")
s.slice(0, 7)
`Total ${n}`                          // template literal
s.replace(/[^0-9]/g, "")              // buang semua yang bukan angka
s.match(/^(\d{4})-(\d{2})-(\d{2})$/) // tangkap bagian-bagian
a.localeCompare(b, "id")              // pembanding teks untuk sort
```

Angka dan tanggal

```
Math.round(x)  Math.floor(x)  Math.ceil(x)  x.toFixed(2)  // toFixed menghasilkan
teks
new Intl.NumberFormat("id-ID", { style: "currency", currency: "IDR",
maximumFractionDigits: 0 }).format(x)
new Intl.NumberFormat("id-ID", { style: "percent", maximumFractionDigits: 1
}).format(0.123)
(1250000).toLocaleString("id-ID")

new Date("2024-04-10")       new Date(2024, 3, 10)   // bulan dari 0!
d.getFullYear()  d.getMonth() + 1  d.getDate()  d.getDay()   // 0 = Minggu
(b - a) / (24 * 60 * 60 * 1000)                               // selisih hari
new Intl.DateTimeFormat("id-ID", { dateStyle: "long" }).format(d)
d.toISOString().slice(0, 10)
```

Galat dan async

```
try { ... } catch (e) { e.message } finally { ... }
throw new Error("Pesan")
await sl.sleep(500)
const [a, b] = await Promise.all([ambilA(), ambilB()])
```

JSON

```
JSON.parse(teks)          // teks → objek
JSON.stringify(obj)       // objek → teks
JSON.stringify(obj, null, 1)
```

Class

```
class Cabang {
  constructor(nama) { this.nama = nama }
  get label() { return "Nusantara " + this.nama }
  sapa() { return `Halo dari ${this.nama}` }
}
const c = new Cabang("Ambon")
```

← A · Ringkasan API Slak di Buku Ini

C · Cheat Sheet SQL →

Lampiran C · Cheat Sheet SQL

Semua kueri di lampiran ini memakai gaya yang jalan di SQLite (`sl.sql`, `sl.sqlite`), DuckDB (`sl.duckdb`), dan PostgreSQL.

Kerangka kueri

```
SELECT kolom, FUNGSI(kolom) AS nama_baru     -- 5. pilih kolom
FROM tabel                                   -- 1. ambil tabel
WHERE syarat_baris                           -- 2. saring baris
GROUP BY kolom                               -- 3. kelompokkan
HAVING syarat_kelompok                       -- 4. saring kelompok
ORDER BY nama_baru DESC                      -- 6. urutkan
LIMIT 10                                     -- 7. batasi
```

Menyaring

```
WHERE kota = 'Ambon'                         -- teks memakai kutip tunggal
WHERE total BETWEEN 100000 AND 500000
WHERE kota IN ('Jayapura', 'Sorong')
WHERE produk LIKE 'Kopi%'                    -- diawali "Kopi"
WHERE kanal IS NULL
WHERE tanggal >= '2024-01-01'
WHERE kota = :kota                           -- parameter dari widget (Bab 21.3)
```

Fungsi ringkasan

`COUNT(*)`, `COUNT(DISTINCT pelanggan_id)`, `SUM(total)`, `AVG(total)`, `MIN(...)`, `MAX(...)`, `ROUND(x, 0)`.

Tanggal (portabel)

```
EXTRACT(YEAR FROM tanggal)        -- tahun
EXTRACT(MONTH FROM tanggal)       -- bulan 1–12
EXTRACT(DOW FROM tanggal)         -- hari: 0 Minggu … 6 Sabtu
to_char(tanggal, 'YYYY-MM')       -- teks bulan
date_trunc('month', tanggal)      -- awal bulan
```

Kondisi

```
CASE WHEN total >= 500000 THEN 'besar'
     WHEN total >= 100000 THEN 'sedang'
     ELSE 'kecil' END AS ukuran
COALESCE(kanal, 'Tidak diketahui')
SUM(CASE WHEN kanal = 'Online' THEN total ELSE 0 END)   -- jumlah bersyarat
```

JOIN

```
SELECT t.produk, p.pemasok, t.total - t.jumlah * p.harga_pokok AS laba
FROM jual AS t
LEFT JOIN produk AS p ON p.produk = t.produk
```

Jenis	Hasil
`JOIN` / `INNER JOIN`	Hanya yang cocok di kedua tabel
`LEFT JOIN`	Semua baris kiri
`RIGHT JOIN`	Semua baris kanan
`FULL OUTER JOIN`	Semua baris dari keduanya

CTE

```
WITH bulanan AS (
  SELECT to_char(tanggal, 'YYYY-MM') AS bulan, SUM(total) AS omzet
  FROM jual GROUP BY 1
)
SELECT * FROM bulanan WHERE omzet > 10000000
```

Window function

```
RANK()       OVER (PARTITION BY kota ORDER BY omzet DESC)   -- peringkat per kota
ROW_NUMBER() OVER (ORDER BY tanggal)                         -- nomor urut
LAG(omzet)   OVER (PARTITION BY kota ORDER BY bulan)         -- nilai sebelumnya
LEAD(omzet)  OVER (PARTITION BY kota ORDER BY bulan)         -- nilai sesudahnya
SUM(omzet)   OVER (PARTITION BY kota ORDER BY bulan)         -- total berjalan
AVG(omzet)   OVER (ORDER BY bulan ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)  --
rata-rata 3 bulan
```

Khusus SQLite yang sebaiknya dihindari

Hindari	Pakai
`strftime('%Y-%m', t)`	`to_char(t, 'YYYY-MM')`
`strftime('%Y', t)`	`EXTRACT(YEAR FROM t)`
`IFNULL(x, 0)`	`COALESCE(x, 0)`
`GROUP_CONCAT(x)`	`string_agg(x, ', ')`

Periksa kuerimu dengan `sl.sql.check(kueri)`.

Di Slak

```
await sl.sql(kueri, { jual: df }, { params: { kota } })     // SQLite atas
DataFrame
const db = await sl.sqlite({ name: "tokoku" }); db.query(kueri, [nilai])   //
SQLite tersimpan
await sl.duckdb(kueri, { jual: df })                        // DuckDB (slak-
duckdb.js)
await sl.duckdb("SELECT ... FROM 'berkas.parquet'")         // DuckDB membaca
berkas
await sl.report(teksMarkdownDenganSQL, { tables: { jual: df } })
```

← B · Cheat Sheet JavaScript untuk Data

D · Kunci Jawaban Latihan →

Lampiran D · Kunci Jawaban Latihan

Lampiran ini berisi jawaban lengkap untuk latihan tingkat **mudah** dan **sedang**, jawaban mini-proyek Bab 9, dan **petunjuk** untuk latihan tingkat **tantangan**. Latihan tantangan sengaja tidak diberi jawaban lengkap: uji otomatis di setiap latihan sudah memberi tahu apakah jawabanmu benar.

Setiap jawaban di bawah memuat ujinya, jadi saat dijalankan semua kotaknya hijau. Jawabanmu tidak harus sama persis; yang penting ujinya lulus.

Bab 1

Latihan 1.1. Jawaban bebas, misalnya:

```
sl.title("Rina Wenda")
sl.write("Hobi saya membaca novel dan bersepeda di tepi Danau Sentani.")
```

Rina Wenda

Hobi saya membaca novel dan bersepeda di tepi Danau Sentani.

Buka di Playground ↗

Latihan 1.2.

```
const hargaBuku = 45000
const hargaPena = 7500
const total = 2 * hargaBuku + 3 * hargaPena
sl.write("Total: " + total)

await sl.testing.test("total belanja benar", () =>
sl.testing.expect(total).toBe(112500))
```

Total: 112500

✓ total belanja benar (0 md)

Buka di Playground ↗

Latihan 1.3 (petunjuk). `let tautan = await Slak.share.link('sl.title("Kuis Ibu Kota")', "https://slak.studio/playground/")`. Jangan lupa `await`.

Bab 2

Latihan 2.1.

```
const produk = "Sagu Lempeng 1kg"
const harga = 30000
const kalimat = `${produk} harganya ${harga}`
sl.write(kalimat)

await sl.testing.test("kalimat benar", () => sl.testing.expect(kalimat).toBe("Sagu Lempeng 1kg harganya 30000"))
```

Sagu Lempeng 1kg harganya 30000

✓ kalimat benar (0 md)

Buka di Playground ↗

Latihan 2.2.

```
const jumlahTeks = "4"
const hargaTeks = "22000"
const total = Number(jumlahTeks) * Number(hargaTeks)

sl.write(total)
await sl.testing.test("total berupa angka", () => sl.testing.expect(typeof total).toBe("number"))
await sl.testing.test("total = 88000", () => sl.testing.expect(total).toBe(88000))
```

88.000

✓ total berupa angka (0 md)

✓ total = 88000 (0 md)

Buka di Playground ↗

Latihan 2.3 (petunjuk). `Number(isian)` menghasilkan `NaN` bila isian bukan angka. `Number.isNaN(...)` bernilai `true` untuk `NaN`, jadi `sah` adalah kebalikannya: pakai `!`.

Bab 3

Latihan 3.1.

```
function lulus(nilai) {
  if (nilai >= 75) return "Lulus"
  return "Belum lulus"
}

await sl.testing.test("80 lulus", () => sl.testing.expect(lulus(80)).toBe("Lulus"))
await sl.testing.test("75 lulus", () => sl.testing.expect(lulus(75)).toBe("Lulus"))
await sl.testing.test("74 belum lulus", () =>
sl.testing.expect(lulus(74)).toBe("Belum lulus"))
```

- ✓ 80 lulus (0 md)
- ✓ 75 lulus (0 md)
- ✓ 74 belum lulus (0 md)

Buka di Playground ↗

Latihan 3.2.

```
function diskon(total, anggota) {
  if (total >= 500000 || anggota) return total * 0.1
  return 0
}

await sl.testing.test("600 ribu, bukan anggota", () =>
sl.testing.expect(diskon(600000, false)).toBe(60000))
await sl.testing.test("200 ribu, anggota", () => sl.testing.expect(diskon(200000,
true)).toBe(20000))
await sl.testing.test("200 ribu, bukan anggota", () =>
sl.testing.expect(diskon(200000, false)).toBe(0))
```

- ✓ 600 ribu, bukan anggota (0 md)
- ✓ 200 ribu, anggota (0 md)
- ✓ 200 ribu, bukan anggota (0 md)

Buka di Playground ↗

Latihan 3.3 (petunjuk). Periksa dari syarat yang paling khusus: habis dibagi 400 → kabisat; habis dibagi 100 → bukan; habis dibagi 4 → kabisat; selain itu bukan. "Habis dibagi 4" ditulis `tahun % 4 === 0`.

Bab 4

Latihan 4.1.

```
let jumlah = 0
for (let i = 1; i <= 100; i++) jumlah += i

sl.write(jumlah)
await sl.testing.test("1 + 2 + ... + 100 = 5050", () =>
sl.testing.expect(jumlah).toBe(5050))
```

5.050

- ✓ 1 + 2 + … + 100 = 5050 (0 md)

Buka di Playground ↗

Latihan 4.2.

```
const omzet = [1250000, 980000, 1430000, 1100000, 1675000, 2010000, 1890000]
let hariRamai = 0
for (const x of omzet) {
  if (x > 1500000) hariRamai++
}

sl.write(hariRamai)
await sl.testing.test("ada 3 hari ramai", () =>
sl.testing.expect(hariRamai).toBe(3))
```

3

✓ ada 3 hari ramai (0 md)

Buka di Playground ↗

Latihan 4.3 (petunjuk). Simpan dua hal di luar perulangan: nilai terendah sejauh ini dan posisinya. Pakai `for (let i = 0; i < omzet2.length; i++)` supaya kamu punya indeks `i`.

Bab 5

Latihan 5.1.

```
function luasPersegiPanjang(p, l) {
  return p * l
}

await sl.testing.test("4 × 5 = 20", () => sl.testing.expect(luasPersegiPanjang(4,
5)).toBe(20))
await sl.testing.test("2,5 × 2 = 5", () =>
sl.testing.expect(luasPersegiPanjang(2.5, 2)).toBe(5))
```

✓ 4 × 5 = 20 (0 md)

✓ 2,5 × 2 = 5 (0 md)

Buka di Playground ↗

Latihan 5.2.

```
const rentang = (data) => Math.max(...data) - Math.min(...data)

await sl.testing.test("rentang [3, 9, 4] = 6", () => sl.testing.expect(rentang([3,
9, 4])).toBe(6))
await sl.testing.test("rentang [5] = 0", () =>
sl.testing.expect(rentang([5])).toBe(0))
```

✓ rentang [3, 9, 4] = 6 (0 md)

✓ rentang [5] = 0 (0 md)

Buka di Playground ↗

Latihan 5.3 (petunjuk). Salin dulu dengan `[...data]`, urutkan dengan `sort((a, b) => a - b)`, lalu buang ujung-ujungnya dengan `slice(1, -1)`. Rata-ratakan sisanya.

Bab 6

Latihan 6.1.

```
const harga = [50000, 80000, 125000]
const hargaPromo = harga.map(h => h * 0.8)

sl.write(hargaPromo)
await sl.testing.test("dipotong 20%", () =>
sl.testing.expect(hargaPromo).toEqual([40000, 64000, 100000]))
```

```
[
  40000,
  64000,
  100000
]
```

✓ dipotong 20% (0 md)

Buka di Playground ↗

Latihan 6.2.

```
const nilai = [80, 62, 91, 75, 58, 88]
const lulus = nilai.filter(n => n >= 75)
const rataLulus = lulus.reduce((a, n) => a + n, 0) / lulus.length

sl.write(rataLulus)
await sl.testing.test("rata-rata yang lulus = 83.5", () =>
sl.testing.expect(rataLulus).toBe(83.5))
```

83,5

✓ rata-rata yang lulus = 83.5 (0 md)

Buka di Playground ↗

Latihan 6.3 (petunjuk). `[...omzet].sort((a, b) => b - a).slice(0, 3)`. Salinan `[...omzet]` menjaga larik aslinya.

Bab 7

Latihan 7.1.

```
const cabang = { kota: "Ambon", buka: 2021 }

sl.write(cabang)
await sl.testing.test("kota Ambon", () =>
sl.testing.expect(cabang.kota).toBe("Ambon"))
await sl.testing.test("buka 2021", () => sl.testing.expect(cabang.buka).toBe(2021))
```

```
{
  "kota": "Ambon",
  "buka": 2021
}
```

✓ kota Ambon (0 md)

✓ buka 2021 (0 md)

Buka di Playground ↗

Latihan 7.2.

```
const rows = sl.dataset("penjualan").to_records()
const omzetPakaian = rows.filter(r => r.kategori === "Pakaian").reduce((a, r) => a
+ r.total, 0)

sl.metric("Omzet pakaian", omzetPakaian)
await sl.testing.test("omzet pakaian benar", () =>
sl.testing.expect(omzetPakaian).toBe(
  rows.filter(r => r.kategori === "Pakaian").reduce((a, r) => a + r.total, 0)))
await sl.testing.test("tidak nol", () =>
sl.testing.expect(omzetPakaian).toBeGreaterThan(0))
```

Omzet pakaian

57.535.000

✓ omzet pakaian benar (0 md)

✓ tidak nol (0 md)

Buka di Playground ↗

Latihan 7.3 (petunjuk). `for (const r of data) omzetPerKota[r.kota] = (omzetPerKota[r.kota] || 0) + r.total`.

Bab 8

Latihan 8.1.

```
const mentah = "  makassar "
const t = mentah.trim()
const rapi = t.charAt(0).toUpperCase() + t.slice(1)

sl.write(`[${rapi}]`)
await sl.testing.test("menjadi Makassar", () =>
sl.testing.expect(rapi).toBe("Makassar"))
```

[Makassar]

menjadi Makassar (0 md)

Buka di Playground ↗

Latihan 8.2.

```
function keAngka(teks) {
  return Number(String(teks).replace(/[^0-9]/g, ""))
}

await sl.testing.test('"Rp 42.000" → 42000', () => sl.testing.expect(keAngka("Rp
42.000")).toBe(42000))
await sl.testing.test('"42000" → 42000', () =>
sl.testing.expect(keAngka("42000")).toBe(42000))
await sl.testing.test('"Rp 1.250.000" → 1250000', () =>
sl.testing.expect(keAngka("Rp 1.250.000")).toBe(1250000))
```

"Rp 42.000" → 42000 (0 md)

"42000" → 42000 (0 md)

"Rp 1.250.000" → 1250000 (0 md)

Buka di Playground ↗

Latihan 8.3 (petunjuk). Selisih hari: `Math.round((new Date(b) - new Date(a)) / (24 * 60 * 60 * 1000))`. Nama hari: `new Intl.DateTimeFormat("id-ID", { weekday: "long" }).format(new Date(teks))`.

Bab 9

Latihan 9.1.

```
function namaHari(nomor) {
  const hari = ["Minggu", "Senin", "Selasa", "Rabu", "Kamis", "Jumat", "Sabtu"]
  return hari[nomor]
}

await sl.testing.test("0 = Minggu", () =>
sl.testing.expect(namaHari(0)).toBe("Minggu"))
await sl.testing.test("5 = Jumat", () =>
sl.testing.expect(namaHari(5)).toBe("Jumat"))
```

✓ 0 = Minggu (0 md)

✓ 5 = Jumat (0 md)

Buka di Playground ↗

Latihan 9.2.

```
function bacaHarga(teks) {
  try {
    return JSON.parse(teks)
  } catch (e) {
    return null
  }
}

await sl.testing.test("JSON sah terbaca", () =>
sl.testing.expect(bacaHarga('{"harga": 5000}')).toEqual({ harga: 5000 }))
await sl.testing.test("JSON rusak menjadi null", () =>
sl.testing.expect(bacaHarga('{"harga": ')).toBeNull())
```

✓ JSON sah terbaca (0 md)

✓ JSON rusak menjadi null (0 md)

Buka di Playground ↗

Latihan 9.3 (petunjuk). Lempar galat lebih dulu: `if (keseluruhan === 0) throw new Error("...")`. Untuk membulatkan satu desimal: `Math.round(x * 10) / 10`.

Mini-proyek Bab 9: kelima bug kasir.

1. `subtotal` menjumlah, padahal seharusnya mengalikan: `item.harga * item.jumlah`.
2. Harga gelang tersimpan sebagai teks `"40000"`: ubah dengan `Number(item.harga)`.
3. Perulangan di `totalBelanja` mulai dari `i = 1`, sehingga barang pertama terlewat: mulai dari `0`.
4. Diskon seharusnya berlaku **mulai tepat** 250.000: pakai `>=`, bukan `>`.
5. PPN 11% adalah `× 0,11`, bukan `× 11`.

```
sl.title("Kasir Toko Nusantara :convenience_store:")

const keranjang = [
  { produk: "Kopi Arabika Wamena 250g", harga: 85000, jumlah: 2 },
  { produk: "Keripik Keladi 200g", harga: 22000, jumlah: 3 },
  { produk: "Gelang Manik Papua", harga: "40000", jumlah: 1 }
]

function subtotal(item) {
  return Number(item.harga) * item.jumlah
}

function totalBelanja(daftar) {
  let total = 0
  for (let i = 0; i < daftar.length; i++) total += subtotal(daftar[i])
  return total
}

function diskon(total) {
  if (total >= 250000) return total * 0.05
  return 0
}

function ppn(setelahDiskon) {
  return setelahDiskon * 0.11
}

const total = totalBelanja(keranjang)
const pot = diskon(total)
const pajak = ppn(total - pot)

sl.dataframe(keranjang)
sl.metric("Total", total, null, { format: "rp" })
sl.metric("Bayar", total - pot + pajak, null, { format: "rp" })

const { test, expect } = sl.testing
```

```
await test("subtotal kopi = 170000", () =>
expect(subtotal(keranjang[0])).toBe(170000))
await test("subtotal gelang = 40000", () =>
expect(subtotal(keranjang[2])).toBe(40000))
await test("total belanja = 276000", () =>
expect(totalBelanja(keranjang)).toBe(276000))
await test("diskon 5% mulai tepat 250000", () =>
expect(diskon(250000)).toBe(12500))
await test("PPN 11% dari 100000 = 11000", () => expect(ppn(100000)).toBe(11000))
```

Kasir Toko Nusantara 🏪

	produk	harga	jumlah
0	Kopi Arabika Wamena 250g	85.000	2
1	Keripik Keladi 200g	22.000	3
2	Gelang Manik Papua	40000	1

3 baris × 3 kolom

Total

Rp 276.000

Bayar

Rp 291.042

- ✓ subtotal kopi = 170000 (0 md)
- ✓ subtotal gelang = 40000 (0 md)
- ✓ total belanja = 276000 (0 md)
- ✓ diskon 5% mulai tepat 250000 (0 md)
- ✓ PPN 11% dari 100000 = 11000 (0 md)

Buka di Playground ↗

Bab 10

Latihan 10.1. Baris `ss.hitung = 0` harus dijalankan hanya bila `hitung` belum ada.

```
const kode = `
const ss = sl.session_state
if (!("hitung" in ss)) ss.hitung = 0
if (sl.button("Tambah")) ss.hitung++
sl.metric("Hitungan", ss.hitung)
`
sl.code(kode)

await sl.testing.test("tiga klik menjadi 3", async () => {
  const at = await sl.testing.run(kode)
  for (let i = 0; i < 3; i++) await at.button("Tambah").click().run()
  sl.testing.expect(at.metric("Hitungan").value).toBe(3)
})
```

```
const ss = sl.session_state
if (!("hitung" in ss)) ss.hitung = 0
if (sl.button("Tambah")) ss.hitung++
sl.metric("Hitungan", ss.hitung)
```

✓ tiga klik menjadi 3 (1 md)

Buka di Playground ↗

Latihan 10.2.

```
const kode2 = `
const s = sl.session_state
if (!s.daftar) s.daftar = ["kopi", "sagu", "abon"]
const hapusTerakhir = () => { s.daftar = s.daftar.slice(0, -1) }
sl.button("Hapus terakhir", { on_click: hapusTerakhir })
sl.write(s.daftar.length + " barang")
`
sl.code(kode2)

await sl.testing.test("satu kali hapus menyisakan 2 barang", async () => {
  const at = await sl.testing.run(kode2)
  await at.button("Hapus terakhir").click().run()
  sl.testing.expect(at.session_state.daftar).toEqual(["kopi", "sagu"])
})
```

```
const s = sl.session_state
if (!s.daftar) s.daftar = ["kopi", "sagu", "abon"]
const hapusTerakhir = () => { s.daftar = s.daftar.slice(0, -1) }
sl.button("Hapus terakhir", { on_click: hapusTerakhir })
sl.write(s.daftar.length + " barang")
```

✓ satu kali hapus menyisakan 2 barang (1 md)

Buka di Playground ↗

Latihan 10.3 (petunjuk). Jalankan `sl.code(sl.crypto.quiz_hash("Manokwari"))` di playground, salin hasilnya (diawali `sq1$`), lalu tempelkan sebagai isi `KUNCI`.

Bab 11

Latihan 11.1.

```
const csv = `nama,harga
Kopi Wamena,85000
Sagu Lempeng,30000
Sambal Roa,45000`

const baris = csv.trim().split("\n").slice(1)
const produk = baris.map(b => {
  const [nama, harga] = b.split(",")
  return { nama, harga: Number(harga) }
})

sl.write(produk)
await sl.testing.test("ada 3 produk", () =>
sl.testing.expect(produk).toHaveLength(3))
await sl.testing.test("harga berupa angka", () =>
sl.testing.expect(produk[1]).toEqual({ nama: "Sagu Lempeng", harga: 30000 }))
```

	nama	harga
0	Kopi Wamena	85.000
1	Sagu Lempeng	30.000
2	Sambal Roa	45.000

3 baris × 2 kolom

✓ ada 3 produk (0 md)

✓ harga berupa angka (0 md)

Buka di Playground ↗

Latihan 11.2.

```
const cabang = (await sl.read_csv("cabang.csv")).to_records()
const kotaPapua = cabang.filter(c => c.provinsi.startsWith("Papua")).map(c =>
c.kota)

sl.write(kotaPapua)
await sl.testing.test("tiga kota di tanah Papua", () =>
sl.testing.expect(kotaPapua).toEqual(["Jayapura", "Sorong", "Manokwari"]))
```

```
[
  "Jayapura",
  "Sorong",
  "Manokwari"
]
```

✓ tiga kota di tanah Papua (0 md)

Buka di Playground ↗

Latihan 11.3 (petunjuk). `const teks = JSON.stringify(laporan)` lalu `const salinan = JSON.parse(teks)`.

Bab 12

Latihan 12.1.

```
async function ambilStok() {
  await sl.sleep(100)
  return 24
}
const stok = await ambilStok()

sl.write(stok)
await sl.testing.test("stok = 24", () => sl.testing.expect(stok).toBe(24))
```

```
24
```

✓ stok = 24 (0 md)

Buka di Playground ↗

Latihan 12.2.

```
const omzetCabang = async (n) => { await sl.sleep(300); return n }
const t0 = performance.now()

const hasil = await Promise.all([omzetCabang(100), omzetCabang(200),
omzetCabang(300)])
const total = hasil.reduce((a, b) => a + b, 0)

const lama = performance.now() - t0
await sl.testing.test("total = 600", () => sl.testing.expect(total).toBe(600))
await sl.testing.test("berjalan bersamaan", () =>
sl.testing.expect(lama).toBeLessThan(500))
```

✓ total = 600 (0 md)

✓ berjalan bersamaan (0 md)

Buka di Playground ↗

Latihan 12.3 (petunjuk). Bungkus `return await fn()` dengan `try { ... } catch (e) { return null }`. `await` di dalam `try` penting: tanpanya, galat dari Promise tidak tertangkap.

Bab 13

Latihan 13.1.

```
function bersihkanKanal(k) {
  if (k == null || String(k).trim() === "") return "Tidak diketahui"
  const t = String(k).trim().toLowerCase()
  return t.charAt(0).toUpperCase() + t.slice(1)
}

const { test, expect } = sl.testing
await test('" online " → Online', () => expect(bersihkanKanal(" online
")).toBe("Online"))
await test('"TOKO" → Toko', () => expect(bersihkanKanal("TOKO")).toBe("Toko"))
await test("null → Tidak diketahui", () => expect(bersihkanKanal(null)).toBe("Tidak
diketahui"))
await test('"" → Tidak diketahui', () => expect(bersihkanKanal("")).toBe("Tidak
diketahui"))
```

" online " → Online (0 md)

"TOKO" → Toko (0 md)

null → Tidak diketahui (0 md)

"" → Tidak diketahui (0 md)

Buka di Playground ↗

Latihan 13.2. Telusuri dari belakang, atau pakai `Map` yang selalu ditimpa dengan baris terbaru.

```
function buangGanda(data) {
  const terakhir = new Map()
  for (const r of data) terakhir.set(r.id, r)
  return [...terakhir.values()]
}

const d = [{ id: 1, v: "lama" }, { id: 2, v: "x" }, { id: 1, v: "baru" }]
await sl.testing.test("tersisa 2 baris", () =>
sl.testing.expect(buangGanda(d)).toHaveLength(2))
await sl.testing.test("id 1 berisi yang baru", () =>
sl.testing.expect(buangGanda(d).find(r => r.id === 1).v).toBe("baru"))
```

tersisa 2 baris (0 md)

id 1 berisi yang baru (0 md)

Buka di Playground ↗

Latihan 13.3 (petunjuk). Tambahkan pola `/^(\d{4})\.(\d{2})\.(\d{2})$/`. Titik di dalam pola harus ditulis `\.`, karena titik tanpa garis miring berarti "karakter apa saja".

Bab 14

Latihan 14.1.

```
const df = await sl.read_csv("toko-nusantara.csv")
const nOnline = df.query("kanal == 'Online'").shape[0]

sl.metric("Transaksi online", nOnline)
await sl.testing.test("banyak transaksi online benar", () =>
  sl.testing.expect(nOnline).toBe(df.to_records().filter(r => r.kanal ===
"Online").length))
```

Transaksi online

1.029

✓ banyak transaksi online benar (4 md)

Buka di Playground ↗

Latihan 14.2.

```
const df = await sl.read_csv("toko-nusantara.csv")
const murah = df
  .filter(r => r.kategori === "Makanan" && r.harga < 40000)
  .select(["produk", "harga"])
  .sort_values("harga")

sl.dataframe(murah.head())
await sl.testing.test("dua kolom", () =>
sl.testing.expect(murah.columns).toEqual(["produk", "harga"]))
await sl.testing.test("semua di bawah 40 ribu", () =>
sl.testing.expect(murah.harga.max()).toBeLessThan(40000))
await sl.testing.test("urut naik", () => sl.testing.expect(murah.harga.to_list()
[0]).toBe(murah.harga.min()))
```

	produk	harga
0	Keripik Keladi 200g	22.000
1	Keripik Keladi 200g	22.000
2	Keripik Keladi 200g	22.000
3	Keripik Keladi 200g	22.000
4	Keripik Keladi 200g	22.000

5 baris × 2 kolom

- ✓ dua kolom (0 md)
- ✓ semua di bawah 40 ribu (0 md)
- ✓ urut naik (0 md)

Buka di Playground ↗

Latihan 14.3 (petunjuk). `df.assign({ margin: r => r.total * 0.4 })`, lalu saring kanal Toko dan ambil `.margin.mean()`.

Bab 15

Latihan 15.1.

```
const df = await sl.read_csv("toko-nusantara.csv")
const nPelanggan = df.pelanggan_id.nunique()

sl.metric("Pelanggan", nPelanggan)
await sl.testing.test("banyak pelanggan benar", () =>
sl.testing.expect(nPelanggan).toBe(new Set(df.pelanggan_id.to_list()).size))
```

Pelanggan

631

banyak pelanggan benar (0 md)

Buka di Playground ↗

Latihan 15.2.

```
const df = await sl.read_csv("toko-nusantara.csv")
const perCabang = df.groupby("cabang").agg({
  omzet: ["total", "sum"],
  pelanggan: ["pelanggan_id", "nunique"]
}).sort_values("omzet", { ascending: false })

sl.dataframe(perCabang)
await sl.testing.test("10 cabang", () =>
sl.testing.expect(perCabang.shape[0]).toBe(10))
await sl.testing.test("kolom benar", () =>
sl.testing.expect(perCabang.columns).toEqual(["cabang", "omzet", "pelanggan"]))
await sl.testing.test("teratas Surabaya", () =>
sl.testing.expect(perCabang.to_records()[0].cabang).toBe("Nusantara Surabaya"))
```

	cabang	omzet	pelanggan
0	Nusantara Surabaya	44.400.600	131
1	Nusantara Jayapura	41.290.150	120
2	Nusantara Jakarta	39.196.450	138
3	Nusantara Makassar	35.906.200	121
4	Nusantara Ambon	33.681.000	84
5	Nusantara Denpasar	33.258.200	93
6	Nusantara Manado	30.546.500	92
7	Nusantara Sorong	25.393.100	86
8	Nusantara Manokwari	24.455.250	75
9	Nusantara Kupang	22.188.500	60

10 baris × 3 kolom

10 cabang (0 md)

kolom benar (0 md)

teratas Surabaya (0 md)

Buka di Playground ↗

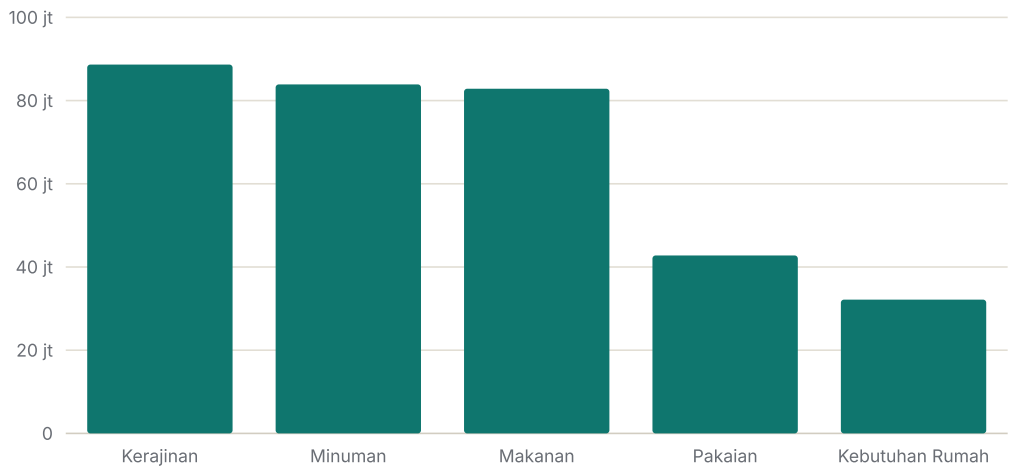
Latihan 15.3 (petunjuk). Saring kategori Minuman dulu ke variabel `m`, lalu bagi omzet online `m` dengan omzet seluruh `m`.

Bab 16

Latihan 16.1.

```
const df = await sl.read_csv("toko-nusantara.csv")
const perKategori = df.groupby("kategori").agg({ omzet: ["total", "sum"] })

sl.bar_chart(perKategori, { x: "kategori", y: "omzet", sort: true })
await sl.testing.test("lima kategori", () =>
sl.testing.expect(perKategori.shape[0]).toBe(5))
await sl.testing.test("kolom omzet ada", () =>
sl.testing.expect(perKategori.columns).toContain("omzet"))
```



lima kategori (0 md)

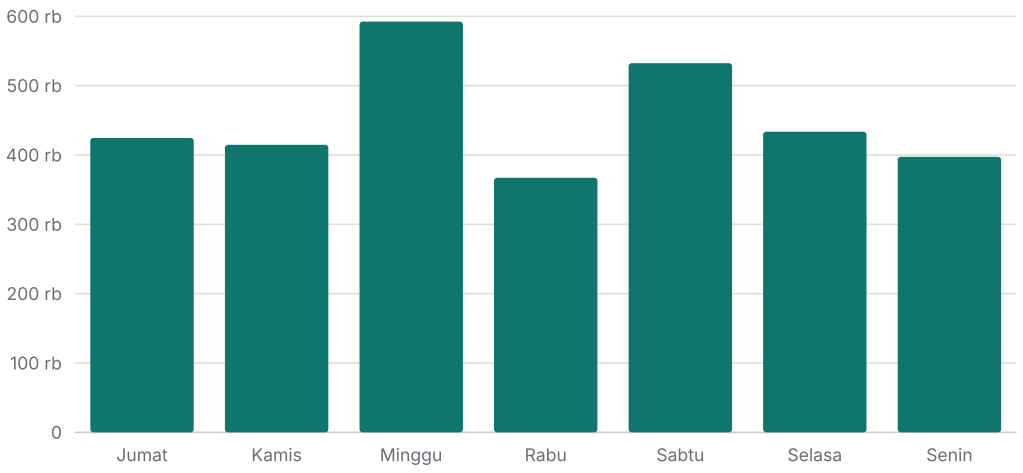
kolom omzet ada (0 md)

Buka di Playground ↗

Latihan 16.2.

```
const df = await sl.read_csv("toko-nusantara.csv")
const harian = df.resample("tanggal", "D").agg({ omzet: ["total", "sum"] })
const perHari = harian
  .assign({ hari: () => harian.tanggal.dt.day_name() })
  .groupby("hari").agg({ rata_omzet: ["omzet", "mean"] })

sl.bar_chart(perHari, { x: "hari", y: "rata_omzet" })
await sl.testing.test("tujuh hari", () =>
sl.testing.expect(perHari.shape[0]).toBe(7))
await sl.testing.test("Sabtu lebih ramai dari Selasa", () => {
  const r = Object.fromEntries(perHari.to_records().map(x => [x.hari,
x.rata_omzet]))
  sl.testing.expect(r.Sabtu).toBeGreaterThan(r.Selasa)
})
```



✓ tujuh hari (0 md)

✓ Sabtu lebih ramai dari Selasa (0 md)

Buka di Playground ↗

Latihan 16.3 (petunjuk). `df.assign({ bulan: () => df.tanggal.dt.month, tahun: () => df.tanggal.dt.year.astype("str") })`, lalu `groupby(["tahun", "bulan"])`.

Bab 17

Latihan 17.1.

```
const df = await sl.read_csv("toko-nusantara.csv")
function saring(data, kanal) {
  if (kanal === "Semua") return data
  return data.filter(r => r.kanal === kanal)
}

await sl.testing.test("Semua tidak menyaring", () => sl.testing.expect(saring(df,
"Semua").shape[0]).toBe(df.shape[0]))
await sl.testing.test("Online saja", () => sl.testing.expect(saring(df,
"Online").kanal.unique()).toEqual(["Online"]))
```

✓ Semua tidak menyaring (0 md)

✓ Online saja (3 md)

Buka di Playground ↗

Latihan 17.2.

```
const aplikasi = `
const df = await sl.read_csv("toko-nusantara.csv")
const tahun = Number(sl.selectbox("Tahun", ["2024", "2023"]))
const data = df.filter(r => r.tanggal.getFullYear() === tahun)
sl.metric("Omzet", data.total.sum())
`
sl.code(aplikasi)

const df = await sl.read_csv("toko-nusantara.csv")
await sl.testing.test("pilih 2023", async () => {
  const at = await sl.testing.run(aplikasi)
  await at.selectbox("Tahun").set_value("2023").run()
  sl.testing.expect(at.metric("Omzet").value).toBe(df.filter(r =>
r.tanggal.getFullYear() === 2023).total.sum())
})
```

```
const df = await sl.read_csv("toko-nusantara.csv")
const tahun = Number(sl.selectbox("Tahun", ["2024", "2023"]))
const data = df.filter(r => r.tanggal.getFullYear() === tahun)
sl.metric("Omzet", data.total.sum())
```

pilih 2023 (11 md)

Buka di Playground ↗

Latihan 17.3 (petunjuk). Kelompokkan per cabang, ambil yang teratas, lalu format porsinya dengan `(porsi * 100).toLocaleString("id-ID", { minimumFractionDigits: 1, maximumFractionDigits: 1 })`.

Bab 18

Latihan 18.1.

```
const trx = await sl.read_csv("toko-nusantara.csv")
const cabang = await sl.read_csv("cabang.csv")
const hasil = trx.merge(cabang.select(["cabang", "kode"]), { on: "cabang", how:
"left" })

await sl.testing.test("banyak baris tetap", () =>
sl.testing.expect(hasil.shape[0]).toBe(trx.shape[0]))
await sl.testing.test("kolom kode ada", () =>
sl.testing.expect(hasil.columns).toContain("kode"))
await sl.testing.test("tidak ada kode kosong", () =>
sl.testing.expect(hasil.kode.isna().sum()).toBe(0))
```

banyak baris tetap (0 md)

kolom kode ada (0 md)

tidak ada kode kosong (0 md)

Buka di Playground ↗

Latihan 18.2.

```
function tambahKategori(trx, produk) {
  const kamus = new Map(produk.map(p => [p.produk, p.kategori]))
  return trx.map(t => ({ ...t, kategori_produk: kamus.get(t.produk) ?? null }))
}

const trx = [{ produk: "Topi Rotan", total: 85000 }, { produk: "Sambal Roa 200g",
total: 45000 }, { produk: "Tidak Ada", total: 1 }]
const produk = (await sl.read_csv("produk.csv")).to_records()
const h = tambahKategori(trx, produk)
await sl.testing.test("Topi Rotan → Pakaian", () =>
sl.testing.expect(h[0].kategori_produk).toBe("Pakaian"))
await sl.testing.test("produk tidak dikenal → null", () =>
sl.testing.expect(h[2].kategori_produk).toBeNull())
await sl.testing.test("data asli tidak berubah", () =>
sl.testing.expect("kategori_produk" in trx[0]).toBe(false))
```

- ✓ Topi Rotan → Pakaian (0 md)
- ✓ produk tidak dikenal → null (0 md)
- ✓ data asli tidak berubah (0 md)

Buka di Playground ↗

Latihan 18.3 (petunjuk). Tiru contoh 18.4: kelompokkan omzet per `["cabang", "bulan"]`, gabungkan dengan `target` memakai dua kunci, lalu hitung baris dengan `omzet >= target` dibagi seluruh baris.

Bab 19

Latihan 19.1.

```
const df = await sl.read_csv("toko-nusantara.csv")
const [q1, q3] = sl.stats.quantile(df.total, [0.25, 0.75])
const iqr = q3 - q1

sl.write(iqr)
await sl.testing.test("IQR benar", () =>
sl.testing.expect(iqr).toBe(sl.stats.iqr(df.total)))
```

86.000

✓ IQR benar (1 md)

Buka di Playground ↗

Latihan 19.2.

```
function batasPencilan(x) {
  const [q1, q3] = sl.stats.quantile(x, [0.25, 0.75])
  const iqr = q3 - q1
  return [q1 - 1.5 * iqr, q3 + 1.5 * iqr]
}

const df = await sl.read_csv("toko-nusantara.csv")
await sl.testing.test("batas sama dengan Slak", () =>
sl.testing.expect(batasPencilan(df.total)).toEqual(df.outliers("total").bounds))
```

✓ batas sama dengan Slak (3 md)

Buka di Playground ↗

Latihan 19.3 (petunjuk). Saring dua cabang dulu: `df.filter(r => r.kota === "Jayapura" || r.kota === "Jakarta")`, lalu `sl.stats.ttest_ind(data, { value: "total", group: "kota" })`. `berbeda` adalah `hasil.p_value < 0.05`.

Bab 20

Latihan 20.1.

```
const df = await sl.read_csv("toko-nusantara.csv")
const kuartalan = df.resample("tanggal", "Q").agg({ omzet: ["total", "sum"] })

sl.dataframe(kuartalan)
await sl.testing.test("8 kuartal", () =>
sl.testing.expect(kuartalan.shape[0]).toBe(8))
await sl.testing.test("jumlahnya sama dengan total omzet", () =>
sl.testing.expect(kuartalan.omzet.sum()).toBe(df.total.sum()))
```

	tanggal	omzet
0	1 Jan 2023	32.477.000
1	1 Apr 2023	36.824.800
2	1 Jul 2023	36.011.000
3	1 Okt 2023	39.817.000
4	1 Jan 2024	38.171.050
5	1 Apr 2024	50.735.600
6	1 Jul 2024	36.087.000
7	1 Okt 2024	60.192.500

8 baris × 2 kolom

✓ 8 kuartal (0 md)

✓ jumlahnya sama dengan total omzet (0 md)

Buka di Playground ↗

Latihan 20.2.

```
function pertumbuhan(x) {
  return x.map((v, i) => i === 0 ? null : v / x[i - 1] - 1)
}

await sl.testing.test("[100, 110, 99]", () => {
  const p = pertumbuhan([100, 110, 99])
  sl.testing.expect(p[0]).toBeNull()
  sl.testing.expect(p[1]).toBeCloseTo(0.1, 6)
  sl.testing.expect(p[2]).toBeCloseTo(-0.1, 6)
})
```

✓ [100, 110, 99] (0 md)

Buka di Playground ↗

Latihan 20.3 (petunjuk). Untuk setiap tahun, `resample` per bulan, cari baris dengan omzet terbesar dengan `reduce`, lalu simpan `tanggal.getMonth() + 1`.

Bab 21

Latihan 21.1.

```
const df = await sl.read_csv("toko-nusantara.csv")
const hasil = await sl.sql(`
  SELECT kanal, SUM(total) AS omzet
  FROM jual
  GROUP BY kanal
  ORDER BY omzet DESC
`, { jual: df })

sl.dataframe(hasil)
const r = hasil.to_records()
await sl.testing.test("dua kanal", () => sl.testing.expect(r).toHaveLength(2))
await sl.testing.test("Toko teratas", () =>
sl.testing.expect(r[0].kanal).toBe("Toko"))
await sl.testing.test("omzet benar", () => sl.testing.expect(r[0].omzet +
r[1].omzet).toBe(df.total.sum()))
```

	kanal	omzet
0	Toko	219.365.050
1	Online	110.950.900

2 baris × 2 kolom

- ✓ dua kanal (0 md)
- ✓ Toko teratas (0 md)
- ✓ omzet benar (0 md)

Buka di Playground ↗

Latihan 21.2.

```
const df = await sl.read_csv("toko-nusantara.csv")
const hasil = await sl.sql(`
  WITH per_produk AS (
    SELECT kota, produk, SUM(jumlah) AS terjual,
           RANK() OVER (PARTITION BY kota ORDER BY SUM(jumlah) DESC) AS peringkat
    FROM jual
    GROUP BY kota, produk
  )
  SELECT kota, produk, terjual
  FROM per_produk
  WHERE peringkat = 1
  GROUP BY kota
  ORDER BY kota
`, { jual: df })

sl.dataframe(hasil)
await sl.testing.test("sepuluh kota", () =>
sl.testing.expect(hasil.shape[0]).toBe(10))
await sl.testing.test("kolom benar", () =>
sl.testing.expect(hasil.columns).toEqual(["kota", "produk", "terjual"]))
```

	kota	produk	terjual
0	Ambon	Keripik Keladi 200g	50
1	Denpasar	Kopi Arabika Wamena 250g	64
2	Jakarta	Sabun Sereh Wangi	68
3	Jayapura	Keripik Keladi 200g	57
4	Kupang	Keripik Keladi 200g	37
5	Makassar	Kopi Robusta Toraja 250g	59
6	Manado	Sabun Sereh Wangi	52
7	Manokwari	Minyak Kayu Putih 60ml	38
8	Sorong	Keripik Keladi 200g	45
9	Surabaya	Sabun Sereh Wangi	60

10 baris × 3 kolom

✓ sepuluh kota (0 md)

kolom benar (0 md)

Buka di Playground ↗

`GROUP BY kota` di bagian akhir memastikan satu baris per kota walau ada dua produk yang terjual sama banyak.

Latihan 21.3 (petunjuk). CTE berisi `to_char(tanggal, 'YYYY-MM') AS bulan` dan `SUM(total) AS omzet` dengan `WHERE EXTRACT(YEAR FROM tanggal) = 2024`, lalu `SUM(omzet) OVER (ORDER BY bulan) AS kumulatif`.

Bab 22

Latihan 22.1.

```
function regresi(x, y) {
  const n = x.length
  const rx = x.reduce((a, v) => a + v, 0) / n
  const ry = y.reduce((a, v) => a + v, 0) / n
  let atas = 0, bawah = 0
  for (let i = 0; i < n; i++) {
    atas += (x[i] - rx) * (y[i] - ry)
    bawah += (x[i] - rx) ** 2
  }
  const b = atas / bawah
  return { a: ry - b * rx, b }
}

const hasil = regresi([1, 2, 3, 4], [3, 5, 7, 9])
await sl.testing.test("kemiringan 2", () =>
sl.testing.expect(hasil.b).toBeCloseTo(2, 6))
await sl.testing.test("titik potong 1", () =>
sl.testing.expect(hasil.a).toBeCloseTo(1, 6))
```

kemiringan 2 (0 md)

titik potong 1 (0 md)

Buka di Playground ↗

Latihan 22.2.

```
const df = await sl.read_csv("toko-nusantara.csv")
const rfmKu = sl.analysis.rfm(df, { customer: "pelanggan_id", date: "tanggal",
amount: "total" })
const porsiBerisiko = rfmKu.filter(r => r.segmen === "Berisiko" || r.segmen ===
"Jangan Sampai Hilang").shape[0] / rfmKu.shape[0]

sl.metric("Pelanggan berisiko", (porsiBerisiko * 100).toFixed(1) + "%")
const rfm = sl.analysis.rfm(df, { customer: "pelanggan_id", date: "tanggal",
amount: "total" })
await sl.testing.test("porsi benar", () =>
sl.testing.expect(porsiBerisiko).toBeCloseTo(
  rfm.filter(r => ["Berisiko", "Jangan Sampai Hilang"].includes(r.segmen)).shape[0]
/ rfm.shape[0], 6))
```

Pelanggan berisiko

13.9%

✓ porsi benar (1 md)

Buka di Playground ↗

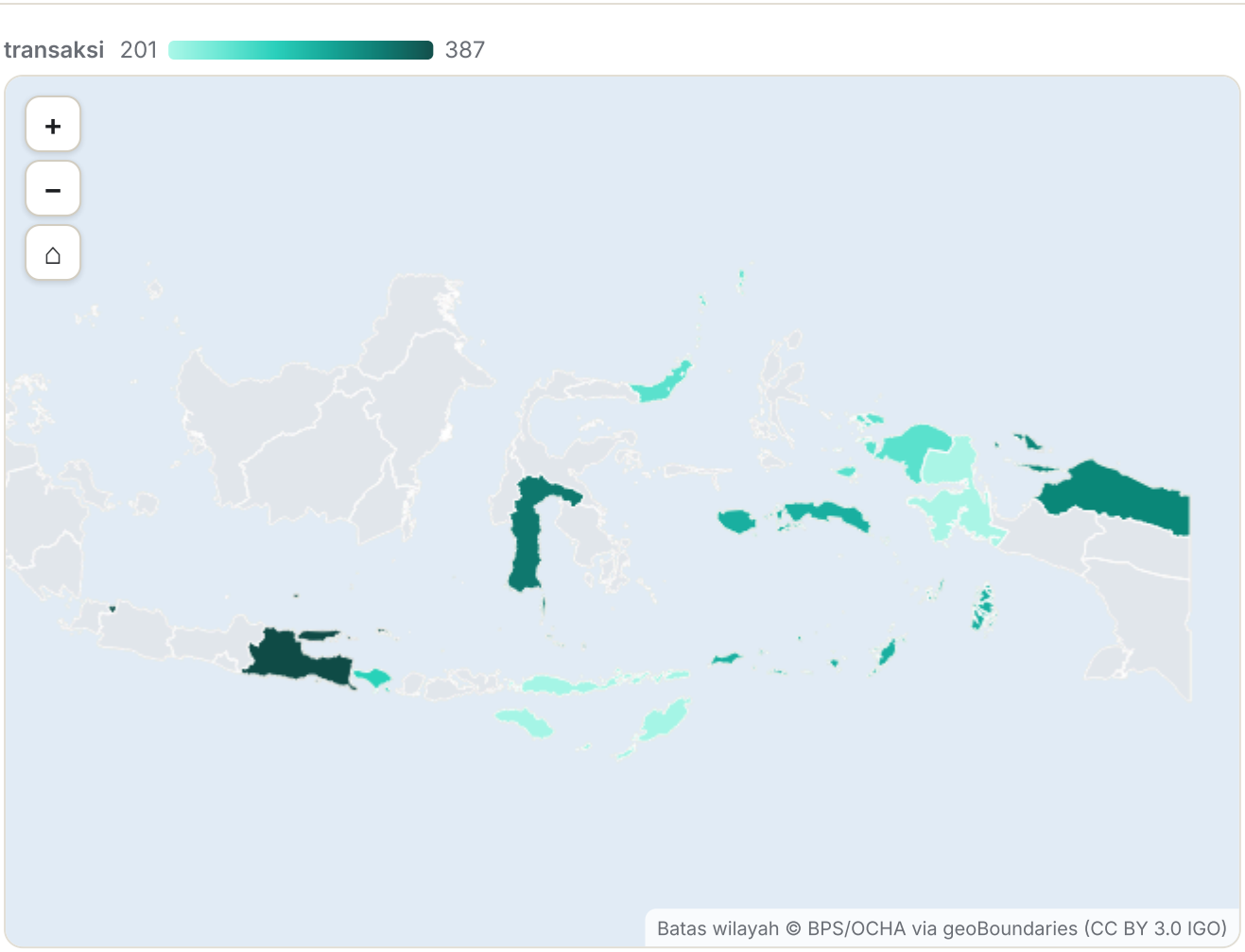
Latihan 22.3 (petunjuk). Tiru contoh pohon keputusan di 22.5, ganti dengan `sl.ml.RandomForestClassifier({ n_estimators: 100, max_depth: 8 })`, lalu `sl.ml.accuracy_score(uji.churn, tebakan)`.

Bab 23

Latihan 23.1.

```
const trx = await sl.read_csv("toko-nusantara.csv")
const perProvinsi = trx.groupby("provinsi").agg({ transaksi: ["id_transaksi",
"nunique"] })

sl.choropleth(perProvinsi, { region: "provinsi", value: "transaksi" })
await sl.testing.test("10 provinsi", () =>
sl.testing.expect(perProvinsi.shape[0]).toBe(10))
await sl.testing.test("jumlahnya benar", () =>
sl.testing.expect(perProvinsi.transaksi.sum()).toBe(trx.shape[0]))
```



✓ 10 provinsi (0 md)

✓ jumlahnya benar (0 md)

Buka di Playground ↗

Latihan 23.2.

```
const cabang = (await sl.read_csv("cabang.csv")).to_records()
const jkt = cabang.find(c => c.kota === "Jakarta")
const terjauh = cabang
  .map(c => ({ kota: c.kota, km: sl.geo.distance([jkt.lat, jkt.lon], [c.lat,
c.lon]) }))
  .reduce((a, c) => c.km > a.km ? c : a).kota

sl.write(terjauh)
await sl.testing.test("terjauh dari Jakarta", () =>
sl.testing.expect(terjauh).toBe("Jayapura"))
```

Jayapura

✓ terjauh dari Jakarta (0 md)

Buka di Playground ↗

Latihan 23.3 (petunjuk). `daftar.map(n => { const m = sl.geo.match(n); return m ? m.nama : null })`.

Bab 24

Latihan 24.1.

```
const Format = {
  rupiah(x) { return new Intl.NumberFormat("id-ID", { style: "currency", currency:
"IDR", maximumFractionDigits: 0 }).format(x) },
  persen(x) { return new Intl.NumberFormat("id-ID", { style: "percent",
maximumFractionDigits: 1 }).format(x) }
}

await sl.testing.test("rupiah", () =>
sl.testing.expect(Format.rupiah(1250000)).toBe(new Intl.NumberFormat("id-ID", {
style: "currency", currency: "IDR", maximumFractionDigits: 0 }).format(1250000)))
await sl.testing.test("persen", () =>
sl.testing.expect(Format.persen(0.125)).toBe("12,5%"))
```

✓ rupiah (0 md)

✓ persen (0 md)

Buka di Playground ↗

Latihan 24.2.

```
class Produk {
  constructor(nama, hargaJual, hargaPokok) {
    this.nama = nama
    this.hargaJual = hargaJual
    this.hargaPokok = hargaPokok
  }
  get margin() { return (this.hargaJual - this.hargaPokok) / this.hargaJual }
  laku(n) { return (this.hargaJual - this.hargaPokok) * n }
}

const kopi = new Produk("Kopi Wamena", 85000, 52000)
await sl.testing.test("margin kopi", () =>
sl.testing.expect(kopi.margin).toBeCloseTo(0.388, 3))
await sl.testing.test("laba 10 buah", () =>
sl.testing.expect(kopi.laku(10)).toBe(330000))
```

✓ margin kopi (0 md)

✓ laba 10 buah (0 md)

Buka di Playground ↗

Latihan 24.3 (petunjuk). `await at.slider("Diskon (%)").set_value(25).run()`, lalu `sl.testing.expect(at.metric("Bayar").value).toBe(75000)`.

Bab 25

Latihan Bab 25 terbuka. Contoh arah penyelesaiannya:

Latihan 25.1. Tambahkan fungsi halaman berikut, lalu daftarkan `sl.page("Pelanggan", halPelanggan, { icon: "👥" })` di kelompok "Analisis".

```
function halPelanggan(sl) {
  sl.title("Pelanggan")
  sl.caption(keterangan)
  const rfm = sl.analysis.rfm(ini, { customer: "pelanggan_id", date: "tanggal",
amount: "total" })
  sl.bar_chart(rfm.value_counts("segmen"), { x: "segmen", y: "jumlah", horizontal:
true, sort: true })
  sl.dataframe(rfm.filter(r => r.segmen === "Berisiko").sort_values("monetary", {
ascending: false }).head(10))
}
```

Latihan 25.2. Tambahkan `const pembanding = sl.sidebar.radio("Bandingkan dengan", ["Tahun lalu", "Target"], { key: "f_banding" })` di bagian filter. Di halaman Ringkasan, hitung total target tahun terpilih dari `target.csv` (saring `bulan` yang diawali tahun itu, dan cabang terpilih), lalu pakai sebagai `a` di `banding(a, r.omzet)` bila pembandingnya "Target".

Latihan 25.3. Di dalam uji, ulangi untuk setiap kombinasi `["2024", "2023"]` × `["Semua", "Toko", "Online"]`. Untuk setiap kombinasi, set widgetnya, baca `at.metric("Omzet").value` di halaman Ringkasan, pindah ke halaman Penjualan dengan `at.switch_page("Penjualan")`, lalu bandingkan dengan jumlah kolom `omzet` grafik per cabang: `at.bar_chart[0].value`. Bila datanya berupa DataFrame, jumlahkan dengan `.omzet.sum()`.

← C · Cheat Sheet SQL

E · Dataset Toko Nusantara →

Lampiran E · Dataset Toko Nusantara

Toko Nusantara adalah jaringan toko oleh-oleh **fiktif** dengan sepuluh cabang dari Jakarta sampai Jayapura. Semua datanya dibuat dengan program, dengan pola yang mirip data nyata: omzet tumbuh dari tahun ke tahun, ramai menjelang Lebaran dan akhir tahun, lebih ramai di akhir pekan, dan kanal online yang porsinya terus naik. Nama pelanggan tidak ada; setiap pelanggan hanya punya kode. Dataset ini bebas dipakai untuk belajar dan untuk portofolio, asalkan disebut sebagai data fiktif.

Berkas

Berkas	Baris	Dipakai di
`toko-nusantara.csv`	2.929	Bab 11–25
`toko-nusantara-kotor.csv`	394	Bab 13–14
`produk.csv`	20	Bab 18, 21, 25
`cabang.csv`	10	Bab 11, 18, 23, 25
`target.csv`	240	Bab 18, 25
`toko-nusantara-besar.csv`	250.006 (29 MB)	Bab 12, 21 (diunduh terpisah)

Setiap berkas kecil punya pasangan `.csv.js` (data yang dibungkus), supaya `sl.read_csv` tetap jalan saat halaman dibuka dengan klik dua kali. Bila kamu menyalin datanya ke proyekmu sendiri, salin kedua berkasnya ke folder yang sama dengan halamanmu.

toko-nusantara.csv

Satu baris adalah satu transaksi, 1 Januari 2023 sampai 31 Desember 2024, dari 631 pelanggan.

Kolom	Tipe	Contoh	Keterangan
`id_transaksi`	teks	`TN000001`	Unik untuk setiap baris
`tanggal`	tanggal	`2023-01-01`	Format ISO
`pelanggan_id`	teks	`P0039`	Pelanggan bisa belanja di beberapa cabang
`cabang`	teks	`Nusantara Surabaya`	Sepuluh cabang
`kota`	teks	`Surabaya`	
`provinsi`	teks	`Jawa Timur`	Nama 38 provinsi terbaru
`kanal`	teks	`Toko` / `Online`	

Kolom	Tipe	Contoh	Keterangan
`kategori`	teks	`Minuman`	Minuman, Makanan, Kerajinan, Kebutuhan Rumah, Pakaian
`produk`	teks	`Kopi Arabika Wamena 250g`	20 produk
`jumlah`	bilangan bulat	`2`	Banyak barang
`harga`	bilangan bulat	`85000`	Harga satuan; naik sekitar 5% di 2024
`diskon_persen`	bilangan bulat	`0` / `10`	Diskon 10% di sebagian transaksi menjelang Lebaran
`total`	bilangan bulat	`170000`	`jumlah × harga` setelah diskon

toko-nusantara-kotor.csv

Sebagian transaksi tahun 2024 dari Jayapura, Makassar, dan Surabaya, yang sengaja dibuat berantakan seperti data dari banyak kasir:

Masalah	Contoh
Tanggal tiga format	`2024-02-14`, `14/02/2024`, `14 Februari 2024`
Nama kota tidak seragam	`jayapura`, `JAYAPURA`, `Jayapra`, `Makasar`, `Surabya`
Harga sebagian berupa teks	`Rp 42.000`
Spasi dan huruf besar-kecil	`GELANG MANIK PAPUA`, `sabun sereh wangi`
Nilai kosong	Kolom `jumlah` dan `kanal`
Transaksi ganda	14 `id_transaksi` muncul dua kali
Pencilan salah ketik	`jumlah` 250

Kolomnya: `id_transaksi`, `tanggal`, `kota`, `kanal`, `kategori`, `produk`, `jumlah`, `harga`.

produk.csv

Kolom	Keterangan
`produk`	Nama produk; kunci untuk digabung dengan transaksi
`kategori`	Kategori produk
`harga_jual`	Harga jual 2023

Kolom	Keterangan
`harga_pokok`	Harga beli dari pemasok
`pemasok`	Sembilan pemasok, misalnya Koperasi Baliem dan Sanggar Noken Abepura

cabang.csv

Kolom	Keterangan
`kode`	Kode tiga huruf, misalnya `JYP`
`cabang`	Nama cabang; kunci untuk digabung dengan transaksi
`kota`, `provinsi`	Lokasi
`lat`, `lon`	Koordinat kota
`tanggal_buka`	Tanggal cabang mulai beroperasi

target.csv

Target omzet bulanan setiap cabang, 2023–2024. Kuncinya dua kolom: `cabang` dan `bulan` (teks `YYYY-MM`). Kolom `target` dalam rupiah.

toko-nusantara-besar.csv

Kolom sama dengan `toko-nusantara.csv`, tetapi berisi sekitar 250 ribu transaksi dari 20.000 pelanggan. Dipakai untuk merasakan pembaca CSV di Web Worker (Bab 12) dan DuckDB (Bab 21). Berkas ini tidak dibungkus menjadi `.csv.js` karena terlalu besar; buka lewat Slak Studio, hosting, atau pilih berkasnya saat diminta.

Dataset bawaan Slak yang dipakai

Selain Toko Nusantara, buku ini memakai dataset bawaan `slak-data.js` (selalu tersedia tanpa berkas):

Nama	Isi	Dipakai di
`penjualan`	3.278 transaksi toko di Papua 2023–2024	Bab 7, 12
`penjualan_bulanan`	Omzet bulanan 2019–2024	Bab 20
`karyawan`	300 karyawan dan gajinya	Bab 19, 22
`ab_test`	Eksperimen checkout, 4.000 pengunjung	Bab 19
`pelanggan`	1.000 pelanggan dan status churn	Bab 22

Cara membuat ulang

Dataset dibuat dengan skrip `buat_data.py` di folder bahan buku, dengan benih acak tetap, sehingga angkanya selalu sama dengan yang tertulis di buku.

← D · Kunci Jawaban Latihan

F · Daftar Pesan Galat Umum →

Lampiran F · Daftar Pesan Galat Umum

Pesan galat JavaScript ditulis dalam bahasa Inggris. Slak Studio menambahkan penjelasan berbahasa Indonesia di konsol (baris 💡). Tabel ini mengumpulkan galat yang paling sering ditemui pembaca buku ini.

Galat JavaScript

Pesan	Artinya	Biasanya karena	Bab
`x is not defined`	Variabel `x` tidak dikenal	Salah ketik, beda huruf besar-kecil, atau dipakai di luar fungsinya	2, 5, 9
`Assignment to constant variable`	Mengisi ulang variabel `const`	Perlu `let`, atau `=` yang seharusnya `===`	2, 3, 9
`Cannot read properties of undefined (reading 'y')`	Membaca `sesuatu.y`, padahal `sesuatu` kosong	`find` tidak menemukan apa-apa, indeks di luar batas	6, 7, 9
`x.map is not a function`	`x` bukan larik	Data berupa objek, teks, atau DataFrame	6, 9
`x is not a function`	Memanggil sesuatu yang bukan fungsi	Salah nama, atau lupa tanda kurung	5
`Unexpected token`	Penulisan tidak sah	Kurung, kutip, atau koma tidak berpasangan	1, 9
`Unexpected end of input`	Kode berhenti di tengah	`{` atau `(` belum ditutup	9
`missing ) after argument list`	Kurung tutup hilang	Kurang `)` di pemanggilan fungsi	9
`Invalid Date`	Tanggal tidak terbaca	Format bukan ISO `YYYY-MM-DD`	8
`NaN` (bukan galat, tetapi hasil)	Hitungan gagal	Teks seperti `"Rp 5.000"` diubah ke angka	2, 8, 13
`Unexpected token ... in JSON`	Teks bukan JSON sah	Kutip tunggal, koma di akhir	9, 11
`await is only valid in async functions`	`await` di fungsi biasa	Tandai fungsinya `async`	12
`[object Promise]` di layar	Nilai belum ditunggu	Lupa `await`	12
`Failed to fetch`	Pengambilan data gagal	Tidak ada internet, alamat salah, atau CORS	12

Pesan	Artinya	Biasanya karena	Bab
`reduce of empty array with no initial value`	`reduce` pada larik kosong	Lupa nilai awal	6

Galat dan pesan Slak

Pesan	Artinya	Perbaikan	Bab
`sl is not defined`	Kode tidak dijalankan Slak	Tulis di `<script type="text/slak">`, dan muat `slak.js` lebih dulu	1
`Fungsi sl.metrik tidak ada. Maksudmu sl.metric?`	Salah ketik nama fungsi	Ikuti saran Slak Studio	9
`Key widget dipakai lebih dari sekali`	Dua widget dengan `key` sama	Buat setiap `key` unik	10
`Kolom "x" tidak ada. Kolom yang tersedia: …`	Nama kolom salah	Lihat `df.columns`	14
`sl.groupby is not a function`	Awalan tertukar	Mengolah tabel memakai nama tabel: `df.groupby(...)`	14
Peringatan "Data … baris sudah di atas batas nyaman"	Data di atas 100 ribu baris	Pertimbangkan DuckDB atau ringkas dulu	11, 12
Permintaan memilih berkas	Halaman dibuka dengan klik dua kali dan tidak ada `.csv.js`	Bungkus data, atau buka lewat hosting	11
`no such column`, `no such table` (SQL)	Nama kolom atau tabel salah	Teks memakai kutip tunggal; nama tabel sesuai objek kedua `sl.sql`	21
`misuse of aggregate` (SQL)	`SUM` di `WHERE`	Pindahkan ke `HAVING`	21
`sl.duckdb is not a function`	`slak-duckdb.js` belum dimuat	Tambahkan `<script src="slak-duckdb.js">`	21
"1 wilayah tidak dikenali: …" di bawah peta	Nama wilayah tidak cocok	Perbaiki ejaan, atau pakai koordinat	23
`Failed to fetch dynamically imported module`	ES Module dari halaman klik dua kali	Pakai berkas `.js` biasa, atau buka lewat hosting	24

Pesan	Artinya	Perbaikan	Bab
`sl.testing: … tidak ditemukan. Yang ada: …`	Label widget di uji tidak sama	Samakan dengan label di aplikasi	24

Langkah umum saat bingung

1. Baca pesan sampai habis, termasuk nomor barisnya.
2. Intip isi variabel dengan `sl.write(x)` atau `console.log(x)`.
3. Jadikan baris yang dicurigai komentar satu per satu.
4. Coba dengan data kecil.
5. Jelaskan kodenya baris demi baris, keras-keras.

← E · Dataset Toko Nusantara

G · Memakai Contoh Streamlit dari Internet →

Lampiran G · Memakai Contoh Streamlit dari Internet

Gaya Slak terinspirasi **Streamlit**, kerangka aplikasi data populer di Python. Ribuan contoh Streamlit tersedia di internet, dan sebagian besar bisa kamu terjemahkan ke Slak dengan mudah, karena nama perintahnya dibuat sama.

Nama perintah yang sama

Hampir semua perintah `st.*` di Streamlit punya padanan `sl.*` dengan nama yang sama persis: `st.title` menjadi `sl.title`, `st.slider` menjadi `sl.slider`, `st.dataframe` menjadi `sl.dataframe`, `st.session_state` menjadi `sl.session_state`, dan seterusnya. Paket Slak 1.9.1 berisi `PARITAS.md`, daftar 104 perintah Streamlit beserta padanannya, yang diperiksa otomatis dengan `sl.testing`: 80 sama, 23 mirip (jalan dengan perbedaan kecil yang dicatat), dan semua 42 perintah yang paling sering dipakai pemula jalan.

Menerjemahkan Python ke JavaScript

Yang berbeda adalah **bahasanya**. Berikut padanan yang paling sering diperlukan:

Streamlit (Python)	Slak (JavaScript)
`import streamlit as st`	(tidak perlu)
`nama = st.text_input("Nama")`	`const nama = sl.text_input("Nama")`
`n = st.slider("Angka", 0, 100, 50)`	`const n = sl.slider("Angka", 0, 100, 50)`
`st.write(f"Halo {nama}")`	`` `sl.write( Halo ${nama} )` ``
`if st.button("Kirim"):` lalu baris menjorok	`if (sl.button("Kirim")) { ... }`
`for x in daftar:`	`for (const x of daftar) { ... }`
`col1, col2 = st.columns(2)`	`const [col1, col2] = sl.columns(2)`
`with col1:` lalu `st.write(...)`	`col1.write(...)`
`with st.sidebar:` lalu `st.slider(...)`	`sl.sidebar.slider(...)`
`st.slider("A", min_value=0, max_value=10)`	`sl.slider("A", { min: 0, max: 10 })`
`df = pd.read_csv("data.csv")`	`const df = await sl.read_csv("data.csv")`
`df[df.total > 100]`	`df.filter(df.total.gt(100))` atau `df.query("total > 100")`

Streamlit (Python)	Slak (JavaScript)
`df.groupby("k").agg(n=("a", "sum"))`	`df.groupby("k").agg({ n: ["a", "sum"] })`
`df.sort_values("a", ascending=False)`	`df.sort_values("a", { ascending: false })`
`True`, `False`, `None`	`true`, `false`, `null`
`and`, `or`, `not`	`&&`, `||`, `!`
`len(daftar)`	`daftar.length`
`# komentar`	`// komentar`

Aturan umumnya:

- **Argumen bernama** Python (`min_value=0`) ditulis sebagai **objek** di Slak (`{ min: 0 }`). Bentuk berurutan seperti `sl.slider("A", 0, 100, 50)` juga diterima.
- **Blok** `with` Python ditulis dengan memanggil fungsi pada wadahnya: `col1.write(...)`, `sl.sidebar.slider(...)`.
- **Membaca data** butuh `await`, karena di browser membaca berkas adalah pekerjaan asinkron (Bab 12).
- **"Magic"** Streamlit, yaitu variabel yang ditulis sendirian di satu baris lalu otomatis tampil, tidak ada di Slak. Tulis `sl.write(x)`.

Contoh terjemahan

Streamlit:

```
import streamlit as st
import pandas as pd

st.title("Penjualan")
df = pd.read_csv("penjualan.csv")
kota = st.sidebar.multiselect("Kota", df["kota"].unique())
if kota:
    df = df[df["kota"].isin(kota)]
st.metric("Omzet", f"{df['total'].sum():,}")
st.bar_chart(df.groupby("kategori")["total"].sum())
```

Slak:

```
sl.title("Penjualan")
let df = await sl.read_csv("toko-nusantara.csv")
const kota = sl.sidebar.multiselect("Kota", df.kota.unique())
if (kota.length) df = df.filter(r => kota.includes(r.kota))
sl.metric("Omzet", df.total.sum())
sl.bar_chart(df.groupby("kategori").agg({ total: ["total", "sum"] }), { x:
"kategori", y: "total" })
```

Nama lama Streamlit

Contoh Streamlit yang lebih tua kadang memakai nama yang sudah diganti di Streamlit sendiri. Slak 1.9 tetap mengenali 12 nama lama ini, dengan peringatan sekali di konsol yang menyebut nama barunya. Nama lama tetap bekerja sampai Slak 2.0, tetapi untuk kode baru pakai nama barunya.

Nama lama	Nama baru
`sl.beta_columns`	`sl.columns`
`sl.beta_expander`	`sl.expander`
`sl.beta_container`	`sl.container`
`sl.beta_tabs`	`sl.tabs`
`sl.experimental_rerun`	`sl.rerun`
`sl.experimental_dialog`	`sl.dialog`
`sl.experimental_fragment`	`sl.fragment`
`sl.experimental_get_query_params`	`sl.query_params`
`sl.experimental_set_query_params`	`sl.query_params`
`sl.cache`	`sl.cache_data`
`sl.experimental_memo`	`sl.cache_data`
`sl.experimental_singleton`	`sl.cache_resource`

Daftar ini juga tersedia dari kode: `Slak.old_names`.

Yang tidak bisa diterjemahkan langsung

Karena Slak berjalan di browser tanpa server, beberapa hal Streamlit berbeda:

Streamlit	Di Slak
`st.secrets`	Sengaja tidak menyimpan apa pun: rahasia harus disimpan di server (slak-bridge), karena isi halaman bisa dibaca pengunjung
Koneksi langsung ke MySQL/PostgreSQL	Lewat `slak-bridge.php` atau `slak-bridge.js` (Bab 26)
Pustaka Python apa pun	Slak Data untuk analisis umum; Python asli lewat `sl.python` (butuh internet)
`st.progress` berskala 0–1	`sl.progress` berskala 0–100
`st.date_input` mengembalikan objek tanggal	`sl.date_input` mengembalikan teks `YYYY-MM-DD`

← F · Daftar Pesan Galat Umum

Glosarium →

Glosarium

Istilah pemrograman dan analisis data yang dipakai di buku ini, beserta padanan bahasa Inggrisnya. Angka di akhir menunjuk bab tempat istilah itu dijelaskan.

Istilah	Inggris	Arti
agregasi	aggregation	Meringkas banyak nilai menjadi satu, misalnya jumlah atau rata-rata (15)
akumulator	accumulator	Variabel penampung yang diperbarui di setiap putaran (4, 6)
argumen	argument	Nilai yang dikirim saat memanggil fungsi (5)
asinkron	asynchronous	Pekerjaan yang hasilnya datang belakangan (12)
boolean	boolean	Tipe data bernilai `true` atau `false` (2)
callback	callback	Fungsi yang dikirim untuk dipanggil nanti, misalnya saat tombol diklik (10)
class	class	Cetakan untuk membuat objek sejenis (24)
CTE	common table expression	Hasil antara bernama di SQL, ditulis dengan `WITH` (21)
CSV	comma-separated values	Teks berisi tabel, kolom dipisah koma (11)
DataFrame	DataFrame	Tabel milik Slak Data, gaya pandas (14)
deret waktu	time series	Data yang diurutkan menurut waktu (20)
destructuring	destructuring	Mengambil beberapa isi objek atau larik sekaligus ke variabel (7)
DOM	Document Object Model	Struktur halaman web yang bisa diubah JavaScript (10, 26)
fungsi	function	Kumpulan perintah bernama yang bisa dipanggil berulang (5)
fungsi panah	arrow function	Cara ringkas menulis fungsi: `x => x * 2` (5)
galat	error	Kesalahan yang menghentikan program (9)
hipotesis nol	null hypothesis	Anggapan awal "tidak ada perbedaan" dalam uji statistik (19)
indeks	index	Posisi isi larik, dimulai dari 0 (6)

Istilah	Inggris	Arti
jalan ulang	rerun	Slak menjalankan ulang seluruh skrip setiap ada interaksi (10)
JOIN / gabung	join / merge	Menggabungkan dua tabel lewat kunci (18, 21)
JSON	JavaScript Object Notation	Teks berbentuk objek dan larik JavaScript (11)
klaster	cluster	Kelompok data yang mirip, ditemukan sendiri oleh model (22)
klasifikasi	classification	Menebak kelompok, misalnya churn atau tidak (22)
komentar	comment	Teks di kode yang diabaikan komputer, diawali `//` (2)
korelasi	correlation	Ukuran seberapa kuat dua variabel bergerak bersama (19)
kuartil	quartile	Nilai yang membagi data menjadi empat bagian sama banyak (19)
kunci	key	Kolom penghubung dua tabel; juga identitas widget di Slak (10, 18)
larik	array	Daftar nilai berurutan (6)
machine learning	machine learning	Program yang belajar aturan dari contoh (22)
median	median	Nilai tengah data yang diurutkan (5)
metode	method	Fungsi milik sebuah objek, misalnya `teks.trim()` (8, 24)
modul	module	Berkas JavaScript yang membagikan fungsi dengan `export` (24)
modus	mode	Nilai yang paling sering muncul (5)
objek	object	Kumpulan pasangan nama dan nilai (7)
overfitting	overfitting	Model menghafal data latih sehingga buruk pada data baru (22)
p-value	p-value	Peluang melihat hasil sebesar ini bila hipotesis nol benar (19)
parameter	parameter	Nama penerima nilai di dalam fungsi (5)
pencilan	outlier	Nilai yang jauh berbeda dari kebanyakan data (13, 19)
percabangan	branching	Memilih jalan program dengan `if` atau `switch` (3)

Istilah	Inggris	Arti
perulangan	loop	Mengulang perintah dengan `for` atau `while` (4)
persentil	percentile	Nilai yang di bawahnya terdapat persen tertentu dari data (19)
pratinjau	preview	Bagian yang menampilkan aplikasi yang sedang berjalan (1)
Promise	Promise	Janji bahwa hasil akan datang nanti (12)
properti	property	Satu pasangan nama dan nilai di dalam objek (7)
R^2	R-squared	Seberapa banyak naik-turunnya data dijelaskan model, 0 sampai 1 (22)
rata-rata bergerak	moving average	Rata-rata beberapa periode terakhir untuk menghaluskan data (20)
regex	regular expression	Pola untuk mencari atau mengganti teks (8, 13)
regresi	regression	Model untuk menebak angka, misalnya garis tren (22)
scope	scope	Wilayah tempat sebuah variabel dikenal (5)
Series	Series	Satu kolom DataFrame (14)
simpangan baku	standard deviation	Ukuran seberapa jauh nilai menyimpang dari rata-rata (19)
spread	spread	Menyalin isi objek atau larik dengan `...` (7)
state	state	Nilai yang diingat antar-jalan ulang (10)
string / teks	string	Tipe data berisi teks (2)
tautan berbagi	share link	Alamat playground yang memuat kode (1)
template literal	template literal	Teks bertanda backtick dengan sisipan `${...}` (2)
tipe data	data type	Jenis nilai: angka, teks, boolean, dan seterusnya (2)
uji A/B	A/B test	Eksperimen membandingkan dua versi secara acak (19)
uji otomatis	automated test	Kode yang memeriksa hasil kode lain (9, 24)
variabel	variable	Nama untuk menyimpan nilai (2)
widget	widget	Kontrol interaktif: tombol, penggeser, kotak isian (3)
window function	window function	Fungsi SQL yang menghitung di sekitar setiap baris (21)

← G · Memakai Contoh Streamlit dari Internet

Write and Run · Belajar JavaScript untuk Analisis Data dengan Slak · Sesuai Slak 1.9.1 · Data Toko Nusantara adalah data fiktif